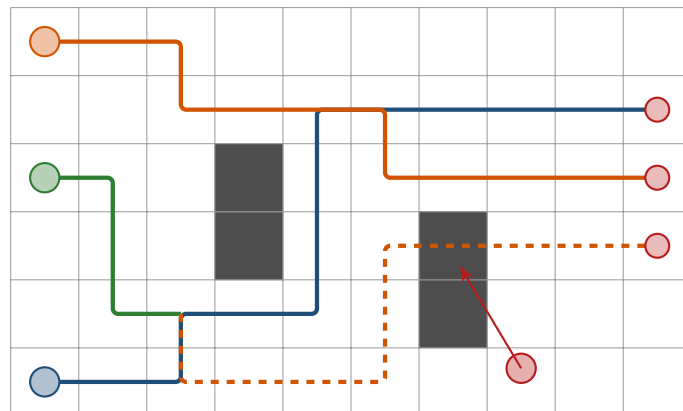


Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms:
From Graph Search to Hybrid Swarm Systems



The Searchbook Project

September 3, 2026

Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms: From Graph Search to Hybrid Swarm Systems

Typeset with L^AT_EX (pdfT_EX), TikZ and pgfplots. Figures were drawn in TikZ or generated from the Python reference implementations that accompany the book.

This book was written to accompany a twelve-week research training plan on multi-agent path planning and collision avoidance for drone swarms. It is intended for self-study. All algorithms described here are established published methods; the original sources are cited in each chapter and collected in the bibliography.

Source code, figures and L^AT_EX sources are distributed together with this book so that every example can be reproduced and every figure regenerated.

Compiled on September 3, 2026.

Contents

Preface	vii
Notation	x
I Foundations	1
1 Planning for Drone Swarms	2
1.1 The problem: many drones, one airspace, and a stranger	2
1.2 Vocabulary: the words you need before anything else	6
1.3 A preview of the hybrid architecture	8
1.4 A map of the book	12
1.5 What is established and where research remains	14
1.6 How to read this book	15
1.7 Summary	18
1.8 Exercises	19
2 The Toolbox: Graphs, Grids, Time, Motion and Uncertainty	21
2.1 Why this matters for a drone swarm	21
2.2 Graphs, grids and lattices	22
2.3 Workspace and configuration space	25
2.4 Time: space-time states, paths, plans and trajectories	26
2.5 Cost measures	28
2.6 Motion models for a drone	30
2.7 A geometry toolkit	32
2.8 Uncertainty: random vectors, Gaussians and Bayes' rule	35
2.9 Computation: complexity, priority queues and hashing	37
2.10 The Python stack and the code of the book	39
2.11 Where this fits in the drone system	41
2.12 Summary	42
2.13 Exercises	42
II Single-Agent Graph Search	44
3 Dijkstra's Algorithm	45
3.1 Why this matters for a drone swarm	45
3.2 The idea in plain words	46
3.3 Problem statement and notation	47
3.4 The algorithm	48
3.5 A worked example	49
3.6 Properties: correctness, optimality, complexity	52
3.7 Variants and extensions	54
3.8 Implementation notes	58

3.9	Where this fits in the drone system	61
3.10	Summary	61
3.11	Exercises	62
4	A* Search	64
4.1	Why this matters for a drone swarm	64
4.2	The idea in plain words	65
4.3	Problem statement and notation	66
4.4	The algorithm	68
4.5	A worked example	69
4.6	Properties: correctness, optimality, complexity	70
4.7	Variants and extensions	73
5	Incremental Search: LPA* and D* Lite	76
5.1	Why this matters for a drone swarm	76
5.2	The idea in plain words	77
5.3	Problem statement and notation	78
5.4	The algorithm: LPA*	79
5.5	A worked example: LPA* repairs a path	80
5.6	The algorithm: D* Lite	82
5.7	A worked example: the robot moves and an obstacle appears	84
5.8	Properties: correctness, optimality, complexity	85
5.9	Variants and extensions	88
5.10	Implementation notes	88
5.11	Where this fits in the drone system	92
5.12	Summary	92
5.13	Exercises	93
6	Anytime Search: ARA*	99
III	Multi-Agent Path Finding	100
7	The Multi-Agent Path Finding Problem	101
8	Prioritized Planning and Space-Time A*	102
8.1	Why this matters for a drone swarm	102
8.2	The idea in plain words	103
8.3	Problem statement and notation	104
8.4	The algorithm	105
9	Conflict-Based Search	107
10	Bounded-Suboptimal Search: ECBS	108
11	Coupling and Constructive Methods: M*, Push-and-Swap, Push-and-Rotate	109
IV	Local and Reactive Collision Avoidance	110
12	Velocity Obstacles	111
13	Reciprocal Avoidance: RVO and ORCA	112

14 The Dynamic Window Approach	113
15 Artificial Potential Fields	114
V Sampling-Based Motion Planning	115
16 Rapidly-Exploring Random Trees	116
17 Optimal Sampling-Based Planning: RRT* and Informed RRT*	117
VI Tracking and Prediction	118
18 The Kalman Filter	119
19 Nonlinear Filtering: EKF, UKF and Particle Filters	120
20 Trajectory Prediction: From Constant Velocity to LSTMs and Transformers	121
VII Optimization and Control	122
21 Model Predictive Control	123
22 Mixed-Integer Linear Programming for Planning	124
23 Consensus and Formation Control	125
VIII Putting It Together	126
24 The Hybrid Collision-Avoidance Architecture	127
25 Evaluating a Hybrid Planner: Experiments, Metrics and Benchmarks	128
IX Appendices	129
A The Twelve-Week Study Plan	130
A.1 How to use this plan	130
A.2 The schedule at a glance	131
A.3 The twelve weeks on one timeline	133
A.4 Study notes, week by week	133
A.5 The capstone: a hybrid collision-avoidance prototype	138
A.6 Completion checklist	140
A.7 Traceability: from the plan to the chapters	140
A.8 Reading order and primary sources	142
B Mathematical Refresher	144
C Hints and Solutions to Selected Exercises	145
Glossary	151

Bibliography	152
Index	155

List of Algorithms

2.1	Priority queue with lazy deletion	38
3.1	Dijkstra's algorithm with a binary heap and lazy deletion. Without a target it computes $\delta(s, v)$ for every vertex; with a target t it stops as soon as t is settled (uniform-cost search).	48
3.2	Backward Dijkstra: the true-distance heuristic table for a goal t	56
4.1	A* graph search with a closed set and a lazy priority queue	68
5.1	Lifelong Planning A* (LPA*) [23]	95
5.2	D* Lite, final version [21]	97
8.1	Prioritized planning	105
8.2	Cooperative A*: space-time A* against a reservation table	106

Preface

Why this book exists

Imagine a small fleet of drones that must fly through the same airspace at the same time. Each drone has a job to do and a route to fly. Before take-off, a planner must make sure that no two routes collide. During the flight, something unplanned happens: another drone, not part of the fleet and not talking to it, crosses the formation. The fleet must notice it, guess where it is going, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

Every part of that story is a well-studied algorithmic problem. Planning a route is *graph search*. Planning routes for many drones at once without collisions is *multi-agent path finding*. Reacting to a surprise in a fraction of a second is *local collision avoidance*. Guessing where the intruder is going is *tracking and trajectory prediction*. Making a new plan without starting from scratch is *incremental replanning*. Keeping the fleet in formation and within radio range while doing all of this is *control*. The individual algorithms are established and published. What is not settled is how to combine them into one system that keeps the guarantees of the global plan, reacts fast enough to be safe, and can be evaluated honestly.

This book teaches all of those algorithms, one at a time, from first principles, and then shows how they fit together into a hybrid swarm planner. It was written to accompany a twelve-week research training plan; the plan is reproduced, with week-by-week reading and coding assignments, in [Appendix A](#).

Who should read it

The book is for a graduate student or engineer who is starting research on drone swarms, or on multi-robot motion planning more generally, and who wants to *implement* the algorithms rather than merely recognise their names. You should be comfortable with Python and with the mathematics of a first course in linear algebra, calculus and probability. Everything else is introduced when it is needed, and [Appendix B](#) collects the mathematical facts that the book relies on. No prior knowledge of robotics or planning is assumed.

What you will be able to do

When you have worked through the book and its exercises you will be able to

- implement and explain A^* and space-time A^* , and say exactly why they return optimal paths under the required conditions;
- repair a path with D^* Lite instead of replanning from scratch, and measure when that is worth it;
- represent and detect the conflicts of a multi-agent plan, and solve the problem optimally with conflict-based search or approximately, with a guaranteed bound, with ECBS;
- explain velocity obstacles, reciprocal velocity obstacles and ORCA using relative position and velocity, and use them to keep drones apart;

- track an unknown drone from noisy observations with a Kalman filter, and compare a constant-velocity predictor with a learned one;
- trigger local avoidance from a time-to-collision or safety-horizon rule, return to the global route, or replan;
- enforce formation and communication-range constraints, and
- evaluate a hybrid planner with reproducible metrics and controlled scenarios.

How the book is organised

Part I, Foundations.

[Chapter 1](#) states the problem and previews the hybrid architecture that the whole book builds towards. [Chapter 2](#) is a toolbox: graphs, grids, configuration space, time-indexed paths, simple motion models, the geometry of moving discs, Gaussians and priority queues.

Part II, Single-Agent Graph Search.

Dijkstra’s algorithm ([Chapter 3](#)), A^* with admissible and consistent heuristics and its space-time version ([Chapter 4](#)), incremental replanning with LPA^* and D^* Lite ([Chapter 5](#)) and anytime search with ARA^* ([Chapter 6](#)).

Part III, Multi-Agent Path Finding.

The problem and its conflicts ([Chapter 7](#)), prioritized planning ([Chapter 8](#)), conflict-based search ([Chapter 9](#)), bounded-suboptimal search with ECBS ([Chapter 10](#)), and the coupled and constructive alternatives M^* , Push-and-Swap and Push-and-Rotate ([Chapter 11](#)).

Part IV, Local and Reactive Collision Avoidance.

Velocity obstacles ([Chapter 12](#)), reciprocal avoidance with RVO and ORCA ([Chapter 13](#)), the dynamic window approach ([Chapter 14](#)) and artificial potential fields ([Chapter 15](#)).

Part V, Sampling-Based Motion Planning.

RRT ([Chapter 16](#)) and the asymptotically optimal RRT^* with informed sampling ([Chapter 17](#)).

Part VI, Tracking and Prediction.

The Kalman filter ([Chapter 18](#)), its nonlinear relatives and the particle filter ([Chapter 19](#)), and trajectory prediction from constant-velocity baselines to LSTMs and Transformers ([Chapter 20](#)).

Part VII, Optimization and Control.

Model predictive control ([Chapter 21](#)), mixed-integer linear programming ([Chapter 22](#)) and consensus and formation control ([Chapter 23](#)).

Part VIII, Putting It Together.

The hybrid collision-avoidance architecture ([Chapter 24](#)) and how to evaluate it ([Chapter 25](#)).

How to read it

Read the chapters in order if you can: each part builds on the previous ones, and the notation is introduced once. If you are short of time, the training plan’s reading order is a good fast path: [Chapter 4](#) (A^*), [Chapter 5](#) (D^* Lite), [Chapter 9](#) (CBS), [Chapter 7](#) (the MAPF problem), [Chapters 12 and 13](#) (velocity obstacles and ORCA), [Chapter 14](#) (DWA), [Chapters 16 and 17](#) (RRT and RRT^*), [Chapter 21](#) (MPC) and [Chapter 20](#) (trajectory prediction), then [Chapter 24](#). Every chapter ends with a box that says where its algorithm sits in the hybrid system, so you can also read by *layer*: the global planner (Parts II and III), the local safety layer (Part IV), the prediction layer (Part VI) and the constraints (Part VII).

Conventions

Every chapter opens with a box of *learning objectives* and closes with a *summary* box and exercises. Inside a chapter you will meet five kinds of coloured boxes: *key idea* (green) states the one thing to remember; *in the drone system* (purple) says how the algorithm is used in the hybrid architecture; *pitfall* (orange) warns about a mistake that is easy to make; *historical note* (grey) gives context; and plain notes carry their own titles. Definitions, examples, theorems and remarks share one counter per chapter, so “Example 4.3” comes after “Definition 4.2”. Exercises are marked with one to three stars (★ routine, ★★ needs thought, ★★★ a small project), and hints or solutions for a selection of them are in [Appendix C](#). Where the training plan assigns a priority to an algorithm it is shown as a tag such as [Essential](#). Pseudocode is written in a uniform style with numbered lines; the text refers to line numbers. Vectors are bold ($\mathbf{p}$, $\mathbf{v}$), matrices are bold capitals ($\mathbf{A}$), and the full notation is listed in the table that follows this preface.

The code

Every algorithm in the book comes with a Python implementation in the `code/` folder of the L^AT_EX project. The implementations are deliberately simple—plain Python and NumPy, a few hundred lines each, no framework—because the goal is that you can read them in one sitting and then write your own. Each file runs a self-test when executed (`python3 code/ch04_astar.py`), and the numbers quoted in the worked examples were produced by that code. The figures that show algorithm output were generated by the scripts in `code/figures/` from fixed random seeds, so you can regenerate and modify them. The listings in the text are excerpts; the files contain the complete programs.

Sources

None of the algorithms in this book is new. Each chapter cites the paper in which its algorithm was introduced and a textbook that treats it at greater length, and the *further reading* paragraph at the end of every chapter points to the developments that the chapter leaves out. The bibliography lists all sources. Where the book simplifies an algorithm for teaching, it says so and names the full version.

Errata

The manuscript was reviewed in several rounds for technical accuracy, completeness against the training plan, and clarity. Mistakes will nevertheless remain; if you find one, the L^AT_EX sources and code are distributed with the book precisely so that it can be corrected.

Notation

Symbol	Meaning
$G = (V, E)$	a graph with vertex set V and edge set E
$g(n), h(n), f(n)$	cost-so-far, heuristic estimate, and $f = g + h$ for node n

Part I

Foundations

Planning for Drone Swarms

Learning objectives

- Describe the problem that this book solves: a swarm of controlled drones flying planned, time-indexed routes through a shared space in which unknown, non-cooperative moving objects appear.
- Tell a *planned conflict* between two drones of the swarm from an *unexpected obstacle*, and say which of the two is resolved before take-off and which one during the flight.
- Use the vocabulary of the field correctly: workspace and configuration space, discrete and continuous planning, global and local planning, deliberative and reactive, centralized and decentralized, cooperative and non-cooperative, tracking and prediction, and the properties completeness, optimality, bounded suboptimality, anytime, incremental and real-time.
- Name the four layers of the hybrid architecture, say what each layer receives and delivers, and give the time scale on which each one works.
- Locate every algorithm of the training plan in this book, state its priority, and say which parts of the field are established and where research questions remain.
- Choose a reading path through the book and set up the Python stack that its code uses.

1.1 The problem: many drones, one airspace, and a stranger

Imagine that you are responsible for a fleet of eight quadrotors that inspect the roof of a large factory every morning. Each drone has its own list of places to photograph, and all of them share the same airspace. Before take-off you need a route for every drone, and you need to be sure that no two routes bring two drones to the same place at the same time. During the flight something unplanned happens: a drone that is not yours crosses the site. It does not talk to your fleet, it does not follow your plan, and you cannot know exactly where it is going. Your drones must notice it, guess where it is heading, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

This book is about exactly that problem, and this chapter states it carefully. Every later chapter teaches one algorithm that solves one piece of it. The setting has four ingredients.

- **A swarm of controlled drones.** There are k drones that we command. We know their positions, we can give each of them a route, and they do what we tell them. In this book they are called the *agents* of the swarm, or simply drones $A, B, C, \dots$
- **A shared space, represented on a grid or a graph.** The drones fly through one three-dimensional volume. For planning we discretise that volume into cells of a grid, or

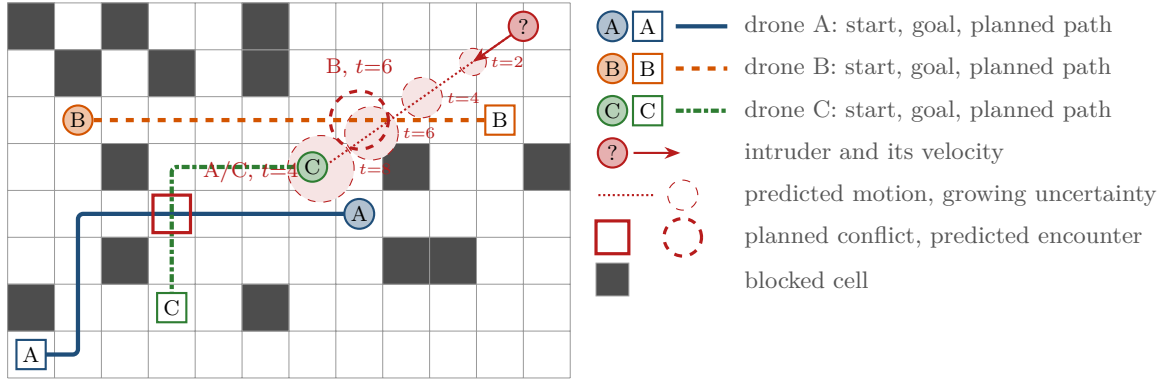


Figure 1.1. The scenario of Example 1.3: a 12×8 grid with 14 blocked cells, three controlled drones with their independently planned shortest paths, and one intruder entering from the top right with a velocity of $(-0.54, -0.38)$ cells per step. Two things go wrong. The paths of A and C both pass through cell (3,3) at $t = 4$ (red square): a *planned conflict*, visible before take-off. The intruder’s predicted motion (dotted, with growing uncertainty discs) passes within 0.38 cells of drone B at $t = 6$ (dashed circle): an *unexpected obstacle*, visible only during the flight.

more generally into the vertices of a graph $G = (V, E)$ whose edges are the moves that a drone can make in one time step. Chapter 2 explains the representations in detail; most figures of the book show a horizontal slice of the space, that is, a two-dimensional grid at the altitude of the swarm.

- **Planned, time-indexed routes.** A route is not just a curve. It says *where* a drone is at *each* time step. Only with time attached can we say whether two routes collide, and only then can a planner remove the collision before anybody takes off.
- **Unknown, non-cooperative moving objects.** During the flight, objects that are not part of the swarm may appear: other drones, most often. We do not know their plans. We observe them with noisy sensors, so even their current state is uncertain, and their future motion is more uncertain still. The book calls such an object an **intruder**.

1.1.1 Two kinds of trouble

Figure 1.1 shows a small instance of the problem, and it shows the two kinds of trouble that this book is about. The first kind is visible before anybody takes off. Drone A wants to fly from (7,3) to (0,0) and drone C from (6,4) to (3,1). Each of them, planning alone, chooses a shortest path, and both shortest paths use cell (3,3) at time step 4. Nothing is unknown here: we know both routes, we know both clocks, and we can see the collision coming while the drones are still on the ground. The second kind of trouble is invisible before take-off. The intruder in the top right corner was not in anybody’s plan. Once it is observed, we can estimate its velocity and extrapolate: if it keeps flying straight, it will pass within 0.38 cells of drone B at $t = 6$. But it may also turn, slow down or stop, and the further we look ahead, the less we know. The two kinds are defined next.

Definition 1.1 (Planned conflict). A **planned conflict** is a collision between two controlled drones that follows from their planned routes alone: at some time step both routes occupy the same vertex, or they traverse the same edge in opposite directions. A planned conflict can be detected, and resolved, before the flight, by a *multi-agent planner* that knows all routes.

Definition 1.2 (Unexpected obstacle). An **unexpected obstacle** is an object that was not part of the plan and that is observed only during the flight. Its current state is known only through noisy measurements and its future motion is uncertain. It must be handled

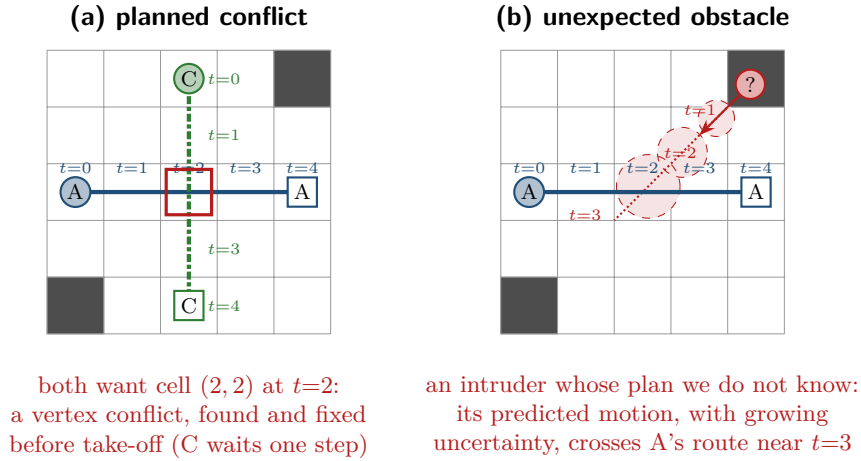


Figure 1.2. The two kinds of trouble on a 5×5 grid. (a) Drones A and C both want cell (2,2) at $t = 2$: a planned conflict. Everything about it is known before take-off, and letting C wait one step removes it. (b) An intruder whose plan we do not know: from its observed velocity we predict where it will be at $t = 1, 2, 3$, with a disc of uncertainty that grows with the horizon, and the prediction crosses the route of A near $t = 3$. Nothing about it could have been planned for.

during the flight, by whatever the drone can decide in the time it has.

The difference matters because the two kinds of trouble call for different tools. A planned conflict is a problem of *coordination* among agents that we fully control; it is solved once, before the flight, by a planner that may take a few seconds and that can guarantee a collision-free plan. An unexpected obstacle is a problem of *reaction* under uncertainty; it must be solved many times per second, by each drone on its own, and no algorithm can guarantee safety against an object whose motion is unknown. Figure 1.2 contrasts the two side by side.

The core idea of the book

Use a *global planner* for the planned routes, a *local collision-avoidance method* for the surprises, and a *replanner* only when the local response is insufficient.

This one sentence is the plan of the whole book. Chapters 3 to 11 build the global planner, from single-agent graph search to multi-agent path finding. Chapters 12 to 15 build the local layer. Chapters 18 to 20 build the tracking and prediction that the local layer needs in order to see the surprise coming. Chapters 5, 16, 17 and 21 to 23 provide the replanning, the continuous-space planning and the control that hold everything together, and Chapters 24 and 25 assemble and evaluate the result.

1.1.2 A scenario in numbers

Numbers make the distinction concrete. The scenario of Figure 1.1 was produced by the scenario generator `code/ch01_scenario.py` of this chapter with the fixed seed 1462; every number quoted below is printed by that program.

Example 1.3 (Three drones and one intruder). The grid has 12×8 cells, 14 of which are blocked. Cell (x, y) covers the unit square with lower-left corner (x, y) , its centre is $(x+0.5, y+0.5)$, and the grid is 4-connected: in one time step a drone moves to a horizontally or vertically adjacent free cell, or it waits. A drone that has reached its goal stays there.

- Drone A flies from (7,3) to (0,0). Its shortest path has 10 steps: (7,3), (6,3), ..., (1,3), (1,2), (1,1), (1,0), (0,0).

Table 1.1. Time line of [Example 1.3](#): the cell of each drone, the predicted position of the intruder, and its distance to the centre of each drone’s cell (in cells). The planned conflict is the row in which A and C share a cell; the encounter is the row in which d_B drops to 0.38. Drones stay at their goals after arriving.

t	A	B	C	Intruder (x, y)	d_A	d_B	d_C
0	(7,3)	(1,5)	(6,4)	(11.00, 7.50)	5.32	9.71	5.41
1	(6,3)	(2,5)	(5,4)	(10.46, 7.12)	5.37	8.12	5.61
2	(5,3)	(3,5)	(4,4)	(9.92, 6.74)	5.48	6.54	5.86
3	(4,3)	(4,5)	(3,4)	(9.38, 6.36)	5.66	4.96	6.17
4	(3,3)	(5,5)	(3,3)	(8.84, 5.98)	5.89	3.37	5.89
5	(2,3)	(6,5)	(3,2)	(8.30, 5.60)	6.17	1.80	5.71
6	(1,3)	(7,5)	(3,1)	(7.76, 5.22)	6.49	0.38	5.66
7	(1,2)	(8,5)	(3,1)	(7.22, 4.84)	6.18	1.44	5.00
8	(1,1)	(9,5)	(3,1)	(6.68, 4.46)	5.97	3.01	4.34
9	(1,0)	(10,5)	(3,1)	(6.14, 4.08)	5.86	4.59	3.69
10	(0,0)	(10,5)	(3,1)	(5.60, 3.70)	6.02	5.22	3.04

- Drone B flies from (1, 5) to (10, 5) along row 5 in 9 steps.
- Drone C flies from (6, 4) to (3, 1) in 6 steps: (6, 4), (5, 4), (4, 4), (3, 4), (3, 3), (3, 2), (3, 1).

Planned independently, the three paths have a sum of costs of $10 + 9 + 6 = 25$ steps and a makespan of 10 steps ([Chapter 2](#) defines both measures). They contain exactly one planned conflict: a vertex conflict of A and C in cell (3, 3) at $t = 4$. If either drone waits one step before entering (3, 3), the conflict disappears and the sum of costs rises to 26.

The intruder is first observed at (11.0, 7.5) with velocity $(-0.54, -0.38)$ cells per step, that is, a speed of 0.66 cells per step. If it keeps that velocity, then at $t = 6$ it is at (7.76, 5.22) while drone B occupies cell (7, 5) with centre (7.5, 5.5): a distance of 0.38 cells, far less than the safety distance that any real swarm would demand. [Table 1.1](#) lists the whole time line.

Look at what the table does and does not tell you. The conflict in row $t = 4$ is a fact: given the two routes, it will happen. The encounter in row $t = 6$ is a *prediction*: it assumes that the intruder flies straight at constant velocity, which is the simplest model we could adopt and, as [Chapter 20](#) shows, not a bad one over a short horizon. The dashed discs in [Figure 1.1](#) grow with t to remind you that the prediction gets worse the further ahead it looks. A good swarm planner acts on the conflict now and on the encounter only when it is close enough in time to be believed—which is the subject of the safety horizon in [Section 1.3](#).

An intruder is not a wall

A tempting shortcut is to treat an intruder as a static obstacle: block the cells it currently occupies and replan around them. This fails twice. The intruder will have moved by the time the new plan is executed, so the plan avoids a place where nothing is, and it may steer the drone straight into where the intruder is heading. A moving object must be avoided in *space and time*, using a prediction of where it will be, not a snapshot of where it was.

1.1.3 Why not one method for everything?

Why not pick the most powerful planner and run it whenever anything changes? Two arguments answer this, and they recur throughout the book. First, *time*: a multi-agent planner that guarantees a collision-free plan for the whole swarm needs seconds, sometimes far longer, and

Table 1.2. The contrasts that organise the book. Most algorithms sit at one end of each pair; the hybrid architecture of [Chapter 24](#) deliberately combines both ends.

Pair	One end	The other end
Workspace / configuration space	the physical volume the drone flies in	the set of all configurations of the drone; obstacles are inflated by its size
Discrete / continuous	finite vertices and time steps; graph search	real-valued positions and time; sampling, optimisation
Global / local	whole route from start to goal, all obstacles known	next few seconds, only what the sensors see
Planning / control	decides <i>where</i> to go and <i>when</i>	makes the vehicle actually follow the decision
Deliberative / reactive	thinks ahead, uses a model, takes time	maps the current situation directly to an action
Centralized / decentralized	one computer knows and decides everything	every drone decides from what it knows and hears
Cooperative / non-cooperative	the other agent shares the avoidance rules	the other agent does what it likes
Tracking / prediction	estimating the state <i>now</i> from noisy data	extrapolating the state into the future

a drone flying at a few metres per second cannot wait that long when another drone is two metres away. Second, *guarantees*: the global plan is collision-free among the swarm, optimal or provably close to it, and respects formation and communication constraints; throwing it away at every surprise means paying for those properties again, and possibly losing them in the meantime. The opposite extreme is equally poor: a swarm that only reacts never plans, and [Exercise 1.4](#) asks you to show that it can lock up in a corridor with nobody willing to go first. The hybrid architecture of [Section 1.3](#) keeps the global plan as the default, reacts locally when it must, and replans only when the reaction cannot bring the drone back.

1.2 Vocabulary: the words you need before anything else

Planning has a vocabulary of contrasts. Most of them are pairs of words that describe two ends of a spectrum, and every algorithm in this book sits somewhere between them. [Table 1.2](#) collects the pairs; the paragraphs below explain each one once, and the rest of the book uses them without further comment.

Workspace and configuration space. The **workspace** is the physical space in which the drone moves. The **configuration space** $\mathcal{C}$ is the set of all configurations of the drone; for a drone modelled as a point with a safety radius, a configuration is a position, and the obstacle region $\mathcal{C}_{\text{obs}}$ is the workspace obstacle inflated by that radius. Planning takes place in $\mathcal{C}$, so that a path of the point in $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$ is a collision-free motion of the real drone. [Chapter 2](#) gives the definitions and the inflation operation [25].

Discrete and continuous planning. **Discrete planning** works on a graph: finitely many vertices and moves, and integer time steps. Dijkstra’s algorithm, A^* , D^* Lite and all multi-agent planners of the book are discrete. **Continuous planning** works with real-valued positions, velocities and times: RRT, RRT^* , MPC, MILP and the velocity-space methods of

local avoidance. A grid is a sampling of the continuous workspace, and a discrete plan becomes a continuous trajectory when the drone flies between the cell centres.

Time-indexed paths. The single most important object in this book is a path with time attached.

Definition 1.4 (Time-indexed path). A **time-indexed path** of an agent on a graph $G = (V, E)$ is a sequence $\pi = (v_0, v_1, \dots, v_T)$ of vertices such that v_0 is the agent's start, v_T is its goal, and for every $t < T$ either $v_{t+1} = v_t$ (a *wait* action) or $\{v_t, v_{t+1}\} \in E$ (a *move* action). The agent occupies v_t during time step t and stays at v_T for all $t \geq T$. A **plan** for a swarm is one time-indexed path per agent.

Without the index t two paths can cross without a collision, because the drones pass the crossing at different times; with the index, a collision is a precise statement, $v_t^A = v_t^C$ for some t . [Chapter 2](#) turns this into the space-time graph and [Chapter 7](#) lists every kind of conflict that time-indexed paths can have [39].

Global and local planning. A **global planner** computes a complete route from the start to the goal from a map of all known obstacles; it may take time and it may be optimal. A **local planner** looks a few seconds ahead, uses only what the sensors currently see, and outputs the next velocity. Global planners know the goal but not the surprise; local planners know the surprise but not the goal. The local methods of the book are those of [Chapters 12 to 15](#); the search and sampling methods of Parts II, III and V are global.

Planning and control. **Planning** decides *where* the drone should be and *when*; **control** makes the vehicle follow that decision by turning the error between the desired and the measured state into motor commands. The book plans, and treats the flight controller as a black box that tracks a velocity or a waypoint. Model predictive control ([Chapter 21](#)) is the exception that does both at once, which is why it is such a natural place to enforce constraints.

Deliberative and reactive. A **deliberative** method uses a model of the world to search or optimise over possible futures before it acts. A **reactive** method maps the current situation to an action with little or no search, which makes it fast and robust to modelling errors, but short-sighted [35]. Global planners are deliberative, local avoidance is reactive, and the hybrid architecture layers the one on the other.

Centralized and decentralized. A **centralized** system has one computer that knows the state of every drone and decides for all of them; in a **decentralized** system every drone decides from its own observations and from what it hears from its neighbours within communication range. The global planner of the book is centralized, because conflict-based search needs all routes in one place. Local avoidance is decentralized, because each drone runs ORCA on what it sees. Consensus and formation control ([Chapter 23](#)) are decentralized by design, and the communication radius that limits them is one of the constraints that [Section 1.5](#) is about.

Cooperative and non-cooperative agents. Two agents are **cooperative** if both follow the same avoidance rules, so that each may count on the other to do its share; reciprocal velocity obstacles and ORCA rest on exactly this assumption [2]. A **non-cooperative** agent does whatever it does, and a drone that meets it must take the full responsibility for avoiding it. The drones of the swarm are cooperative with one another and non-cooperative with the intruder; [Chapter 13](#) handles both cases with a change of a single factor.

Table 1.3. Properties of planning algorithms, informally. The formal versions appear as theorems in the chapters of the last column. No algorithm in the book has all six.

Property	Informal meaning	Examples in this book
Complete	if a solution exists, the algorithm finds one, and if none exists it says so; sampling-based planners are <i>probabilistically</i> complete: the probability of finding a solution tends to one as the number of samples grows	Dijkstra, A* (Chapters 3 and 4); CBS (Chapter 9); RRT, probabilistically (Chapter 16); prioritized planning is <i>not</i> complete (Chapter 8)
Optimal	the solution it returns has the least possible cost; RRT* is <i>asymptotically</i> optimal, that is, optimal in the limit of infinitely many samples	A* with an admissible heuristic (Chapter 4); CBS for the sum of costs (Chapter 9); RRT* (Chapter 17)
Bounded suboptimal	the cost is at most w times the optimal cost, for a factor $w \geq 1$ chosen by the user; $w = 1$ recovers optimality, $w = 1.5$ is often orders of magnitude faster	weighted A* (Chapter 4); ECBS (Chapter 10)
Anytime	returns a first, possibly poor, solution quickly and improves it while time remains; interrupt it whenever you like and take the best solution so far	ARA*, which lowers w step by step (Chapter 6)
Incremental	reuses the previous search when the graph changes a little, an obstacle appears or an edge cost changes, instead of starting over	LPA*, D* Lite (Chapter 5): the replanning layer
Real-time	the work per call is bounded, so an answer is guaranteed within a fixed, short time budget; such methods do not search for a route, which is why they can run at tens of hertz	ORCA, one small linear program per cycle (Chapter 13); DWA, a fixed number of candidate velocities (Chapter 14)

Tracking and prediction. **Tracking** estimates the *current* state of a moving object, position and velocity, from a sequence of noisy measurements; the Kalman filter of Chapter 18 is the classical tool [41]. **Prediction** extrapolates that estimate into the *future* over a horizon of a few seconds and reports the growing uncertainty along the way (Chapter 20). Tracking ends at the present moment; prediction begins there.

1.2.1 Properties of algorithms

When the book says that an algorithm is complete, or optimal, or anytime, it means something precise. The precise statements come with the algorithms; here are the informal meanings, so that you can read the map of the book in Section 1.4 with them in mind. Table 1.3 summarises them.

These properties conflict with one another, and choosing among them is the designer’s job. An optimal multi-agent planner is not real-time; a real-time local method is not complete; an incremental planner can only repair what a complete planner has built. The four layers of the next section are chosen so that each property is provided by the layer that can afford it.

1.3 A preview of the hybrid architecture

The book builds towards one system, and it helps to see the whole of it before studying its parts. The training plan that this book follows (reproduced in Appendix A) proposes a **hybrid**

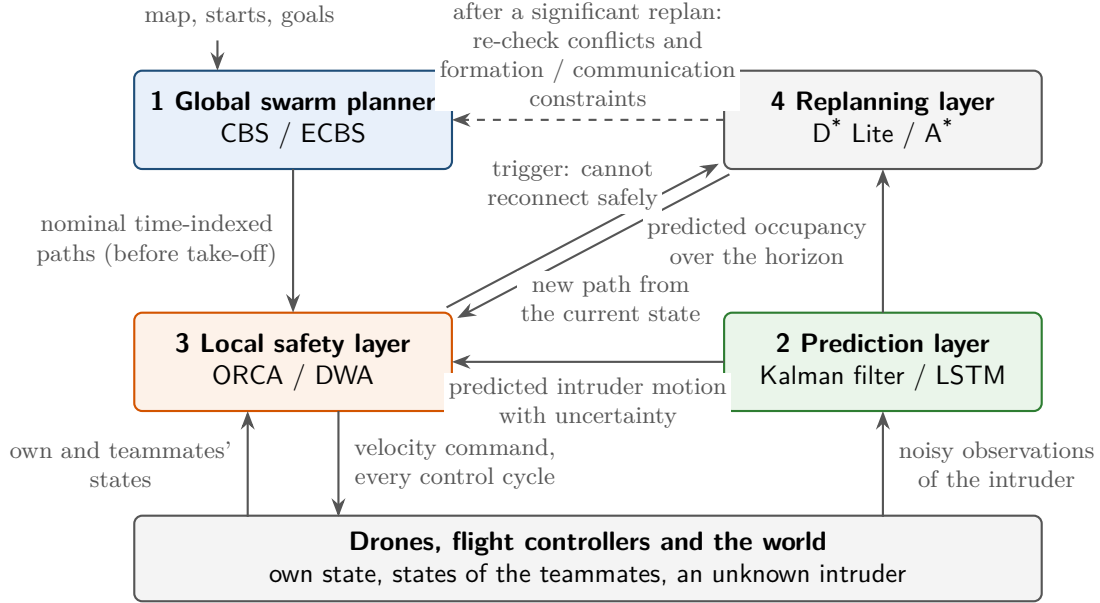


Figure 1.3. The four layers of the hybrid architecture and what flows between them. The global planner runs before take-off and hands nominal time-indexed paths to the local layer, which alone talks to the flight controllers. The prediction layer turns noisy observations of an intruder into a predicted motion with uncertainty, used by both the local layer and the replanner. The replanner is called only when the local layer cannot return to the nominal path; after a significant replan the swarm-level conflicts are checked again (dashed arrow).

collision-avoidance architecture with four layers, shown in Figure 1.3. This section is a preview in plain words; Chapter 24 gives the full treatment, with the state machine, the triggers, the constraints and a discussion of which guarantees survive the combination.

1.3.1 The four layers

1. Global swarm planner (CBS, ECBS). Before the flight, the global planner receives the map, the start and goal of every controlled drone, and any constraints on formation or communication range. It returns a nominal, time-indexed path for each drone such that no two drones are ever in conflict. The book’s global planners are conflict-based search (CBS), which is optimal [36], and its bounded-suboptimal variant ECBS, which is much faster for a chosen factor w (Chapters 9 and 10). Both build on single-agent search in space and time (Chapter 4). This layer resolves the planned conflicts of Definition 1.1; it never sees the intruder.

2. Prediction layer (Kalman filter, LSTM). During the flight, the prediction layer observes every object that is not part of the swarm. From noisy position measurements it estimates the object’s current position and velocity with a Kalman filter [41] (Chapters 18 and 19) and then predicts a short future trajectory, either by extrapolating at constant velocity or with a learned model such as an LSTM (Chapter 20). The output is not a single line but a line *with uncertainty*: a sequence of predicted positions, each with a covariance that grows along the horizon, exactly as the discs in Figure 1.1 do.

3. Local safety layer (ORCA, DWA). Every control cycle, each drone runs a local method that takes its own state, the states of its teammates and the predicted motion of any intruder, and produces a safe velocity for the next fraction of a second. The book’s local methods are ORCA [2], which chooses the velocity closest to the preferred one inside

a set of half-planes (Chapter 13), and the dynamic window approach, which scores a small set of dynamically feasible velocities (Chapter 14). When nothing threatens, the preferred velocity is the one that follows the nominal path, and the local layer simply passes it on. When a predicted collision enters the safety horizon, the local layer deviates temporarily *without discarding the global route*.

4. Replanning layer (D^* Lite, A^*). Sometimes the local manoeuvre cannot bring the drone safely back to its nominal path: the intruder parks in the corridor, the detour has become too long, or the deviation has broken the formation. Then the drone replans from its current state to its goal. D^* Lite [21] does this incrementally, reusing the previous search (Chapter 5); plain A^* in space and time is the fallback (Chapter 4). A new path for one drone may conflict with the others, so a significant replan is followed by a conflict re-check at the swarm level—the dashed arrow back to the global planner in Figure 1.3.

1.3.2 The decision logic in five steps

The layers are only useful with a rule that says which one acts when. The training plan states that rule in five steps, and Figure 1.4 draws it. Two quantities appear in it. The **time to collision** τ of a drone with a predicted object is the earliest future time at which the prediction brings the two closer than the safety distance (infinite if it never does). The **safety horizon** τ_{safe} is a look-ahead time chosen by the designer, a few seconds at most; a predicted conflict counts as a *near-term risk* when $\tau < \tau_{\text{safe}}$. Chapter 12 computes τ from relative position and velocity.

1. **Follow the nominal path.** As long as no predicted conflict lies inside the safety horizon, every drone follows the time-indexed path that the global planner computed. This is the normal state, and most of the time nothing else happens.
2. **Avoid locally.** If an unknown moving object creates a near-term risk, the drone invokes ORCA or DWA to produce a local avoidance action: a velocity that keeps it clear of the predicted motion of the object and of its teammates, and that deviates from the nominal path as little as possible.
3. **Reconnect.** When the conflict has cleared, the drone tries to rejoin its nominal path at a *safe future waypoint*: a point on the path that it can reach in time without a new conflict. If it succeeds, the global plan is intact and nothing else needs to change.
4. **Replan.** If reconnection is impossible, too costly, or would violate a formation or communication constraint, the drone runs D^* Lite or A^* from its current state and follows the new path.
5. **Re-check.** After any significant replan, the swarm checks the new path against the paths of the other drones, and against the formation and communication constraints, and repairs what the replan has broken.

Notice the order of preference built into the rule. Doing nothing is best; a local manoeuvre is second best because it keeps the global plan; a replan is a last resort because it costs computation and may invalidate the plans of the others. Deciding *automatically* where the line between step 3 and step 4 lies is one of the open questions of Section 1.5.

1.3.3 Time scales

Why four layers rather than one? Because they work on different time scales, and no single algorithm can serve all of them. Figure 1.5 places the layers on a logarithmic time axis. The numbers are typical orders of magnitude for the kind of implementation this book describes,

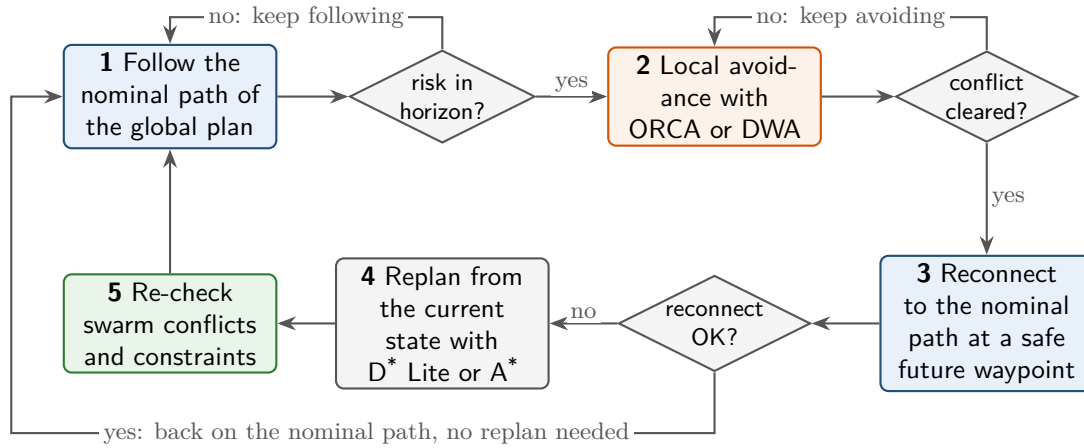


Figure 1.4. The five-step decision logic as a loop. The normal path is the top row: follow, and keep following while no risk lies inside the safety horizon. Only when a local avoidance action cannot be reconnected to the nominal path does the bottom row run: replan, re-check the swarm, and continue.

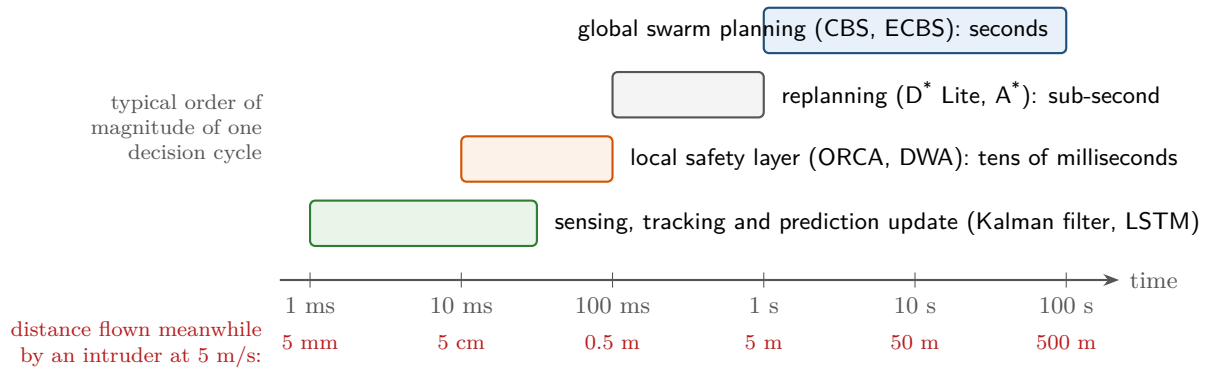


Figure 1.5. Typical orders of magnitude of one decision cycle of each layer, on a logarithmic time axis. The reactive layer acts in tens of milliseconds, a replan takes a fraction of a second, and a global multi-agent plan takes seconds. The lower row shows how far an intruder flying at 5 m/s moves in the meantime: a layer that needs one second cannot be the one that reacts to an object two metres away.

not measurements; your own numbers will depend on the hardware, the size of the swarm and the map, and [Chapter 25](#) shows how to measure them.

- The **reactive layer** must deliver a velocity every control cycle, typically every few tens of milliseconds; ORCA and DWA can, because their work per cycle is small and bounded.
- **Replanning** takes a fraction of a second: D* Lite repairs only the part of the search that the change affected, and the local layer keeps the drone safe while the replanner works.
- **Global planning** for the whole swarm takes seconds, on hard instances much longer; it runs before take-off and after a significant replan, never inside the control loop.
- **Sensing, tracking and prediction** run at the rate of the sensors; a Kalman update is a handful of small matrix operations and a trained LSTM evaluates in milliseconds.

The lower row of [Figure 1.5](#) explains why the order matters. An intruder flying at 5 m/s covers 10 cm during a 20 ms control cycle, 2.5 m during a half-second replan, and 15 m during a three-second global plan. A safety horizon of a few seconds, an avoidance layer that acts

within one control cycle, and a replanner that is allowed to take longer because the avoidance layer covers for it: that is the division of labour of Figure 1.3.

Numbers on a logarithmic axis are not measurements

It is easy to read Figure 1.5 as a promise. It is not. A Python implementation of ORCA for fifty drones may take longer than ten milliseconds; a CBS instance with eight agents in a corridor may not finish in a minute. Treat the figure as a statement of *requirements*: whatever runs in the reactive layer must fit into a control cycle, and whatever cannot must be moved to a slower layer that the reactive layer can cover for. Chapter 25 explains how to measure computation time honestly, and that is where real numbers belong.

Interfaces between the layers

Read the architecture as a set of interfaces. The global planner (Chapters 9 and 10) receives the map, the starts, the goals and the swarm constraints and delivers one time-indexed path per drone. The prediction layer (Chapters 18 to 20) receives noisy observations of the intruder and delivers predicted positions with covariances over the safety horizon. The local layer (Chapters 13 and 14) receives the nominal path, the teammates' states and the predictions and delivers one velocity command per control cycle. The replanner (Chapters 4 and 5) receives the current state and the changed map and delivers a new path, which the global layer re-checks (Chapter 7). The system is assembled in Chapter 24, evaluated in Chapter 25, and is the capstone of week 12 of the study plan in Appendix A.

1.4 A map of the book

The training plan groups the algorithms of a complete autonomous-drone system into six families, labelled A to F, and gives each algorithm a priority: **Essential** for what a working prototype cannot do without, **High** for what you should be able to implement, **Medium** for what you should understand, and **Awareness** for what you should recognise. Table 1.4 lists every algorithm of the plan with its role in one line, its priority and the chapter that teaches it. Every chapter repeats the priority tag of its algorithm at the start, and the twelve-week schedule of Section 1.6 tells you when to read it.

Table 1.4. Map of the book: every algorithm of the training plan, its role, its priority in the plan, and the chapter that teaches it. Groups A–C are to be learned deeply, D and E to be known well, and F well enough to integrate.

Algorithm	Role in one line	Priority	Chapter
A. Single-agent graph search (learn deeply)			
Dijkstra	Baseline shortest-path search on graphs with non-negative edge weights; the template that A* refines	High	Chapter 3
A*	The main deterministic shortest-path planner, guided by a heuristic; with time as a state variable it plans one drone around known moving obstacles	Essential	Chapter 4
D* Lite	Efficient replanning when the environment changes after planning; the replanning layer of the architecture	Essential	Chapter 5

continued on the next page

Table 1.4 (continued)

Algorithm	Role in one line	Priority	Chapter
LPA [*]	Conceptual foundation of D [*] Lite and of incremental search	Medium	Chapter 5
ARA [*]	Anytime search: a quick suboptimal path, improved while time remains	Medium	Chapter 6
B. Multi-agent path finding (learn deeply)			
Prioritized planning	Plans the agents one after another in a fixed order; fast, but incomplete and suboptimal in general	High	Chapter 8
CBS	High-level conflict resolution over a constraint tree plus low-level per-agent search; optimal	Essential	Chapter 9
ECBS	Bounded-suboptimal variant of CBS based on focal search; much faster for a chosen factor w	Essential	Chapter 10
M [*]	Couples only the agents that are actually in conflict (subdimensional expansion)	Medium	Chapter 11
Push-and-Swap, Push-and-Rotate	Constructive, rule-based methods; complete on a large class of instances but not optimal; a conceptual contrast to CBS	Awareness	Chapter 11
C. Local and reactive collision avoidance (learn deeply)			
Velocity obstacles (VO)	The set of velocities that lead to a future collision with a moving obstacle	Essential	Chapter 12
Reciprocal VO (RVO)	Extends VO to the case in which both agents share the avoidance effort; removes oscillation	High	Chapter 13
ORCA	Efficient reciprocal avoidance: one half-plane constraint per neighbour and a small linear program	Essential	Chapter 13
Dynamic window approach (DWA)	Chooses a safe short-horizon velocity command under acceleration limits	High	Chapter 14
Artificial potential fields (APF)	Attractive and repulsive forces; simple, but prone to local minima and oscillation	High	Chapter 15
D. Sampling-based motion planning (know well)			
RRT	Explores continuous configuration spaces quickly by random sampling; probabilistically complete	High	Chapter 16
RRT [*]	Asymptotically optimal version of RRT, obtained by rewiring the tree	High	Chapter 17
Informed RRT [*]	Focuses the sampling on an ellipsoid once a first solution is known	Medium	Chapter 17
E. Tracking and prediction (know well)			
Kalman filter	Tracks linear motion under Gaussian noise; the tracker of the prediction layer	Essential	Chapter 18
Extended Kalman filter (EKF)	Handles nonlinear motion and measurement models by linearisation	High	Chapter 19
Unscented Kalman filter (UKF)	Nonlinear estimator based on sigma points; needs no Jacobians	Medium	Chapter 19

continued on the next page

Table 1.4 (continued)

Algorithm	Role in one line	Priority	Chapter
Particle filter	Nonlinear, non-Gaussian tracking with weighted samples	Medium	Chapter 19
LSTM prediction	Learns temporal motion patterns from recorded trajectories; the plan marks it essential for prediction-based research	Essential	Chapter 20
Transformer prediction	Attention-based alternative for longer horizons and interaction-aware prediction	Awareness-High	Chapter 20
F. Optimization and control (learn enough to integrate)			
Model predictive control (MPC)	Optimises the controls over a receding horizon; handles constraints naturally	Essential	Chapter 21
Mixed-integer linear programming (MILP)	Exact constrained formulations of scheduling and path problems; can be expensive	Medium	Chapter 22
Consensus, formation control	Maintains relative positions and shared swarm objectives; the plan marks it essential for formation-preserving work	Essential	Chapter 23

Two chapters do not appear in the table because they teach no algorithm of their own. [Chapter 2](#) is the toolbox: graphs, grids, configuration space, space-time states, path costs, drone motion models, the geometry of moving discs, Gaussians and priority queues. [Chapter 7](#) defines the multi-agent path finding problem, its conflicts and its benchmarks, following the standard terminology of Stern et al. [39]; every planner of group B is stated in that language.

How old is all this?

Older than you may think. Dijkstra published his algorithm in 1959 [7] and Kalman his filter in 1960 [19]; A* dates from 1968 [16]. The dynamic window approach appeared in 1997 [12], velocity obstacles and RRT in 1998 [11, 24], D* Lite in 2002 [21], ORCA in 2011 [2] and conflict-based search in 2012, with the journal version in 2015 [36]. The learned predictors are the youngest members: the LSTM cell is from 1997 [18], its use for trajectory prediction from the mid-2010s, and the Transformer from 2017 [42].

1.5 What is established and where research remains

Let us be honest about what is new here and what is not. Every algorithm in [Table 1.4](#) is established: published, proved, implemented many times, and taught. Nobody starting research on drone swarms needs to invent a new shortest-path algorithm or a new Kalman filter, and a thesis that did would have to compete with sixty years of prior work. The training plan says it plainly, and this book repeats it: *the contribution does not need to be a completely new planner. Strong research opportunities remain in how the components are combined, constrained, predicted and evaluated.*

The plan names seven such directions. Each of them is a place where the established algorithms, put together, leave a question open, and each is discussed at length in [Chapters 24](#) and [25](#) once the algorithms are in place.

Unknown, non-cooperative drones whose future motion is uncertain rather than shared.

Multi-agent planners assume that all routes are known, and reciprocal avoidance assumes that the other agent cooperates; an intruder offers neither. Tracking, prediction and

full-responsibility avoidance are established piece by piece (Chapters 13, 18 and 20); how well they work together against real, unpredictable intruders is not.

Combining CBS-style coordination with real-time local avoidance without destroying the global guarantees unnecessarily.

The moment a drone leaves its nominal path, the conflict-freeness that CBS proved no longer holds for it. Which guarantees survive a deviation, for how long, and when a deviation can be reconnected without replanning the swarm, is a question about the combination, not about either algorithm (Chapter 24).

Formation preservation and communication-range constraints during avoidance and replanning.

A swarm that must keep a formation, or stay within radio range of its neighbours, cannot let each drone dodge as it likes. The tools exist (Chapters 21 and 23); trading a constraint violation against a collision risk during a manoeuvre is open.

Prediction-aware constraints that include uncertainty rather than treating the predicted path as exact.

A prediction is a distribution, not a line. Inflating obstacles by a multiple of the predicted standard deviation is the simplest use of it (Chapters 20 and 24); doing so without making the planner hopelessly conservative at long horizons is not solved.

Deciding automatically when a local avoidance action is sufficient and when a global replan is necessary.

Step 3 versus step 4 of the decision logic. A threshold on the detour length or on the time to collision is a start; a rule that adapts to the situation, and that can be shown to be safe, is a research topic.

Reducing computation for larger swarms while maintaining safety in dynamic 3D environments.

CBS is exponential in the number of conflicts in the worst case, and even ECBS struggles with dozens of agents in tight spaces (Chapters 10 and 11); larger swarms, three dimensions and moving obstacles all make the problem harder.

Benchmarks that compare local-only, replan-only and hybrid approaches under the same conditions.

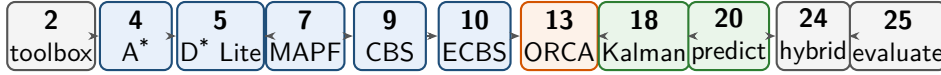
Most published systems are evaluated on their own scenarios. A shared benchmark with controlled numbers of drones and intruders, controlled prediction quality and a fixed set of metrics would allow honest comparison; Chapter 25 shows how to build one.

You will meet these directions again as *dronebox* notes at the end of many chapters, where each algorithm's role in the combination is spelled out. Keep a list of them; by the time you reach Chapter 24 you will have opinions about every one.

1.6 How to read this book

The chapters are ordered so that each one uses only what came before it, and reading them in order is always safe. But the book was written to accompany a training plan with a schedule, and the plan suggests two shorter routes, drawn in Figure 1.6.

The fast path. The training plan gives a reading order of twelve items, of which the first six are the highest priority: A* (Chapter 4), D* Lite (Chapter 5), CBS (Chapter 9), the MAPF overview (Chapter 7), velocity obstacles (Chapter 12) and ORCA (Chapter 13); then DWA (Chapter 14), RRT (Chapter 16), RRT* (Chapter 17), a planning reference, MPC (Chapter 21) and trajectory prediction (Chapter 20). Two remarks. The plan lists CBS before the MAPF overview; if you have never seen a multi-agent path finding instance, read Chapter 7 first, it is

Fast path: the training plan's reading order**Builder's path: the chapters behind the capstone prototype**

 global planner
 local safety layer
 prediction layer
 foundations, control, integration

Figure 1.6. Two reading paths. The top row is the reading order of the training plan; the first six chapters of it are the highest priority. The bottom row is the shortest route to a working capstone prototype. Colours mark the layer of the architecture that a chapter belongs to.

short and [Chapter 9](#) assumes it. The “planning reference” of the plan is not a chapter but a book to keep at hand: we recommend LaValle [25] for motion planning and Russell and Norvig [35] for search, and the book’s own reference material is the toolbox of [Chapter 2](#) and the mathematical refresher of [Appendix B](#).

The builder’s path. If your aim is the capstone prototype as soon as possible, follow the bottom row of [Figure 1.6](#): the toolbox ([Chapter 2](#)), A* and D* Lite ([Chapters 4 and 5](#)), the MAPF problem and CBS with ECBS ([Chapters 7, 9 and 10](#)), ORCA ([Chapter 13](#)), the Kalman filter and trajectory prediction ([Chapters 18 and 20](#)), and then the two integration chapters ([Chapters 24 and 25](#)). Everything else can be read when the prototype demands it.

The twelve-week path. The plan’s schedule is reproduced in full, with study notes and a traceability table, in [Appendix A](#); [Table 1.5](#) is the one-page version. Each week has a coding exercise, which reappears as the coding exercise of the corresponding chapter, and a milestone sentence that you should be able to say truthfully at the end of the week.

1.6.1 Conventions, code and tools

Every algorithm chapter has the same shape: why the algorithm matters for a drone swarm, the idea in plain words, the problem statement, pseudocode with a walkthrough, a worked example with a figure and a trace table, the properties with proofs, variants, implementation notes with a listing, the algorithm’s place in the drone system, a summary, further reading and exercises. Once you know the shape you can find the proof of optimality or the pitfalls of any algorithm without searching.

Conventions. Coloured boxes carry the parts of a chapter that you will return to: the learning objectives (blue), the key idea (green), the algorithm’s place in the drone system (purple), pitfalls (orange) and the chapter summary (grey). Definitions, examples and theorems share one counter per chapter; pseudocode is numbered per chapter and referred to by line; listings are verbatim excerpts of the chapter’s code file. Exercises are marked ★ (minutes), ★★ (about an hour) or ★★★ (an afternoon with code), and [Appendix C](#) gives hints and solutions to selected ones. Every algorithm carries the priority tag of the training plan, from [Essential](#) to [Awareness](#), at the start of its chapter.

Table 1.5. The twelve-week schedule of the training plan in one page. Chapters in parentheses are optional in that week. The full plan, with the coding exercises and milestones, is in [Appendix A](#).

Wk	Topic	What you build	Chapters
1	Foundations: Dijkstra and A*	grid A* with time as a state variable	Chapters 3 and 4 (Chapters 1 and 2)
2	Dynamic replanning: LPA*, D* Lite	a path that is repaired, not recomputed	Chapter 5 (Chapter 6)
3	MAPF basics and conflict types	prioritized planning for 2–5 agents	Chapters 7 and 8
4	Conflict-based search	CBS from scratch on a grid	Chapter 9
5	ECBS and practical MAPF	ECBS benchmarked against CBS	Chapter 10 (Chapter 11)
6	VO, RVO and ORCA	2D crossing simulation: none / VO / ORCA	Chapters 12 and 13
7	DWA and potential fields	a local planner and its failure cases	Chapters 14 and 15
8	RRT, RRT* and continuous 3D planning	RRT* in a 3D obstacle field	Chapters 16 and 17
9	Tracking unknown moving obstacles	a Kalman filter tracking a noisy intruder	Chapters 18 and 19
10	Trajectory prediction	constant velocity versus LSTM, ADE/FDE	Chapter 20
11	MPC and formation constraints	a constrained controller keeping a formation	Chapters 21 and 23 (Chapter 22)
12	Hybrid system integration	the capstone and its experiment matrix	Chapters 24 and 25

Code files. Each chapter has one Python file, `code/chNN_slug.py`, that implements the chapter’s algorithm in full and runs a self-test when executed. The listings in the text are excerpts of these files, never separate code, so what you read is what runs. Every number in a worked example was produced by the corresponding file, and the figures that show algorithm output were generated by scripts in `code/figures/`. The file of this chapter is `code/ch01_scenario.py`, a small scenario generator that needs only the standard library. [Listing 1.1](#) shows the function that found the planned conflict of [Table 1.1](#).

Listing 1.1. Finding the planned conflicts of a set of time-indexed paths (from `code/ch01_scenario.py`). A drone that has reached its goal stays there, which `position_on_path` implements by clamping the time index.

```

1 def position_on_path(path: list, t: int) -> Cell:
2     """Cell occupied at step ``t``; the drone stays at its goal afterwards."""
3     return path[min(t, len(path) - 1)]
4
5
6 def find_conflicts(paths: list) -> list:
7     """All vertex and edge (swap) conflicts between the given paths."""
8     conflicts = []
9     horizon = max(len(p) for p in paths)
10    for i in range(len(paths)):
11        for j in range(i + 1, len(paths)):
12            for t in range(horizon):
13                a0 = position_on_path(paths[i], t)
14                b0 = position_on_path(paths[j], t)
15                if a0 == b0:

```

```

16         conflicts.append(Conflict(t, "vertex", (i, j), a0))
17         a1 = position_on_path(paths[i], t + 1)
18         b1 = position_on_path(paths[j], t + 1)
19         if a0 == b1 and b0 == a1 and a0 != a1:
20             conflicts.append(Conflict(t, "edge", (i, j), (a0, a1)))
21     conflicts.sort(key=lambda c: (c.time, c.agents))
22     return conflicts

```

`find_conflicts` compares exact plans and reports facts: a vertex conflict when two drones occupy the same cell at the same step, an edge conflict when they swap cells. Its companion `intruder_encounters` in the same file compares a plan with a *prediction*, the intruder extrapolated at constant velocity, within a radius, and its answer is only as good as the prediction: the asymmetry of [Section 1.1.1](#) in two functions. Running the file executes a self-test (reproducibility of the generator, validity of the scenarios, shortest paths, the conflict rules including stay-at-goal, the intruder motion, a JSON round trip) and prints the scenario of [Example 1.3](#). [Exercise 1.8](#) asks you to write a generator of your own.

The Python stack. The training plan suggests Python, NumPy, NetworkX, Matplotlib and PyBullet or gym-pybullet-drones. The book’s reference implementations use only Python 3.10 or newer and NumPy (SciPy for the QP and MILP solvers of [Chapters 21](#) and [22](#)), so that you can read every line. NetworkX [\[14\]](#) is useful for quick experiments with graphs and for checking your shortest paths against an independent implementation. Matplotlib draws what the exercises ask you to look at: open and closed sets, trajectories, velocity obstacles, benchmark curves. PyBullet, through the gym-pybullet-drones environment [\[29\]](#), provides a physics simulation of quadrotors for the day when a point-mass model is no longer enough; nothing in the book requires it before [Chapter 24](#). [Chapter 2](#) describes the small conventions that the book’s code shares: how a grid is stored, how a path is represented, how a scenario is written to a file.

1.7 Summary

Chapter summary

- The problem: a swarm of controlled drones flies planned, time-indexed routes through a shared space represented on a grid or a graph, while unknown, non-cooperative moving objects appear whose future motion is uncertain.
- Two kinds of trouble: *planned conflicts* between drones of the swarm are facts, visible before take-off and resolved by a multi-agent planner; *unexpected obstacles* are predictions, visible only during the flight and handled by whatever the drone can decide in a control cycle.
- The core idea: a global planner for the routes, a local collision-avoidance method for the surprises, a replanner only when the local response is insufficient.
- The vocabulary: workspace and configuration space, discrete and continuous, global and local, planning and control, deliberative and reactive, centralized and decentralized, cooperative and non-cooperative, tracking and prediction; and the properties complete, optimal, bounded suboptimal with factor w , anytime, incremental and real-time, which no single algorithm has all at once.
- The architecture: global swarm planner (CBS/ECBS), prediction layer (Kalman filter/LSTM), local safety layer (ORCA/DWA), replanning layer (D^* Lite/ A^*), with a five-step decision logic—follow, avoid, reconnect, replan, re-check—and time scales

of tens of milliseconds, sub-second and seconds.

- The map: twenty-seven algorithms in six groups, each with a priority from the training plan and a chapter in this book.
- The honesty: the algorithms are established; the research lies in their combination, their constraints, the use of uncertain predictions, and honest evaluation.

Further reading. For the general background of search and agents, Russell and Norvig [35] is the standard text, and LaValle [25] is the reference for motion planning in configuration space; both are freely available online from their authors. The multi-agent path finding problem, its conflict types and its benchmarks are defined by Stern et al. [39], and conflict-based search by Sharon et al. [36]. Reciprocal local avoidance is introduced by Berg et al. [2], incremental replanning by Koenig and Likhachev [21], and state estimation with the Kalman filter and its relatives by Thrun, Burgard, and Fox [41]. The simulation environment gym-pybullet-drones is described by Panerati et al. [29]. The full training plan, with its schedule, capstone description and completion checklist, is [Appendix A](#).

1.8 Exercises

Exercise 1.1 (★). For each of the following events, say which layer of the architecture of [Figure 1.3](#) handles it (global planner, prediction layer, local safety layer, replanning layer), or whether it is handled by none of them (for example by the flight controller, or by nobody at all). Give a one-line reason each.

- Before take-off, the routes of drones B and D both use cell $(4, 7)$ at $t = 12$.
- A bird appears, predicted to cross the route of drone A in 1.5 seconds.
- The intruder stops and hovers in the only corridor that drone C must use.
- A new no-fly zone is uploaded to the swarm before take-off.
- Drone A has replanned, and its new route now conflicts with the route of drone C .
- The sensor delivers a noisy position of the intruder every 50 ms.
- A gust of wind pushes drone B twenty centimetres off its path.
- Drone D must stay within 30 m of drone B while it dodges the intruder.

Exercise 1.2 (★). Copy the following list of algorithms and, using [Table 1.4](#) and the informal definitions of [Section 1.2.1](#), mark for each one which of the six properties it has: Dijkstra, weighted A^* with $w = 1.5$, prioritized planning, CBS, ECBS, ARA^* , D^* Lite, ORCA, RRT, RRT^* . Then answer: which two properties never appear together in your table, and why is that not a coincidence?

Exercise 1.3 (★★). Sketch, on a grid of at most 8×8 cells, a scenario in which local avoidance alone cannot bring a drone to its goal and the replanning layer must act. Draw the nominal path, the intruder and its motion, and explain in a few sentences why every velocity that the local layer can choose either collides or makes no progress. Then say what the replanner does instead and why step 5 of the decision logic is needed afterwards.

Exercise 1.4 (★★). Explain why a swarm that uses *only* a reactive local method, with no global plan, can deadlock. Use two settings: (a) two drones that want to pass each other in a corridor one cell wide, and (b) four drones at the corners of a square, each wanting to reach the opposite corner, all running the same symmetric avoidance rule. What information does a global plan add that a reactive rule lacks? Would giving the reactive rule a random component fix the problem, and at what cost?

Exercise 1.5 (★★). Work with [Table 1.1](#) and [Example 1.3](#).

- Verify from the two paths that A and C have a vertex conflict at $t = 4$ and that they have no edge (swap) conflict.
- Let C wait one step in cell $(3, 4)$ before entering $(3, 3)$. Write down the new path of C , check that the conflict is gone, and compute the new sum of costs and makespan. Would letting A wait instead be cheaper?
- Treat the motion of B as continuous: at time $t \in [0, 9]$ its centre is at $(1.5 + t, 5.5)$. Write the relative position of the intruder with respect to B as a function of t , find the time of closest approach and the minimum distance, and compare with the entry 0.38 at $t = 6$ in the table.
- How many steps must B wait at its start for the intruder, if it keeps its velocity, to stay at least one cell away from the centre of B 's cell at every integer time step? Which layer of the architecture would make such a decision, and what is wrong with making it before take-off?

Exercise 1.6 (★★). The function `intruder_encounters` of `code/ch01_scenario.py` checks, by default, only the time steps up to the makespan plus one, which is $t = 11$ in [Example 1.3](#). Using $\mathbf{p}(t) = \mathbf{p}_0 + t \mathbf{v}$ with $\mathbf{p}_0 = (11.0, 7.5)$ and $\mathbf{v} = (-0.54, -0.38)$, compute the intruder's predicted position at $t = 14, 15$ and 19 , and its distance to the goal cells of C and A , where those drones are parked. What does the result say about the choice of the horizon, and about the rule that a drone stays at its goal? Which layer should protect a parked drone?

Exercise 1.7 (★). An intruder flies at 5 m/s. (a) How far does it move during one 20 ms control cycle, during a 0.5 s replan, and during a 3 s global plan? (b) Drone A flies towards it at 3 m/s, the required separation is 2 m, and the local layer needs at least ten control cycles to complete an avoidance manoeuvre. What is the smallest safety horizon τ_{safe} that gives the local layer that time, and at what distance is the risk then detected? (c) Why would a safety horizon of 30 s be useless with a constant-velocity prediction, even though it gives the local layer plenty of time?

Exercise 1.8 (★★★ Coding). Write a scenario generator of your own, without reading `code/ch01_scenario.py` first. Given a grid size, an obstacle density, a number of drones k and a seed, it must return blocked cells, pairwise distinct free start and goal cells, each goal reachable from its start, and one intruder that enters from a border and flies straight across the grid at constant velocity without touching a blocked cell. Then:

- Check your generator with the tests of the self-test in the book's file: the same seed gives the same scenario, different seeds give different ones, the obstacle count is right, every goal is reachable, and the intruder's line is free.
- Add a function that plans a shortest path for every drone independently (breadth-first search is enough on a 4-connected grid) and a function that finds the vertex and edge conflicts among the paths.
- On a 12×8 grid with 15% blocked cells, generate 200 scenarios each for $k = 2, 4$ and 8 drones and report the fraction of scenarios whose independent shortest paths contain at least one planned conflict. What does the trend tell you about the need for a multi-agent planner as swarms grow?
- Extend the generator to several intruders and write the scenario to a JSON file; [Chapter 25](#) will read such files when it builds an experiment matrix.

The Toolbox: Graphs, Grids, Time, Motion and Uncertainty

Learning objectives

- Define graphs, grid graphs and 3D lattices precisely, including the cost of a diagonal move and the corner-cutting rule.
- Explain why a disc-shaped drone can be planned as a point once the obstacles have been inflated by a Minkowski sum.
- Tell a path, a plan and a trajectory apart; build the time-expanded graph with wait actions; compute path length, travel time, makespan, sum of costs and minimum separation.
- Write down the single- and double-integrator models of a drone with velocity and acceleration limits, and compute its stopping distance.
- Use the geometric primitives of later chapters: distance to a segment, ray-circle intersection, tangents to a circle and the time of closest approach.
- Read a covariance matrix as an ellipse, state Bayes' rule, and use the lazy-deletion priority queue and the hashed states on which every search in this book relies.

2.1 Why this matters for a drone swarm

Picture eight quadrotors inspecting the racks of a warehouse. A global planner has given each of them a route. Halfway through the mission a forklift-mounted drone that nobody planned for crosses the main aisle. To handle this scene, the four layers of [Chapter 24](#) need a surprising amount of shared vocabulary. The global planner needs a *map* it can search: a graph, usually a grid. It needs to know how *big* a drone is, so that a route through a narrow gap is not a route into a shelf. It needs *time*, because two drones may use the same aisle if they do so at different moments. It needs *cost measures* to compare routes and whole multi-agent plans. The local avoidance layer needs the drone's *motion limits* (how fast, how hard it can brake) and a small kit of *geometry*: when do two moving discs touch, and which velocities steer clear of an obstacle? The prediction layer describes the intruder with a mean and a covariance, so the planner must understand *uncertainty*. Finally, everything has to run in a few milliseconds, which is a question of *computation*: complexity, priority queues and hashing.

This chapter collects these tools in one place. It is the only chapter of the book that does not present an algorithm; instead it fixes the definitions and conventions that every later chapter cites by number. Read it once from beginning to end, then come back to it as a reference. [Table 2.1](#) shows where each tool is used again. Every formal object is introduced in

Table 2.1. Where the tools of this chapter are used again. Every later chapter cites at least one definition from here.

Tool	Section	Used again in
Graphs, grids, lattices	Section 2.2	Chapters 3 to 7
Configuration space, inflation	Section 2.3	Chapters 4, 16, 17 and 24
Space-time states, paths, plans	Section 2.4	Chapters 4 and 7 to 10
Cost measures	Section 2.5	Chapters 7, 9, 10 and 25
Motion models, stopping distance	Section 2.6	Chapters 14, 18, 21 and 22
Geometry toolkit	Section 2.7	Chapters 12 to 14 and 16
Gaussians, covariance, Bayes' rule	Section 2.8	Chapters 18 to 20
Big-O, priority queues, hashing	Section 2.9	Chapters 3 to 5 and 9
Python stack, code layout	Section 2.10	every chapter

a numbered definition, and every number quoted in the text is produced by the companion file `code/ch02_toolbox.py`.

2.2 Graphs, grids and lattices

Key idea

Every planner in this book searches a graph. The graph may be explicit (a road map stored in memory), implicit (a grid whose neighbours are computed on demand) or time-expanded (one copy of the map per time step). The search algorithms of Parts II and III do not care which; they only ask a vertex for its successors and their costs.

2.2.1 Graphs, paths and shortest paths

Definition 2.1 (Graph, weighted graph). A **directed graph** $G = (V, E)$ consists of a finite set V of **vertices** (also called *nodes*) and a set $E \subseteq V \times V$ of **edges**; the edge (u, v) leads from u to v . The graph is **undirected** if $(u, v) \in E$ implies $(v, u) \in E$; we then write $\{u, v\}$ for the pair of opposite edges and count it once. A **weighted graph** carries a cost function $c: E \rightarrow [0, \infty)$; $c(u, v)$ is the **edge cost**. The **successors** of u are $\text{Succ}(u) = \{v : (u, v) \in E\}$ and its **predecessors** are $\text{Pred}(u) = \{v : (v, u) \in E\}$; $|\text{Succ}(u)|$ is the out-degree of u , and the largest out-degree in the graph is the **branching factor** b .

Edge costs in this book are non-negative, and almost always positive: a move on a grid costs its length, a wait costs one time step. Dijkstra's algorithm (Chapter 3) needs exactly this non-negativity.

A graph can be stored in three ways. An *adjacency matrix* keeps a $|V| \times |V|$ table of costs; it answers “is there an edge from u to v ?” in constant time but needs $\mathcal{O}(|V|^2)$ memory. An **adjacency list** keeps, for every vertex, the list of its successors with their costs; in Python this is a dictionary such as `{'a': [('b', 1.0), ('c', 2.5)], 'b': [('c', 1.0)], 'c': []}`, and it needs $\mathcal{O}(|V| + |E|)$ memory. An *implicit graph* stores nothing at all: a successor function computes $\text{Succ}(u)$ on demand. Grids are implicit graphs, and so are the time-expanded graphs of Section 2.4. All the search code of this book is written against the implicit interface, a function `neighbors(v)` that returns pairs (v', cost) , because an adjacency list can always be wrapped in such a function.

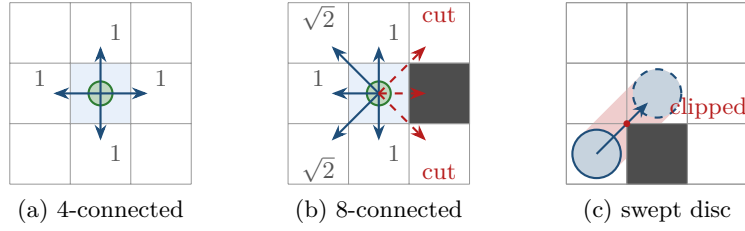


Figure 2.1. Grid connectivity. (a) The four straight moves of G_4 cost 1 each. (b) In G_8 the diagonal moves cost $\sqrt{2}$; the move into the obstacle and the two diagonals that would pass its corners are forbidden by the corner-cutting rule. (c) Why: a disc that slides diagonally past an obstacle corner clips it, even though the start and end cells are free.

Definition 2.2 (Path, path cost). A **path** from s to g in G is a sequence $\pi = (v_0, v_1, \dots, v_k)$ of vertices with $v_0 = s$, $v_k = g$ and $(v_{i-1}, v_i) \in E$ for $i = 1, \dots, k$. Its **cost** is the sum of its edge costs,

$$\text{cost}(\pi) = \sum_{i=1}^k c(v_{i-1}, v_i), \quad (2.1)$$

and k is its number of edges. A path is *simple* if no vertex appears twice. The one-vertex sequence (s) is a path from s to s of cost 0.

Definition 2.3 (Shortest path, distance). A **shortest path** from s to g is a path of minimum cost among all paths from s to g . Its cost is the **distance** $\text{dist}(s, g) = \min_{\pi} \text{cost}(\pi)$; we set $\text{dist}(s, g) = \infty$ when no path exists, and $\text{dist}(s, s) = 0$. The distance from s to g is also written $h^*(s)$ when g is fixed (Chapter 4).

Proposition 2.4 (Triangle inequality). For all vertices u, v, w of a graph with non-negative costs, $\text{dist}(u, w) \leq \text{dist}(u, v) + \text{dist}(v, w)$.

Proof. If either distance on the right is infinite there is nothing to show. Otherwise take a shortest path from u to v and append a shortest path from v to w . The result is a path from u to w of cost $\text{dist}(u, v) + \text{dist}(v, w)$, and the shortest path from u to w cannot cost more. $\square$

This small fact is the reason why consistent heuristics work in Chapter 4 and why the backward search of Chapter 3 yields an exact heuristic; you will see it again.

2.2.2 Grid graphs

Most maps in this book are **occupancy grids**: the workspace is cut into unit squares, and each square is either free or blocked. A grid is a graph in disguise, as Definition 2.5 makes precise. Our coordinate convention is the mathematical one: x grows to the right, y grows upwards, the cell $(0, 0)$ sits in the lower-left corner, and we identify a cell with its centre when we measure distances. The ASCII maps used by the code (`.` for free, `#` for blocked) list the *top* row first, so the code flips the row index when it reads them.

Definition 2.5 (Grid graph, 4- and 8-connectivity, corner-cutting rule). A **grid** of width W and height H is the set of **cells** $C = \{(x, y) : x \in \{0, \dots, W-1\}, y \in \{0, \dots, H-1\}\}$, together with a set $O \subseteq C$ of **obstacle cells**; the cells in $C \setminus O$ are *free*. The **4-connected grid graph** G_4 has the free cells as vertices and an undirected edge of cost 1 between two free cells that share a side (a *straight move*). The **8-connected grid graph** G_8 has, in addition, an edge of cost $\sqrt{2}$ between two free cells that share only a corner (a *diagonal move*), subject to the **corner-cutting rule**: the diagonal move from (x, y) to $(x + d_x, y + d_y)$ is allowed only if the two cells $(x + d_x, y)$ and $(x, y + d_y)$ it passes between are both free.

Three choices in this definition deserve a comment. First, the diagonal cost is $\sqrt{2}$ and not

Table 2.2. Neighbourhoods on grids and lattices. The stretch is the worst-case ratio between the cost of the cheapest lattice path and the straight-line distance in an empty space; it is attained in the direction that the available moves imitate worst, e.g. 22.5° for 8-connectivity. The numbers are computed by `lattice_stretch` in `ch02_toolbox.py`.

Lattice	Moves	Move costs	Corner-cutting check	Stretch
2D, 4-connected	4	1	none	$\sqrt{2} \approx 1.414$
2D, 8-connected	8	1, $\sqrt{2}$	2 side cells free	1.082
3D, 6-connected	6	1	none	$\sqrt{3} \approx 1.732$
3D, 26-connected	26	1, $\sqrt{2}$, $\sqrt{3}$	2 or 6 box cells free	1.128

1 because $\sqrt{2}$ is the distance between the centres of two diagonal neighbours. With costs 1 and $\sqrt{2}$ the cost of a grid path is exactly the Euclidean length of the polyline through the cell centres, and the straight-line and octile heuristics of [Chapter 4](#) never overestimate it. Second, the corner-cutting rule exists because a drone is not a point: [Figure 2.1\(c\)](#) shows a disc sliding diagonally past an obstacle corner and clipping it. Some code bases use the weaker rule that forbids the diagonal only when *both* side cells are blocked; that rule is unsafe for any robot of positive size, and this book never uses it. Third, [Definition 2.5](#) treats the grid as undirected: every move can be reversed. Later chapters add directed edges only when time enters ([Section 2.4](#)).

Which connectivity should you use? [Table 2.2](#) lists the **stretch** of each lattice, the worst-case ratio between the cost of the cheapest lattice path and the straight-line distance in an obstacle-free space. On a 4-connected grid the worst direction is the diagonal, where the grid path is $\sqrt{2} \approx 1.41$ times longer than the straight line; the 8-connected grid brings the worst case down to 1.08, in the direction 22.5° from an axis. Single-agent planners therefore prefer G_8 . The multi-agent benchmarks of [Chapter 7](#), following Stern et al. [39], use G_4 , because a unit cost per move makes “one move per time step” and “one unit of cost per time step” the same thing, which keeps conflicts simple.

2.2.3 Lattices in three dimensions

A drone flies in three dimensions, and the same construction works there.

Definition 2.6 (Lattice in three dimensions, 6- and 26-connectivity). A **lattice** of size $W \times H \times D$ has the cells (x, y, z) with $0 \leq x < W$, $0 \leq y < H$, $0 \leq z < D$, some of which are obstacle cells. A *move* changes each coordinate by -1 , 0 or $+1$ and is not the zero move. The **6-connected lattice** allows the six moves with one non-zero component, at cost 1. The **26-connected lattice** allows all 26 moves; a move with k non-zero components costs $\sqrt{k}$, and the corner-cutting rule requires every cell of the axis-aligned box spanned by the two endpoints to be free. The intermediate *18-connected* lattice allows the moves with at most two non-zero components.

The box rule generalises the two side cells of the plane: a move with two non-zero components spans a 2×2 box and must check the two other cells in it, a move with three non-zero components spans a $2 \times 2 \times 2$ box and must check six cells. In an empty 26-connected lattice the cheapest path to a cell at displacement (a, b, c) with $a \geq b \geq c \geq 0$ uses c triple-diagonal, $b - c$ double-diagonal and $a - b$ straight moves, so it costs $(a - b) + \sqrt{2}(b - c) + \sqrt{3}c$; maximising the ratio of this cost to $\sqrt{a^2 + b^2 + c^2}$ over all directions gives the stretch 1.128 of [Table 2.2](#). The price of 26-connectivity is a branching factor of 26 instead of 6, which multiplies the work of every search by more than four. [Chapters 16](#) and [17](#) show the alternative for 3D problems: do not discretise at all and sample the free space instead.

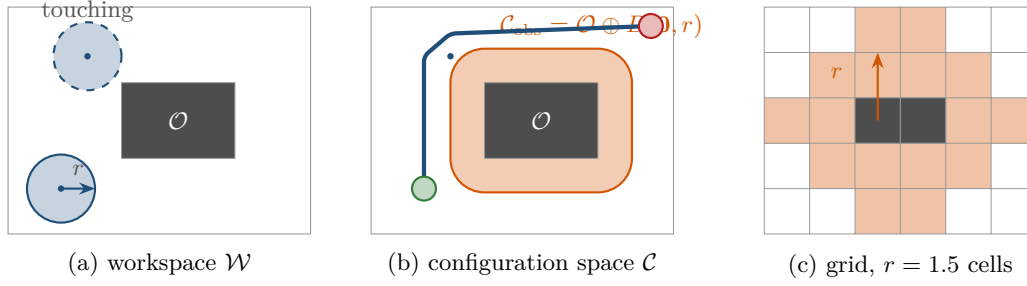


Figure 2.2. Obstacle inflation. (a) In the workspace the disc robot of radius r can at best touch the obstacle. (b) In the configuration space the obstacle grows by r (note the rounded corners) and the robot shrinks to a point; the path that touches the inflated boundary corresponds to the touching disc in (a). (c) On a grid, every cell whose centre lies within $r = 1.5$ cells of an obstacle square is blocked.

2.3 Workspace and configuration space

The map lives in the **workspace** $\mathcal{W} \subseteq \mathbb{R}^2$ or $\mathbb{R}^3$, the physical space in which obstacles occupy a region $\mathcal{O} \subset \mathcal{W}$. A drone is not a point of this space; it is a body that occupies a region. Planning a route for the point at the centre of the drone and then flying it would scrape every wall. The classical remedy, due to Lozano-Pérez and presented in every planning textbook [4, 25], is to move the problem into the configuration space.

Definition 2.7 (Configuration space, free space, obstacle region). A **configuration** q of a robot is a tuple of parameters that fixes the position of every point of the robot; the set of all configurations is the **configuration space** $\mathcal{C}$. Write $\mathcal{A}(q) \subset \mathcal{W}$ for the region of the workspace occupied by the robot in configuration q . The **obstacle region** and the **free space** are

$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} : \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}, \quad \mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}. \quad (2.2)$$

A path of configurations is *collision-free* if it lies entirely in $\mathcal{C}_{\text{free}}$.

For a disc-shaped robot of radius r whose orientation does not matter, the configuration is the centre $\mathbf{p} \in \mathbb{R}^2$, $\mathcal{C} = \mathbb{R}^2$ and $\mathcal{A}(\mathbf{p}) = D(\mathbf{p}, r)$, the closed disc of radius r around $\mathbf{p}$. The obstacle region then has a simple description, which needs one more definition.

Definition 2.8 (Minkowski sum, inflation). The **Minkowski sum** of two sets $A, B \subseteq \mathbb{R}^n$ is $A \oplus B = \{\mathbf{a} + \mathbf{b} : \mathbf{a} \in A, \mathbf{b} \in B\}$. **Inflating** a set A by the radius r means forming $A \oplus D(\mathbf{0}, r) = \{\mathbf{x} : \text{dist}(\mathbf{x}, A) \leq r\}$, the set of points within distance r of A .

Proposition 2.9 (Point-robot planning on the inflated map is exact). *For a disc (in 3D: a ball) robot of radius r whose configuration is its centre $\mathbf{p}$,*

$$\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus D(\mathbf{0}, r). \quad (2.3)$$

Consequently $\mathbf{p} \in \mathcal{C}_{\text{free}}$ if and only if $\text{dist}(\mathbf{p}, \mathcal{O}) > r$, and a path of the centre is collision-free for the disc robot if and only if it is collision-free for a point robot on the map whose obstacles are inflated by r .

Proof. The disc $D(\mathbf{p}, r)$ meets $\mathcal{O}$ if and only if there is an obstacle point $\mathbf{o} \in \mathcal{O}$ with $\|\mathbf{p} - \mathbf{o}\| \leq r$, that is, if and only if $\mathbf{p} = \mathbf{o} + \mathbf{b}$ for some $\mathbf{o} \in \mathcal{O}$ and some $\mathbf{b}$ with $\|\mathbf{b}\| \leq r$. The last statement says precisely that $\mathbf{p} \in \mathcal{O} \oplus D(\mathbf{0}, r)$. The second claim applies the first to every point of the path. $\square$

Figure 2.2 shows the construction. Inflating a polygon by a disc rounds its corners, and

inflating a union of obstacles inflates each of them. The same idea serves twice more in this book. In [Chapter 12](#) the two drones of an encounter are both discs; viewed from drone A , drone B is a disc of radius $r_A + r_B$ (the Minkowski sum of the two discs, [Proposition 2.28](#)) and A is a point. In [Chapter 24](#) the obstacle is a *predicted* position with a covariance, and the inflation radius grows with the uncertainty of the prediction.

The drone as a bounding sphere. A quadrotor with its rotor guards is not a disc, and it rotates about its vertical axis while flying. Modelling it exactly would add the yaw angle to the configuration, giving $\mathcal{C} = \mathbb{R}^3 \times S^1$ and an obstacle region that depends on the heading; for a car-like ground robot this is unavoidable [\[25\]](#). For a drone it is unnecessary: the body fits into a **bounding sphere** of radius r that is the same in every heading, so the configuration is the position alone, $\mathcal{C} = \mathbb{R}^3$ (or $\mathbb{R}^2$ at constant altitude), and [Proposition 2.9](#) applies with the sphere radius. In this book every drone is such a sphere. Its radius includes a safety margin, as the following pitfall explains.

Inflation on a grid. On a grid the inflation is approximated cell by cell: a cell is added to O if its centre lies within r of some obstacle square, which is what `Grid.inflate` does. Blocking a single cell and inflating by $r = 0.6, 1.0$ and 1.6 cell widths blocks 5, 9 and 21 cells respectively: a plus shape, a 3×3 block, and the block with a ring of 12 further cells. The grid version is only a sampled Minkowski sum; a free cell may still contain points that are closer than r to an obstacle. This is harmless for straight moves between cell centres, because for grid-aligned squares the distance to the obstacle along such a move is smallest at one of its endpoints, and the corner-cutting rule guards the diagonal moves.

Inflating by the body radius alone

The radius you inflate by is a promise that the drone’s centre will never leave the inflated free space. Three things break that promise: the drone is larger than its nominal radius when propellers and guards are counted, the position estimate is off by the localisation error, and the controller does not track the plan exactly. Use $r = r_{\text{body}} + \text{margin}$, where the margin covers the localisation and tracking errors you expect, and on a grid round the result *up* to the next cell width. A margin that is too small shows up as scraped walls; one that is too large closes narrow passages and makes the planner report that no path exists. [Chapter 25](#) measures the resulting minimum separation.

2.4 Time: space-time states, paths, plans and trajectories

A path says *where* to go; a multi-agent plan must also say *when*. The book handles time in two ways. Search-based planners (Parts II and III) use discrete time: they fix a time step Δt and let every action take exactly one step. Controllers and reactive methods (Parts IV, VI and VII) use continuous time and describe motion by trajectories. This section defines both and the words that connect them.

2.4.1 Discrete time and the time-expanded graph

Definition 2.10 (Space-time state, move and wait actions). Fix a graph $G = (V, E)$ and a time step $\Delta t > 0$. A **space-time state** is a pair (v, t) with $v \in V$ and $t \in \mathbb{N} = \{0, 1, 2, \dots\}$; it means “the agent is at vertex v at time $t \Delta t$ ”. Two kinds of **actions** lead from (v, t) to time $t + 1$: a **move** along an edge $(v, v') \in E$ leads to $(v', t + 1)$, and the **wait action** leads to $(v, t + 1)$. The move costs $c(v, v')$ and the wait costs c_{wait} ; in the unit-cost problems of [Chapter 7](#), every action costs 1.

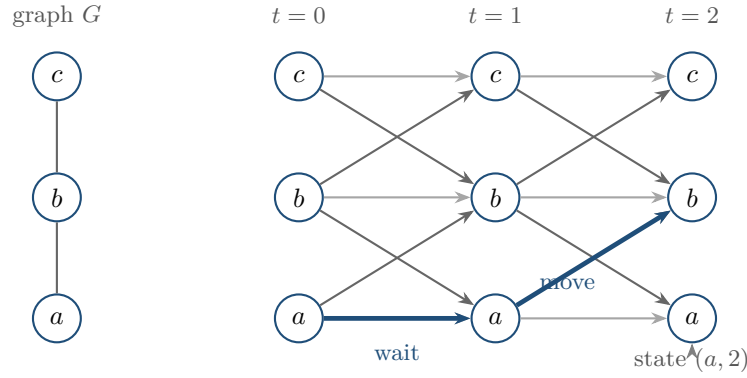


Figure 2.3. A three-vertex graph G (left) and its time-expanded graph with three time layers (right). Grey arrows are wait edges, black arrows are move edges; every edge goes from layer t to layer $t + 1$, so the graph is acyclic. The highlighted time-indexed path waits at a for one step and then moves to b , ending in the state $(b, 2)$.

Space-time states are the reason why a planner can let two agents share a corridor: the states $(v, 3)$ and $(v, 7)$ are different vertices of the graph below, so occupying one does not block the other.

Definition 2.11 (Time-expanded graph). The **time-expanded graph** of G with horizon T is the directed graph with vertex set $V \times \{0, \dots, T\}$ and, for every $t < T$, the **wait edges** $((v, t), (v, t + 1))$ for all $v \in V$ and the **move edges** $((u, t), (v, t + 1))$ for all $(u, v) \in E$, weighted as in Definition 2.10.

Figure 2.3 shows a base graph with three vertices and its time-expanded graph with three layers. Note what the construction costs: $|V| (T + 1)$ vertices and $T (|V| + 2 |E|)$ edges for an undirected G (Exercise 2.4). Nobody stores this graph. The planner keeps G and generates the successors of (v, t) on demand with a function like `space_time_successors` in the toolbox, which returns the wait successor and one move successor per free neighbour, all at time $t + 1$. Since all edges point forward in time, the time-expanded graph is acyclic; a search in it terminates even without a closed set, though a closed set still saves work.

Definition 2.12 (Time-indexed path, arrival time, stay-at-goal convention). A **time-indexed path** for an agent with start s and goal g is a sequence $\pi = (v_0, v_1, \dots, v_T)$ with $v_0 = s$, $v_T = g$ and, for every $t < T$, either $v_{t+1} = v_t$ (a wait) or $(v_t, v_{t+1}) \in E$ (a move). The agent occupies v_t at time t , and by the **stay-at-goal convention** it occupies g at every time $t > T$. The **arrival time** $T(\pi)$ is the smallest t such that $v_{t'} = g$ for all $t' \geq t$; an agent that passes through g and leaves again has not arrived.

The stay-at-goal convention matters for multi-agent planning: an agent that has landed still occupies its cell, and a later agent must route around it (Chapters 7 and 8). The arrival time is the individual cost that the multi-agent objectives of Section 2.5 add up. It is computed by `path_time` in the toolbox, which walks backwards from the end of the sequence while the vertex equals the goal.

2.4.2 Plans and trajectories

Definition 2.13 (Plan). A **plan** for k agents is a tuple $\Pi = (\pi_1, \dots, \pi_k)$ of time-indexed paths, one per agent, all with the same time step. The position of agent i at time t is v_t^i , extended by the stay-at-goal convention when t exceeds the length of π_i . Which pairs of positions count as a *conflict* is a modelling choice fixed in Chapter 7; this chapter only measures how close the agents come.

Table 2.3. Path, time-indexed path, plan and trajectory. The columns say what each object fixes, which chapters produce it and which chapters consume it.

Object	Fixes	Produced by	Consumed by
Path (Definition 2.2)	where, not when	Chapters 3 to 5 and 16	timing, a tracking controller
Time-indexed path (Definition 2.12)	where and when, in steps of Δt	Chapters 4, 8 and 9	conflict checks, execution
Plan (Definition 2.13)	one time-indexed path per agent	Chapters 8 to 11	execution, the metrics of Chapter 25
Trajectory (Definition 2.14)	position at every time	Chapters 13, 14 and 21	the flight controller

Definition 2.14 (Trajectory). A **trajectory** is a function $\mathbf{p}: [0, t_f] \rightarrow \mathcal{C}_{\text{free}}$ that gives the position at every continuous time. In this book a trajectory is represented by a sequence of **waypoints** $(t_j, \mathbf{p}_j, \mathbf{v}_j)$ with strictly increasing times and a rule for the motion between them: constant velocity, or the double integrator of Section 2.6. A time-indexed path becomes a trajectory by straight-line motion at constant speed between consecutive vertices,

$$\mathbf{p}((t+s)\Delta t) = (1-s)\mathbf{p}(v_t) + s\mathbf{p}(v_{t+1}), \quad s \in [0, 1], \quad (2.4)$$

where $\mathbf{p}(v)$ is the position of vertex v in the workspace.

Table 2.3 summarises the vocabulary. Keep the three words apart: a *path* is a geometric object, a *plan* is a set of time-indexed paths for several agents, and a *trajectory* is a continuous-time motion. The hybrid architecture of Chapter 24 converts one into the next: the global planner produces a plan, each drone follows its time-indexed path as a trajectory, and the local layer edits that trajectory when an intruder appears.

2.5 Cost measures

Definition 2.15 (Path length). The **length** of a path $\pi = (v_0, \dots, v_k)$ whose vertices have workspace positions $\mathbf{p}(v_i)$ is

$$L(\pi) = \sum_{i=1}^k \|\mathbf{p}(v_i) - \mathbf{p}(v_{i-1})\|. \quad (2.5)$$

Waiting adds nothing to the length. On a grid with the costs of Definition 2.5, $L(\pi) = \text{cost}(\pi)$.

Definition 2.16 (Travel time). The **travel time** of a time-indexed path π is its arrival time $T(\pi)$ of Definition 2.12, measured in steps; in seconds it is $T(\pi)\Delta t$. It counts moves and waits alike.

Definition 2.17 (Makespan). The **makespan** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ is the arrival time of the last agent,

$$\text{MS}(\Pi) = \max_{1 \leq i \leq k} T(\pi_i). \quad (2.6)$$

Definition 2.18 (Sum of costs). The **sum of costs** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ is the total

Table 2.4. Trace of the plan of [Example 2.20](#). The separation is evaluated at integer times and, in the last column, at the closest approach during the following step.

t	A	B	$d_{AB}(t)$	closest approach in $[t, t + 1]$	Comment
0	(0, 1)	(1, 2)	$\sqrt{2} \approx 1.414$	1.000 at $t = 1$	B waits
1	(1, 1)	(1, 2)	1.000	0.707 at $t = 1.5$	perpendicular crossing
2	(2, 1)	(1, 1)	1.000	1.000 at $t = 2$	
3	(3, 1)	(1, 0)	$\sqrt{5} \approx 2.236$	—	both have arrived

travel time of all agents,

$$\text{SoC}(\Pi) = \sum_{i=1}^k T(\pi_i). \quad (2.7)$$

Later chapters write these two objectives with the macros MS and SoC, exactly as above. They are the two standard objectives of multi-agent path finding [39]; the sum of costs is the default in [Chapters 9](#) and [10](#). They do not agree in general: a plan that minimises one may not minimise the other ([Exercise 2.5](#)).

Definition 2.19 (Separation, minimum separation, collision distance). For two agents with positions $\mathbf{p}_i(t)$ and $\mathbf{p}_j(t)$, the **separation** at time t is $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|$. The **minimum separation** of a plan or of a set of trajectories is $d_{\min} = \min_t \min_{i < j} d_{ij}(t)$ over the whole execution. Two disc (or sphere) agents of radii r_i and r_j **collide** at time t if $d_{ij}(t) < r_i + r_j$; the sum $R_{ij} = r_i + r_j$ is their **collision distance**.

For a plan on a graph the separation can be evaluated at the integer times only (the toolbox function `min_separation`) or along the straight-line motion of [Equation \(2.4\)](#) between consecutive steps (`min_separation_continuous`, which uses the closest-approach formula of [Section 2.7](#) on every interval). The two answers differ, as the worked example shows.

Example 2.20 (Two agents on a small grid). Take an empty 4-connected grid with $W = 4$ and $H = 3$. Agent A goes from (0, 1) to (3, 1) along the middle row, agent B from (1, 2) down to (1, 0). If both move without waiting, $\pi_A = ((0, 1), (1, 1), (2, 1), (3, 1))$ and $\pi'_B = ((1, 2), (1, 1), (1, 0))$ put both agents in cell (1, 1) at $t = 1$: the separation is 0. Let B wait one step first, $\pi_B = ((1, 2), (1, 2), (1, 1), (1, 0))$. [Table 2.4](#) traces the plan $\Pi = (\pi_A, \pi_B)$. The lengths are $L(\pi_A) = 3$ and $L(\pi_B) = 2$, the travel times are $T(\pi_A) = 3$ and $T(\pi_B) = 3$, hence $\text{MS}(\Pi) = 3$ and $\text{SoC}(\Pi) = 6$. At integer times the minimum separation is 1. Between $t = 1$ and $t = 2$, however, A moves from (1, 1) to (2, 1) while B moves from (1, 2) to (1, 1); in the middle of that step, at $t = 1.5$, the agents are at (1.5, 1) and (1, 1.5), only $\sqrt{0.5} \approx 0.707$ apart. With a body radius of 0.3 cells (collision distance 0.6) the plan is safe; with a radius of 0.4 (collision distance 0.8) the two agents touch although they never share a cell.

A conflict-free plan is not automatically collision-free

The conflict definitions of [Chapter 7](#) look at integer times and at swaps along an edge. [Example 2.20](#) shows a plan without any such conflict in which two agents of radius 0.4 cells still touch halfway through a step. Grid planning is physically safe only if the agents are small compared with the cell (a collision distance below $\sqrt{0.5} \approx 0.71$ cells for perpendicular crossings), or if you inflate the cells, add a check along the straight-line motion, or forbid such crossings as an extra conflict type. [Chapter 25](#) reports the minimum separation of an execution precisely so that this gap between the model and the physics becomes visible.

2.6 Motion models for a drone

Graph search ignores dynamics: a move is a move, whatever the speed. The reactive and control layers cannot afford that, because a drone at 5 m/s needs metres to stop. This section fixes the two motion models of the book. Both are *linear*, which is what makes the Kalman filter of Chapter 18 and the model predictive controller of Chapter 21 tractable.

Definition 2.21 (Single integrator). The **single integrator** (point-mass model) has the state $\mathbf{p} \in \mathbb{R}^n$, $n \in \{2, 3\}$, and the velocity $\mathbf{v} \in \mathbb{R}^n$ as its control input, limited by $\|\mathbf{v}\| \leq v_{\max}$. In continuous time $\dot{\mathbf{p}} = \mathbf{v}$; in discrete time with step Δt ,

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k. \quad (2.8)$$

The single integrator assumes that the drone can change its velocity instantly. That is the model behind velocity obstacles and optimal reciprocal collision avoidance (ORCA; Chapters 12 and 13), whose output is a velocity, and it is implicitly the model of a grid planner, where a move of one cell per step is a velocity of one cell per Δt .

Definition 2.22 (Double integrator). The **double integrator** has the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2n}$ and the acceleration $\mathbf{a} \in \mathbb{R}^n$ as its control input, limited by $\|\mathbf{a}\| \leq a_{\max}$; the speed is limited by $\|\mathbf{v}\| \leq v_{\max}$. In continuous time $\dot{\mathbf{p}} = \mathbf{v}$ and $\dot{\mathbf{v}} = \mathbf{a}$. If the acceleration is held constant during each step of length Δt (a *zero-order hold*), the discrete-time model is exactly

$$\mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{a}_k, \quad \mathbf{A} = \begin{pmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{pmatrix}, \quad (2.9)$$

where $\mathbf{I}$ is the $n \times n$ identity.

To see where $\mathbf{A}$ and $\mathbf{B}$ come from, integrate a constant acceleration $\mathbf{a}$ over one step: the velocity becomes $\mathbf{v} + \Delta t \mathbf{a}$ and the position, which is the integral of the velocity, becomes $\mathbf{p} + \Delta t \mathbf{v} + \frac{1}{2} \Delta t^2 \mathbf{a}$. The two rows of Equation (2.9) say exactly this. The model is exact, not an approximation; the forward-Euler update $\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k$ that many programs use drops the term $\frac{1}{2} \Delta t^2 \mathbf{a}$ and is exact only when the acceleration is zero. The Kalman filter of Chapter 18 uses the same $\mathbf{A}$ with the acceleration replaced by noise (the *constant-velocity model*), and the controller of Chapter 21 uses $\mathbf{A}$ and $\mathbf{B}$ as its prediction model.

Holonomic and non-holonomic. Not every robot can move in every direction; the distinction has a name.

Definition 2.23 (Holonomic and non-holonomic systems). A robot is **holonomic** if every constraint on its motion restricts only the configurations it may occupy, so that from any configuration it can move in every direction of the configuration space. It is **non-holonomic** if some constraint restricts its velocity without restricting the reachable configurations.

A car is the textbook non-holonomic system: it cannot slide sideways, yet it can reach any parking spot by manoeuvring. Planning for such systems needs the tools of LaValle [25], which this book does not cover. A quadrotor, at the level of position, is holonomic: it can accelerate in any direction by tilting, and it can hover. That is why a drone can be abstracted as a point mass at all, while a fixed-wing aircraft, which must keep flying forward, cannot.

Why a quadrotor is a double integrator, and when it is not. A quadrotor produces one thrust force along its body axis and controls its direction by tilting. Its centre of mass therefore obeys $m\ddot{\mathbf{p}} = \mathbf{f}_{\text{thrust}} - mg\mathbf{e}_z$: given a desired acceleration $\mathbf{a}$, the flight controller solves for the thrust magnitude and the attitude that produce it. Because the attitude loop of a small quadrotor settles in a fraction of a second, the position dynamics seen by a planner that

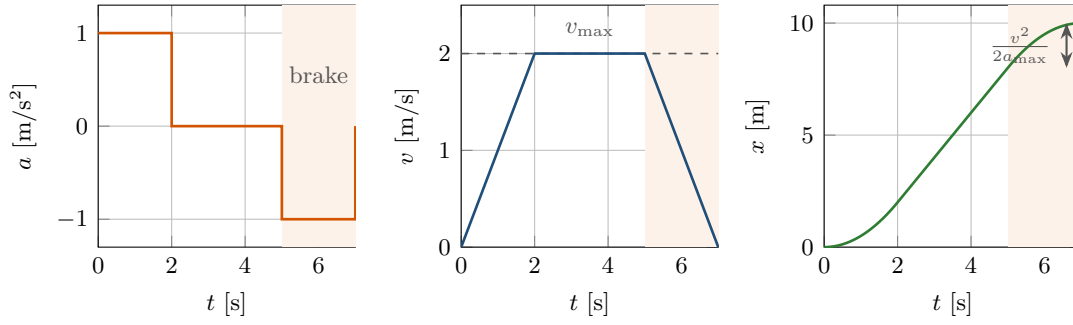


Figure 2.4. A one-dimensional double integrator with $a_{\max} = 1 \text{ m/s}^2$, $v_{\max} = 2 \text{ m/s}$ and $\Delta t = 0.1 \text{ s}$ travels 10 m. It accelerates until it reaches $v_{\max}$ at $t = 2 \text{ s}$, cruises, and starts braking at $t = 5 \text{ s}$ when the remaining distance equals the stopping distance $v_{\max}^2/(2a_{\max}) = 2 \text{ m}$ (shaded). It stops exactly at 10 m at $t = 7 \text{ s}$, because Equation (2.9) is exact for piecewise-constant acceleration.

replans every $\Delta t = 0.1 \text{ s}$ or slower are very nearly the double integrator, with $a_{\max}$ set by the thrust-to-weight ratio and the maximum tilt. The theory of differential flatness makes this precise: any sufficiently smooth position trajectory can be tracked by a quadrotor [28]. The caveats are that the acceleration cannot jump (the attitude must first rotate), that the limits are not symmetric (gravity helps descending and hinders climbing), that aerodynamic drag grows with speed, and that a heavy payload slows the attitude loop. Choose $a_{\max}$ and $v_{\max}$ with a margin below the physical limits, and the abstraction holds.

Proposition 2.24 (Stopping distance). *A double integrator moving at speed v that brakes with the maximal deceleration $a_{\max}$ comes to rest after the time $v/a_{\max}$ and the **stopping distance***

$$d_{\text{stop}}(v) = \frac{v^2}{2a_{\max}}. \quad (2.10)$$

Consequently a drone that must be able to stop before an obstacle at distance d straight ahead may not move faster than $\sqrt{2a_{\max}d}$.

Proof. Along the direction of motion the speed is $v(t) = v - a_{\max}t$, which reaches zero at $t_s = v/a_{\max}$. The distance covered is $\int_0^{t_s} v(t) dt = vt_s - \frac{1}{2}a_{\max}t_s^2 = v^2/a_{\max} - v^2/(2a_{\max})$. Solving $d_{\text{stop}}(v) \leq d$ for v gives the bound. $\square$

Figure 2.4 shows the formula at work: the drone starts braking when the remaining distance equals $d_{\text{stop}}(v_{\max}) = 2^2/(2 \cdot 1) = 2 \text{ m}$ and stops on the mark. The stopping distance is the single most important number of the reactive layer. It sets the safety horizon after which an intruder must trigger avoidance (Chapter 24), it defines the admissible velocities of the dynamic window approach (Chapter 14), and it tells you how far ahead a prediction must look (Chapter 20). Table 2.5 lists the models side by side.

Clipping breaks linearity

The limits $\|\mathbf{a}\| \leq a_{\max}$ and $\|\mathbf{v}\| \leq v_{\max}$ are not part of the linear model Equation (2.9); they are constraints on top of it. Code that first applies $\mathbf{A}$ and $\mathbf{B}$ and then clips the velocity has silently changed the position update too, because the position moved with the unclipped velocity for half a step. The toolbox function `double_integrator_step` avoids the inconsistency by clipping the input, computing the new velocity, and then advancing the position with the average of the old and the new velocity, which coincides with $\mathbf{Ax} + \mathbf{Ba}$ whenever no limit is active. Model predictive control (Chapter 21) handles the limits properly, as constraints of an optimisation problem.

Table 2.5. Motion models used in the book. The graph model has no dynamics at all; the two integrator models are linear; the full quadrotor model appears only inside a physics simulator.

Model	State	Input	Limits	Used in
Graph move	vertex v	move or wait	one edge per step	Parts II and III
Single integrator	$\mathbf{p}$	$\mathbf{v}$	$\ \mathbf{v}\ \leq v_{\max}$	Chapters 12, 13 and 15 to 17
Double integrator	$(\mathbf{p}, \mathbf{v})$	$\mathbf{a}$	$\ \mathbf{a}\ \leq a_{\max},$ $\ \mathbf{v}\ \leq v_{\max}$	Chapters 14, 18, 21 and 22
Unicycle (ground robot)	$(\mathbf{p}, \theta)$	speed, turn rate	non-holonomic	contrast only
Full quadrotor	$\mathbf{p}, \mathbf{v}$, attitude, rates	thrust, torques	motor limits	simulation only

2.7 A geometry toolkit

The reactive layer of the book is built from a handful of geometric primitives. They are collected here with their formulas and proofs; the toolbox implements each of them on plain tuples of floats, so that positions can also serve as dictionary keys.

Definition 2.25 (Vectors, dot product, 2D cross product). For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ the **dot product** is $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \phi$, where ϕ is the angle between the vectors and $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$. The **projection** of $\mathbf{p}$ onto a unit vector $\mathbf{u}$ is $(\mathbf{p} \cdot \mathbf{u}) \mathbf{u}$, and $\mathbf{p} - (\mathbf{p} \cdot \mathbf{u}) \mathbf{u}$ is the part of $\mathbf{p}$ perpendicular to $\mathbf{u}$. In the plane, the **2D cross product** $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x = \|\mathbf{a}\| \|\mathbf{b}\| \sin \phi$ is the signed area of the parallelogram spanned by the two vectors; it is positive when $\mathbf{b}$ lies counter-clockwise from $\mathbf{a}$ (a left turn). The vector $\mathbf{a}^\perp = (-a_y, a_x)$ is $\mathbf{a}$ rotated by 90° counter-clockwise. In three dimensions $\mathbf{a} \times \mathbf{b}$ is the vector perpendicular to both with length $\|\mathbf{a}\| \|\mathbf{b}\| |\sin \phi|$.

Definition 2.26 (Distance from a point to a segment). Let $\mathbf{a} \neq \mathbf{b}$ be the endpoints of a segment and $\mathbf{p}$ a point. With

$$s = \min\left(1, \max\left(0, \frac{(\mathbf{p} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|^2}\right)\right), \quad \mathbf{q} = \mathbf{a} + s(\mathbf{b} - \mathbf{a}), \quad (2.11)$$

the point $\mathbf{q}$ is the **closest point** of the segment to $\mathbf{p}$ and $\text{dist}(\mathbf{p}, \overline{\mathbf{a}\mathbf{b}}) = \|\mathbf{p} - \mathbf{q}\|$. When $\mathbf{a} = \mathbf{b}$ we set $s = 0$.

The unclamped fraction in Equation (2.11) is the projection of $\mathbf{p}$ onto the line through $\mathbf{a}$ and $\mathbf{b}$, expressed as a multiple of $\mathbf{b} - \mathbf{a}$. Clamping it to $[0, 1]$ handles the three cases: the foot of the perpendicular lies inside the segment ($0 < s < 1$), or before $\mathbf{a}$ ($s = 0$), or beyond $\mathbf{b}$ ($s = 1$). This one function is the collision checker of the book: a disc obstacle of radius r blocks a straight motion from $\mathbf{a}$ to $\mathbf{b}$ if and only if the distance from its centre to the segment is at most r , and the clearance of a candidate motion in Chapter 14 is the smallest such distance over all obstacles.

Proposition 2.27 (Ray–circle and segment–circle intersection). Let $\mathbf{o} + t\mathbf{d}$, $t \geq 0$, be a ray and $\|\mathbf{x} - \mathbf{c}\| = r$ a circle, and put $\mathbf{f} = \mathbf{o} - \mathbf{c}$. The ray meets the circle where

$$(\mathbf{d} \cdot \mathbf{d}) t^2 + 2(\mathbf{f} \cdot \mathbf{d}) t + (\mathbf{f} \cdot \mathbf{f} - r^2) = 0, \quad (2.12)$$

$$t_{1,2} = \frac{-(\mathbf{f} \cdot \mathbf{d}) \mp \sqrt{\Delta}}{\mathbf{d} \cdot \mathbf{d}}, \quad \Delta = (\mathbf{f} \cdot \mathbf{d})^2 - (\mathbf{d} \cdot \mathbf{d})(\mathbf{f} \cdot \mathbf{f} - r^2).$$

If $\Delta < 0$ the ray misses the circle; otherwise the first contact is at the smaller root t_1 , provided $t_1 \geq 0$. If $\mathbf{f} \cdot \mathbf{f} \leq r^2$ the ray starts inside the disc and the contact time is 0. The segment

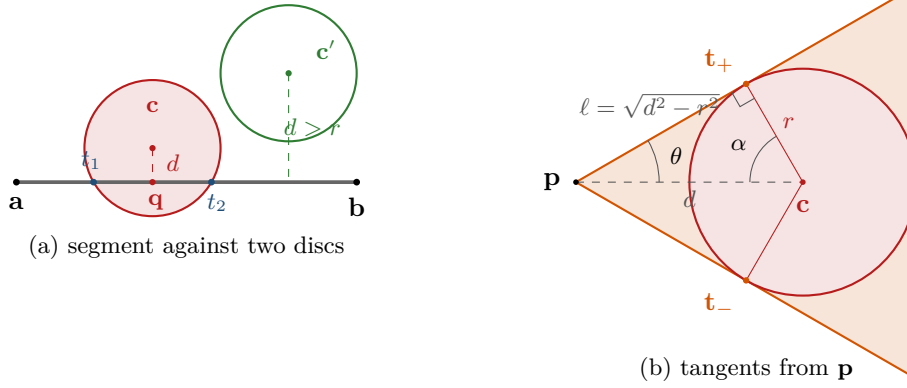


Figure 2.5. (a) The segment $\overline{ab}$ meets the disc around $\mathbf{c}$ because the closest point $\mathbf{q}$ is at distance $d \leq r$; the ray $\mathbf{a} + t(\mathbf{b} - \mathbf{a})$ crosses the circle at t_1 and t_2 . The disc around $\mathbf{c}'$ is missed because $d' > r$. (b) The two tangents from $\mathbf{p}$ to a circle touch it at $\mathbf{t}_{\pm}$; the radius is perpendicular to the tangent, so $\cos \alpha = r/d$ and $\sin \theta = r/d$. Every ray from $\mathbf{p}$ inside the shaded cone hits the disc; this cone is the velocity obstacle of Chapter 12.

from $\mathbf{a}$ to $\mathbf{b}$ meets the disc $D(\mathbf{c}, r)$ if and only if $\text{dist}(\mathbf{c}, \overline{ab}) \leq r$; it crosses the circle at the roots of Equation (2.12) with $\mathbf{o} = \mathbf{a}$, $\mathbf{d} = \mathbf{b} - \mathbf{a}$ that lie in $[0, 1]$.

Proof. Substitute $\mathbf{x} = \mathbf{o} + t\mathbf{d}$ into $\|\mathbf{x} - \mathbf{c}\|^2 = r^2$ and expand $\|\mathbf{f} + t\mathbf{d}\|^2$; the quadratic formula gives the roots. The segment statement restates the definition of the distance to a segment. $\square$

Proposition 2.28 (Minkowski sum of two discs). $D(\mathbf{c}_1, r_1) \oplus D(\mathbf{c}_2, r_2) = D(\mathbf{c}_1 + \mathbf{c}_2, r_1 + r_2)$. In particular, two discs $D(\mathbf{p}_A, r_A)$ and $D(\mathbf{p}_B, r_B)$ overlap if and only if $\|\mathbf{p}_B - \mathbf{p}_A\| \leq r_A + r_B$, i.e. if and only if the point $\mathbf{p}_B - \mathbf{p}_A$ lies in $D(\mathbf{0}, r_A + r_B)$.

Proof. If $\mathbf{x} = \mathbf{a} + \mathbf{b}$ with $\|\mathbf{a} - \mathbf{c}_1\| \leq r_1$ and $\|\mathbf{b} - \mathbf{c}_2\| \leq r_2$, the triangle inequality gives $\|\mathbf{x} - \mathbf{c}_1 - \mathbf{c}_2\| \leq r_1 + r_2$. Conversely, if $\mathbf{w} = \mathbf{x} - \mathbf{c}_1 - \mathbf{c}_2$ has $\|\mathbf{w}\| \leq r_1 + r_2$, take $\mathbf{a} = \mathbf{c}_1 + \frac{r_1}{r_1 + r_2} \mathbf{w}$ and $\mathbf{b} = \mathbf{c}_2 + \frac{r_2}{r_1 + r_2} \mathbf{w}$; then $\mathbf{a} + \mathbf{b} = \mathbf{x}$ and both points lie in their discs. The overlap statement is Proposition 2.9 applied with $\mathcal{O} = D(\mathbf{p}_B, r_B)$ and the robot $D(\mathbf{p}_A, r_A)$. $\square$

This is the *relative frame* that Chapters 12 and 13 use throughout: put drone A at the origin as a point, and replace drone B by a disc of radius $r_A + r_B$ at the relative position $\mathbf{p}_B - \mathbf{p}_A$. The next primitive describes the directions in which that disc is visible from the origin.

Proposition 2.29 (Tangent lines from a point to a circle). Let $d = \|\mathbf{p} - \mathbf{c}\| > r$ and $\mathbf{u} = (\mathbf{p} - \mathbf{c})/d$. The two tangents from $\mathbf{p}$ to the circle $\|\mathbf{x} - \mathbf{c}\| = r$ touch it at

$$\mathbf{t}_{\pm} = \mathbf{c} + r (\cos \alpha \mathbf{u} \pm \sin \alpha \mathbf{u}^{\perp}), \quad \cos \alpha = \frac{r}{d}, \quad \sin \alpha = \sqrt{1 - \frac{r^2}{d^2}}. \quad (2.13)$$

Both tangents have length $\ell = \sqrt{d^2 - r^2}$ and make the half-angle $\theta = \arcsin(r/d)$ with the line from $\mathbf{p}$ to $\mathbf{c}$; a ray from $\mathbf{p}$ meets the disc if and only if its direction is within θ of $\mathbf{c} - \mathbf{p}$.

Proof. A radius is perpendicular to the tangent at its endpoint, so $\mathbf{p}$, $\mathbf{c}$ and $\mathbf{t}_{\pm}$ form a right triangle with the right angle at $\mathbf{t}_{\pm}$, hypotenuse d and legs r and ℓ . Hence $\cos \alpha = r/d$ for the angle α at $\mathbf{c}$, $\ell = \sqrt{d^2 - r^2}$, and $\sin \theta = r/d$ for the angle θ at $\mathbf{p}$. Rotating $\mathbf{u}$ by $\pm \alpha$ and scaling by r gives the two tangent points relative to $\mathbf{c}$. The rays that hit the disc are exactly those between the two tangents. $\square$

Figure 2.5(b) shows the geometry for $\mathbf{p} = (0, 0)$, $\mathbf{c} = (4, 0)$ and $r = 2$: the tangent points are $(3, \pm\sqrt{3})$, the tangent length is $\sqrt{12}$ and the half-angle is 30° (Exercise 2.8). When $\mathbf{p}$

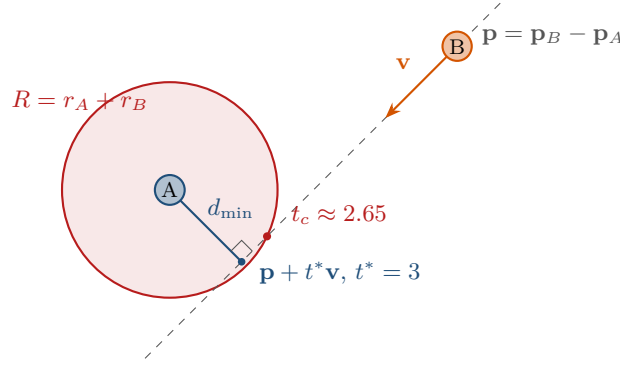


Figure 2.6. Closest approach in the relative frame of [Example 2.32](#). Drone *A* sits at the origin as a point; drone *B* starts at $\mathbf{p} = (4, 2)$ and moves with the relative velocity $\mathbf{v} = (-1, -1)$. The perpendicular from the origin to the line of motion gives $d_{\min} = \sqrt{2}$ at $t^* = 3$. With collision distance $R = 1.5$ the relative motion enters the disc at $t_c \approx 2.65$.

approaches the circle, θ grows towards 90° and the cone opens into a half-plane; when $\mathbf{p}$ is inside, there are no tangents, and the toolbox returns `None`.

Definition 2.30 (Closest approach and time to collision). Let two objects move with constant velocities, $\mathbf{p}_A(t) = \mathbf{p}_A + t\mathbf{v}_A$ and $\mathbf{p}_B(t) = \mathbf{p}_B + t\mathbf{v}_B$, and write $\mathbf{p} = \mathbf{p}_B - \mathbf{p}_A$ and $\mathbf{v} = \mathbf{v}_B - \mathbf{v}_A$ for the **relative position and velocity**. Their distance is $D(t) = \|\mathbf{p} + t\mathbf{v}\|$. The **time of closest approach** is $t^* = \arg \min_{t \geq 0} D(t)$, the **minimum distance** is $d_{\min} = D(t^*)$, and, for a collision distance R , the **time to collision** is $t_c = \min \{t \geq 0 : D(t) \leq R\}$, or ∞ if no such time exists. With a horizon τ the minimisation is restricted to $t \in [0, \tau]$.

Proposition 2.31 (Closest approach of two constant-velocity objects). *If $\mathbf{v} \neq \mathbf{0}$ then*

$$t^* = \max\left(0, -\frac{\mathbf{p} \cdot \mathbf{v}}{\mathbf{v} \cdot \mathbf{v}}\right), \quad d_{\min} = \|\mathbf{p} + t^*\mathbf{v}\|, \quad (2.14)$$

and when the maximum is not clamped, $d_{\min} = |\mathbf{p} \times \mathbf{v}| / \|\mathbf{v}\|$ in the plane (in 3D, $\|\mathbf{p} \times \mathbf{v}\| / \|\mathbf{v}\|$). The objects come within R of each other if and only if $d_{\min} \leq R$, and then

$$t_c = \max\left(0, t^* - \frac{\sqrt{R^2 - d_{\min}^2}}{\|\mathbf{v}\|}\right). \quad (2.15)$$

If $\mathbf{v} = \mathbf{0}$ the distance is constant, $t^ = 0$ and t_c is 0 if $\|\mathbf{p}\| \leq R$ and ∞ otherwise.*

Proof. $D(t)^2 = \|\mathbf{p}\|^2 + 2t(\mathbf{p} \cdot \mathbf{v}) + t^2\|\mathbf{v}\|^2$ is a parabola in t with positive leading coefficient; its derivative $2(\mathbf{p} \cdot \mathbf{v}) + 2t\|\mathbf{v}\|^2$ vanishes at $t = -(\mathbf{p} \cdot \mathbf{v}) / \|\mathbf{v}\|^2$. If this value is negative, the objects are already separating and the closest approach on $t \geq 0$ is at $t = 0$, which the maximum expresses. At the unclamped minimum, $D^2 = \|\mathbf{p}\|^2 - (\mathbf{p} \cdot \mathbf{v})^2 / \|\mathbf{v}\|^2 = (\|\mathbf{p}\|^2\|\mathbf{v}\|^2 - (\mathbf{p} \cdot \mathbf{v})^2) / \|\mathbf{v}\|^2 = (\mathbf{p} \times \mathbf{v})^2 / \|\mathbf{v}\|^2$ by Lagrange's identity. For the time to collision, $D(t)^2 = R^2$ is the quadratic of [Proposition 2.27](#) with $\mathbf{o} = \mathbf{p}$, $\mathbf{d} = \mathbf{v}$, $\mathbf{c} = \mathbf{0}$; it has real roots exactly when $d_{\min} \leq R$, symmetric about t^* at distance $\sqrt{R^2 - d_{\min}^2} / \|\mathbf{v}\|$, and the smaller root is the first contact, clamped at 0 when the discs already overlap. $\square$

Example 2.32 (Two drones on crossing courses). Drone *A* is at the origin and flies along the x -axis at 1 m/s; drone *B* is at $(4, 2)$ and flies straight down at 1 m/s. Then $\mathbf{p} = (4, 2)$ and $\mathbf{v} = (0, -1) - (1, 0) = (-1, -1)$. From [Equation \(2.14\)](#), $t^* = -(\mathbf{p} \cdot \mathbf{v}) / (\mathbf{v} \cdot \mathbf{v}) = -(-6)/2 = 3$ s and $d_{\min} = |\mathbf{p} \times \mathbf{v}| / \|\mathbf{v}\| = |4 \cdot (-1) - 2 \cdot (-1)| / \sqrt{2} = 2/\sqrt{2} = \sqrt{2} \approx 1.414$ m. If both drones have radius 0.75 m, the collision distance is $R = 1.5 > \sqrt{2}$, and [Equation \(2.15\)](#) gives the first contact at $t_c = 3 - \sqrt{2.25 - 2} / \sqrt{2} \approx 2.646$ s. If both have radius 0.5 m then $R = 1 < \sqrt{2}$

and they pass each other safely. The toolbox functions `time_of_closest_approach` and `time_to_collision` return exactly these numbers; Figure 2.6 draws the situation.

Three remarks close the toolkit. First, the time to collision is a ray–circle intersection in the relative frame: `time_to_collision` in the toolbox is a one-line call to `ray_circle_intersection` with $\mathbf{o} = \mathbf{p}$, $\mathbf{d} = \mathbf{v}$ and the disc $D(\mathbf{0}, R)$; the set of relative velocities for which that ray hits the disc is the cone of Proposition 2.29, and that cone is the velocity obstacle of Chapter 12. Second, a horizon τ turns “will they ever collide?” into “will they collide within the next τ seconds?”, which is the question the safety layer of Chapter 24 asks; the toolbox clamps t^* to $[0, \tau]$. Third, everything in this section carries over to three dimensions without change, except that the cross product becomes a vector (use its norm in Equation (2.14)) and the two tangent lines become a tangent cone with the same half-angle $\theta = \arcsin(r/d)$.

2.8 Uncertainty: random vectors, Gaussians and Bayes' rule

The tracker of Chapter 18 does not hand the planner a position; it hands over an estimate *and* a statement of how much to trust it. That statement is a covariance matrix, and the language for it is probability. This section fixes the few facts that Part VI builds on and that Chapter 24 uses to inflate constraints.

Definition 2.33 (Random vector, mean, covariance). A **random vector** $\mathbf{x} \in \mathbb{R}^n$ has a probability density $p(\mathbf{x})$. Its **mean** and **covariance** are

$$\mu = \mathbb{E}[\mathbf{x}] = \int \mathbf{x} p(\mathbf{x}) d\mathbf{x}, \quad \Sigma = \text{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^\top]. \quad (2.16)$$

Σ is symmetric and positive semidefinite; its diagonal entries are the variances $\Sigma_{ii} = \text{Var}(x_i) = \sigma_i^2$, and Σ_{ij} is the covariance of x_i and x_j , with correlation coefficient $\rho_{ij} = \Sigma_{ij}/(\sigma_i \sigma_j) \in [-1, 1]$. Under an affine map $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$,

$$\mathbb{E}[\mathbf{y}] = \mathbf{A}\mu + \mathbf{b}, \quad \text{Cov}(\mathbf{y}) = \mathbf{A}\Sigma\mathbf{A}^\top. \quad (2.17)$$

Definition 2.34 (Multivariate Gaussian). The **multivariate Gaussian** (normal) distribution $\mathcal{N}(\mu, \Sigma)$ with positive definite Σ has the density

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)\right). \quad (2.18)$$

The quadratic form in the exponent is the squared **Mahalanobis distance** of $\mathbf{x}$ from μ . An affine map of a Gaussian vector is Gaussian, with the mean and covariance of Equation (2.17), and the sum of two independent Gaussian vectors is Gaussian with the sum of the means and the sum of the covariances.

These two closure properties are the whole reason the Kalman filter has a closed form: pushing a Gaussian through the linear motion model of Definition 2.22 and adding Gaussian noise gives another Gaussian, so the filter only has to update a mean and a covariance (Chapter 18). A covariance matrix is hard to read as numbers; it is easy to read as an ellipse.

Definition 2.35 (Covariance ellipse). The $k\sigma$ **covariance ellipse** (in 3D: ellipsoid) of $\mathcal{N}(\mu, \Sigma)$ is the set of points at Mahalanobis distance k from the mean,

$$\mathcal{E}_k = \left\{ \mathbf{x} : (\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu) = k^2 \right\}. \quad (2.19)$$

Proposition 2.36 (Axes of the covariance ellipse). Let $\Sigma = \mathbf{E}\mathbf{\Lambda}\mathbf{E}^\top$ be the eigen-decomposition of a 2×2 covariance, with orthonormal eigenvectors $\mathbf{e}_1, \mathbf{e}_2$ as the columns of $\mathbf{E}$ and eigenvalues $\lambda_1 \geq \lambda_2 > 0$. Then $\mathcal{E}_k$ is the ellipse centred at μ with semi-axis $k\sqrt{\lambda_1}$ along $\mathbf{e}_1$ and $k\sqrt{\lambda_2}$

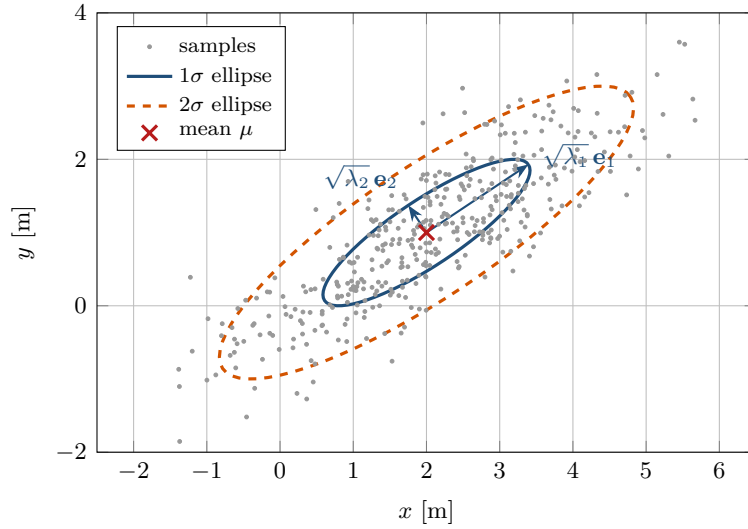


Figure 2.7. Samples from the Gaussian of [Example 2.37](#) with its 1σ and 2σ covariance ellipses. The arrows are the semi-axes $\sqrt{\lambda_i} \mathbf{e}_i$ of the 1σ ellipse; the strong positive correlation shows as the tilt. Only about 39% of the samples lie inside the 1σ ellipse.

along $\mathbf{e}_2$,

$$\mathbf{x}(\varphi) = \mu + k(\sqrt{\lambda_1} \cos \varphi \mathbf{e}_1 + \sqrt{\lambda_2} \sin \varphi \mathbf{e}_2), \quad \varphi \in [0, 2\pi), \quad (2.20)$$

and the major axis makes the angle $\text{atan2}(e_{1,y}, e_{1,x})$ with the x -axis. In two dimensions the probability mass inside $\mathcal{E}_k$ is $1 - e^{-k^2/2}$: 39.3% for $k = 1$, 86.5% for $k = 2$ and 98.9% for $k = 3$.

Proof. Substitute $\mathbf{y} = \mathbf{E}^T(\mathbf{x} - \mu)$. Since $\mathbf{E}$ is orthogonal, $\Sigma^{-1} = \mathbf{E}\mathbf{\Lambda}^{-1}\mathbf{E}^T$ and the quadratic form becomes $y_1^2/\lambda_1 + y_2^2/\lambda_2 = k^2$, an axis-aligned ellipse with semi-axes $k\sqrt{\lambda_i}$, which [Equation \(2.20\)](#) parametrises; $\mathbf{x} = \mu + \mathbf{E}\mathbf{y}$ rotates it back onto the eigenvectors. For the mass, note that $\mathbf{y}$ scaled by $\mathbf{\Lambda}^{-1/2}$ is a standard Gaussian in $\mathbb{R}^2$, whose squared norm is χ^2 -distributed with two degrees of freedom, i.e. exponential with mean 2; hence $\mathbb{P}(\text{inside } \mathcal{E}_k) = 1 - e^{-k^2/2}$. $\square$

To draw the ellipse in practice, call `numpy.linalg.eigh` on Σ (it returns the eigenvalues in ascending order, so reverse them), take the square roots, and evaluate [Equation \(2.20\)](#) at, say, 72 angles. The toolbox does this in `covariance_ellipse` and `ellipse_points`; the self-test checks that every returned point has squared Mahalanobis distance exactly k^2 .

Example 2.37 (Reading a covariance matrix). Let $\mu = (2, 1)$ and $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$, so $\sigma_x = \sqrt{2}$, $\sigma_y = 1$ and $\rho = 1.2/\sqrt{2} \approx 0.85$. The eigenvalues solve $\lambda^2 - 3\lambda + 0.56 = 0$, giving $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$, with $\mathbf{e}_1 \propto (3, 2)$. The 1σ ellipse therefore has semi-axes $\sqrt{2.8} \approx 1.673$ and $\sqrt{0.2} \approx 0.447$, and its major axis is tilted by $\arctan(2/3) \approx 33.7^\circ$. [Figure 2.7](#) shows 400 samples drawn with a fixed seed: 39.8% of them fall inside the 1σ ellipse and 85.8% inside the 2σ ellipse, close to the theoretical 39.3% and 86.5%.

The 68–95 rule is one-dimensional

In one dimension 68.3% of the mass of a Gaussian lies within 1σ and 95.4% within 2σ . In two dimensions the 1σ ellipse holds only 39.3% and the 2σ ellipse 86.5%; in three dimensions the figures drop to 19.9% and 73.9%. If you inflate a predicted obstacle by “one sigma” to be safe, you are covering less than half of the cases. Choose k from the χ^2 distribution with the right number of degrees of freedom for the confidence you need; [Chapter 24](#) does this when it turns predictions into constraints.

Table 2.6. Cost of the basic operations used by the search code of this book. n is the number of stored items. Dictionary and set costs are averages; keys must be hashable.

Operation	Python	Time	Note
append, index, pop from the end	<code>list</code>	$\mathcal{O}(1)$	
pop from the front	<code>list.pop(0)</code>	$\mathcal{O}(n)$	use <code>collections.deque</code>
lookup, insert, delete	<code>dict</code> , <code>set</code>	$\mathcal{O}(1)$	tuples of integers as keys
push, pop-min	<code>heapq</code>	$\mathcal{O}(\log n)$	binary heap on a list
decrease-key	—	—	not available: use lazy deletion
sort	<code>sorted</code>	$\mathcal{O}(n \log n)$	
vector operation of length n	NumPy	$\mathcal{O}(n)$	avoid Python loops over arrays

Bayes’ rule. Everything in Part VI is an application of one formula. If $\mathbf{x}$ is the unknown state and $\mathbf{z}$ a measurement, then

$$p(\mathbf{x} | \mathbf{z}) = \frac{p(\mathbf{z} | \mathbf{x}) p(\mathbf{x})}{p(\mathbf{z})}, \quad p(\mathbf{z}) = \int p(\mathbf{z} | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}, \quad (2.21)$$

where $p(\mathbf{x})$ is the **prior** (what you believed before the measurement), $p(\mathbf{z} | \mathbf{x})$ the **likelihood** (how probable the measurement is for each state) and $p(\mathbf{x} | \mathbf{z})$ the **posterior**; the denominator is a normalising constant that does not depend on $\mathbf{x}$, so in practice one writes posterior $\propto$ likelihood $\times$ prior. Tracking applies the rule again and again: today’s posterior, pushed through the motion model, becomes tomorrow’s prior. With a Gaussian prior, a linear motion model and a linear measurement with Gaussian noise, the posterior is again Gaussian and Equation (2.21) reduces to a few matrix operations, which is the Kalman filter of Chapter 18; Chapter 19 covers what to do when the models are not linear. The standard reference for all of this is Thrun, Burgard, and Fox [41].

2.9 Computation: complexity, priority queues and hashing

Definition 2.38 (Big-O notation). For functions $f, g: \mathbb{N} \rightarrow [0, \infty)$ we write $f(n) = \mathcal{O}(g(n))$ if there are constants $c > 0$ and n_0 such that $f(n) \leq c g(n)$ for all $n \geq n_0$. The running time of a search is expressed in the sizes of its input: the numbers of vertices $|V|$ and edges $|E|$, the branching factor b , the horizon T , the number of agents k .

Big-O hides constants, and in Python the constants are large: a loop iteration costs about a hundred nanoseconds, a NumPy operation on a whole array costs about the same per element but with a hundredfold smaller constant. Table 2.6 lists the operations that the book’s code relies on. Two of them deserve a closer look, because getting them wrong turns an $\mathcal{O}(n \log n)$ search into an $\mathcal{O}(n^2)$ one.

Definition 2.39 (Priority queue). A **priority queue** stores items with numeric keys and supports *insert*(item, key), *pop-min* (remove and return the item with the smallest key) and, optionally, *decrease-key*. A **binary heap** implements insert and pop-min in $\mathcal{O}(\log n)$ time and peeking at the minimum in $\mathcal{O}(1)$ [5]. Python’s `heapq` module is a binary heap over a list; the book stores the tuples (key, counter, item) in it, where the counter breaks ties by insertion order.

The lazy-deletion idiom. Dijkstra’s algorithm and A^* (Chapters 3 and 4) must lower the key of a vertex whenever they find a cheaper path to it, and LPA^* and D^* Lite (Chapter 5) must also remove vertices from the queue. `heapq` offers neither operation, and searching the

list for the entry would cost $\mathcal{O}(n)$. The idiom used throughout this book, called **lazy deletion**, avoids both: never touch an entry that is already in the heap; when a key improves, push a *second* entry with the better key; when popping, skip any entry whose key is no longer the item's best known key. [Algorithm 2.1](#) states the two operations and [Listing 2.3](#) shows the toolbox class LazyPQ.

Algorithm 2.1. Priority queue with lazy deletion

Data: binary heap H of triples $(key, counter, item)$; dictionary $best$ mapping items to their best pushed key; counter c

```

1 Function LazyPush( $item, key$ ):
2   if  $item \in best$  and  $best[item] \leq key$  then
3     return // no improvement: ignore
4    $best[item] \leftarrow key$ ;  $c \leftarrow c + 1$ 
5   push  $(key, c, item)$  onto  $H$  // duplicates are allowed
6 Function LazyPop():
7   while  $H$  is not empty do
8      $(key, \cdot, item) \leftarrow$  pop the minimum of  $H$ 
9     if  $best[item] = key$  then
10      delete  $best[item]$ ; return  $(key, item)$ 
11      // otherwise the entry is stale: a better key was pushed later
12   return empty
  
```

Line 4 of [Algorithm 2.1](#) records the best key of each item, which is what makes a stale entry recognisable at line 8: an entry is stale exactly when its key differs from the recorded best key (or the item has already been popped). Each push costs $\mathcal{O}(\log n)$, each pop costs $\mathcal{O}(\log n)$ plus one extra pop per stale entry it skips; since every entry is popped at most once, the total cost of a search with P pushes is $\mathcal{O}(P \log P)$. A search that relaxes every edge once pushes at most $|E|$ entries, so the heap can hold $|E|$ rather than $|V|$ entries; because $\log |E| \leq 2 \log |V|$, the asymptotic bound of the search is unchanged. Two equivalent variants appear in the literature: some codes compare the popped key with the vertex's current g instead of a separate dictionary, and some skip a popped vertex if it is already in the closed set. All three do the same thing; the toolbox version also counts pushes and stale pops so that you can watch the overhead in the exercises.

Hashing states. The dictionaries of a search (g -values, parents, the closed set, the best keys of the queue) need hashable keys. In this book a state is always a tuple of integers, such as (x, y) for a grid cell, $((x, y), t)$ for a space-time state, or (x, y, z) for a lattice cell. Tuples of integers hash in constant time, compare quickly and cost about a hundred bytes each. Continuous states, as in [Chapters 16](#) and [17](#), are *not* hashed; they live in arrays and are searched with a KD-tree. If you must key a dictionary by a continuous quantity, round it to a resolution first.

Floats and arrays as dictionary keys

`0.1 + 0.2 == 0.3` is `False` in floating point, so two states that are physically identical can hash to different keys and the closed set stops working: the search revisits states forever or until memory runs out. NumPy arrays are not hashable at all (`tuple(a)` is). Keep discrete states as tuples of integers, and if a continuous coordinate has to serve as a key, quantise it: `round(x / res)` with a fixed resolution `res`.

Table 2.7. The software stack. Only the first two rows are required to run the code of the book.

Package	Status	Used for
Python ≥ 3.10	required	all code; tuples, dictionaries, <code>heapq</code> , <code>math</code>
NumPy	required	arrays, linear algebra (<code>eigh</code> , <code>solve</code>), seeded random numbers
SciPy (<code>scipy.optimize</code>)	Part VII only	quadratic programs and <code>milp</code> in Chapters 21 and 22
Matplotlib	optional	your own plots and animations; the book’s figures are TikZ
NetworkX [14]	optional	explicit graphs, Laplacians, quick prototypes
PyBullet, gym-pybullet-drones [29]	optional	physics simulation of quadrotors for the capstone

Why dictionaries are enough. The largest instances in this book are grids of about 100×100 cells with horizons of about 100 steps, i.e. 10^6 space-time states, and swarms of a few dozen agents. A Python dictionary holding 10^6 tuple keys occupies about 150 MB and is filled in a few seconds; a search touches far fewer states than that. This is why no chapter uses a specialised data structure for its closed set, and why the code prefers clarity over speed. Where speed matters in research code, the usual remedies are to encode a state as a single integer $x + W(y + Ht)$, to store g -values in a NumPy array indexed by that integer, and eventually to move the inner loop to a compiled language.

2.10 The Python stack and the code of the book

The training plan behind this book ([Appendix A](#)) prescribes a lean stack, listed in [Table 2.7](#). Every algorithm is implemented with Python and NumPy alone; SciPy appears only for the optimisation problems of Part VII. The code is organised as follows.

- One file per chapter, `Overleaf/code/chNN_slug.py`, e.g. `ch02_toolbox.py` for this chapter and `ch04_astar.py` for A*. Each file is self-contained apart from NumPy and, where useful, an import from an earlier chapter’s file.
- Each file ends with `if __name__ == "__main__":`, which runs a self-test made of `assert` statements against hand-computed values. Run it with `python3 code/ch02_toolbox.py` from the `Overleaf` directory; it must finish in seconds and print that the test passed. The numbers quoted in the worked examples of every chapter come from these files.
- Figures whose content is computed (a trajectory, a set of samples, a benchmark) are produced by scripts `Overleaf/code/figures/gen_chNN_slug.py` that write whitespace-separated data files to `Overleaf/figures/data/`; the TikZ figure reads the data file. The scripts use fixed random seeds, so that the figures are reproducible.
- Positions in the geometry functions are plain tuples, so that they can be dictionary keys and are cheap to create; the linear-algebra functions use NumPy arrays. ASCII maps use `.` for a free cell and `#` for an obstacle, with the top row first.

[Listing 2.1](#) shows the successor function of the toolbox grid, the one function that the searches of [Chapters 3 to 6](#) call most often; note the corner-cutting test and the two edge costs of [Definition 2.5](#). [Listing 2.2](#) shows the geometry of [Propositions 2.29 and 2.31](#), and [Listing 2.3](#) the priority queue of [Algorithm 2.1](#).

Listing 2.1. The successor function of the grid (from `code/ch02_toolbox.py`).

```

1  def neighbors(self, cell):
2      """Return [(neighbour, cost)] for the free neighbours of ``cell``."""
3      x, y = cell
4      moves = MOVES4 if self.connectivity == 4 else MOVES8
5      result = []
6      for dx, dy in moves:
7          nxt = (x + dx, y + dy)
8          if not self.is_free(nxt):
9              continue
10         if dx != 0 and dy != 0:                # diagonal move
11             if not (self.is_free((x + dx, y))
12                     and self.is_free((x, y + dy))):
13                 continue                # corner cutting
14             result.append((nxt, SQRT2))
15         else:
16             result.append((nxt, 1.0))
17     return result

```

Listing 2.2. Tangent points, time of closest approach and time to collision (from code/ch02_toolbox.py).

```

1  def tangent_points(p, c, r):
2      """The two points where the tangent lines from p touch the circle
3      (c, r), or None if p lies on or inside the circle."""
4      u = sub(p, c)
5      d = norm(u)
6      if d <= r:
7          return None
8      u = scale(u, 1.0 / d)                # unit vector from c towards p
9      perp = (-u[1], u[0])                # u rotated by +90 degrees
10     cos_a = r / d                        # angle at the centre
11     sin_a = math.sqrt(1.0 - cos_a * cos_a)
12     t_left = add(c, scale(add(scale(u, cos_a), scale(perp, sin_a)), r))
13     t_right = add(c, scale(add(scale(u, cos_a), scale(perp, -sin_a)), r))
14     return t_left, t_right
15
16
17 def time_of_closest_approach(p_a, v_a, p_b, v_b, horizon=math.inf):
18     """Time t* in [0, horizon] at which two constant-velocity objects are
19     closest, and their distance at that time."""
20     p = sub(p_b, p_a)                    # relative position
21     v = sub(v_b, v_a)                    # relative velocity
22     vv = dot(v, v)
23     t = 0.0 if vv == 0.0 else -dot(p, v) / vv
24     t = max(0.0, min(horizon, t))
25     return t, norm(add(p, scale(v, t)))
26
27
28 def time_to_collision(p_a, v_a, r_a, p_b, v_b, r_b):
29     """First time t >= 0 at which two constant-velocity discs touch, or
30     None. This is a ray-circle test in the relative frame."""
31     return ray_circle_intersection(sub(p_b, p_a), sub(v_b, v_a),
32                                   (0.0, 0.0), r_a + r_b)

```

Listing 2.3. The lazy-deletion priority queue (from code/ch02_toolbox.py).

```

1  class LazyPQ:

```

```

2      """Binary-heap priority queue with lazy deletion.
3
4      ``push`` never searches the heap: a better key for a known item is
5      simply pushed as a second entry.  ``pop`` discards entries whose key
6      is no longer the item's best key (they are 'stale').  Ties are broken
7      by insertion order, so items themselves are never compared.
8      """
9
10     def __init__(self):
11         self._heap = []
12         self._best = {}                # item -> best key pushed so far
13         self._counter = itertools.count()
14         self.pushes = 0
15         self.stale_pops = 0
16
17     def push(self, key, item):
18         """Insert ``item`` with ``key``; ignored unless ``key`` improves."""
19         if item in self._best and self._best[item] <= key:
20             return False
21         self._best[item] = key
22         heapq.heappush(self._heap, (key, next(self._counter), item))
23         self.pushes += 1
24         return True
25
26     def pop(self):
27         """Remove and return (key, item) with the smallest key."""
28         while self._heap:
29             key, _, item = heapq.heappop(self._heap)
30             if self._best.get(item) == key:
31                 del self._best[item]
32                 return key, item
33             self.stale_pops += 1        # stale entry: skip it
34         raise IndexError("pop from an empty LazyPQ")

```

2.11 Where this fits in the drone system

In the drone system

Each of the four layers of the hybrid architecture in [Chapter 24](#) is built from tools of this chapter. The *global planner* searches a grid or lattice ([Definitions 2.5](#) and [2.6](#)) of inflated obstacles ([Proposition 2.9](#)) in space-time ([Definition 2.10](#)), compares plans by makespan and sum of costs ([Definitions 2.17](#) and [2.18](#)), and runs on the lazy priority queue ([Algorithm 2.1](#)). The *prediction layer* describes an intruder by a Gaussian ([Definition 2.34](#)) propagated through the double-integrator model ([Definition 2.22](#)) with Bayes' rule ([Equation \(2.21\)](#)). The *local safety layer* works in the relative frame of two discs ([Proposition 2.28](#)), decides with the time to collision ([Proposition 2.31](#)) whether an intruder is inside the safety horizon, avoids it with the tangent cone ([Proposition 2.29](#)), and respects the stopping distance ([Proposition 2.24](#)). The *replanning layer* re-searches the grid from the current space-time state with hashed states. In the twelve-week study plan of [Appendix A](#) this chapter accompanies Week 1, the foundations week, and is the reference to return to in every later week.

2.12 Summary

Chapter summary

- A planner searches a weighted graph through a successor function; grids and lattices are implicit graphs with move costs 1, $\sqrt{2}$, $\sqrt{3}$ and a corner-cutting rule (Definitions 2.1, 2.5 and 2.6).
- A disc or ball robot becomes a point when the obstacles are inflated by its radius: $\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus D(\mathbf{0}, r)$ (Proposition 2.9); a drone is such a ball, radius plus margin.
- Discrete time adds the wait action and turns the map into the time-expanded graph; a time-indexed path fixes where and when, a plan is one such path per agent, a trajectory is continuous motion (Definitions 2.12 to 2.14).
- Path length, travel time, makespan $\text{MS} = \max_i T_i$, sum of costs $\text{SoC} = \sum_i T_i$ and minimum separation are the cost measures of the book; a plan without grid conflicts can still bring physical agents too close.
- The single integrator chooses velocities, the double integrator chooses accelerations with exact discrete matrices $\mathbf{A}, \mathbf{B}$; a quadrotor is nearly the latter; the stopping distance is $v^2/(2a_{\text{max}})$.
- Distance to a segment, ray–circle intersection, the Minkowski disc, tangent lines and the time of closest approach $t^* = -(\mathbf{p} \cdot \mathbf{v})/\|\mathbf{v}\|^2$ with $d_{\text{min}} = \|\mathbf{p} \times \mathbf{v}\|/\|\mathbf{v}\|$ are the geometry of collision avoidance.
- A covariance is an ellipse with axes $\sqrt{\lambda_i} \mathbf{e}_i$ (Proposition 2.36); in 2D the 1σ ellipse holds only 39% of the mass; Bayes' rule turns a prior and a measurement into a posterior.
- Searches use a binary heap with lazy deletion (push duplicates, skip stale pops) and dictionaries keyed by integer tuples; Python and NumPy are enough for every instance in this book.

Further reading. Graphs, shortest paths, heaps and big-O notation are covered in Cormen et al. [5], and the search view of planning in Russell and Norvig [35]. Configuration space, Minkowski sums, holonomic and non-holonomic systems and motion models are the subject of LaValle [25] and Choset et al. [4]; the latter also contains the geometry of distance computations. The definitions of time-indexed paths, makespan and sum of costs follow Stern et al. [39]. The velocity-obstacle construction that motivates the tangent-line and closest-approach primitives is due to Fiorini and Shiller [11]. Gaussians, covariance ellipses and Bayes' rule in the form used by tracking are treated in Thrun, Burgard, and Fox [41]. For the quadrotor as a double integrator and differential flatness see Mellinger and Kumar [28]; for the simulation stack see Panerati et al. [29].

2.13 Exercises

Exercise 2.1 (★). Write the adjacency list of the 8-connected grid graph of the 4×3 map `["....", ".#..", "...."]` for the cells (0,0), (2,1) and (3,0), including edge costs. Then show that an empty $W \times H$ grid has $W(H-1) + H(W-1)$ undirected edges when 4-connected and $2(W-1)(H-1)$ more when 8-connected; check the totals 17 and 29 for $W = 4$, $H = 3$ against the self-test of the toolbox.

Exercise 2.2 (★). Suppose a diagonal move cost 1 instead of $\sqrt{2}$. Give a start and goal on an empty 8-connected grid for which the cheapest grid path is then not the shortest polyline through cell centres, and explain what this would do to a heuristic that estimates

the straight-line distance ([Chapter 4](#)).

Exercise 2.3 (★★). A single cell of a large grid is blocked. Using the cell-centre rule of `Grid.inflate`, determine by hand which cells are blocked after inflating by $r = 0.6$, $r = 1.0$ and $r = 1.6$, and confirm the counts 5, 9 and 21. At which values of r does the count change next? Compare the blocked area with the area $\pi(r + \frac{1}{2})^2$ of the exact Minkowski sum of a square and a disc for large r .

Exercise 2.4 (★★). Let $G = (V, E)$ be undirected. Show that its time-expanded graph with horizon T has $|V|(T + 1)$ vertices and $T(|V| + 2|E|)$ edges, that it is acyclic, and that its paths from $(s, 0)$ to (g, T) correspond one-to-one to the time-indexed paths of [Definition 2.12](#) of length T .

Exercise 2.5 (★★). Compute the makespan and the sum of costs of the plan in [Example 2.20](#). Then construct an instance with three agents on a corridor in which the plan with the smallest makespan does not have the smallest sum of costs, and vice versa, and prove that no plan is optimal for both objectives.

Exercise 2.6 (★★). A drone flies at $v = 2$ m/s with $a_{\max} = 1$ m/s² and $\Delta t = 0.1$ s. How many steps of full braking does it need, and how far does it travel according to the exact model [Equation \(2.9\)](#)? Repeat with the forward-Euler update $\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k$, $\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t \mathbf{a}_k$ and explain the difference. Finally, write the rule “a velocity v is admissible if the drone can still stop before the nearest obstacle at distance d ” as an inequality; this rule returns in [Chapter 14](#).

Exercise 2.7 (★★). Derive [Equation \(2.14\)](#) by minimising $D(t)^2$, show that $d_{\min} = |\mathbf{p} \times \mathbf{v}| / \|\mathbf{v}\|$ when the minimum is not clamped, and derive the time to collision [Equation \(2.15\)](#). Evaluate all three for $\mathbf{p} = (4, 2)$, $\mathbf{v} = (-1, -1)$ and collision distances $R = 1.5$ and $R = 1$.

Exercise 2.8 (★★). For $\mathbf{p} = (0, 0)$, $\mathbf{c} = (4, 0)$ and $r = 2$ compute the tangent points, the tangent length and the half-angle θ of the cone from [Proposition 2.29](#). Show in general that $\theta = \arcsin(r/d)$, and describe what happens to the cone as $d \rightarrow r^+$ and as $d \rightarrow \infty$. Why does this matter for a drone that is close to another drone?

Exercise 2.9 (★★). For $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$ compute the eigenvalues, the eigenvectors and the orientation of the major axis by hand, and give the semi-axes of the 1σ and 2σ ellipses. Then compute the probability mass inside each ellipse and compare it with the one-dimensional 68% and 95% figures.

Exercise 2.10 (★★★ Coding). Extend the toolbox to three dimensions and measure the priority queue. (a) Write a class `Grid3D` with 6- and 26-connectivity, the move costs of [Definition 2.6](#), the box version of the corner-cutting rule and an `inflate` method that samples the Minkowski sum with a ball. (b) Implement `segment_sphere_intersects` and the 3D versions of `time_of_closest_approach` and `time_to_collision`, and test them with `asserts` against values you compute by hand, in the style of the self-test of `ch02_toolbox.py`. (c) Using `space_time_successors` and `LazyPQ`, write a uniform-cost search over space-time states on a 2D grid ([Chapter 3](#) explains why it is correct), run it on a 50×50 map with random obstacles, and report the number of pushes and stale pops; explain why the stale pops never exceed the pushes.

Part II

Single-Agent Graph Search

Dijkstra's Algorithm

Learning objectives

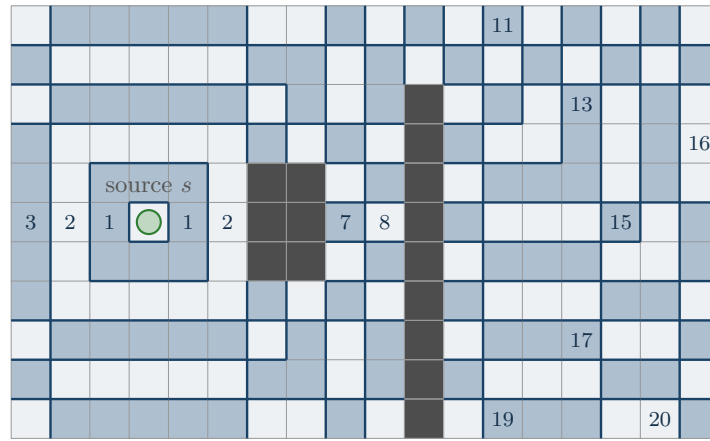
- Explain what Dijkstra's algorithm computes (the cheapest cost from one source to every vertex of a graph with non-negative edge costs) and when it is the right tool.
- Trace it by hand on a small weighted graph and on a grid, keeping track of the priority queue, the settled set and the parent pointers.
- Prove the settled-node invariant, explain why it needs non-negative edge costs, and derive the running time $\mathcal{O}((|V| + |E|) \log |V|)$.
- Implement it in Python with `heapq` and lazy deletion, on explicit graphs and on occupancy grids, with the early exit (uniform-cost search) and path reconstruction.
- Use one backward run from the goal as an exact heuristic, the table that feeds A^* (Chapter 4) and the single-agent searches inside the multi-agent solvers of Chapters 8 and 9.

3.1 Why this matters for a drone swarm

Picture one delivery drone at a depot on the edge of a city. Its map is the occupancy grid of Chapter 2: cells are free or blocked, and moving into a cell costs one unit of time, or more when the cell lies in a wind-exposed corridor or above a schoolyard where the operator has attached a risk penalty. No-fly zones are simply removed from the grid. The customer is twenty blocks away. Before the drone lifts off, the planner has to answer one question: which route is the cheapest, and how do we know that no cheaper route exists?

This is the **single-source shortest-path problem** on a graph with non-negative edge costs, and Dijkstra's algorithm [7] solves it exactly. It is the oldest algorithm in this book, and the one you will use most often, for three reasons.

1. *It is the baseline planner.* Every other single-agent planner in Chapters 4 to 6 returns a path whose cost is judged against the cost that Dijkstra's algorithm would have found. When the experiments of Chapter 25 report a path length "relative to the optimum", the optimum comes from this chapter.
2. *It is the heuristic factory.* A^* (Chapter 4) needs, for every vertex, a lower bound on the cost that remains to the goal. One run of Dijkstra's algorithm *backwards* from the goal produces the exact cost-to-go for every vertex of the map. That table is the perfect heuristic for A^* , and it is the heuristic that the single-agent searches inside prioritized planning (Chapter 8) and conflict-based search (Chapter 9) use when they route one drone at a time through space and time.
3. *It is the ancestor.* A^* is Dijkstra's algorithm with a heuristic added to the priority;



numbers: $\lfloor g \rfloor$, the wave settles band k before band $k+1$

Figure 3.1. Dijkstra's algorithm as a spreading wave on an 8-connected grid (diagonal moves cost $\sqrt{2}$). Cells are shaded by the band $\lfloor g \rfloor$ of their cost from the source; the wave settles every cell of band k before any cell of band $k+1$, bends around the block and passes through the gap at the top of the wall. Behind the wall the bands are centred on the gap, not on the source: cost is measured along paths, not as the crow flies.

uniform-cost search is Dijkstra's algorithm with an early exit; D^* Lite ([Chapter 5](#)) is a backward search from the goal that repairs its distance table when the map changes. Understand this chapter thoroughly and the next three chapters become variations on one theme.

In the four-layer hybrid architecture of [Chapter 24](#), Dijkstra's algorithm therefore lives in the global planning layer: it plans a single drone on a static map, and it supplies the distance tables that the multi-agent solver and the replanning layer consume. It knows nothing about other drones or about moving intruders; those are the subjects of Parts III and IV.

Roadmap. [Section 3.2](#) gives the picture of a wave spreading over the map. [Section 3.3](#) fixes the notation, [Section 3.4](#) states the algorithm with a binary heap and lazy deletion, and [Section 3.5](#) traces it on a seven-vertex graph and on a 5×5 grid. [Section 3.6](#) proves that the algorithm is correct, shows what goes wrong with a negative edge, and derives the running time. [Section 3.7](#) collects the variants you will meet later in the book, above all the backward run that produces exact heuristics. [Section 3.8](#) discusses the Python implementation and its pitfalls, and [Section 3.9](#) places the algorithm in the drone system.

3.2 The idea in plain words

Drop a stone into a pond. The ripple spreads at constant speed, reaches nearby points before distant ones, bends around a rock and squeezes through a gap in a wall. If you write down the moment at which the ripple first reaches each point, you have written down the distance from the stone to every point of the pond, because the ripple always arrives along a shortest route. [Figure 3.1](#) shows this picture on a grid map. The bands are the cells whose cost from the source lies between k and $k+1$, and band k is completely settled before the wave enters band $k+1$.

Dijkstra's algorithm simulates this wave on a graph, one vertex at a time. Every vertex is in one of three states.

- *Unseen*: no path to it has been found; its tentative cost is $g(v) = \infty$.

- *Open*: at least one path to it is known, with tentative cost $g(v)$, but a cheaper path may still turn up. The open vertices form the frontier of the wave and are stored in a priority queue keyed by g .
- *Settled* (or *closed*): the wave has reached it, and $g(v)$ is final.

The loop is the same at every step: take the open vertex with the smallest tentative cost, declare it settled, and offer each of its neighbours the path that goes through it. If that path is cheaper than the neighbour's current tentative cost, update the cost and the parent pointer of the neighbour and put it on the frontier. This update is called **relaxation**.

Key idea

Always settle the open vertex with the smallest tentative cost. Because no edge has a negative cost, every path that is still to be discovered starts from a vertex that is at least as expensive, so nothing found later can undercut it: the cost of a vertex is final the moment it is settled.

Two consequences follow, and both are worth fixing in your mind now. First, vertices are settled in non-decreasing order of cost; that is the wave. Second, if you only want the cost of one target t , you may stop the moment t is settled: everything settled before t was at least as cheap, everything still open is at least as expensive, and the answer cannot change any more. With this early exit the algorithm is what the artificial-intelligence literature calls **uniform-cost search** [35].

3.3 Problem statement and notation

We use the graphs of Chapter 2: an explicit graph is a set of vertices with adjacency lists, and a grid is an implicit graph whose neighbours are generated on demand by a successor function.

Definition 3.1 (Weighted graph, path, path cost). A **weighted directed graph** is a triple $G = (V, E, c)$ with a finite vertex set V , an edge set $E \subseteq V \times V$ and a cost function $c: E \rightarrow \mathbb{R}_{\geq 0}$. We write $c(u, v)$ for the cost of the edge (u, v) and $\text{Succ}(u) = \{v : (u, v) \in E\}$ for the successors of u . An undirected edge $\{u, v\}$ of cost w stands for the two directed edges (u, v) and (v, u) , both of cost w . A **path** from u to v is a sequence $\pi = (v_0, v_1, \dots, v_k)$ with $v_0 = u$, $v_k = v$ and $(v_{i-1}, v_i) \in E$ for $i = 1, \dots, k$; its cost is $c(\pi) = \sum_{i=1}^k c(v_{i-1}, v_i)$.

Definition 3.2 (Shortest-path distance). The **shortest-path distance** $\delta(u, v)$ is the smallest cost of any path from u to v , and $\delta(u, v) = \infty$ if no such path exists. A path with $c(\pi) = \delta(u, v)$ is a **shortest path**; there may be several.

Three facts about δ are used throughout the chapter. For all $u, v, w \in V$,

$$\delta(u, u) = 0, \quad \delta(u, w) \leq \delta(u, v) + \delta(v, w), \quad \delta(u, v) \leq \delta(u, w) + c(w, v) \text{ if } (w, v) \in E. \quad (3.1)$$

The second is the triangle inequality: going via v can only be a detour. The third is the triangle inequality with a single edge as the second leg. A fourth fact is that *subpaths of shortest paths are shortest paths*: if π is a shortest path from u to v and w lies on π , then the prefix of π up to w is a shortest path from u to w , because a cheaper prefix would give a cheaper π .

Definition 3.3 (Single-source shortest-path problem). Given $G = (V, E, c)$ with $c \geq 0$ and a source $s \in V$, compute $\delta(s, v)$ for every $v \in V$ together with a **shortest-path tree**: a parent pointer $\text{parent}(v)$ for every reachable $v \neq s$ such that following the pointers from v back to s traces a shortest path. In the **single-pair** variant a target t is also given, and only $\delta(s, t)$ and one shortest path from s to t are required.

During the search every vertex carries a **tentative cost** $g(v)$, the cost of the best path to v found so far, so that $g(v) \geq \delta(s, v)$ at all times. The open vertices form the priority queue OPEN and the settled vertices the set CLOSED, exactly as in the best-first skeleton of Chapter 2. In later chapters g is joined by a heuristic h ; in this chapter $h = 0$ and the queue is ordered by g alone.

3.4 The algorithm

Algorithm 3.1 is Dijkstra's algorithm in the form that every search algorithm of this book reuses. A binary min-heap holds pairs (g, v) , and instead of decreasing the key of a vertex that is already in the heap, we push a second entry and ignore the older one when it surfaces. This is the **lazy deletion** idiom of Chapter 2. An optional target t turns the algorithm into uniform-cost search.

Algorithm 3.1. Dijkstra's algorithm with a binary heap and lazy deletion. Without a target it computes $\delta(s, v)$ for every vertex; with a target t it stops as soon as t is settled (uniform-cost search).

Input: graph $G = (V, E, c)$ with $c \geq 0$, given by a successor function Succ; source s ; optional target t

Output: tentative costs g , exact for every settled vertex; parent pointers

```

1 Function Dijkstra( $G, s, t$ ):
2    $g(v) \leftarrow \infty$  for all  $v \in V$ ;  $g(s) \leftarrow 0$ ; parent( $s$ )  $\leftarrow$  nil
3   OPEN  $\leftarrow$  min-heap holding the single entry  $(0, s)$ ; CLOSED  $\leftarrow \emptyset$ 
4   while OPEN  $\neq \emptyset$  do
5      $(g, u) \leftarrow$  pop the entry with the smallest  $g$  from OPEN
6     if  $u \in \text{CLOSED}$  then
7       continue // stale entry:  $u$  was settled earlier with a smaller key
8     add  $u$  to CLOSED // settle  $u$ : from now on  $g(u) = \delta(s, u)$ 
9     if  $u = t$  then
10      break // early exit: uniform-cost search
11    foreach  $v \in \text{Succ}(u)$  with  $v \notin \text{CLOSED}$  do
12      if  $g + c(u, v) < g(v)$  then
13         $g(v) \leftarrow g + c(u, v)$ ; parent( $v$ )  $\leftarrow u$  // relaxation
14        push  $(g(v), v)$  onto OPEN // lazy insertion, no decrease-key
15  return  $g$ , parent

```

Walkthrough. Lines 2–3 initialise: only the source has a finite cost and only the source is on the frontier. In an implementation the assignment $g(v) \leftarrow \infty$ for all v is never executed; a dictionary that answers ∞ for a missing key does the same job without touching V , which matters when V is a large grid or the space-time graph of Chapter 8.

Line 5 removes the entry with the smallest key; with a binary heap this costs $\mathcal{O}(\log |\text{OPEN}|)$. Line 6 is the lazy-deletion test. A vertex that was pushed several times sits in the heap several times, and every entry of it after the first one to be popped is **stale**: its key is at least the key with which the vertex was settled, and it is simply discarded. Line 8 settles u ; this is the moment at which Theorem 3.7 guarantees $g(u) = \delta(s, u)$.

Line 9 is the early exit. Note where it sits: the target is tested when it is *popped*, not when it is first discovered on line 13. A target that has just been discovered may still be reached

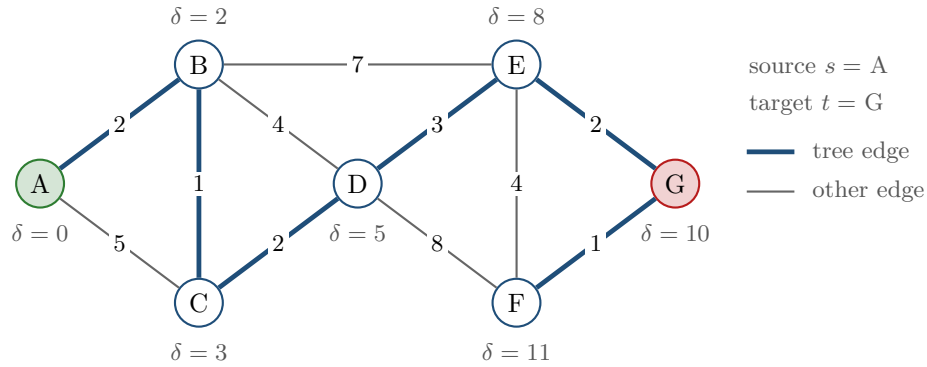


Figure 3.2. The waypoint graph of [Example 3.4](#). Edge labels are costs; after the search the thick blue edges form the shortest-path tree from A , and the grey numbers are the final distances $\delta(A, v)$. Notice that the direct edge $A-C$ of cost 5 is not in the tree, because the detour through B costs only 3, and that F is reached through the target G .

more cheaply through a vertex that is waiting in the queue; a target that has been popped cannot. [Section 3.5](#) shows both cases on a concrete graph, and [Corollary 3.8](#) states exactly which vertices the early exit settles.

Lines 11–14 *expand u* : they generate its successors and relax every edge. The strict inequality on line 12 keeps the first-found path when two paths cost the same; this is a tie-breaking rule, and [Section 3.8](#) discusses its consequences. Skipping settled successors on line 11 is an optimisation, not a necessity: with $c \geq 0$ the test on line 12 would fail for them anyway. The word **expansion** names the unit in which we count the work of a search: one expansion is one non-stale pop together with the generation of its successors. The number of expansions is what [Figure 3.5](#) and the comparisons of [Chapter 4](#) measure.

The result. After the loop, $g(v) = \delta(s, v)$ for every settled vertex, and $g(v) = \infty$ for every vertex the wave never reached because there is no path from s . Without a target every reachable vertex is settled. With a target, only the vertices with $\delta(s, v) \leq \delta(s, t)$ have been settled, and the tentative costs of the vertices left in the queue are upper bounds, not answers. The shortest path to a settled vertex v is read off the parent pointers: start at v , move to $\text{parent}(v)$, and repeat until the source is reached; reversing the list gives the path from s to v (**path reconstruction**). If t is unreachable, the queue runs empty before t is popped and the search reports failure, which for a drone means that the goal is sealed off by obstacles or no-fly zones and the map, not the planner, has to change.

3.5 A worked example

Two examples follow: an explicit graph small enough to trace on paper with every state of the heap written out, and a grid on which the wave of [Section 3.2](#) becomes visible. Both live in `code/ch03_dijkstra.py`, whose self-test checks every number quoted here.

Example 3.4 (Seven waypoints). [Figure 3.2](#) shows a waypoint graph with seven vertices $A, \dots, G$ and eleven undirected edges whose costs are flight times in minutes. The source is A and the target is G . We first run [Algorithm 3.1](#) from A *without* a target, so that the whole shortest-path tree appears, and then look at what the early exit would have changed. The graph is `EXAMPLE_GRAPH` in the chapter code; `python3 ch03_dijkstra.py` prints the trace of [Table 3.1](#).

Table 3.1. Trace of Algorithm 3.1 on Example 3.4 from A without a target. Each row is one pop from the heap. “Push $(3, C)$ (improves 5)” means that the tentative cost of C drops from 5 to 3 and the entry $(3, C)$ is pushed; the old entry $(5, C)$ stays in the heap and is discarded as stale when it surfaces. The last column is the heap after the row, sorted by key. Produced by `dijkstra_trace`.

Step	Popped	Action	OPEN after the step
1	$(0, A)$	settle A ; push $(2, B)$, $(5, C)$	$(2, B)$ $(5, C)$
2	$(2, B)$	settle B ; push $(3, C)$ (improves 5), $(6, D)$, $(9, E)$	$(3, C)$ $(5, C)$ $(6, D)$ $(9, E)$
3	$(3, C)$	settle C ; push $(5, D)$ (improves 6)	$(5, C)$ $(5, D)$ $(6, D)$ $(9, E)$
4	$(5, C)$	stale, C is already settled: discarded	$(5, D)$ $(6, D)$ $(9, E)$
5	$(5, D)$	settle D ; push $(8, E)$ (improves 9), $(13, F)$	$(6, D)$ $(8, E)$ $(9, E)$ $(13, F)$
6	$(6, D)$	stale: discarded	$(8, E)$ $(9, E)$ $(13, F)$
7	$(8, E)$	settle E ; push $(12, F)$ (improves 13), $(10, G)$	$(9, E)$ $(10, G)$ $(12, F)$ $(13, F)$
8	$(9, E)$	stale: discarded	$(10, G)$ $(12, F)$ $(13, F)$
9	$(10, G)$	settle G , the target (an early exit stops here); push $(11, F)$ (improves 12)	$(11, F)$ $(12, F)$ $(13, F)$
10	$(11, F)$	settle F ; nothing improves	$(12, F)$ $(13, F)$
11	$(12, F)$	stale: discarded	$(13, F)$
12	$(13, F)$	stale: discarded	empty

Reading the trace. Step 1 settles the source and discovers its two neighbours. Step 2 is the first improvement: C was discovered from A at cost 5, but through B it costs $2 + 1 = 3$, so $(3, C)$ is pushed while $(5, C)$ stays in the heap; step 4 pops the old entry and discards it, lazy deletion at work. The same pattern repeats for D (steps 3–6) and for E (steps 5–8). Vertex F is the most instructive. It is discovered at cost 13 through D , improved to 12 through E and to 11 through G , and settled only at step 10 with $\delta(A, F) = 11$ along the six-edge path A, B, C, D, E, G, F , which beats both the direct edge $D-F$ ($5 + 8 = 13$) and the edge $E-F$ ($8 + 4 = 12$). The settling order A, B, C, D, E, G, F has costs 0, 2, 3, 5, 8, 10, 11: non-decreasing, as the wave picture promised.

Counting. There are twelve pops, seven of which settle a vertex and five of which are stale, and twelve pushes: one per vertex and one per improvement. The heap never holds more than four entries. Every stale entry is the trace of one improvement, so without an early exit the number of stale pops equals the number of improvements; Theorem 3.10 turns this observation into a bound.

The early exit. With the target G the search stops at step 9, after nine pops. The parent pointers give $\text{parent}(G) = E$, $\text{parent}(E) = D$, $\text{parent}(D) = C$, $\text{parent}(C) = B$ and $\text{parent}(B) = A$; reversed, that is the path A, B, C, D, E, G of cost 10. Every vertex settled before G carries its final cost. But F is still in the heap with the tentative cost 12, not the true 11: the improvement through G is made only when G is expanded, and the early exit skips that expansion. `dijkstra(EXAMPLE_GRAPH, 'A', target='G')` indeed returns `dist['F'] == 12`. The tentative cost of an unsettled vertex is an upper bound and nothing more.

Example 3.5 (A 5×5 grid with slow cells). Figure 3.3 shows a 4-connected 5×5 grid. Cells are named (r, c) by row and column, counted from the top-left corner; the start S is $(0, 0)$ and the target T is $(4, 4)$. Two cells are blocked, the centre $(2, 2)$ and $(2, 4)$ on the

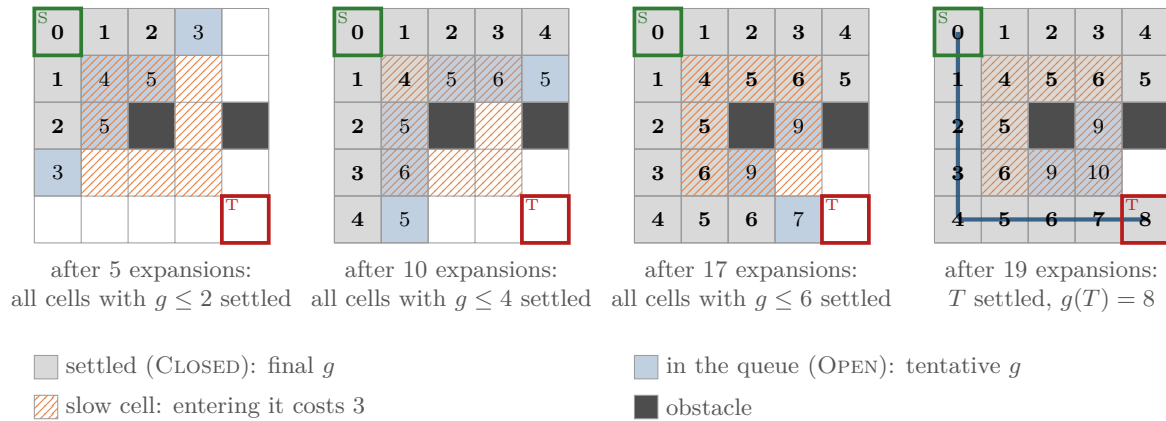


Figure 3.3. The closed set growing on the grid of [Example 3.5](#): snapshots after 5, 10, 17 and 19 expansions of a run with target T . Grey cells are settled and carry their final cost, blue cells are in the heap with their tentative cost. At every moment the settled cells are exactly the cells whose cost is at most the last popped key: this is the wave. It runs along the free border before it seeps into the slow ring, and it stops when T is popped with $g(T) = 8$; the blue line is the path read off the parent pointers.

right edge. The eight hatched cells around the centre are *slow*: entering one of them costs 3 instead of 1, the kind of penalty an operator attaches to a wind-exposed corridor. Leaving a slow cell costs 1, so the grid graph is directed. The instance is `EXAMPLE_GRID` in the chapter code, with 23 free cells. We run [Algorithm 3.1](#) from S with the target T .

Following the wave. The first pop settles S and discovers $(0,1)$ and $(1,0)$ at cost 1. Ties are broken by comparing the cells themselves, so $(0,1)$ is settled before $(1,0)$; it discovers $(0,2)$ at cost 2 and the slow cell $(1,1)$ at cost $1 + 3 = 4$. After five expansions (first panel) every cell with $g \leq 2$ is settled and the heap holds $(0,3)$ and $(3,0)$ at cost 3, $(1,1)$ at 4, and $(1,2)$ and $(2,1)$ at 5. The wave keeps running along the free top row and left column, where every step costs 1, and enters the slow ring only when nothing cheaper is left: $(1,1)$ is settled ninth, at cost 4, after $(0,4)$ and before $(4,0)$, which have the same cost. After ten expansions all cells with $g \leq 4$ are settled (second panel), after seventeen all cells with $g \leq 6$ (third panel), and the nineteenth pop settles T with $g(T) = 8$ (fourth panel). The path read off the parent pointers runs down the left column and along the bottom row, eight moves of cost 1; the route along the top row is blocked by $(2,4)$, and every route through the slow ring enters at least two slow cells and costs 12 or more. The full settling order, with costs, is

$(0,0): 0$, $(0,1): 1$, $(1,0): 1$, $(0,2): 2$, $(2,0): 2$, $(0,3): 3$, $(3,0): 3$, $(0,4): 4$, $(1,1): 4$,
 $(4,0): 4$, $(1,2): 5$, $(1,4): 5$, $(2,1): 5$, $(4,1): 5$, $(1,3): 6$, $(3,1): 6$, $(4,2): 6$, $(4,3): 7$, $(4,4): 8$.

Nineteen of the 23 free cells are settled when the search stops. Three cells, $(2,3)$, $(3,2)$ and $(3,3)$, are in the heap with the tentative costs 9, 9 and 10; a full run without a target settles them at exactly these values, because no cheaper route to them exists, but the search cannot know that when it stops. One cell, $(3,4)$, has not been discovered at all: its only free neighbours are $(3,3)$, which is not expanded, and T , which is popped but, because of the early exit, never expanded. This instance produces no stale entries, since every cell keeps the cost of its first discovery; [Exercise 3.5](#) shows that 8-connectivity changes that.

3.6 Properties: correctness, optimality, complexity

Throughout this section $G = (V, E, c)$ is a finite graph with $c \geq 0$, s is the source, and g , OPEN and CLOSED refer to [Algorithm 3.1](#). Two facts hold at every moment of a run and are used without further comment. First, $g(v) \geq \delta(s, v)$ for every v , because $g(v)$ is always the cost of an actual path from s to v , the one recorded by the parent pointers: line 13 only ever stores the cost of a path to u extended by one edge. Second, $g(v)$ never increases, because line 12 allows only strict improvements.

Lemma 3.6 (Monotone settling order). *The keys of the entries popped on line 5 form a non-decreasing sequence. Moreover, when an entry (g, u) is popped and u is not yet settled, g equals the current tentative cost $g(u)$. Hence vertices are settled in non-decreasing order of the cost they carry at the moment of settling.*

Proof. Let (g, u) be popped. Every entry still in the heap has a key of at least g , because the heap returned the minimum. Every entry pushed while u is expanded has the key $g + c(u, v) \geq g$, because $c \geq 0$. So the next pop has a key of at least g , and by induction so do all later pops. For the second claim, suppose (g, u) is popped, u is not settled, and $g(u) < g$. Then $g(u)$ was lowered after (g, u) was pushed, and at that moment an entry $(g(u), u)$ with a smaller key was pushed. That entry is still in the heap, since an entry of u leaves the heap only by being popped, which either settles u or requires u to be settled already. A heap never returns (g, u) while it holds a smaller key, a contradiction. $\square$

The lemma is the wave: the radius of the settled region, measured by the popped key, never shrinks. Note where $c \geq 0$ entered. With a negative edge, an entry pushed during an expansion could have a smaller key than the entry just popped, and the popped keys would no longer be sorted.

Theorem 3.7 (Settled-node invariant). *When [Algorithm 3.1](#) settles a vertex u on line 8, $g(u) = \delta(s, u)$, and from then on $g(u)$ does not change. Consequently:*

1. *Without a target, the algorithm settles every vertex reachable from s , returns $g(v) = \delta(s, v)$ for all $v \in V$ (with $g(v) = \infty$ exactly for the unreachable vertices), and its parent pointers form a shortest-path tree.*
2. *With a target t , the algorithm returns $g(t) = \delta(s, t)$ and a shortest path from s to t if t is reachable, and ends with an empty heap otherwise.*

Proof. Suppose the claim fails, and let u be the first vertex settled with $g(u) > \delta(s, u)$. Then $u \neq s$, because s is settled first with $g(s) = 0 = \delta(s, s)$, and $\delta(s, u) < \infty$, so a shortest path π from s to u exists. Look at the moment just before u is settled. The first vertex of π , namely s , is settled and the last, u , is not, so π has a first unsettled vertex y ; let x be its predecessor on π (x may be s). We collect three facts.

- (i) $g(x) = \delta(s, x)$, because x was settled before u and u is the first violation.
- (ii) $g(y) = \delta(s, y)$, and the heap holds the entry $(g(y), y)$. When x was settled it was expanded, and the edge (x, y) was relaxed on line 12: afterwards $g(y) \leq g(x) + c(x, y) = \delta(s, x) + c(x, y) = \delta(s, y)$, the last step because the prefix of π up to y is a shortest path to y (subpaths of shortest paths are shortest paths, [Section 3.3](#)). Together with $g(y) \geq \delta(s, y)$ this gives equality. The push that established this value put $(\delta(s, y), y)$ into the heap, and since y is not settled, the entry is still there.
- (iii) $\delta(s, y) \leq \delta(s, u)$, because y precedes u on π and the edges between them cost at least 0. *This is the only place where $c \geq 0$ is used.*

Now u is being settled because the heap returned $(g(u), u)$ as its minimum, so every key in the heap is at least $g(u)$. But by (ii) and (iii) the heap holds the key $\delta(s, y) \leq \delta(s, u) < g(u)$.

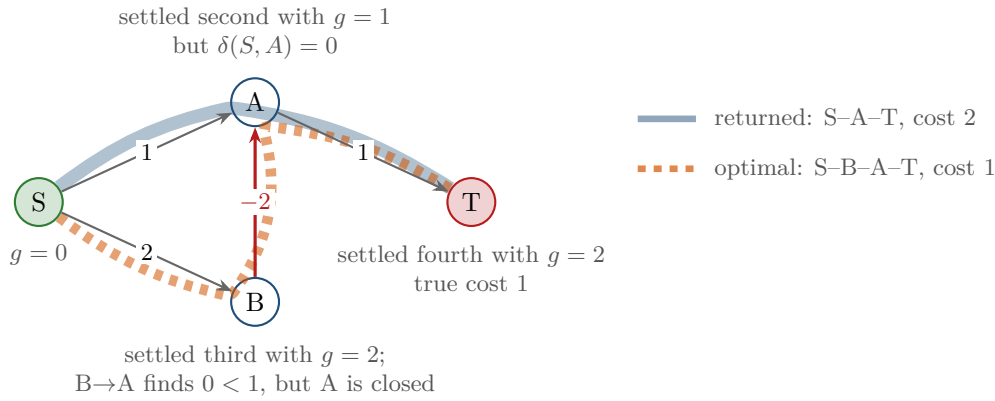


Figure 3.4. The graph of Example 3.9. Vertex A is settled with $g = 1$ before B reveals the route S, B, A of cost 0; a settled value is never revised, so the search returns S, A, T (cost 2) instead of the optimal S, B, A, T (cost 1). Nothing warns the caller.

This contradiction shows that no violation exists. That $g(u)$ does not change after settling follows: it cannot increase, and it cannot decrease below $\delta(s, u)$.

For consequence 1, let v be reachable and let π be a shortest path to v . By induction along π , every vertex of π is settled: the source is, and when a vertex of π is expanded, its successor on π is pushed, or already has a finite cost and an entry in the heap, and is therefore popped before the heap runs empty. Unreachable vertices are never pushed. For the tree: when u is settled, $\text{parent}(u)$ is the vertex x whose expansion produced $g(u) = g(x) + c(x, u)$; x was settled, so $g(x) = \delta(s, x)$ and $\delta(s, u) = \delta(s, x) + c(x, u)$: the edge (x, u) lies on a shortest path, and following the pointers back to s traces one. Consequence 2 is the invariant applied to t : if t is reachable it is pushed and settled by the argument above, and if it is not, it is never pushed, so the loop ends with an empty heap and never breaks on line 9. $\square$

Corollary 3.8 (Uniform-cost search). *With a target t , Algorithm 3.1 settles only vertices v with $\delta(s, v) \leq \delta(s, t)$, and it settles every vertex with $\delta(s, v) < \delta(s, t)$. The tentative costs of the vertices left in the heap are upper bounds on their distances and may be strictly larger.*

Proof. By Lemma 3.6 every vertex settled before t has a key of at most $g(t)$, and by Theorem 3.7 these keys are the true distances. The second claim and the remaining details are Exercise 3.3; Example 3.4 shows the upper bound $g(F) = 12 > 11$ at the exit. $\square$

Why the costs must be non-negative. The proof used $c \geq 0$ exactly once, in step (iii), and a single negative edge is enough to break the algorithm.

Example 3.9 (A negative edge). Figure 3.4 shows a directed graph with four vertices and one negative edge, $B \rightarrow A$ of cost -2 . Run Algorithm 3.1 from S . The first pop settles S and pushes $(1, A)$ and $(2, B)$. The second pop settles A with $g(A) = 1$ and pushes $(2, T)$. The third pop settles B with cost 2 and looks at the edge $B \rightarrow A$: the new cost $2 + (-2) = 0$ would improve A , but A is settled, so line 11 skips it (and had it not, the successor T of A would still keep the cost 2 it inherited from the wrong value). The fourth pop settles T with $g(T) = 2$ and the path S, A, T . The true distances are $\delta(S, A) = 0$ and $\delta(S, T) = 1$, along S, B, A, T . In the proof, the first violation is $u = A$: the shortest path to A is S, B, A , its first unsettled vertex is $y = B$ with $g(B) = \delta(S, B) = 2$, and step (iii) would need $\delta(S, B) \leq \delta(S, A)$, that is $2 \leq 0$, which is false precisely because $c(B, A) < 0$. The chapter code reproduces this run as `NEGATIVE_EXAMPLE: dijkstra` returns 2 for T , and the Bellman–Ford reference returns 1.

The failure is silent, which is what makes it dangerous. On graphs with negative edges but no negative cycles the algorithm of choice is Bellman–Ford, which relaxes every edge $|V| - 1$

times and runs in $\mathcal{O}(|V| |E|)$ [5]; the chapter code uses it only as a reference in the self-test, because in planning negative costs are best avoided altogether (see the pitfalls in [Section 3.8](#)).

Theorem 3.10 (Running time and memory). *On a graph with $|V|$ vertices and $|E|$ directed edges, [Algorithm 3.1](#) with a binary heap and lazy deletion performs at most $|V|$ expansions and at most $|E| + 1$ pushes and pops, runs in $\mathcal{O}((|V| + |E|) \log |V|)$ time, and uses $\mathcal{O}(|V| + |E|)$ memory beyond the graph.*

Proof. A vertex is settled at most once, because line 6 discards every later entry, so the loop of line 11 runs at most once per vertex and relaxes each directed edge at most once. Each relaxation pushes at most one entry, so the heap receives at most $|E| + 1$ entries including the initial one, and every pop removes one, so there are at most $|E| + 1$ pops. The heap never holds more than $|E| + 1$ entries, and $|E| \leq |V|^2$, so each push or pop costs $\mathcal{O}(\log(|E| + 1)) = \mathcal{O}(\log |V|)$. Generating successors costs $\mathcal{O}(|V| + |E|)$ in total, and each dictionary lookup or update costs $\mathcal{O}(1)$ expected time. Adding up gives $\mathcal{O}(|V| + |E| + (|E| + 1) \log |V|) \subseteq \mathcal{O}((|V| + |E|) \log |V|)$. The memory is the heap, at most $|E| + 1$ entries, plus the dictionaries g , parent and CLOSED of at most $|V|$ entries each. $\square$

Reading the bound. On a grid every cell has at most 8 neighbours, so $|E| \leq 8|V|$ and the time is $\mathcal{O}(|V| \log |V|)$, almost linear in the number of cells. The classic version with a decrease-key operation keeps at most $|V|$ entries in the heap but performs the same $|E|$ key updates at $\mathcal{O}(\log |V|)$ each, so its bound is the same and its heap operations are more complex. Two refinements are worth knowing.

- *Fibonacci heaps* [13] make decrease-key cost $\mathcal{O}(1)$ amortised and bring the time to $\mathcal{O}(|E| + |V| \log |V|)$. The gain is real only when $|E| \gg |V|$; on sparse graphs such as grids the binary heap with lazy deletion is faster in practice, and it is the version used throughout this book.
- *Dense graphs.* When $|E| = \Theta(|V|^2)$, for instance on a complete visibility graph between waypoints, the heap bound becomes $\mathcal{O}(|V|^2 \log |V|)$, and the original formulation of Dijkstra [7], which keeps the tentative costs in an array and finds the minimum by scanning it in $\mathcal{O}(|V|)$ time per expansion, is better: $\mathcal{O}(|V|^2)$ in total.

Expansions in practice. The bound counts every vertex, but the work of a run is really the area of the wave. [Figure 3.5](#) measures it on random $n \times n$ grids with 25 % obstacles, 8-connected, with the source in the top-left corner, averaged over ten seeded instances per size (`code/figures/gen_ch03_expansions.py`). Without a target the wave settles every reachable cell, about 99 % of the free cells (7 438 of 7 499 at $n = 100$; the rest are pockets sealed off by obstacles), so the count grows with the area of the map: four times as many cells from $n = 50$ to $n = 100$. With the early exit and the target in the far corner the saving is nil, 7 437 settled cells instead of 7 438, because the far corner is among the last cells the wave reaches, at a cost of 165.5 on average. With the target at the centre of the map, at a cost of 85.3, the wave stops at about half the radius and settles 3 020 cells, 41 % of the full run. The early exit therefore helps exactly when the target is close, and the only way to do better when it is not is to steer the wave towards the target, which is what the heuristic of A^* does in [Chapter 4](#).

3.7 Variants and extensions

The invariant of [Theorem 3.7](#) survives a number of changes to [Algorithm 3.1](#), as long as they alter only where the wave starts, on which graph it runs, or when it stops. [Table 3.2](#) lists the variants used in this book; the rest of the section explains them.

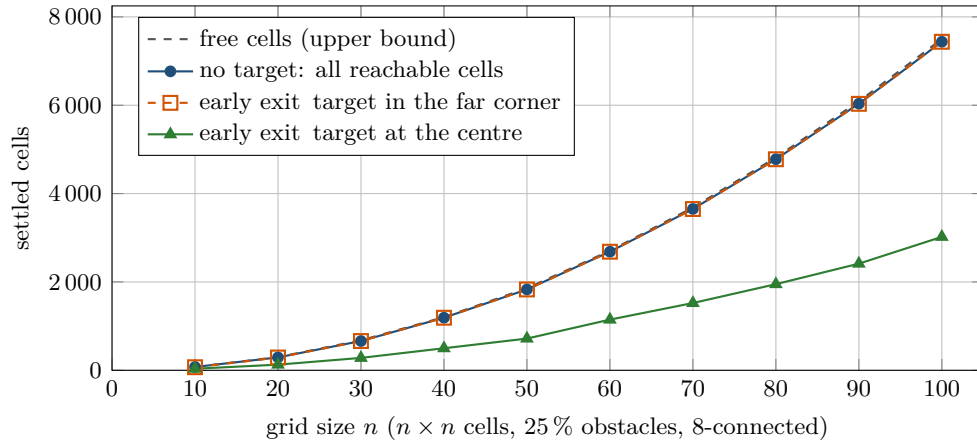


Figure 3.5. Settled cells against grid size on random 8-connected grids with 25 % obstacles, mean of ten seeded instances per size; the source is the top-left cell. Without a target almost every free cell is settled. The early exit saves nothing when the target is the far corner, the last region the wave reaches, and about 60 % of the expansions when the target is the centre of the map, whose distance from the source is about half that of the far corner.

Table 3.2. Variants of Dijkstra's algorithm and what they deliver. G^R is the graph with every edge reversed, and μ is the cost of the best connecting path found so far by a bidirectional search.

Variant	Starts from	Runs on	Stops when	Delivers
Dijkstra	s	G	heap empty	$\delta(s, v)$ for all v , tree
Uniform-cost search	s	G	t settled	$\delta(s, t)$, one path
Breadth-first search	s	G , unit costs	t discovered	hop counts, one path
Multi-source	every $s \in S$	G	heap empty	distance to the nearest source
Backward	t	G^R	heap empty	$h^*(v) = \delta(v, t)$ for all v
Bidirectional	s and t	G and G^R	smallest keys sum to $\geq \mu$	$\delta(s, t)$, one path

3.7.1 Several sources

Push every source of a set $S \subseteq V$ with key 0 instead of the single entry $(0, s)$ on line 3, and leave the rest of the algorithm unchanged. The run is identical to a single-source run from a virtual vertex joined to every $s \in S$ by an edge of cost 0, so [Theorem 3.7](#) applies and the result is

$$g(v) = \delta(S, v) = \min_{s \in S} \delta(s, v),$$

the cost from the source *nearest* to v , with parent pointers that lead back to it. Started from all obstacle cells of a grid with unit costs, the run labels every free cell with its distance to the nearest obstacle, the distance map behind the inflated obstacles of [Chapter 2](#) and behind clearance penalties; started from all depots, it tells every drone which depot is closest in a single run. The function `multi_source_dijkstra` in the chapter code does this, and its self-test checks the table against the element-wise minimum over single-source runs.

3.7.2 Backward search: the true-distance heuristic

A^* ([Chapter 4](#)) orders its queue by $g(v) + h(v)$, where $h(v)$ estimates the cost that remains from v to the target t . The best conceivable estimate is the true cost-to-go $\delta(v, t)$, and Dijkstra's

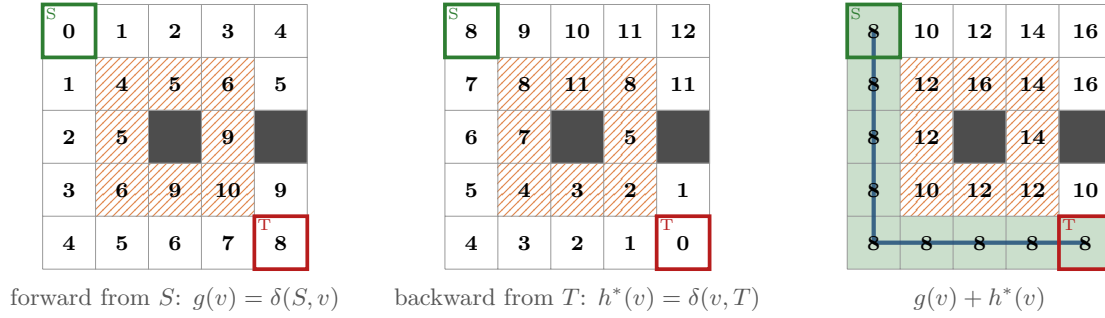


Figure 3.6. Forward and backward tables on the grid of Example 3.5, both from full runs. Left: $g(v) = \delta(S, v)$ from S . Middle: $h^*(v) = \delta(v, T)$ from Algorithm 3.2; because entering a slow cell costs 3 but leaving it costs 1, this is not the distance from T (compare $h^*(3, 3) = 2$ with the 4 that a forward run from T gives). Right: $g(v) + h^*(v)$, the cost of the best path from S to T through v . The green cells, where the sum equals $\delta(S, T) = 8$, are exactly the cells of the optimal path; every other cell has a larger sum, which is how A* will know to avoid them.

algorithm can compute it for *every* vertex at once: run it from t on the graph with all edges reversed. Algorithm 3.2 states the recipe. The resulting table is written h^* and called the **true-distance heuristic** or *perfect heuristic*.

Algorithm 3.2. Backward Dijkstra: the true-distance heuristic table for a goal t .

Input: graph $G = (V, E, c)$ with $c \geq 0$; goal t

Output: table h^* with $h^*(v) = \delta(v, t)$ for every v from which t can be reached, and $h^*(v) = \infty$ otherwise

```

1 Function BackwardDijkstra( $G, t$ ):
2    $G^R \leftarrow (V, E^R, c^R)$  with  $E^R = \{(v, u) : (u, v) \in E\}$  and  $c^R(v, u) = c(u, v)$  // reverse
   every edge
3    $(g^R, \cdot) \leftarrow \text{Dijkstra}(G^R, t, \text{nil})$  // one full run from  $t$ , no target
4   return  $h^* \leftarrow g^R$ 

```

Reversing matters whenever the graph is directed: a path $v = v_0, v_1, \dots, v_k = t$ in G is a path $t = v_k, \dots, v_0 = v$ in G^R with the same cost, so $\delta_G(v, t) = \delta_{G^R}(t, v)$, which is what the run from t on G^R computes. On an undirected graph $G^R = G$, and a plain forward run from t suffices, as in Exercise 3.6(a). The grid of Example 3.5 is directed, and Figure 3.6 shows the difference: a forward run from T would say that $(3, 3)$ costs 4 from T (leave T for 1, enter the slow cell for 3), whereas $h^*(3, 3) = 2$, because from $(3, 3)$ the drone leaves the slow cell for 1 and enters T for 1. An implementation does not need to build G^R : `GridGraph.reversed()` in the chapter code simply charges the cost of a slow cell when the cell is *left* instead of when it is entered, and `reverse_graph` flips an adjacency list in $\mathcal{O}(|V| + |E|)$ time.

Theorem 3.11 (The true-distance heuristic). *Let h^* be the table returned by `BackwardDijkstra( $G, t$ )` for a graph with $c \geq 0$. Then:*

1. *Exactness.* $h^*(v) = \delta(v, t)$ for every $v \in V$, with $h^*(v) = \infty$ if t cannot be reached from v . A vertex v lies on some shortest path from s to t if and only if $\delta(s, v) + h^*(v) = \delta(s, t)$.
2. *Consistency.* $h^*(t) = 0$ and $h^*(u) \leq c(u, v) + h^*(v)$ for every edge $(u, v) \in E$. Consequently $h^*(v) \leq c(\pi)$ for every path π from v to t .
3. *Admissibility in space-time.* Consider the space-time graph of Chapter 2 built on G , in which a state is a pair (vertex, time), a move along (u, v) costs $c(u, v)$, waiting costs a non-negative amount, and any set of states or moves may be forbidden. Every path in

it from a state (v, τ) to a state (t, τ') costs at least $h^*(v)$.

Proof. (1) By the correspondence between paths in G and in G^R , $\delta_{G^R}(t, v) = \delta_G(v, t)$, and Theorem 3.7 applied to G^R gives $h^*(v) = \delta_{G^R}(t, v)$. For the second statement, if v lies on a shortest s - t path, its two halves are shortest paths, so $\delta(s, t) = \delta(s, v) + \delta(v, t)$. Conversely, if equality holds, a shortest s - v path followed by a shortest v - t path costs $\delta(s, t)$; it may repeat vertices, but removing the cycles costs nothing extra ($c \geq 0$) and leaves a shortest path through v . (2) $h^*(t) = \delta(t, t) = 0$. For an edge (u, v) , going from u to v and then along a shortest path from v to t is a path from u to t of cost $c(u, v) + \delta(v, t)$, which is at least $\delta(u, t)$; this is the third fact of Equation (3.1). Applying the inequality to the edges $(v_0, v_1), \dots, (v_{k-1}, v_k)$ of a path π from $v_0 = v$ to $v_k = t$ and adding up, the h^* terms telescope to $h^*(v) \leq c(\pi) + h^*(t) = c(\pi)$. (3) Drop the time index from the space-time path and delete every wait step. What remains is a walk in G from v to t whose cost is at most that of the space-time path, because waiting costs at least 0 and every move keeps its cost. A walk from v to t costs at least $\delta(v, t) = h^*(v)$, since removing its cycles cannot increase the cost. Forbidding states or moves removes paths from the space-time graph and adds none, so the bound is unaffected. $\square$

Chapter 4 calls property 2 of Theorem 3.11 *consistency* and its consequence *admissibility*, and proves that they make A^* optimal. Property 1 is visible in the right-hand panel of Figure 3.6: the cells with $g(v) + h^*(v) = \delta(S, T) = 8$, shaded green, are exactly the cells of the optimal path, and every other cell has a larger sum. An A^* search guided by h^* therefore expands only cells that lie on a shortest path; no heuristic can do better on a static map, which is why h^* is the reference against which the grid heuristics of Chapter 4 are judged. Property 3 is what makes the table useful beyond this chapter. The multi-agent solvers of Chapters 8 and 9 route one drone at a time through *space-time*, where waiting is an action and the reservations or constraints of other drones forbid certain cells at certain times, and they do so thousands of times on the same map with the same goals. One backward run per goal, costing $\mathcal{O}(|V| \log |V|)$ on a grid, produces a table that every one of those searches reads in $\mathcal{O}(1)$ per expansion and that never overestimates, whatever the constraints. It is no longer exact there, since a forced wait or detour is not in the table (Exercise 3.6(c)), but admissibility is all that A^* needs for optimality. D^* Lite (Chapter 5) searches backward from the goal in the same way and repairs the table when the map changes instead of recomputing it.

3.7.3 Bidirectional search

For a single pair (s, t) on a large graph, one may grow two waves at once: a forward wave from s on G with costs g_f , and a backward wave from t on G^R with costs g_b , each with its own heap and closed set, alternating expansions between the two sides. Whenever one side scans an edge (u, v) during an expansion, including edges to vertices that either side has settled, and both $g_f(u)$ and $g_b(v)$ are finite, the concatenation is an s - t path of cost $g_f(u) + c(u, v) + g_b(v)$, and the best such cost seen so far is kept in μ . The search stops when

$$\min_{\text{OPEN}_f} g_f + \min_{\text{OPEN}_b} g_b \geq \mu, \quad (3.2)$$

and returns the path that achieved μ . The tempting rule “stop as soon as one vertex has been settled by both sides” is wrong: the first common vertex need not lie on a shortest path, because the two waves may meet at a vertex that is close to both but not on the cheapest route. The rule of Equation (3.2) is correct because a path cheaper than μ would have to pass through a vertex still open on the forward side and later through one still open on the backward side, which costs at least the sum of the two smallest keys; Exercise 3.8 asks for the counterexample and the proof.

The saving comes from geometry: on a map the number of settled cells grows with the area of the wave, one wave of radius d covers an area proportional to d^2 , and two waves of radius $d/2$ that meet halfway cover $2(d/2)^2 = d^2/2$, half as much (a quarter in three dimensions). The gain is real but modest, and it shrinks when a good heuristic is available, so the book uses bidirectional search only as a benchmark; Russell and Norvig [35] give a careful treatment, including the heuristic version.

3.7.4 Unit costs: breadth-first search

When every edge has the same cost $c_0 > 0$, the keys in the heap are multiples of c_0 and the heap can be replaced by a first-in-first-out queue. The reason is [Lemma 3.6](#): at any moment the queue holds vertices of at most two consecutive levels k and $k+1$ (in units of c_0), all of level k ahead of all of level $k+1$, so the front of the queue always has the smallest key. Two further simplifications follow. A vertex is pushed at most once, because its first discovery is already at the smallest possible level, so there are no stale entries and no relaxation test. And the target may be recognised when it is *discovered*, since no later discovery can be cheaper. The result is **breadth-first search**, which runs in $\mathcal{O}(|V| + |E|)$ time, without the logarithm. The function `bfs` in the chapter code implements it, and the self-test checks it against `dijkstra` on random 4-connected grids with unit costs. On an 8-connected grid the diagonal costs $\sqrt{2}$, the levels interleave, and breadth-first search is wrong; every search from [Chapter 4](#) onwards is built on the heap version for this reason.

3.8 Implementation notes

[Listing 3.1](#) is the function `dijkstra` of `code/ch03_dijkstra.py`, a direct transcription of [Algorithm 3.1](#). The remarks below explain the choices in it; [Listing 3.2](#) adds path reconstruction and the backward run.

The heap. `heapq` is a binary min-heap stored in a list; it offers `heappush` and `heappop` but no decrease-key, which is why the listing pushes duplicates. Do not “fix” this by searching the list for the old entry: that costs $\mathcal{O}(|\text{OPEN}|)$ per relaxation and destroys the bound of [Theorem 3.10](#). Entries are tuples `(g, node)`; Python compares tuples lexicographically, so ties in `g` are broken by comparing the nodes themselves. This works for strings and for `(row, col)` tuples and makes the search deterministic, which is what you want when you compare runs or reproduce a trace. If your nodes are not comparable, insert a running counter as the second element of the tuple, as `LazyPQ` in [Chapter 2](#) does.

Costs and parents. `dist` is a dictionary: a missing key means ∞ , so `dist.get(v, INF)` is the test of line 12 and no array of size $|V|$ is ever allocated. `parent` maps every discovered node to its predecessor and the source to `None`; `reconstruct_path` follows the pointers back and reverses the list. Because the parent is overwritten at every relaxation, the pointers always describe the path that produced the current `dist`.

The closed set. `closed` is a set, and the membership test on pop is the lazy-deletion check. After the loop, `dist[v]` is exact if and only if `v in closed`; with an early exit, the tentative costs of open nodes are upper bounds only (recall F in [Example 3.4](#)). If your caller needs the guarantee, return `closed` as well, or return only the settled entries.

Implicit graphs. The function only ever evaluates `graph[u]`, which must yield `(neighbour, cost)` pairs. A dictionary of lists does this for explicit graphs. The class `GridGraph` in the

same file does it for occupancy grids: it generates the 4 or 8 neighbours of a cell on the fly, with cost 1 or $\sqrt{2}$, forbids corner cutting as in [Chapter 2](#), and multiplies the cost of a move by an optional factor attached to the cell being entered. Its method `reversed()` flips the direction in which the factors are applied, so that [Algorithm 3.2](#) can run on a grid without ever building the reversed graph explicitly.

Tie-breaking. Among several shortest paths, the one returned depends on two rules: the strict `<` on line 12, which keeps the first path found at equal cost, and the order of the heap among equal keys. Both are deterministic here, but neither is canonical: a different neighbour order yields a different, equally optimal path. Planners that must be reproducible fix the neighbour order, as `MOVES` in the code does. A^* in [Chapter 4](#) adds a tie-breaking rule on g that changes the number of expansions substantially.

Floating-point costs. On 8-connected grids the costs 1 and $\sqrt{2}$ are added in different orders along different paths, so two paths of the same true cost can end with values that differ in the last binary digit. Never test costs for equality: compare with a tolerance, or scale the costs to integers (many grid planners use 10 for a straight move and 14 for a diagonal one). The self-test of the chapter code compares against a brute-force enumeration with a tolerance of 10^{-9} and checks that h^* from the backward run agrees with a forward search from every cell.

Listing 3.1. The core of the implementation (from `code/ch03_dijkstra.py`): the main loop with `heapq`, a dictionary of tentative costs, a closed set tested on every pop, and lazy insertion instead of decrease-key.

```

1  def dijkstra(graph, source, target=None):
2      """Shortest paths from source; every edge cost must be >= 0.
3
4      graph[u] yields the (v, cost) pairs of the out-edges of u; a dict of
5      lists or a GridGraph both work. Returns (dist, parent): dist[v] is the
6      cost of the best path found to v and parent[v] its predecessor on that
7      path (parent[source] is None). Unreached nodes are absent (infinity).
8      With a target the loop stops as soon as the target is settled
9      (uniform-cost search); dist is then exact for settled nodes only.
10     """
11     dist = {source: 0.0}
12     parent = {source: None}
13     closed = set()
14     queue = [(0.0, source)]           # min-heap of (g, node) entries
15     while queue:
16         g, u = heapq.heappop(queue)   # smallest g first, ties by node
17         if u in closed:                # stale entry: u was settled earlier
18             continue
19         closed.add(u)                 # settle u: dist[u] == g is final
20         if u == target:
21             break                    # early exit
22         for v, cost in graph[u]:
23             if v in closed:
24                 continue
25             g_new = g + cost
26             if g_new < dist.get(v, INF):
27                 dist[v] = g_new        # relaxation
28                 parent[v] = u
29                 heapq.heappush(queue, (g_new, v))  # lazy insertion
30     return dist, parent

```

Listing 3.2. Path reconstruction from the parent pointers and the backward run that produces the true-distance heuristic table (from `code/ch03_dijkstra.py`).

```

1  def reconstruct_path(parent, target):
2      """Follow parent pointers back from target; None if unreachable."""
3      if target not in parent:
4          return None
5      path = []
6      node = target
7      while node is not None:
8          path.append(node)
9          node = parent[node]
10     path.reverse()
11     return path
12
13
14  def backward_dijkstra_heuristic(graph, goal):
15      """True-distance heuristic  $h^*(v) = \text{dist}(v, \text{goal})$  for every node  $v$ .
16
17      One run of dijkstra() from the goal over the reversed graph. The
18      table is a perfect heuristic: admissible and consistent on the static
19      map, and still admissible when wait actions or constraints are added
20      (Chapters 4, 8 and 9). Unreached nodes are absent: read as infinity.
21      """
22      if hasattr(graph, "reversed"):
23          rev = graph.reversed()          # GridGraph knows how to flip itself
24      else:
25          rev = reverse_graph(graph)
26      h_star, _ = dijkstra(rev, goal)
27      return h_star

```

Testing for the goal at the wrong time

The most common bug in a first implementation is to return as soon as the target is *discovered*, on line 13, instead of when it is *popped*. In [Example 3.4](#) the vertex *F* is discovered with cost 13, later improved to 12 and then to 11. A search that stops at discovery would report 13, almost twenty percent too much, and the “optimal” path it returns would be *A, B, C, D, F*. Breadth-first search may test at discovery because every edge costs the same; Dijkstra’s algorithm and A^* may not. The same rule applies to the early exit of every search in [Chapters 4, 8 and 9](#).

Negative costs and the constant-shift trap

Dijkstra’s algorithm is silently wrong when an edge has a negative cost ([Example 3.9](#)): it does not crash, it returns a path that is not optimal. Negative costs sneak into planners as rewards, for instance a bonus for flying through a cell that must be surveyed. Adding a constant to all edge costs does not help, because paths with more edges are penalised more and the optimal path changes ([Exercise 3.4](#)). Model rewards as reduced non-negative costs, or use the Bellman–Ford algorithm. Zero costs are fine.

Forgetting the stale check

Without the test on line 6 every duplicate entry is expanded again. The costs stay correct, because a relaxation from a stale entry can never improve anything, but a vertex is then

“settled” several times: the count of expansions is wrong, a visualisation of the closed set shows cells being settled twice, and in the incremental algorithms of [Chapter 5](#), where an expansion has side effects, the result is wrong. Test membership in the closed set, or compare the popped key with the current `dist[u]`; either works, but one of them must be there.

3.9 Where this fits in the drone system

In the drone system

In the hybrid architecture of [Chapter 24](#) Dijkstra's algorithm is part of the global planning layer, and it plays two roles there. As the *baseline planner* it routes a single drone across a static occupancy grid or waypoint graph whose edge costs encode flight time, energy or risk; its output, an optimal path with its cost, is the yardstick against which every faster or more capable planner of the book is measured, and the experiments of [Chapter 25](#) report path lengths relative to it. As the *heuristic factory* it runs backwards from every drone's goal once, on the static map, and delivers the true-distance table h^* that A^* ([Chapter 4](#)), the space-time searches of prioritized planning ([Chapter 8](#)) and the low level of conflict-based search ([Chapter 9](#)) consult at every expansion. The replanning layer inherits the same backward wave in incremental form as D^* Lite ([Chapter 5](#)). What Dijkstra's algorithm does not do is react: it does not see other drones, and it does not see the intruder that appears after take-off; those are handled by the multi-agent solver, by the prediction layer and by the local avoidance layer. In the study plan ([Appendix A](#)) this chapter is the first half of Week 1; the second half is A^* .

3.10 Summary

Chapter summary

- Dijkstra's algorithm computes $\delta(s, v)$ for every vertex of a graph with non-negative edge costs, together with a shortest-path tree, by settling vertices in non-decreasing order of tentative cost: a wave spreading from the source.
- The settled-node invariant ([Theorem 3.7](#)) says that a vertex's cost is final when it is popped. Its proof uses $c \geq 0$ exactly once, and a single negative edge breaks it ([Example 3.9](#)).
- With a binary heap and lazy deletion the running time is $\mathcal{O}((|V| + |E|) \log |V|)$, the number of expansions grows with the area of the wave, and the early exit (uniform-cost search) stops the wave at the radius of the target.
- The implementation needs a heap of (g, v) pairs, a dictionary of tentative costs, a closed set that is tested on every pop, parent pointers for the path, and tolerance-based comparisons of floating-point costs.
- One backward run from the goal on the reversed graph gives the true-distance heuristic $h^*(v) = \delta(v, t)$, which is consistent, exact on the static map, and still admissible in space-time and under constraints: the heuristic used throughout [Chapters 4, 8 and 9](#).

Further reading. The original three-page paper is [\[7\]](#); the array-scanning version in it is still the right choice for dense graphs. Cormen et al. [\[5\]](#) give the standard textbook treatment with

the proof of the invariant, the Bellman–Ford algorithm for negative costs and the priority-queue trade-offs, and Fredman and Tarjan [13] introduce the Fibonacci heap. Russell and Norvig [35] present the algorithm as uniform-cost search alongside breadth-first and bidirectional search, and Felner [9] explains why that presentation is the better one for implicit graphs. The step from Dijkstra's algorithm to A^* was taken by Hart, Nilsson, and Raphael [16]; Pearl [30] is the classic reference on heuristics, including the exact heuristic of Section 3.7.2.

3.11 Exercises

Exercise 3.1 (★). In Example 3.4 reduce the cost of the edge $B-E$ from 7 to 4 and trace Algorithm 3.1 from A by hand in the format of Table 3.1. Give the final distances, the shortest path to G , the number of pops and the number of stale entries. Check your trace with `dijkstra_trace` in the chapter code.

Exercise 3.2 (★). Suppose Algorithm 3.1 returned as soon as the target is pushed on line 14 rather than when it is popped. Using Table 3.1, give the cost and path it would return for the target F , and for the target G . Which step of the proof of Theorem 3.7 no longer applies to a vertex that has been discovered but not settled?

Exercise 3.3 (★★). Use Lemma 3.6 and Theorem 3.7 to prove that uniform-cost search with target t settles every vertex v with $\delta(s, v) < \delta(s, t)$, no vertex with $\delta(s, v) > \delta(s, t)$, and that which vertices with $\delta(s, v) = \delta(s, t)$ it settles depends only on tie-breaking. Then show that the tentative cost of every vertex left in the queue at the exit is at least $\delta(s, t)$, and explain why it may nevertheless differ from $\delta(s, v)$. Finally, show that edges of cost 0, including zero-cost cycles, do not endanger termination.

Exercise 3.4 (★★). Let G have some negative edge costs but no negative cycle, and let G' be obtained by adding the same constant $K > 0$ to every edge cost so that all costs become non-negative. Show with a graph of four vertices that a shortest path in G' need not be a shortest path in G . Then explain, in one sentence, what property of paths makes the shift harmless when all shortest paths of interest have the same number of edges.

Exercise 3.5 (★★). Repeat Example 3.5 on the same grid with 8-connectivity: a diagonal move costs $\sqrt{2}$, entering a hatched cell multiplies the cost of the move by 3, and corner cutting is forbidden. Determine $\delta(S, T)$ and an optimal path by hand, then run `dijkstra_trace` on `GridGraph(cells, 8, cost)` and count the expansions and the stale entries. Why does this instance produce stale entries when the 4-connected one did not?

Exercise 3.6 (★★). (a) Compute the true-distance heuristic $h^*(v) = \delta(v, G)$ for every vertex of Example 3.4 by running Algorithm 3.2 by hand, and verify the consistency inequality $h^*(u) \leq c(u, v) + h^*(v)$ on every edge. (b) On the grid of Example 3.5, explain why the forward table from T and the backward table to T differ, and name every cell where they differ. (c) A drone must reach T on that grid but is forbidden to enter the cell $(4, 2)$ at time step 6 (a vertex constraint of the kind used in Chapter 9). Explain why the table of Figure 3.6 is still an admissible heuristic for the constrained space-time search, and why it is no longer exact.

Exercise 3.7 (★★★ Coding). This is the first half of the Week 1 exercise of the study plan (Appendix A); Chapter 4 completes it. Implement Dijkstra's algorithm on an occupancy grid from scratch, with 4- and 8-connectivity, a successor function that forbids corner cutting, lazy deletion, the early exit and path reconstruction. (a) Visualise the closed set at several moments of a search on a 30×30 grid with a few walls, as in Figure 3.3, and check that the settled cells at any moment are exactly the cells whose cost is at most the last popped key. (b) Count expansions against grid size for random grids with and without the early exit, and reproduce the shape of Figure 3.5. (c) Add a function that returns the true-distance table

for a goal and check, on random grids, that $h^*(v)$ equals the cost of a forward search from v for every free cell v . Keep the code: the next chapter turns it into A^* by changing the key of the heap.

Exercise 3.8 (★★★). Implement bidirectional Dijkstra as described in [Section 3.7.3](#). (a) Construct a small graph on which stopping at the first vertex reached by both waves returns a suboptimal path, and show that the stopping rule with μ returns the optimal one. (b) Prove that the rule is correct: when the two smallest keys sum to at least μ , no path cheaper than μ exists. (c) On the random grids of [Figure 3.5](#), measure the number of settled cells of bidirectional search relative to uniform-cost search for the far-corner target, and compare with the factor of one half predicted by the disc argument.

A* Search

Learning objectives

- Explain best-first search with $f = g + h$ and trace A* by hand on a small grid while keeping track of OPEN and CLOSED.
- State the exact definitions of admissible and consistent heuristics, prove that consistency implies admissibility, and prove that A* returns an optimal path.
- Choose a heuristic for a 4- or 8-connected grid (Manhattan, octile, Euclidean, Chebyshev), break ties well, and predict what a weight w does to cost and effort.
- Add time to the state: run A* in space-time with wait actions, vertex and edge constraints, a reservation table and a correct goal test.
- Implement grid A* and space-time A* in Python, visualise the open and closed sets, and recognise the failure modes that matter in a drone planner.

4.1 Why this matters for a drone swarm

Picture a delivery drone that has to cross a city district represented by a 100×100 grid of cells, a quarter of them blocked by buildings. Dijkstra's algorithm from [Chapter 3](#) finds the shortest route, but it does so by settling almost every free cell: in the experiment of [Section 4.7](#) it expands about 7 450 of the 7 500 free cells on such maps. The drone knows something Dijkstra ignores: the goal lies to the north-east, and a cell far to the south-west is unlikely to be on the way. A* puts this knowledge into the search. With the Manhattan distance as a guide it solves the same instances with about 750 expansions, ten times fewer, and it still guarantees the shortest route.

For a single query this is a convenience. For a swarm it is a necessity. The global planner of the hybrid architecture in [Chapter 24](#) is CBS or ECBS ([Chapters 9](#) and [10](#)), and every node of their constraint tree calls a single-agent search for one drone under a set of constraints. A run on a few dozen drones can easily make thousands of such calls, and each of them is an A* search in *space-time*: the state of the drone is a cell together with the time at which it is there, the drone may wait, and it must avoid the cells and edges that other drones have reserved. The replanning layer of [Chapter 24](#) calls A* again whenever a local avoidance manoeuvre has pushed a drone off its nominal route, and the incremental planners of [Chapter 5](#) (LPA*, D* Lite) and the anytime planner of [Chapter 6](#) (ARA*) are best understood as A* with extra bookkeeping. This chapter is therefore the foundation of everything that follows in [Chapters 5](#), [6](#) and [8](#) to [10](#).

The chapter proceeds as follows. [Section 4.2](#) gives the idea, [Section 4.3](#) the definitions of admissible and consistent heuristics and the standard grid heuristics. [Section 4.4](#) presents

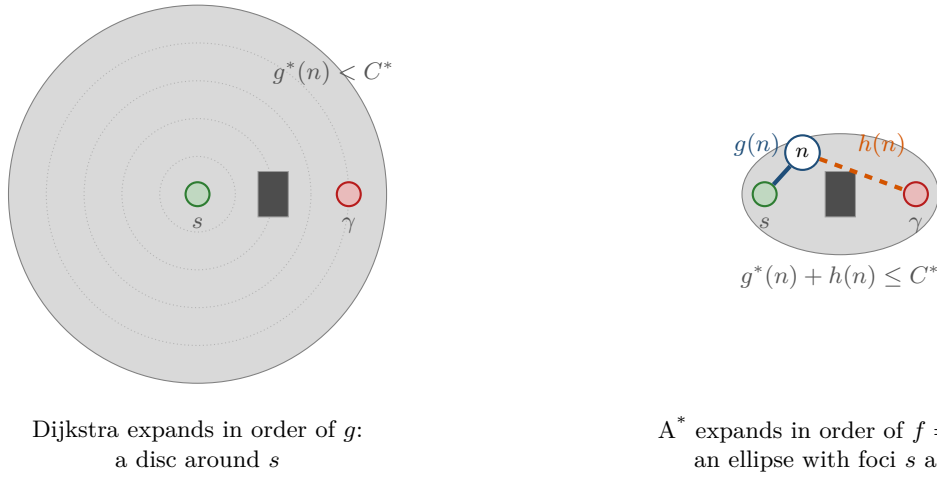


Figure 4.1. Left: Dijkstra expands nodes in order of g , so its closed set is a disc around the start s that reaches the goal γ only when it has covered every cheaper node. Right: A* expands nodes in order of $f = g + h$. With the straight-line distance as h , the nodes with $f \leq C^*$ lie inside an ellipse with foci s and γ ; the search never leaves it. For the node n , the solid segment is the known cost $g(n)$ and the dashed segment the estimate $h(n)$.

the graph-search version of A* with a closed set, [Section 4.5](#) runs it by hand on a 6×6 grid, and [Section 4.6](#) proves the optimality theorems that the Week 1 milestone of the study plan ([Appendix A](#)) asks you to be able to explain exactly. [Section 4.7](#) covers tie-breaking, weighted A* and an experiment on random grids. ?? is a full treatment of space-time A*, the exact form of the search used by the multi-agent planners of [Chapters 8 and 9](#). ?? discusses the implementation and ?? its place in the drone system.

4.2 The idea in plain words

Dijkstra's algorithm always expands the node that is cheapest to reach from the start. Its frontier grows like a ripple in a pond: a disc around the start that spreads evenly in all directions until it touches the goal ([Figure 4.1](#), left). A* adds a second ingredient. For every node n it keeps the cost $g(n)$ of the best path found so far from the start *and* an estimate $h(n)$ of the cost that remains from n to the goal. Their sum

$$f(n) = g(n) + h(n) \quad (4.1)$$

estimates the cost of the cheapest solution that passes through n . A* is a **best-first search** on f : it always expands the node with the smallest f . The function h is called a **heuristic** because it is a rule of thumb, computed without looking at the obstacles, for example the straight-line distance to the goal. A node far from the goal has a large h and is expanded late or not at all. The frontier stops being a disc and becomes an elongated region pointing at the goal ([Figure 4.1](#), right).

Two questions decide whether this idea is safe. First, can a heuristic mislead the search into returning a path that is not the shortest? Yes, if it *overestimates*: a node on the true shortest path may then look worse than it is and never be expanded. Second, can a node be expanded twice? Yes, if the heuristic is not *consistent*, because a cheaper path to an already expanded node may show up later. The two conditions that rule out these problems, admissibility and consistency, are the heart of this chapter.

Key idea

Expand the node with the smallest $f = g + h$. If h never overestimates the remaining cost, the first goal that A* expands is reached by an optimal path. If in addition h obeys the triangle inequality along every edge, no node is ever expanded twice, and A* is Dijkstra's algorithm with its effort focused towards the goal.

4.3 Problem statement and notation

4.3.1 Search problems and heuristics

Definition 4.1 (Heuristic search problem). A **heuristic search problem** consists of a finite directed graph $G = (V, E)$ with edge costs $c(u, v) \geq 0$, a start node $s \in V$, a non-empty set of goal nodes $V_\gamma \subseteq V$ (usually a single node γ), and a **heuristic function** $h: V \rightarrow \mathbb{R}_{\geq 0}$. The cost of a path is the sum of its edge costs. We write $g^*(n)$ for the cost of a cheapest path from s to n , $h^*(n)$ for the cost of a cheapest path from n to a goal ($h^*(n) = \infty$ if no goal is reachable), and $C^* = h^*(s)$ for the optimal solution cost.

During the search, $g(n)$ denotes the cost of the best path from s to n found *so far*; it satisfies $g(n) \geq g^*(n)$, with equality once the best path has been found. The successor function $\text{Succ}(n)$ returns the pairs $(n', c(n, n'))$ for all edges $(n, n') \in E$, exactly as in Chapter 3. For grids the nodes are cells (x, y) , x the column and y the row, with $(0, 0)$ in the bottom-left corner as in Chapter 2.

Definition 4.2 (Admissible heuristic). A heuristic h is **admissible** if it never overestimates the true cost to go:

$$0 \leq h(n) \leq h^*(n) \quad \text{for every } n \in V. \quad (4.2)$$

In particular $h(\gamma) = 0$ for every goal node γ .

Definition 4.3 (Consistent heuristic). A heuristic h is **consistent** (also called **monotone**) if $h(\gamma) = 0$ for every goal node γ and, for every edge $(n, n') \in E$,

$$h(n) \leq c(n, n') + h(n'). \quad (4.3)$$

Equation (4.3) is a triangle inequality: the estimate from n may not exceed the cost of first stepping to n' and estimating from there. An equivalent formulation is that f never decreases along an edge when g is computed along the path: $f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n)$. Consistency is the stronger property.

Lemma 4.4 (Consistency implies admissibility). *Every consistent heuristic is admissible. The converse is false.*

Proof. Let h be consistent and let $n \in V$. If no goal is reachable from n , then $h^*(n) = \infty$ and Equation (4.2) holds. Otherwise let $n = n_0, n_1, \dots, n_k = \gamma$ be a cheapest path from n to a goal, so that $h^*(n) = \sum_{i=0}^{k-1} c(n_i, n_{i+1})$. Applying Equation (4.3) to each edge in turn,

$$h(n_0) \leq c(n_0, n_1) + h(n_1) \leq c(n_0, n_1) + c(n_1, n_2) + h(n_2) \leq \dots \leq \sum_{i=0}^{k-1} c(n_i, n_{i+1}) + h(\gamma) = h^*(n),$$

because $h(\gamma) = 0$. For the converse, take the graph with nodes s, a, b, γ and edges $s \rightarrow a$ (cost 1), $s \rightarrow b$ (cost 3), $a \rightarrow b$ (cost 1), $b \rightarrow \gamma$ (cost 2). The heuristic $h(a) = 3$, $h(s) = h(b) = h(\gamma) = 0$ is admissible, since $h^*(a) = 3$, but $h(a) = 3 > c(a, b) + h(b) = 1$ violates Equation (4.3). We return to this graph in Section 4.6. $\square$

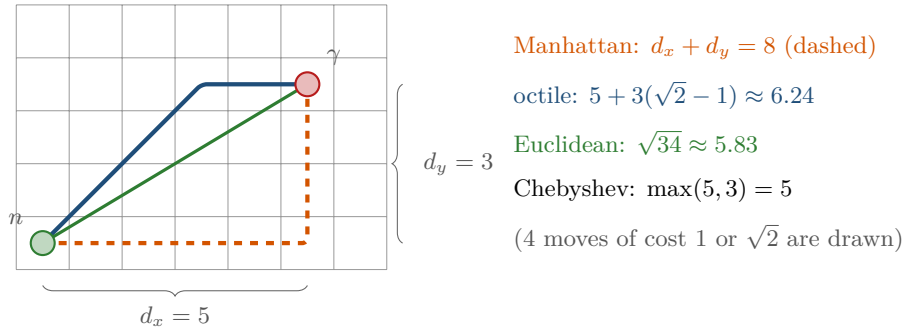


Figure 4.2. The four grid heuristics from the cell n to the goal γ with $d_x = 5$ and $d_y = 3$. The Manhattan path (orange, dashed) has length 8; the octile path (blue) uses three diagonal moves and two straight ones, length $5 + 3(\sqrt{2} - 1) \approx 6.24$; the Euclidean distance (green) is $\sqrt{34} \approx 5.83$; the Chebyshev distance counts only the longer side, 5. On a 4-connected grid all four are admissible; on an 8-connected grid the Manhattan distance overestimates every diagonal move.

4.3.2 Heuristics for grids

Let $n = (x_n, y_n)$ be a cell and $\gamma = (x_\gamma, y_\gamma)$ the goal, and write $d_x = |x_n - x_\gamma|$ and $d_y = |y_n - y_\gamma|$. Four distances are in everyday use (Figure 4.2):

$$h_M(n) = d_x + d_y \quad \text{Manhattan distance,} \quad (4.4)$$

$$h_O(n) = \max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y) \quad \text{octile distance,} \quad (4.5)$$

$$h_E(n) = \sqrt{d_x^2 + d_y^2} \quad \text{Euclidean distance,} \quad (4.6)$$

$$h_C(n) = \max(d_x, d_y) \quad \text{Chebyshev distance.} \quad (4.7)$$

The Manhattan distance is the length of a cheapest path on an empty 4-connected grid, where every move costs 1. The octile distance is the length of a cheapest path on an empty 8-connected grid, where a straight move costs 1 and a diagonal move $\sqrt{2}$: use $\min(d_x, d_y)$ diagonal moves and then $\max(d_x, d_y) - \min(d_x, d_y)$ straight moves. The Chebyshev distance is the same count when diagonal moves also cost 1. Each of the four is the norm of the displacement vector (ℓ_1 , octagonal, ℓ_2 and ℓ_∞ norm), so each satisfies the triangle inequality.

Proposition 4.5 (Metric heuristics are consistent). *Let d be a metric on V such that $d(n, n') \leq c(n, n')$ for every edge $(n, n') \in E$, and let $h(n) = d(n, \gamma)$. Then h is consistent, hence admissible.*

Proof. $h(\gamma) = d(\gamma, \gamma) = 0$, and by the triangle inequality $h(n) = d(n, \gamma) \leq d(n, n') + d(n', \gamma) \leq c(n, n') + h(n')$. $\square$

To apply Proposition 4.5 we only have to compare each distance with the cost of a single move. A straight move costs 1, and all four distances assign 1 to it. A diagonal move on an 8-connected grid costs $\sqrt{2}$: the octile and Euclidean distances assign $\sqrt{2}$, the Chebyshev distance 1, but the Manhattan distance assigns $2 > \sqrt{2}$. Table 4.1 summarises the result. The Manhattan distance is the classic example of a heuristic that is perfect for one grid and wrong for another: in the self-test of `ch04_astar.py`, A* with the Manhattan heuristic on 8-connected random grids returns a path longer than optimal on 6 of 40 instances.

None of the four sees obstacles, so with walls in the way they all underestimate, sometimes badly: the worked example of Section 4.5 shows the search walking into a dead end because the Manhattan distance does not know about a wall. The exact heuristic h^* itself can be computed by a backward Dijkstra search from the goal, as in Chapter 3. That costs a full search and is pointless for a single query, but it becomes the heuristic of choice in space-time (??), where the same static distances serve every time layer and every constrained re-run of

Table 4.1. Grid heuristics and their properties. “Exact” means equal to h^* on an obstacle-free grid. A heuristic that is exact on empty grids stays admissible with obstacles, because obstacles can only lengthen paths.

Heuristic	4-connected (cost 1)	8-connected (costs 1, $\sqrt{2}$)
Manhattan $d_x + d_y$	exact on empty grids; consistent	not admissible (diagonal: $2 > \sqrt{2}$)
Octile $\max + (\sqrt{2} - 1) \min$	consistent (below Manhattan)	exact on empty grids; consistent
Euclidean $\sqrt{d_x^2 + d_y^2}$	consistent	consistent; dominated by octile
Chebyshev $\max(d_x, d_y)$	consistent; weakest of the four	consistent; exact if diagonals cost 1

the search.

4.4 The algorithm

Algorithm 4.1 is A* as a *graph search*: it remembers the nodes it has expanded in the set CLOSED so that a node is not expanded again unless a strictly cheaper path to it is found. The nodes that have been generated but not yet expanded form the frontier OPEN, a priority queue keyed on f . The queue is *lazy*: when the g of a node improves, the algorithm pushes a fresh entry instead of searching the queue for the old one, and the old entry is discarded when it is popped and found to be stale. The weight w is 1 for plain A*; [Section 4.7](#) discusses $w > 1$.

Algorithm 4.1. A* graph search with a closed set and a lazy priority queue

Input: graph $G = (V, E)$ with costs c , start s , goal test, heuristic h , weight $w \geq 1$
 ($w = 1$ for A*)

Output: a cheapest path from s to a goal, or *failure*

```

1 Function AStar( $G, s, h, w$ ):
2    $g(s) \leftarrow 0$ ;  $\text{parent}(s) \leftarrow \text{nil}$ ;  $g(n) \leftarrow \infty$  for all other nodes (implicitly)
3   OPEN  $\leftarrow$  priority queue holding  $s$  with key  $(wh(s), 0)$ ; CLOSED  $\leftarrow \emptyset$ 
4   while OPEN  $\neq \emptyset$  do
5      $(n, g_n) \leftarrow$  pop the entry with the smallest key  $(f, -g)$  from OPEN
6     if  $g_n > g(n)$  then
7       continue // stale entry: a cheaper path to  $n$  was found later
8     if  $n$  is a goal then
9       return ReconstructPath( $\text{parent}, n$ )
10    add  $n$  to CLOSED
11    foreach  $(n', c(n, n')) \in \text{Succ}(n)$  do
12       $g' \leftarrow g(n) + c(n, n')$ 
13      if  $g' < g(n')$  then
14        if  $n' \in \text{CLOSED}$  then
15          remove  $n'$  from CLOSED // re-open  $n'$ ; never happens if  $h$  is
16          consistent
17           $g(n') \leftarrow g'$ ;  $\text{parent}(n') \leftarrow n$ 
18          push  $(n', g')$  into OPEN with key  $(g' + wh(n'), -g')$ 
19  return failure

```

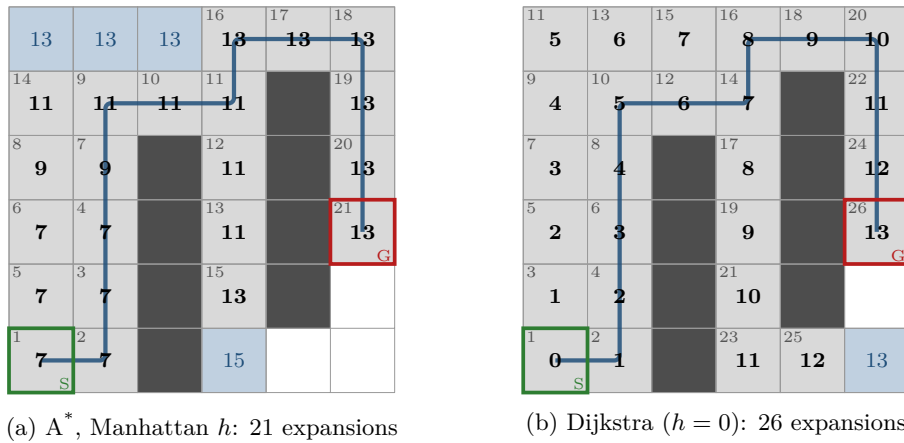


Figure 4.3. Example 4.6 after termination. Grey cells are CLOSED (expanded), blue cells are still in OPEN; the bold number is f at the time of expansion (for Dijkstra $f = g$) and the small grey number in the corner is the expansion order. The blue line is the returned path of cost 13. (a) A* with the Manhattan heuristic expands 21 cells; note the four wasted expansions in the column $x = 3$, where the heuristic cannot see the wall at $x = 4$. (b) Dijkstra’s algorithm ($h = 0$) expands 26 cells, including the whole south-west corner.

Walkthrough. Line 3 seeds the frontier with the start, whose f is $h(s)$. The loop repeatedly takes the most promising entry (line 5). The key is the pair $(f, -g)$, compared lexicographically: smaller f first and, among equal f , larger g first. This tie-breaking rule is not needed for correctness, but Section 4.7 shows that it can make a tenfold difference in effort. Line 7 implements lazy deletion: a node may have several entries in OPEN, one per improvement of its g ; only the entry carrying the current value is valid. The goal test happens when the goal is *popped* (line 9), not when it is first generated. This detail is what makes the optimality proof of Section 4.6 work; testing at generation time returns the first path found to the goal, which need not be the cheapest. Line 10 records the expansion. The inner loop relaxes every outgoing edge (line 13) exactly as Dijkstra’s algorithm does; the only differences from Chapter 3 are the key of the priority queue and line 15, which re-opens a closed node when a cheaper path to it appears. With a consistent heuristic this line is never executed (Theorem 4.10), and many implementations drop it together with the ability to correct a closed node; with an admissible but inconsistent heuristic it is required for optimality (Theorem 4.9). The function `ReconstructPath` follows the parent pointers back from the goal to s and reverses the list.

Setting $h \equiv 0$ turns the key into $(g, -g)$ and Algorithm 4.1 into Dijkstra’s algorithm with early exit; every property of Chapter 3 is the special case $h = 0$ of a property in this chapter.

4.5 A worked example

Example 4.6 (A* on a 6×6 grid). Figure 4.3 shows a 6×6 4-connected grid with two walls: the cells $(2, 0)$ to $(2, 3)$ and $(4, 1)$ to $(4, 4)$ are blocked. The start is $s = (0, 0)$ in the bottom-left corner and the goal $\gamma = (5, 2)$. Every move costs 1 and the heuristic is the Manhattan distance Equation (4.4), so $h(s) = 5 + 2 = 7$. Ties in f are broken towards the larger g , and among equal keys the entry pushed first is popped first. The instance is `worked_example_grid()` in `ch04_astar.py`, and every number below is produced by its self-test.

Table 4.2 lists the first ten expansions. Steps 1–4 walk up the column $x = 1$: every cell there has $f = 7$, because moving towards the goal decreases h by exactly the move cost. At $(1, 2)$ the search would like to continue east, but $(2, 2)$ is blocked; going north to $(1, 3)$ moves

Table 4.2. The first ten expansions of Algorithm 4.1 on Example 4.6. The last column lists the cells in OPEN after the step with their current f ; a cell whose g improves, such as $(0, 2)$ in step 5, keeps a single valid entry.

Step	Expanded n	g	h	f	OPEN after the step (cell: f)
1	(0, 0)	0	7	7	(0, 1):7, (1, 0):7
2	(1, 0)	1	6	7	(0, 1):7, (1, 1):7
3	(1, 1)	2	5	7	(0, 1):7, (1, 2):7
4	(1, 2)	3	4	7	(0, 1):7, (0, 2):9, (1, 3):9
5	(0, 1)	1	6	7	(0, 2):7, (1, 3):9
6	(0, 2)	2	5	7	(0, 3):9, (1, 3):9
7	(1, 3)	4	5	9	(0, 3):9, (1, 4):11
8	(0, 3)	3	6	9	(0, 4):11, (1, 4):11
9	(1, 4)	5	6	11	(0, 4):11, (1, 5):13, (2, 4):11
10	(2, 4)	6	5	11	(0, 4):11, (1, 5):13, (2, 5):13, (3, 4):11

away from the goal row, and f rises to 9. A* therefore turns to the remaining cells with $f = 7$, the column $x = 0$ (steps 5 and 6), before it accepts any cell with $f = 9$ (steps 7 and 8). From step 9 on the frontier has $f = 11$. The search passes the first wall at $(2, 4)$ and, led by the decreasing h , dives down the column $x = 3$ to $(3, 2)$, where the second wall stops it: the four cells $(3, 4)$, $(3, 3)$, $(3, 2)$, $(3, 1)$ are a dead end that the Manhattan distance cannot foresee. Only after these and $(0, 4)$ are exhausted does the frontier reach $f = 13$, and the search crosses the top row $(3, 5)$, $(4, 5)$, $(5, 5)$ and descends to the goal, which is popped at step 21 with $g = 13$. At that moment three cells with $f = 13$ but smaller g are still in OPEN together with $(3, 0)$ at $f = 15$; none of them is ever expanded. Dijkstra's algorithm needs 26 expansions for the same instance, and A* with ties broken towards the *smaller* g needs 24. On the 8-connected version of the grid, A* with the octile heuristic finds a path of cost $9 + 2\sqrt{2} \approx 11.83$ with 24 expansions.

Two things in the table deserve a second look. In step 4 the cell $(0, 2)$ enters OPEN with $g = 3$ (through $(1, 2)$) and $f = 9$; in step 5 the expansion of $(0, 1)$ finds the cheaper route with $g = 2$, the entry is replaced and f drops to 7. This is the relaxation of line 13, and it happens while $(0, 2)$ is still open, so no re-opening is involved. Second, the f of the expanded nodes reads 7, 7, 7, 7, 7, 7, 9, 9, 11, 11, $\dots$, 13: it never decreases. This is not luck; Theorem 4.10 shows that it holds for every consistent heuristic.

4.6 Properties: correctness, optimality, complexity

Throughout this section G is finite, costs are non-negative and Algorithm 4.1 runs with $w = 1$. The proofs follow Hart, Nilsson and Raphael [16, 17] in the form given by Russell and Norvig [35]; Pearl [30] treats the general case.

4.6.1 Termination and completeness

Proposition 4.7 (Termination and completeness). *Algorithm 4.1 terminates. It returns a path if and only if some goal is reachable from s .*

Proof. Every push into OPEN strictly decreases $g(n')$ for some node n' (line 13). The path recorded by the parent pointers of n' is simple: if it returned to n' , the non-negative costs along the cycle would give $g(n) + c(n, n') \geq g(n')$, contradicting the strict decrease. A finite graph has finitely many simple paths, so each node's g can decrease only finitely often, there

are finitely many pushes, and the loop ends. If no goal is reachable, no goal is ever popped and the algorithm returns *failure* when OPEN runs empty. If a goal is reachable, Lemma 4.8 below shows that OPEN is non-empty until a goal is expanded, so the algorithm returns a path. $\square$

4.6.2 Optimality with an admissible heuristic

The proofs rest on one invariant. It concerns a cheapest path to an arbitrary node m ; we apply it with m a goal and later with m an ordinary node.

Lemma 4.8 (An optimal-path node is always open). *Let $P = (s = n_0, n_1, \dots, n_k = m)$ be a cheapest path from s to a node m . At any time before Algorithm 4.1 expands m with $g(m) = g^*(m)$, OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$.*

Proof. Every prefix of a cheapest path is a cheapest path, so $g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$. Let i be the smallest index such that n_i has not yet been expanded while holding $g(n_i) = g^*(n_i)$; the index exists because $m = n_k$ qualifies. If $i = 0$, then s has not been expanded (it always holds $g(s) = 0 = g^*(s)$), so its initial entry is still in OPEN. If $i > 0$, then n_{i-1} was expanded with the optimal value, and at that moment line 13 set $g(n_i) \leq g^*(n_{i-1}) + c(n_{i-1}, n_i) = g^*(n_i)$. Since every g value is the cost of an actual path, $g(n_i) \geq g^*(n_i)$, hence $g(n_i) = g^*(n_i)$ from then on. The push that established this value (either at that relaxation, which also re-opened n_i if it was closed, or earlier) put an entry with $g^*(n_i)$ into OPEN. That entry is never stale, and by the choice of i it has not been popped for expansion; so it is still in OPEN. $\square$

Theorem 4.9 (Optimality of A*; Hart, Nilsson and Raphael 1968). *If h is admissible and a goal is reachable from s , then Algorithm 4.1 (with re-opening, line 15) returns a path of cost C^* .*

Proof. By Proposition 4.7 the algorithm returns a path to some goal γ' ; its cost is $g(\gamma')$ at the moment γ' is popped. Suppose $g(\gamma') > C^*$. Let $P = (s, \dots, \gamma)$ be an optimal solution, of cost C^* . The goal γ has not been expanded with $g(\gamma) = C^*$, since the algorithm would have stopped there. By Lemma 4.8, at the moment γ' is popped OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$. The rest of P from n_i to γ costs $C^* - g^*(n_i)$ and is one particular path to a goal, so $h^*(n_i) \leq C^* - g^*(n_i)$, and admissibility gives

$$f(n_i) = g^*(n_i) + h(n_i) \leq g^*(n_i) + h^*(n_i) \leq C^*.$$

But $f(\gamma') = g(\gamma') + h(\gamma') = g(\gamma') > C^* \geq f(n_i)$, so line 5 would have popped n_i , or some other entry with a smaller key, before γ' . This contradiction shows $g(\gamma') \leq C^*$, and $g(\gamma') \geq C^*$ because it is the cost of a real path. $\square$

The proof used admissibility only in one place: to bound $f(n_i)$ by C^* . It used re-opening, through Lemma 4.8, to make sure that the optimal-path node is in OPEN even if it had been expanded earlier with a worse g . With a consistent heuristic the second precaution is unnecessary.

4.6.3 Consistent heuristics: no node is expanded twice

Theorem 4.10 (Consistent heuristics). *Let h be consistent. Then in Algorithm 4.1:*

- (i) *whenever a node n is expanded, $g(n) = g^*(n)$; hence line 15 is never executed and every node is expanded at most once;*
- (ii) *the f values of the expanded nodes, in order of expansion, are non-decreasing.*

Proof. (i) Suppose n is popped for expansion with $g(n) > g^*(n)$. Let $P = (s, \dots, n)$ be a cheapest path to n . By Lemma 4.8 with $m = n$, OPEN contains a node n_i of P with $g(n_i) =$

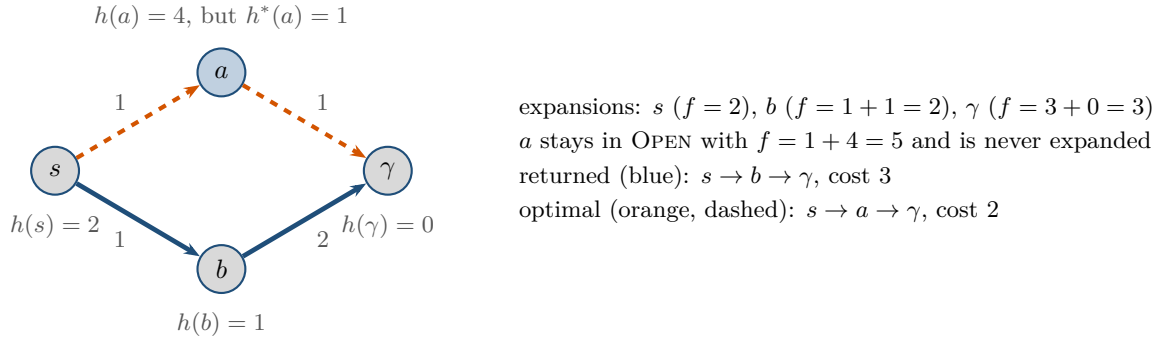


Figure 4.4. An inadmissible heuristic misleads A*. The value $h(a) = 4$ overestimates $h^*(a) = 1$, so a looks worse than the detour through b and is never expanded (blue fill: still in OPEN). A* returns the path of cost 3 although a path of cost 2 exists.

$g^*(n_i)$, and $n_i \neq n$ because $g(n)$ is not optimal. Applying Equation (4.3) along the edges of P from n_i to n , exactly as in the proof of Lemma 4.4, gives $h(n_i) \leq (g^*(n) - g^*(n_i)) + h(n)$. Therefore

$$f(n_i) = g^*(n_i) + h(n_i) \leq g^*(n) + h(n) < g(n) + h(n) = f(n),$$

and line 5 would have chosen n_i instead of n : a contradiction. So every expanded node carries its optimal g , no later path can be cheaper, and a closed node is never re-opened.

(ii) Let n be expanded and let n' be the node expanded next. When n was popped, every other entry in OPEN had $f \geq f(n)$. The expansion of n changes OPEN in two ways: it pushes successors n'' with $f(n'') = g(n) + c(n, n'') + h(n'') \geq g(n) + h(n) = f(n)$ by consistency, and it leaves the other entries unchanged. Hence every entry in OPEN has $f \geq f(n)$ and in particular $f(n') \geq f(n)$. $\square$

Corollary 4.11 (Which nodes A* expands). *With a consistent heuristic, A* expands every node n with $g^*(n) + h(n) < C^*$, expands no node with $g^*(n) + h(n) > C^*$, and may or may not expand nodes with $g^*(n) + h(n) = C^*$ depending on tie-breaking.*

Proof. By Theorem 4.10(ii) and $f(\gamma) = C^*$ at termination, no expanded node has $f > C^*$. Conversely let $g^*(n) + h(n) < C^*$ and let P be a cheapest path to n . Along P the value $g^* + h$ is non-decreasing (consistency), so it is below C^* at every node of P . By induction along P , each node of P is generated with its optimal g by the expansion of its predecessor and, having $f < C^* = f(\gamma)$, is popped before the goal. $\square$

Corollary 4.11 is the reason why the closed set can be a plain “expanded” flag when h is consistent, why the trace in Table 4.2 shows non-decreasing f , and why the f values in Figure 4.3 never exceed 13.

4.6.4 What goes wrong without the conditions

Inadmissible heuristics. Figure 4.4 shows the smallest kind of failure. The heuristic is wrong at a single node, a , where it says 4 while the true cost to go is 1. The search expands s , then b (with $f = 2$), then the goal with $g = 3$, and it stops. The node a sits in OPEN with $f = 5$ and the optimal path $s \rightarrow a \rightarrow \gamma$ of cost 2 is never found. Nothing in the run signals the error: an overestimate makes A* *confidently* wrong, which is why the Manhattan heuristic on an 8-connected grid (Table 4.1) is a real bug and not a matter of taste.

Admissible but inconsistent heuristics. Take the graph from the proof of Lemma 4.4: $s \rightarrow a$ (1), $s \rightarrow b$ (3), $a \rightarrow b$ (1), $b \rightarrow \gamma$ (2), with $h(a) = 3$ and $h = 0$ elsewhere. A* expands s ($f = 0$) and pushes a with $f = 1 + 3 = 4$ and b with $f = 3 + 0 = 3$. It expands b first and

pushes γ with $g = 5$. Next it expands a and finds b with $g = 2 < 3$: b is closed, and line 15 re-opens it. The expansion of b (now $f = 2$) improves γ to $g = 4$, and the goal is returned with the optimal cost 4. The sequence of f values, 0, 3, 4, 2, 4, is not monotone, and b is expanded twice. If re-opening is disabled, the same run returns the path $s \rightarrow b \rightarrow \gamma$ of cost 5. The self-test of `ch04_astar.py` reproduces both runs.

4.6.5 Complexity

With a consistent heuristic every node is expanded at most once and every edge relaxed at most once, so [Algorithm 4.1](#) performs at most $|E|$ pushes and $|V|$ expansions; with a binary heap the running time is $\mathcal{O}((|V| + |E|) \log |V|)$ and the memory $\mathcal{O}(|V| + |E|)$, the same bounds as for Dijkstra’s algorithm in [Chapter 3](#). The bounds are worst cases. What decides the actual effort is the number of expanded nodes, that is, by [Corollary 4.11](#), the number of nodes with $g^*(n) + h(n) < C^*$. With $h = 0$ this is every node cheaper than the goal; with the exact heuristic $h = h^*$ it is empty, and with larger- g -first tie-breaking A^* then expands only the nodes of one optimal path. Between these extremes the effort grows with the error $h^* - h$: on a grid with obstacles every detour that the heuristic does not foresee adds a band of cells with $f < C^*$.

4.6.6 Heuristic dominance

Definition 4.12 (Dominance). A heuristic h_2 **dominates** h_1 if $h_2(n) \geq h_1(n)$ for every node n . If both are admissible, h_2 is the *more informed* of the two.

Theorem 4.13 (More informed, fewer expansions). *Let h_1 and h_2 be consistent and let h_2 dominate h_1 . Every node that A^* with h_2 expands with $f < C^*$ is also expanded by A^* with h_1 . Consequently the number of expansions with h_2 is at most the number with h_1 , up to nodes with $f = C^*$.*

Proof sketch. By [Corollary 4.11](#), A^* with h_2 expands with $f < C^*$ exactly the nodes with $g^*(n) + h_2(n) < C^*$. For such a node, $g^*(n) + h_1(n) \leq g^*(n) + h_2(n) < C^*$, so [Corollary 4.11](#) applied to h_1 says that it is expanded there too. Nodes with $f = C^*$ escape the argument: whether they are expanded depends on tie-breaking, and h_2 may expand a few of them that h_1 skips. Pearl [30] and Dechter and Pearl [6] extend the statement to admissible heuristics and show that in this sense no admissible algorithm can do better than A^* with a consistent heuristic. $\square$

On grids the theorem orders the heuristics of [Table 4.1](#): on an 8-connected grid the octile distance dominates the Euclidean distance, which dominates the Chebyshev distance, and the self-test of `ch04_astar.py` checks the theorem on random grids. The theorem also shows why the maximum of several admissible heuristics is admissible and never worse than any of them, and it gives the rule of thumb for designing heuristics: relax the problem (drop the obstacles, drop a constraint), solve the relaxed problem exactly, and use its cost. That is precisely how the Manhattan and octile distances arise.

4.7 Variants and extensions

4.7.1 Tie-breaking

[Corollary 4.11](#) leaves the nodes with $f = C^*$ to the tie-breaking rule, and near the goal there are many of them: every node on an optimal path has $g^*(n) + h^*(n) = C^*$, and with an exact heuristic all of them tie. On an obstacle-free grid the Manhattan distance is exact, and every cell of the 20×20 grid in [Figure 4.5](#) lies on some optimal path from the bottom-left to the

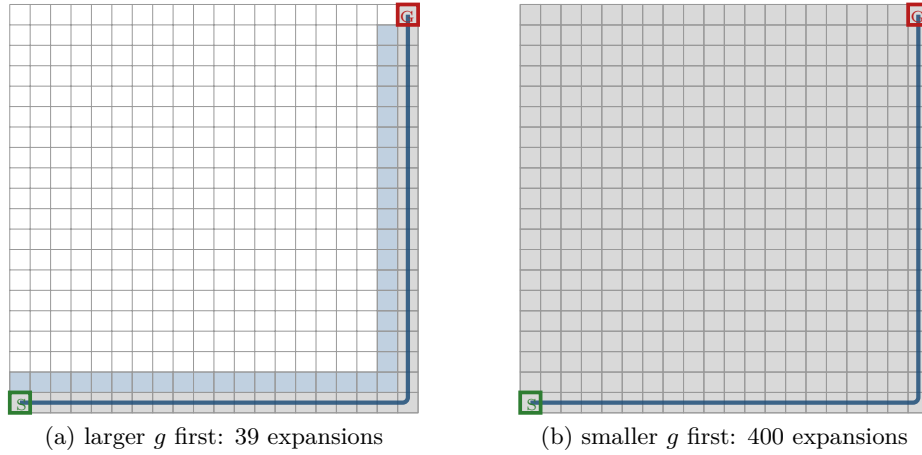


Figure 4.5. Tie-breaking on an empty 20×20 grid with the Manhattan heuristic. Every cell has $f = 38 = C^*$. (a) Breaking ties towards the larger g , A^* expands only the 39 cells of one optimal path. (b) Breaking ties towards the smaller g , it expands all 400 cells. Both runs return a path of cost 38.

top-right corner: all 400 cells have $f = 38$. Which of them are expanded is entirely up to the tie-breaking rule. Preferring the larger g (equivalently, the smaller h) means: among nodes that look equally good, take the one that is furthest along. The search then behaves like a depth-first dive along one optimal path and expands $2 \cdot 20 - 1 = 39$ cells. Preferring the smaller g , or breaking ties first-in-first-out without looking at g , expands all 400 cells before the goal is popped. This is why [Algorithm 4.1](#) uses the key $(f, -g)$ and why the implementation in ?? adds a counter as a third component: the counter keeps the order deterministic and stops Python from comparing nodes when the first two components tie. In the worked example the rule saves three of 24 expansions; in the last stretch of any search with a good heuristic it saves far more.

4.7.2 Weighted A^*

Pohl [32] proposed to inflate the heuristic: **weighted A^*** runs [Algorithm 4.1](#) with the key

$$f_w(n) = g(n) + w h(n), \quad w \geq 1. \quad (4.8)$$

For $w = 1$ this is A^* ; as $w \rightarrow \infty$ the g term becomes irrelevant and the search turns into *greedy best-first search*, which always expands the node that seems closest to the goal. In between, a larger w makes the search more greedy: it trusts the heuristic more, expands fewer nodes and accepts a longer path. The price is bounded.

Theorem 4.14 (Bounded suboptimality of weighted A^*). *If h is admissible and a goal is reachable, weighted A^* with re-opening returns a path of cost at most $w \cdot C^*$.*

Proof. [Lemma 4.8](#) did not use the heuristic at all, so it holds for the key [Equation \(4.8\)](#): when the goal γ' is popped, OPEN contains a node n_i of an optimal path P with $g(n_i) = g^*(n_i)$. As in the proof of [Theorem 4.9](#), $h(n_i) \leq h^*(n_i) \leq C^* - g^*(n_i)$, hence

$$f_w(n_i) = g^*(n_i) + w h(n_i) \leq g^*(n_i) + w (C^* - g^*(n_i)) \leq w C^*,$$

where the last step uses $w \geq 1$ and $g^*(n_i) \geq 0$. The goal was popped before n_i , so $g(\gamma') = f_w(\gamma') \leq f_w(n_i) \leq w C^*$. $\square$

Note that wh is in general not admissible and not consistent, even when h is, so weighted A^* may re-open nodes and [Theorem 4.10](#) does not apply. Likhachev, Gordon and Thrun [27]

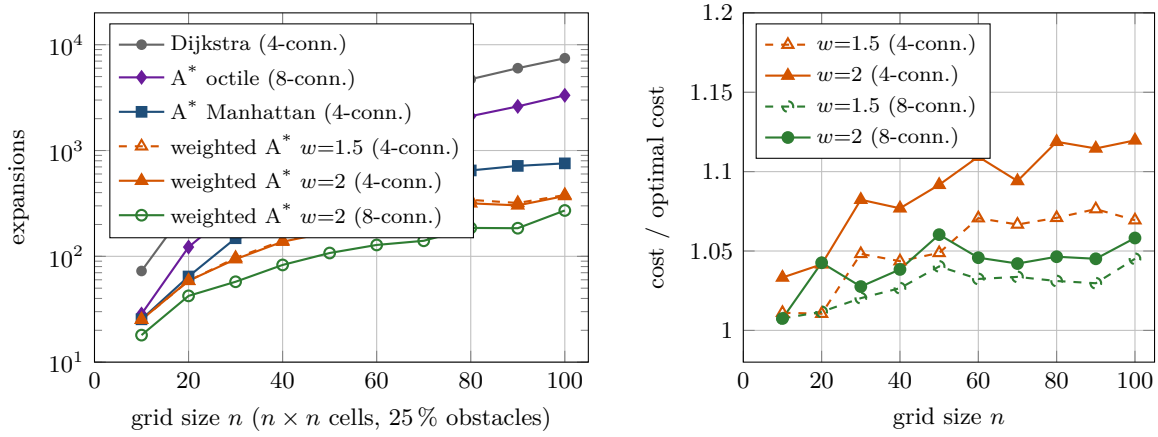


Figure 4.6. Left: mean number of expansions (log scale) on random $n \times n$ grids; Dijkstra’s algorithm grows with the number of free cells, A* grows much more slowly, and the weight w cuts the effort again. Right: the mean ratio between the cost of the path returned by weighted A* and the optimal cost; the guaranteed bounds are 1.5 and 2, the observed ratios stay below 1.13.

showed that the bound wC^* survives even when re-opening is switched off, and that observation is the basis of ARA* in Chapter 6, which decreases w step by step while reusing the search. The focal search of ECBS in Chapter 10 obtains the same guarantee by a different mechanism.

4.7.3 An experiment on random grids

Figure 4.6 compares the effort of the searches in this chapter on random grids with 25% obstacles (script `gen_ch04_expansions.py`, ten instances per size, start and goal in opposite corners). Three observations. First, Dijkstra’s algorithm expands nearly every free cell (7452 of 7513 at $n = 100$), while A* with the Manhattan heuristic on the 4-connected grid expands 756 on average: the heuristic removes about 90% of the work. Second, on the 8-connected grid A* with the octile heuristic still expands 3324 cells. Diagonal moves make detours around obstacles cheap, so many cells have $g^* + h < C^*$; the error $h^* - h$, not the connectivity, is what the search pays for. Third, weighted A* with $w = 2$ needs 372 expansions on the 4-connected and 271 on the 8-connected grid, while its paths are on average only 12% and 6% longer than optimal, far inside the guaranteed factor of 2; with $w = 1.5$ the average excess is 7% and 5%. Bounded suboptimality is cheap on these maps, which is why ECBS (Chapter 10) is so much faster than CBS.

4.7.4 Other relatives

A* has a large family. LPA* and D* Lite (Chapter 5) reuse the search when edge costs change; ARA* (Chapter 6) is weighted A* with a decreasing weight; focal search (Chapter 10) keeps a second queue of nodes whose f is within a factor w of the best and picks among them by a different criterion. Jump point search [15] exploits the symmetry of uniform grids to skip most cells, and safe interval path planning [31] compresses the time dimension of the search described next. All of them keep the structure of Algorithm 4.1; what changes is the key, the successor function or what is remembered between searches.

Incremental Search: LPA* and D* Lite

Learning objectives

- Explain why re-running A* after a small map change wastes work, and what an *incremental* search reuses instead.
- Define the *g*- and *rhs*-values of LPA*, local consistency, over- and underconsistency, and the two-component priority key, and trace LPA* by hand on a small grid.
- Explain why D* Lite searches backward from the goal, what the key modifier k_m repairs when the robot moves, and reproduce its pseudocode.
- State the conditions under which D* Lite returns the same path costs as re-running A*, and bound its number of vertex expansions.
- Implement D* Lite in Python, change obstacles while the agent executes its plan, and measure the saving relative to re-running A*.

5.1 Why this matters for a drone swarm

Picture one drone of a swarm on its nominal route through a warehouse. The route was computed by the global planner (Chapters 9 and 10) before take-off, on a grid of 200×200 cells at ten flight levels. Halfway through the mission the prediction layer (Chapters 18 and 20) announces that an unknown drone will cross the corridor ahead within the next few seconds. The local layer (Chapters 13 and 14) can dodge for a moment, but the intruder is going to hover in the corridor, and the local manoeuvre cannot return the drone to its nominal route. Now the replanning layer must deliver a new route from the drone's *current* position, and it has one control cycle, say 50 ms, to do so.

Re-running A* (Chapter 4) from scratch is the obvious answer. It is also wasteful: of the 400 000 cells of the map, only a few dozen have changed. Every distance that A* computed last time is still valid, except in a small region around the intruder. An *incremental* search keeps the old search results and repairs only what the change invalidated. D* Lite, the algorithm of this chapter, does exactly that, and on top of it handles the fact that the drone keeps moving while it replans. It is the replanning algorithm of the hybrid architecture in Chapter 24, and the training plan rates it **Essential**; its conceptual parent LPA* is rated **Medium**.

The chapter proceeds in two halves. The first half develops LPA*, which repairs a shortest path from a fixed start to a fixed goal when edge costs change. It introduces the two numbers stored per vertex (*g* and *rhs*), the notion of local consistency, and the two-component key that orders the work. The second half turns LPA* into D* Lite by reversing the search direction and adding the key modifier k_m , so that the start vertex may move. Both halves have a fully worked grid example whose numbers come from the chapter's Python code, and the chapter

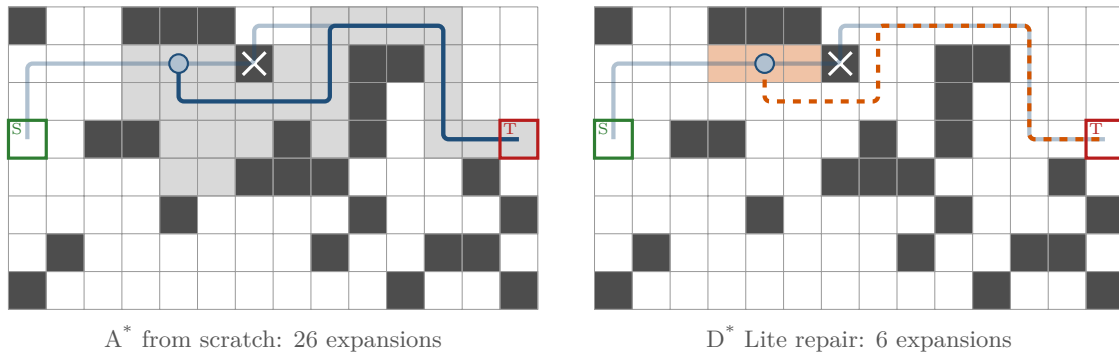


Figure 5.1. An obstacle (black, with a cross) appears two cells ahead of the drone (blue disc) on its planned route (faded blue). Left: A* re-run from scratch from the drone’s cell expands all grey cells. Right: D* Lite expands only the orange cells, those whose goal distance was invalidated or is needed for the detour (dashed). The number of expansions is printed under each panel.

ends with an experiment that compares D* Lite with repeated A* on a 50×50 grid as obstacles appear one by one.

5.2 The idea in plain words

Figure 5.1 shows the situation of the motivation on a small grid. The drone has a shortest path to its goal and has already flown a third of it when the cell two steps ahead of it turns out to be blocked. In the left panel, A* is re-run from scratch from the drone’s cell: it expands every cell shaded grey, 26 of them, most of which lie far from the obstacle and whose distance to the goal did not change at all. In the right panel, an incremental search touches only the orange cells, four cells in six expansions: the ones whose stored distance became stale because the obstacle removed the edge they relied on, plus the few cells needed to find the detour.

How does the incremental search know which cells are stale? It stores *two* numbers per vertex instead of one. The first, $g(s)$, is the distance value from the last search, exactly as in A*. The second, $rhs(s)$, is what the neighbours of s currently say the value *should* be: one step of lookahead, “the best neighbour’s g plus the edge cost”. As long as the two numbers agree, nothing at s needs attention. When an edge cost changes, the rhs -value of the affected vertex changes but its g -value does not, and the disagreement flags the vertex as *locally inconsistent*. Inconsistent vertices go into a priority queue, ordered by the same kind of key that A* uses ($g + h$), and are processed in that order. Processing a vertex makes it consistent and may make its neighbours inconsistent, so the repair spreads outward from the change, but only as far as distance values actually differ, and only until the goal is consistent and no queued key is smaller than the goal’s key. Cells far from the change, or cells whose distance happens to stay the same, are never touched.

Key idea

Store, for every vertex, the value from the last search (g) next to the value its neighbours currently imply (rhs). Only vertices where the two disagree need work. Process them in A* order, from the change outward, and stop as soon as the query vertex agrees with its neighbours and nothing with a smaller key is left. The result equals what A* would compute from scratch, at a fraction of the expansions when the change is small.

D* Lite adds one more idea. A moving robot changes the *start* of the search at every step. If distance values were measured from the start, every one of them would change with

every step. Distances *to the goal* do not change when the robot moves. So D* Lite searches backward from the goal, and the robot's motion only affects the heuristic part of the keys, which a single running offset k_m can absorb.

5.3 Problem statement and notation

We use the graph notation of Chapters 2 and 3: a finite directed graph $G = (V, E)$ with edge costs $c(u, v) \in (0, \infty]$ for $(u, v) \in E$, a start vertex s_{start} and a goal vertex s_{goal} . $\text{Pred}(s)$ and $\text{Succ}(s)$ are the predecessors and successors of s . A cost of ∞ means that the edge is currently unusable; we write $c(u, v) = \infty$ also for pairs that are not adjacent, which keeps formulas uniform. The true distance from u to v under the current costs is $\text{dist}(u, v)$, and $\text{dist}(u, v) = \infty$ if no path exists.

Definition 5.1 (Incremental shortest-path problem). An **incremental shortest-path problem** is a sequence of shortest-path queries between the same two vertices s_{start} and s_{goal} on a graph whose edge costs change between queries. An **incremental search algorithm** answers each query using information saved from the previous queries, and it must return a path of cost $\text{dist}(s_{\text{start}}, s_{\text{goal}})$ under the costs current at the time of the query.

The grids of this chapter are 4-connected with unit costs (Chapter 2). A cell is either *free* or *blocked*. Blocking a cell b is modelled as an edge-cost change: *every* edge that enters b and every edge that leaves b changes from cost 1 to cost ∞ . Unblocking reverses the change. We keep blocked cells as vertices of the graph, with all their edges at ∞ , rather than deleting them; that way the graph structure is fixed and only the costs vary, which is exactly what the algorithms assume. The heuristic on these grids is the Manhattan distance, which is consistent (Chapter 4).

LPA* stores two values per vertex.

Definition 5.2 (g -value and rhs -value). For every vertex $s \in V$, $g(s) \in [0, \infty]$ is the **g -value**, an estimate of $\text{dist}(s_{\text{start}}, s)$ that is only changed when s is expanded. The **rhs -value** (“right-hand side”, after the Bellman equation it comes from) is the one-step lookahead

$$rhs(s) = \begin{cases} 0 & \text{if } s = s_{\text{start}}, \\ \min_{s' \in \text{Pred}(s)} (g(s') + c(s', s)) & \text{otherwise.} \end{cases} \quad (5.1)$$

The rhs -value is always kept up to date with respect to the current g -values of the predecessors and the current edge costs; the g -value is allowed to lag behind. The gap between them is the central concept.

Definition 5.3 (Local consistency). A vertex s is **locally consistent** if $g(s) = rhs(s)$, and **locally inconsistent** otherwise. An inconsistent vertex is **overconsistent** if $g(s) > rhs(s)$ (its neighbours now offer a cheaper way to reach it than it knows) and **underconsistent** if $g(s) < rhs(s)$ (it believes in a cost that its neighbours can no longer support).

A freshly discovered vertex has $g = \infty$ and a finite rhs , so it is overconsistent; this is the only kind of inconsistency A* ever meets. Underconsistent vertices appear when a cost *increases*, for example when an obstacle appears on the path: the vertex behind the obstacle still holds its old, now too optimistic g -value. The following proposition says why consistency is the right target.

Proposition 5.4 (Consistency everywhere means correct distances). *Suppose all edge costs are positive. If every vertex is locally consistent, then $g(s) = \text{dist}(s_{\text{start}}, s)$ for all $s \in V$, and a shortest path from s_{start} to any reachable s is obtained by moving backward from s to a predecessor s' that attains the minimum in Equation (5.1), until s_{start} is reached.*

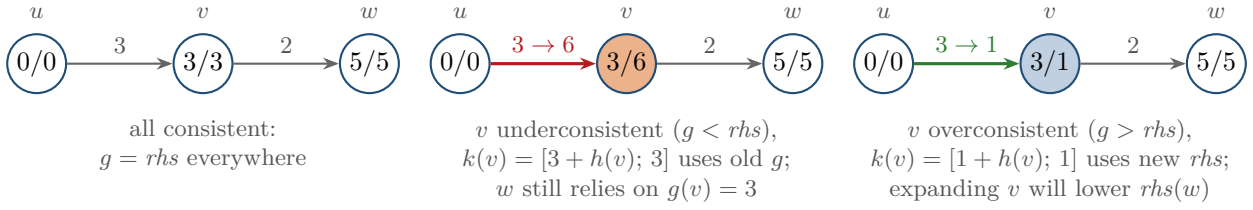


Figure 5.2. Local consistency on a three-vertex fragment (edge costs on the arrows, g/rhs inside the vertices). Left: all consistent. Middle: the edge into v became expensive, so $rhs(v)$ rose and v is underconsistent; its key uses the old $g(v) = 3$ and puts v before w , whose rhs still relies on $g(v)$. Right: a new cheap edge into v makes v overconsistent; its key uses $rhs(v) = 1$.

Proof. If all vertices are consistent, the g -values satisfy $g(s_{\text{start}}) = 0$ and $g(s) = \min_{s' \in \text{Pred}(s)} (g(s') + c(s', s))$ for all other s : the Bellman equations of the shortest-path problem. With positive edge costs, the true distances are the unique solution of these equations (Chapter 3), so $g \equiv \text{dist}(s_{\text{start}}, \cdot)$. Following a minimising predecessor decreases g by exactly the edge cost at every step, so the walk terminates at s_{start} and has cost $g(s)$. $\square$

LPA* does not make *all* vertices consistent; like A* it stops early. What it guarantees is that the goal is consistent and that no queued vertex could still improve the goal, and that suffices for the goal's g -value and the path traced from it to be correct (Theorem 5.8). To decide the order of work it uses a key with two components.

Definition 5.5 (Priority key). The **key** of a vertex s is the pair

$$k(s) = [k_1(s); k_2(s)] = [\min(g(s), rhs(s)) + h(s); \min(g(s), rhs(s))], \quad (5.2)$$

where $h(s)$ estimates $\text{dist}(s, s_{\text{goal}})$ and is nonnegative and consistent: $h(s_{\text{goal}}) = 0$ and $h(s) \leq c(s, s') + h(s')$ for all $(s, s') \in E$. Keys are compared **lexicographically**: $k \leq k'$ iff $k_1 < k'_1$, or $k_1 = k'_1$ and $k_2 \leq k'_2$.

Read k_1 as the f -value of A* and k_2 as its g -value used for tie-breaking, both computed from the *smaller* of the two stored values. For an overconsistent vertex the smaller value is rhs , the tentative new distance, exactly as in A*. For an underconsistent vertex it is the old g , which is too small. Using the too-small value on purpose puts the vertex *early* in the queue, before every vertex whose value depends on it, so that the stale value is retracted before it can be propagated further. Figure 5.2 illustrates the three states.

The priority queue U holds exactly the locally inconsistent vertices, each with the key it had when it was last inserted. $U.\text{TopKey}()$ returns the smallest key in U (or $[\infty; \infty]$ if U is empty), $U.\text{Pop}()$ removes and returns the vertex with that key, $U.\text{Insert}(s, k)$ inserts s with key k , and $U.\text{Remove}(s)$ removes s .

5.4 The algorithm: LPA*

Algorithm 5.1 is Lifelong Planning A* (LPA*) of Koenig, Likhachev and Furcy [23], in the form of the original paper. It searches *forward* from s_{start} ; g -values are start distances, and rhs -values look at predecessors.

Walkthrough. Initialize sets every g and rhs to ∞ except $rhs(s_{\text{start}}) = 0$ (line 6); the start is thus the only inconsistent vertex and the only entry of U , just as it is the only entry of OPEN at the start of A*.

`UpdateVertex( $u$ )` is the maintenance routine. Line 9 recomputes $rhs(u)$ from Equation (5.1); the start keeps $rhs = 0$ forever. Lines 10–11 then restore the invariant “ U contains exactly the inconsistent vertices, with current keys”: u is taken out, and put back with a fresh key only if it is inconsistent. Every path of the algorithm that touches a g -value or an edge cost calls `UpdateVertex` on the vertices whose rhs could have changed, and nothing else ever modifies rhs .

`ComputeShortestPath` is the search proper. Its loop condition (line 13) mirrors the termination of A^* : continue while some queued vertex has a key smaller than the goal’s, or while the goal itself is inconsistent. Each iteration pops the vertex u with the smallest key (line 14) and treats the two kinds of inconsistency differently:

- *Overconsistent* ($g(u) > rhs(u)$, line 15): accept the better value, $g(u) \leftarrow rhs(u)$. This is the expansion of A^* . The successors of u may now have a better predecessor, so they are updated (line 17). After this step u is consistent.
- *Underconsistent* ($g(u) < rhs(u)$, line 19): the old value cannot be trusted, so it is thrown away, $g(u) \leftarrow \infty$. Now every successor that computed its rhs through u must recompute it, and u itself must recompute its own rhs and re-enter the queue if it is not ∞ (line 20, note the $\cup \{u\}$). After this step u is either consistent (both ∞) or overconsistent with a fresh, larger key, and will be expanded a second time later as an ordinary A^* expansion.

`Main` glues the pieces together: after each search, wait for the environment to report changed edges, update their costs, and call `UpdateVertex` on the *head* v of every changed edge (line 28), because it is v whose rhs depends on $c(u, v)$. Then search again. The first call of `ComputeShortestPath` is exactly an A^* search (Theorem 5.9); each later call is a repair.

Why the two-component key works. Consider what the loop condition must guarantee: when it stops, the goal’s g -value must equal the true distance. The first key component $k_1 = \min(g, rhs) + h$ is a lower bound on the cost of any path to the goal through the queued vertex, so as long as $\text{TopKey}() \geq k(s_{\text{goal}})$ no queued vertex can lead to a cheaper path to the goal than the goal’s own value, exactly the argument that proves A^* optimal with a consistent heuristic (Chapter 4). The second component breaks ties in favour of *smaller* $\min(g, rhs)$, which matters more here than in A^* : among vertices with equal k_1 , an underconsistent vertex u with a small old $g(u)$ must be processed before a successor w whose $rhs(w) = g(u) + c(u, w)$ was derived from it. Because $c(u, w) > 0$, $k_2(u) < k_2(w)$ whenever $k_1(u) = k_1(w)$, so the stale value is withdrawn first and w is recomputed from correct information. Without the second component, w could be expanded with a value that is about to be retracted, and would have to be expanded a third time.

Comparing keys as single numbers

A key is a pair, and both components matter. Comparing only k_1 (or, worse, only k_2) breaks the ordering argument above and can make LPA* expand vertices with stale values, or terminate with a wrong $g(s_{\text{goal}})$. In Python, store keys as tuples (`k1`, `k2`); tuple comparison is lexicographic, which is what the algorithm needs. Also never compare keys for equality with floating-point tolerance tricks: the algorithm only needs $<$ and $\leq$.

5.5 A worked example: LPA* repairs a path

Example 5.6 (An obstacle appears on the path). Figure 5.3 shows a 5×3 grid of free cells. Cells are named (x, y) with x the column from the left and y the row from the bottom, both starting at 0. The start is $S = (0, 1)$, the goal is $T = (4, 1)$, moves are 4-connected with unit cost, and h is the Manhattan distance to T . Ties between equal keys are broken toward the

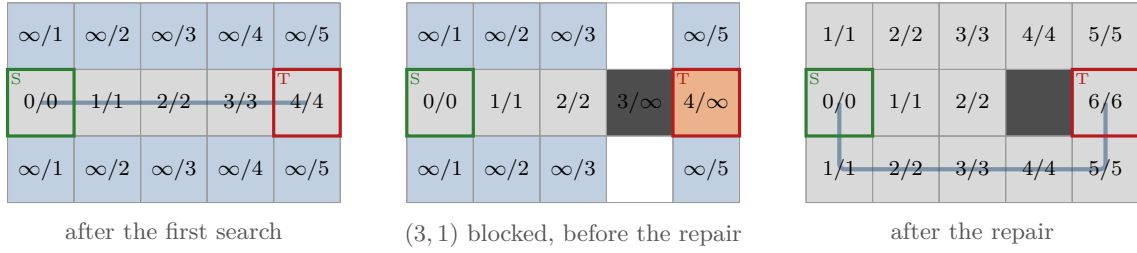


Figure 5.3. LPA* on Example 5.6. Each cell shows g/rhs ; grey cells are locally consistent with finite values (expanded), blue cells are overconsistent and orange cells underconsistent (both kinds are in the queue U); cells without numbers have $g = rhs = \infty$. Left: after the first search, path of cost 4. Middle: right after cell (3, 1) is blocked and `UpdateVertex` has run on the affected vertices, before `ComputeShortestPath`. Right: after the repair, the new path of cost 6.

smaller x , then the smaller y . First LPA* computes a shortest path. Then cell (3, 1), which lies on the path, becomes blocked, and LPA* repairs the path.

The first search. Table 5.1 traces the first call of `ComputeShortestPath`. Every popped vertex is overconsistent, so this is an A* search: the cells of the middle row are expanded one after the other, each with key $[4; x]$, because for a cell on the straight line $g + h = 4$. The side cells receive finite rhs -values and enter the queue with $k_1 = 6$, but they are never expanded: after T is popped and made consistent, the smallest queued key is $[6; 1]$, which is not smaller than $k(T) = [4; 4]$, and the loop stops. Five expansions, ten cells left in the queue with correct rhs -values but $g = \infty$.

The change. Blocking (3, 1) changes eight directed edges: the four that enter (3, 1) and the four that leave it. `Main` calls `UpdateVertex` on the head of each. For the entering edges the head is (3, 1) itself: all its predecessors are now at cost ∞ , so $rhs(3, 1) = \infty$ while $g(3, 1) = 3$; the cell is underconsistent and enters the queue with key $[3 + 1; 3] = [4; 3]$. For the leaving edges the heads are the neighbours. $T = (4, 1)$ loses its only finite predecessor, $rhs(T) = \infty$ against $g(T) = 4$: underconsistent, key $[4; 4]$. (2, 1) still has S 's side as predecessor and keeps $rhs = 2$. (3, 0) and (3, 2) had $rhs = 4$ through (3, 1); now their only finite predecessor is gone, $rhs = \infty = g$, and they leave the queue. The middle panel of Figure 5.3 shows this state: two underconsistent cells, eight queued cells, and every g -value untouched.

The repair. Table 5.2 traces the second call. The two underconsistent cells come first, because their keys use the old, small g -values. Popping (3, 1) sets $g(3, 1) = \infty$; its successors are updated but nothing changes, since their rhs -values already ignored (3, 1). Popping T sets $g(T) = \infty$, and now the cells (4, 0) and (4, 2), whose $rhs = 5$ came through T , also drop out of the queue. From here on the search is a plain A* over the side rows: the queued cells are expanded in order of their keys, all with $k_1 = 6$ and increasing k_2 , the detour $(2, 0) \rightarrow (3, 0) \rightarrow (4, 0)$ is discovered, and T is finally re-expanded with $g(T) = 6$. The repair costs 13 expansions, of which the first two are underconsistent. Compare this with A* from scratch on the changed grid, which expands only 7 cells: on this small instance the repair costs almost twice as much as starting over. Two effects are at work. The stale values must be retracted first, which costs two expansions that A* does not have. And the key order expands every queued cell with $k_1 = 6$ in order of increasing g , both side rows in full, whereas A* breaks ties among equal f -values toward the larger g , the usual rule on grids (Chapter 4), and runs down one side row straight to the goal. What LPA* reuses are the g -values of S , (1, 1) and (2, 1), which are never expanded again; on a 5×3 grid that is not much. Section 5.8.1 shows

that on larger maps, when the change is local, the reused part dominates, and [Section 5.8](#) returns to the tie-breaking.

The path is read off at the end by walking backward from T : from each cell move to the predecessor that attains the minimum in [Equation \(5.1\)](#), ties broken toward the smaller coordinates. In the right panel of [Figure 5.3](#) this gives $T \rightarrow (4, 0) \rightarrow (3, 0) \rightarrow (2, 0) \rightarrow (1, 0) \rightarrow (0, 0) \rightarrow S$, the blue path of cost 6. Several other paths, through $(2, 1)$ and $(1, 1)$ or along the top row, have the same cost; the tie-break selects this one.

5.6 The algorithm: D* Lite

LPA* assumes that start and goal stay fixed. A drone that executes its plan moves, and every step moves the start. D* Lite [\[21, 22\]](#) is LPA* adapted to a moving start with two changes: the search runs backward from the goal, and a key modifier k_m keeps the priority queue valid across moves.

5.6.1 Reversing the search

Suppose we kept the forward search of LPA* and simply moved s_{start} one cell after each step. The g -values are distances *from the start*; after the move, every one of them is off by the length of the step (or more), so every vertex would become inconsistent and the “repair” would redo the whole search. Distances *to the goal*, on the other hand, are unaffected by where the robot is. D* Lite therefore lets $g(s)$ estimate $\text{dist}(s, s_{\text{goal}})$ and searches from the goal toward the robot. Everything in [Section 5.4](#) is mirrored, as [Table 5.3](#) summarises:

- the *rhs*-value looks at *successors*: $\text{rhs}(s) = \min_{s' \in \text{Succ}(s)} (c(s, s') + g(s'))$ for $s \neq s_{\text{goal}}$, and $\text{rhs}(s_{\text{goal}}) = 0$;
- the queue is initialised with s_{goal} , and expanding a vertex updates its *predecessors*;
- when the cost of edge (u, v) changes, it is the *tail* u whose *rhs* depends on it, so $\text{UpdateVertex}(u)$ is called;
- the heuristic $h(s_{\text{start}}, s)$ estimates $\text{dist}(s_{\text{start}}, s)$, the distance from the robot to s , and the termination test compares against $k(s_{\text{start}})$;
- the path is read off *forward* from the robot: from s , move to the successor s' that minimises $c(s, s') + g(s')$.

The heuristic must now be consistent with respect to the *start*: nonnegative, $h(s_{\text{start}}, s_{\text{start}}) = 0$ and $h(s_{\text{start}}, s) \leq h(s_{\text{start}}, s') + c(s', s)$ for all s and $s' \in \text{Pred}(s)$. In addition, because the start will move, we require the triangle inequality between any three vertices, $h(a, c) \leq h(a, b) + h(b, c)$; this is what the key modifier relies on. The Manhattan distance on a 4-connected unit-cost grid satisfies all of these (it is a metric that never exceeds the true distance), and so does the octile distance on an 8-connected grid.

Wrong heuristic direction

In D* Lite the heuristic argument is the *robot's* position, not the goal: $h(s_{\text{start}}, s)$. A common slip is to keep the A* habit and use the distance from s to the goal. The search then still terminates and often still finds a path, because the heuristic only orders the queue, but it explores the wrong region (around the goal instead of around the robot), can violate the consistency condition relative to the start, and destroys the lower-bound property that the loop test and the key modifier rely on. Whenever you swap the search direction, swap the heuristic with it.

5.6.2 The key modifier k_m

Reversing the search removes the robot's position from the g -values, but not from the keys: $k_1(s) = \min(g(s), rhs(s)) + h(s_{\text{start}}, s)$ contains the distance from the robot to s . After the robot moves, every key stored in the queue is computed with the old start. The obvious fix, recomputing all keys and rebuilding the heap after every step, costs time proportional to the queue size at every step, which is exactly the kind of work we set out to avoid.

D* Lite avoids the rebuild with an observation. Let the robot move from s_{old} to s_{new} . By the triangle inequality,

$$h(s_{\text{old}}, s) \leq h(s_{\text{old}}, s_{\text{new}}) + h(s_{\text{new}}, s) \quad \text{for every vertex } s, \quad (5.3)$$

so every old key is at most the new key plus the constant $h(s_{\text{old}}, s_{\text{new}})$. Instead of lowering the old keys, D* Lite *raises* all keys computed from now on by that constant: it keeps a running sum k_m , adds $h(s_{\text{old}}, s_{\text{new}})$ to it whenever edge costs are processed after a move, and defines

$$k(s) = \lceil \min(g(s), rhs(s)) + h(s_{\text{start}}, s) + k_m; \min(g(s), rhs(s)) \rceil. \quad (5.4)$$

With this definition every key that sits in the queue is a *lower bound* on the key that `CalculateKey` would return for the same vertex now. Lower bounds are harmless as long as the algorithm checks them before acting: when a vertex is popped whose stored key is smaller than its current key, D* Lite simply reinserts it with the current key and pops again (lines 15–18 of Algorithm 5.2). Because keys are only ever compared with one another and with $k(s_{\text{start}})$, which also contains k_m , adding the same constant to all new keys changes no comparison among them.

Table 5.4 gives numbers. Vertex u was inserted before the move with key $[4 + 6; 4] = [10; 4]$. The robot then moves two cells toward u , so its heuristic drops to 4 and the correct key becomes $[8; 4]$. A new vertex v is inserted after the move. Without the modifier it gets $[6 + 3; 6] = [9; 6]$ and sits *above* u in the queue, although u should be processed first ($[8; 4] < [9; 6]$). The search would expand v using a stale rhs -value that may still depend on u , and the loop could even terminate with u buried under an inflated key. With $k_m = 2$ the new vertex gets $[11; 6]$, the stored key $[10; 4]$ of u is exactly its current key $[4 + 4 + 2; 4]$, and the order is correct. In general the stored key is only a lower bound, and the reinsert check repairs the rest.

5.6.3 Pseudocode

Algorithm 5.2 is the final, optimised version of D* Lite as published by Koenig and Likhachev [21]. Compared with LPA*, read Succ for Pred and vice versa, note the k_m term in the key, the reinsert check, and the main loop that moves the robot.

Walkthrough. `Initialize` seeds the queue with the goal (line 8), the mirror image of LPA*'s start. `UpdateVertex` recomputes $rhs(u)$ over the successors of u (line 10); the goal keeps $rhs = 0$.

`ComputeShortestPath` runs until the robot's vertex is consistent and no queued key is smaller than the robot's key (line 14). It records the top key before popping (line 15) and compares it with the freshly computed key. If the stored key was a strict lower bound, the vertex is reinserted with its current key and is *not* expanded (line 18); this costs one heap operation, never an expansion. Otherwise the vertex is expanded as in LPA*, with predecessors in place of successors (lines 20–24).

`Main` is the execution loop. After the initial search (line 28), the robot repeatedly takes the greedy step toward the successor with the smallest $c + g$ (line 31); by Theorem 5.8 this follows a shortest path. Then it looks for cost changes (line 33). If there are none, it keeps

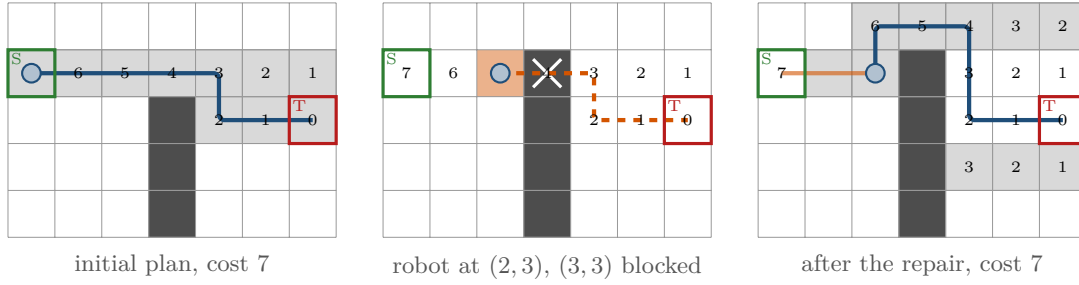


Figure 5.4. D* Lite on Example 5.7. Numbers are g -values (goal distances); blank cells were never touched. Left: the initial search from T and the planned path. Middle: the robot (blue disc) has moved two steps and $(3, 3)$ becomes blocked; orange cells are the ones `UpdateVertex` made inconsistent, the old path is dashed. Right: after the repair, with the cells expanded by the repair shaded and the new path; the robot's traversed route is drawn in orange.

walking without touching the queue, and s_{last} stays where it was. If there are changes, the key modifier is increased by the heuristic distance from s_{last} , the position at which the queue keys were last valid, to the current position (line 35); note that this is done once per batch of changes, not once per step, which is why s_{last} is needed. Then every changed edge is processed at its tail (line 39) and the search is repaired (line 40). Line 30 is the exit when no path exists: the drone must then hover, land or ask the swarm planner for a new goal.

Forgetting the vertex itself on underconsistency

Line 24 updates $\text{Pred}(u) \cup \{u\}$, not just $\text{Pred}(u)$ (and line 20 of LPA* updates $\text{Succ}(u) \cup \{u\}$). After $g(u)$ is set to ∞ , u itself is usually overconsistent again, with a finite rhs through another neighbour, and must go back into the queue to be expanded properly later. Dropping the $\cup \{u\}$ leaves u inconsistent but absent from the queue, and the vertices behind it never receive the corrected value: the robot follows a path through a cost that no longer exists. The symmetric mistake, updating only u and forgetting its predecessors (successors in LPA*), leaves the vertices that depended on u with stale rhs -values.

5.7 A worked example: the robot moves and an obstacle appears

Example 5.7 (D* Lite with a moving robot). Figure 5.4 shows a 7×5 grid. A wall occupies $(3, 0)$, $(3, 1)$ and $(3, 2)$, leaving a gap of two cells at the top of column 3. The robot starts at $S = (0, 3)$ and the goal is $T = (6, 2)$; the heuristic is the Manhattan distance from the robot's current cell. D* Lite plans, the robot takes two steps, and then cell $(3, 3)$, the lower cell of the gap and the next cell on the robot's path, becomes blocked. D* Lite repairs the plan and the robot continues through the remaining gap cell $(3, 4)$.

Table 5.5 lists the events. The initial search (Algorithm 5.2, line 28) expands 10 cells, working outward from T until S is consistent and no queued key is smaller than $k(S) = [7; 7]$, and finds the path of cost 7 through the gap. The g -values in the left panel are goal distances; cells off the corridor between T and S were never expanded and keep $g = \infty$, although several of them sit in the queue with finite rhs -values and keys $[9; \cdot]$, which will matter in a moment.

The robot moves to $(1, 3)$ and then to $(2, 3)$ without any queue work: no edge cost changed, so lines 35–40 are skipped, and the remaining cost read off at the robot's cell falls from 7 to 6 to 5. Then $(3, 3)$ becomes blocked. Line 35 first sets $k_m = h((0, 3), (2, 3)) = 2$: every key computed from now on is inflated by 2, and the keys already in the queue, computed when

the robot stood at S , become lower bounds. Then the eight directed edges of $(3, 3)$ change cost and **UpdateVertex** runs on their tails. For $(3, 3)$ itself all outgoing edges are now ∞ , so $rhs(3, 3) = \infty$ while $g(3, 3) = 4$: underconsistent, key $[4 + 1 + 2; 4] = [7; 4]$. The robot's own cell $(2, 3)$ loses its best successor; its rhs rises from 5 to 7 through $(1, 3)$, so it is underconsistent too, with key $[5 + 0 + 2; 5] = [7; 5]$. The gap cell $(3, 4)$ was queued with $rhs = 5$ through $(3, 3)$ and now has $rhs = \infty = g$, so it leaves the queue. $(4, 3)$ keeps $rhs = 3$ through $(4, 2)$, and the wall cell $(3, 2)$ stays at ∞ . Three vertices changed, as the middle panel of Figure 5.4 shows.

Table 5.6 traces the repair, which expands 12 vertices. The two underconsistent cells come first. Retracting $g(3, 3)$ changes nothing else, since no rhs -value still relies on it. Retracting $g(2, 3)$ does: $(1, 3)$, whose rhs came through the robot's cell, now has $rhs = 8$ through S and is underconsistent with key $[6 + 1 + 2; 6] = [9; 6]$; $(2, 4)$ and $(2, 2)$, queued with values derived from $g(2, 3) = 5$, drop out; and $(2, 3)$ itself re-enters as overconsistent with key $[7 + 0 + 2; 7] = [9; 7]$, which is now the key the loop compares against. Next come the cells that the initial search had left in the queue with keys $[9; \cdot]$, among them $(6, 1)$, $(5, 1)$ and $(4, 1)$ in the bottom right, which have nothing to do with the detour: their k_2 is smaller than 7, so the key order expands them before the robot's cell (the band effect discussed in Section 5.8). For every one of them the stored key equals the current key, because the robot moved two cells straight toward them and $k_m = 2$ compensates exactly; the reinsert check of line 18 therefore never fires in this example. Expanding $(4, 4)$ gives the gap cell $(3, 4)$ its value 5, expanding $(3, 4)$ gives $(2, 4)$ the value 6, the stale $(1, 3)$ is retracted on the way (which briefly removes the robot's cell from the queue, until $(2, 4)$ is expanded and hands it $rhs = 7$ again), and finally $(2, 3)$ is expanded with $g(2, 3) = 7$: the detour through $(3, 4)$ costs 2 more than the old plan. A* from scratch from $(2, 3)$ on the changed grid also expands 12 cells. From here the robot walks $(2, 4) \rightarrow (3, 4) \rightarrow (4, 4) \rightarrow (4, 3) \rightarrow (4, 2) \rightarrow (5, 2) \rightarrow T$ with no further changes and reaches the goal at $t = 9$.

5.8 Properties: correctness, optimality, complexity

Throughout this section the assumptions are: all edge costs are positive or ∞ , the graph is finite, and the heuristic is nonnegative and consistent with respect to the search direction (Definition 5.5 for LPA*, Section 5.6.1 for D* Lite, including the triangle inequality). The results are those of Koenig, Likhachev, and Furcy [23] and Koenig and Likhachev [21]; we state them for D* Lite and give the proof ideas, which transfer to LPA* by mirroring.

Theorem 5.8 (Termination and correctness). *Every call of **ComputeShortestPath** in Algorithm 5.2 terminates. When it returns, $g(s_{\text{start}}) = \text{dist}(s_{\text{start}}, s_{\text{goal}})$ under the current edge costs, and if this value is finite, moving from the current vertex s to any successor s' that minimises $c(s, s') + g(s')$, starting at s_{start} , reaches s_{goal} along a shortest path. Consequently the robot in **Main** follows, at every moment, a path whose cost equals the cost that A* would compute from scratch from its current position.*

Proof sketch. Three invariants hold throughout. (i) rhs is always the one-step lookahead of the current g -values and costs, because every change of a g -value or a cost is followed by **UpdateVertex** on all vertices whose rhs could be affected. (ii) U contains exactly the locally inconsistent vertices, and every stored key is a lower bound on the vertex's current key: at insertion the two are equal, and afterwards the current key can only grow, either because k_m grew by $h(s_{\text{last}}, s_{\text{start}})$ while $h(s_{\text{start}}, s)$ fell by at most that much (Equation (5.3)), or because a vertex's key is left untouched between insertions. (iii) Vertices are expanded in nondecreasing order of their current keys, because the reinsert check never expands a vertex whose stored key is stale.

From (iii) and the consistency of h one shows, as for A*, that when an overconsistent vertex

u is expanded its new $g(u) = rhs(u)$ equals $dist(u, s_{goal})$ under the current costs, provided all vertices with smaller keys have already been made consistent; the underconsistent expansions ensure that no stale finite value survives among those. When the loop stops, s_{start} is consistent and every remaining queued vertex has a key at least $k(s_{start})$, so no queued vertex can offer a shorter path to the goal than $g(s_{start})$ itself. Every vertex on a minimising successor chain from s_{start} has a key no larger than $k(s_{start})$ and is therefore consistent, so the chain is a shortest path by the argument of [Proposition 5.4](#). Termination follows from [Theorem 5.9](#): each vertex is expanded at most twice, and a reinsert without expansion happens at most once per stale entry. $\square$

Theorem 5.9 (Expansion bound). *Within one call of `ComputeShortestPath`, every vertex is expanded at most twice: at most once when it is underconsistent and at most once when it is overconsistent. Vertices that are consistent before the call and whose g -value does not need to change are never expanded. The first call, with all g -values at ∞ , expands the same vertices in the same order as A^* with the same heuristic and the same tie-breaking (ties among equal f -values broken toward the smaller g) would, on the reversed graph.*

Proof sketch. An overconsistent expansion of u sets $g(u) = rhs(u)$ with key $[g(u) + h(s_{start}, u) + k_m; g(u)]$. Any vertex s' expanded afterwards has a key at least as large, so $g(s') + h(s_{start}, s') \geq g(u) + h(s_{start}, u)$. By consistency of h , $h(s_{start}, u) \leq h(s_{start}, s') + c(u, s')$ for the edge (u, s') , hence $c(u, s') + g(s') \geq g(u)$: no later expansion can lower $rhs(u)$ below $g(u)$, so u is never popped as overconsistent again in this call. An underconsistent expansion sets $g(u) = \infty$; a vertex with $g = \infty$ cannot be underconsistent, and $g(u)$ only becomes finite again through an overconsistent expansion, after which the first argument applies. The statement about the first call follows because with all $g = \infty$ there are no underconsistent vertices, every expansion is an A^* expansion with rhs playing the role of the tentative g , and the keys order vertices exactly as f with g tie-breaking does. The full proofs are in Koenig, Likhachev, and Furcy [23]. $\square$

Two remarks put the bound in perspective. First, “at most twice” is a worst case; the typical repair after a local change expands a small fraction of the vertices, as the worked examples and [Figure 5.5](#) show. Second, the bound does not say that D* Lite is always cheaper than A^* : if a change invalidates most of the stored values, D* Lite may expand vertices twice and pay for queue maintenance that A^* does not need (see the pitfall in [Section 5.10](#)). The key modifier adds no expansions at all; its only cost is at most one extra pop-and-reinsert per queue entry that was inserted before a move. Third, the theorem compares with A^* with the same tie-breaking. The second key component orders vertices with equal k_1 by increasing g ; on a grid, where many cells share the same f -value, this expands the whole band of cells with $f \leq dist(s_{start}, s_{goal})$ in order of their distance, whereas the usual A^* rule of breaking ties toward the larger g ([Chapter 4](#)) runs to the goal along one path of that band. The first search of D* Lite is therefore often several times more expensive than a well-tuned A^* ([Section 5.8.1](#) measures a factor of 4.6 on a 50×50 grid), and a repair that raises the path cost re-expands the part of the band that the new cost uncovers, as the worked examples show. This is the price of the second component, and it cannot be dropped: it is what guarantees that stale values are retracted before the vertices that depend on them are expanded ([Exercise 5.3](#)).

5.8.1 An experiment: D* Lite versus repeated A^*

[Figure 5.5](#) was produced by `code/figures/gen_ch05_replanning.py`. On a 50×50 grid with 20% random obstacles, the robot starts in the top-left corner and heads for the bottom-right corner. At each of 40 events it first takes one step along its current path; then a new obstacle appears, with probability one half on the robot’s current path (which forces a repair) and otherwise at a random free cell, never disconnecting the goal. D* Lite processes the change

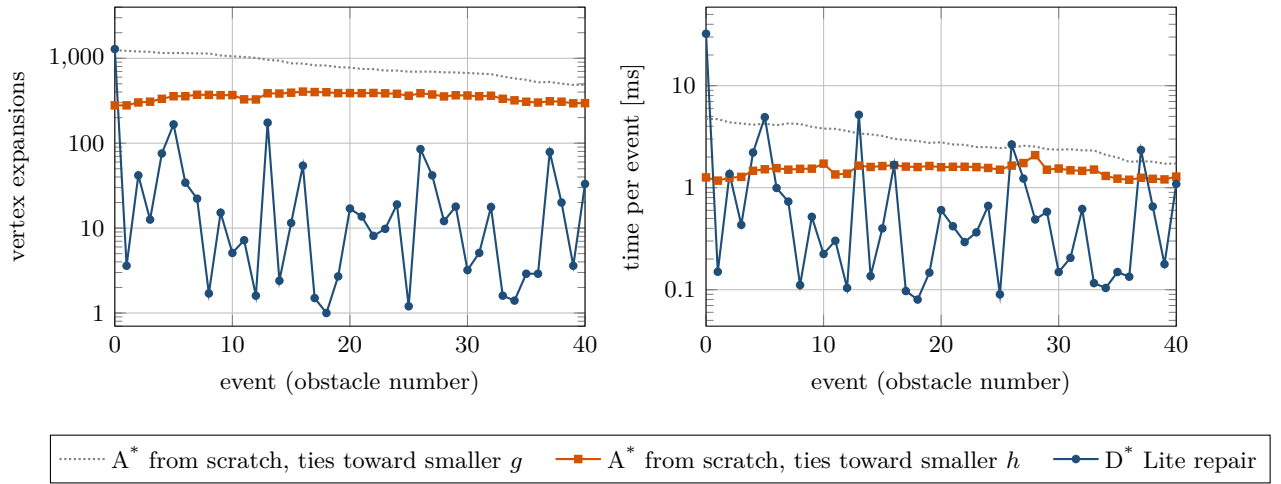


Figure 5.5. Replanning cost as obstacles appear one by one on a 50×50 grid (means over 10 grids, logarithmic scales; event 0 is the initial search). Left: vertex expansions per event. A* re-expands a few hundred cells every time; D* Lite expands almost nothing when the obstacle is off the path and a small region when it is on the path, but its initial search costs as much as A* with the same tie-breaking (dotted). Right: wall-clock time per event of the Python implementation; the gap is smaller than for expansions because D* Lite does more work per expansion.

with [Algorithm 5.2](#); the baseline runs A* from scratch from the robot’s current cell on the changed grid, once with the usual grid tie-breaking (ties among equal f -values toward the smaller h) and once with the tie-breaking of the D* Lite key (toward the smaller g). All three count vertex expansions and wall-clock time; the curves average over 10 random grids, and event 0 is the initial search.

Over the 40 replanning events, A* expanded 14 166 vertices in total and D* Lite 1 031, a ratio of about 14; in wall-clock time the ratio was only about 1.8 (60 ms against 33 ms), because one D* Lite expansion costs about $28 \mu\text{s}$ in this implementation against $4 \mu\text{s}$ for an A* expansion: `UpdateVertex` recomputes the rhs -value of every neighbour from all of its neighbours, and the lazy queue does extra bookkeeping. Three features of the curves are worth noticing. First, the initial search (event 0) costs D* Lite 1 285 expansions and 32 ms, against 279 expansions and 1.3 ms for A*. The dotted curve shows that A* with the tie-breaking of the D* Lite key needs 1 247 expansions for the same search, so the difference is the band effect of [Section 5.8](#), not the incremental machinery. In cumulative expansions D* Lite overtakes A* after four events; in cumulative time A* is still marginally ahead at the end of the run (61 ms against 65 ms), so on this instance and in this implementation the incremental search has not quite recovered the cost of its first search after 40 events, although it is faster at every single one of them (0.8 ms against 1.5 ms on average). Second, events whose obstacle lands off the path cost D* Lite next to nothing: over all grids and events, 197 such events needed 1.5 expansions on average and none at all in 68 % of the cases, because `UpdateVertex` finds every affected vertex consistent or irrelevant and `ComputeShortestPath` returns immediately. Third, the 203 events on the path needed 49 expansions on average but only 4 in the median: most repairs are tiny, and the mean is dominated by a few events in which the obstacle closes a corridor and the detour is long. The largest single repair expanded 1 704 vertices, several times the cost of an A* search, which is the “almost everything changed” case of [Theorem 5.9](#) and the situation of the last pitfall in [Section 5.10](#).

5.9 Variants and extensions

The original D*. Stentz’s D* (“Dynamic A*”) [38] was the first algorithm to repair a robot’s path incrementally as its sensors revealed unknown terrain, and its Focussed variant added a heuristic. It works backward from the goal, like D* Lite, but propagates cost changes with RAISE and LOWER states and a set of case distinctions that are hard to get right. D* Lite computes the same paths, is provably at least as efficient in expansions, and fits on half a page; this is why it replaced D* in practice [22].

Anytime D*. Chapter 6 presents ARA*, which finds a path quickly with an inflated heuristic εh and improves it while time remains. Anytime D* [26] combines that schedule with the repair machinery of this chapter: it keeps g and rhs across both ε changes and edge-cost changes, lowering ε when the world is quiet and raising it again after a large change. It is the natural choice when the replanning layer has a hard deadline and a suboptimal path now is worth more than an optimal one later.

Field D*. Paths on a 4- or 8-connected grid are restricted to a few headings and are up to 8% longer than the true shortest path. Field D* [10] keeps the D* Lite update scheme but lets a path leave a cell through any point of an edge, computing the cost of a cell by linear interpolation between the g -values of the two corner vertices of that edge. The result is smooth, nearly any-angle paths at little extra cost; it was used on the Mars rovers and on several autonomous cars.

Moving Target D* Lite. D* Lite handles a moving start. When the *goal* moves too, for example when a drone must intercept or follow another vehicle, the goal distances change with every goal step and the backward search loses its advantage. Moving Target D* Lite [40] extends D* Lite to targets that move: it reuses the part of the previous search tree that remains valid after both the agent and the target have moved, deletes the rest, and handles edge-cost changes with the same g/rhs machinery.

5.10 Implementation notes

The priority queue. The algorithms need Insert, Pop, TopKey, membership tests and, in UpdateVertex, Remove of an arbitrary element. Python’s `heapq` offers no removal, so Listing 5.1 uses *lazy deletion*: a dictionary maps each queued vertex to its current key, and heap entries are only trusted if their key equals the dictionary entry. Remove deletes the dictionary entry; the stale heap entry is skipped when it surfaces. Since a vertex is re-inserted only after it has been removed, the heap holds at most one live entry per vertex, and all operations stay $\mathcal{O}(\log |U|)$ amortised. An alternative is an indexed binary heap with an explicit `decrease_key` and `remove`; it is faster in compiled languages but longer to write. Store keys as tuples so that comparison is lexicographic, and break ties among equal keys deterministically (here by the vertex coordinates), which makes traces reproducible.

Infinity. Use `float("inf")`: it compares, adds and takes minima correctly, and $\infty = \infty$ is true, which the consistency test $g(u) = rhs(u)$ relies on. Never encode blocked cells as a large finite cost such as 10^9 in the incremental algorithms; a “blocked” path would then have a finite cost, the algorithms would happily compute it, and the robot would walk into the wall when nothing else is available.

Scanning changed edges. The pseudocode “scans the graph”. In a grid world the environment reports the cells whose status flipped; the planner converts each such cell b into its eight directed edges and calls `UpdateVertex` on each tail, that is on b and on each of its four neighbours. Doing this once per changed cell is enough; calling `UpdateVertex` twice on the same vertex is harmless. Edge-cost *decreases* (an obstacle disappears or a region is no longer predicted to be occupied) are handled by the same code and produce only overconsistent vertices.

When does a key change? Only inside `UpdateVertex`, which removes and reinserts the vertex, and implicitly for all queued vertices when k_m grows. Nothing else touches keys, and there is no `decrease_key` call anywhere in the algorithm. This is what makes lazy deletion sufficient.

Termination and the no-path case. `ComputeShortestPath` stops with $g(s_{\text{start}}) = \infty$ if the goal is unreachable; the queue is then empty or holds only keys $\geq [\infty; \infty]$. The main loop must check this before taking a step (Algorithm 5.2, line 30). When the obstacle disappears later, the same queue continues to work: no re-initialisation is needed, ever.

Listing 5.1. The priority queue with lazy deletion (from `code/ch05_dstar_lite.py`).

```

1  class PriorityQueue:
2      """Min-heap of (key, vertex) entries with Remove by lazy deletion.
3
4      key_of maps each queued vertex to its current key; a heap entry is live
5      only if its key equals key_of[vertex], and stale entries are skipped
6      when they surface. Keys are tuples (lexicographic order); equal keys
7      are ordered by the vertex so that traces are deterministic.
8      """
9
10     def __init__(self):
11         self.heap = []
12         self.key_of = {}
13
14     def __contains__(self, s):
15         return s in self.key_of
16
17     def __len__(self):
18         return len(self.key_of)
19
20     def insert(self, s, key):
21         self.key_of[s] = key
22         heapq.heappush(self.heap, (key, s))
23
24     def remove(self, s):
25         del self.key_of[s]          # the heap entry becomes stale
26
27     def _purge(self):
28         while self.heap and self.key_of.get(self.heap[0][1]) != self.heap[0][0]:
29             heapq.heappop(self.heap)
30
31     def top_key(self):
32         self._purge()
33         return self.heap[0][0] if self.heap else (INF, INF)
34
35     def pop(self):
36         self._purge()

```

```

37     key, s = heapq.heappop(self.heap)
38     del self.key_of[s]
39     return s

```

Listing 5.2. Algorithm 5.2 line by line: key, UpdateVertex and ComputeShortestPath of class DStarLite (from code/ch05_dstar_lite.py). Missing dictionary entries stand for ∞ ; `_trace` records the events for the worked examples.

```

1     def calculate_key(self, s):
2         m = min(self.g_of(s), self.rhs_of(s))
3         return (m + self.h(self.start, s) + self.k_m, m)
4
5     def update_vertex(self, u):
6         rhs_old, queued_old = self.rhs_of(u), u in self.U
7         if u != self.goal:
8             self.rhs[u] = min(self.grid.cost(u, s) + self.g_of(s)
9                               for s in self.grid.neighbors(u))
10
11         if u in self.U:
12             self.U.remove(u)
13         if self.g_of(u) != self.rhs_of(u):
14             self.U.insert(u, self.calculate_key(u))
15         self._trace("update", u, rhs_old, self.rhs_of(u), queued_old,
16                     u in self.U, self.U.key_of.get(u))
17
18     def compute_shortest_path(self):
19         """Repair the search; returns the number of vertex expansions."""
20         n = 0
21         while (self.U.top_key() < self.calculate_key(self.start)
22               or self.rhs_of(self.start) != self.g_of(self.start)):
23             k_old = self.U.top_key()
24             u = self.U.pop()
25             k_new = self.calculate_key(u)
26             if k_old < k_new: # stale lower bound
27                 self.U.insert(u, k_new)
28                 self._trace("reinsert", u, k_old, k_new)
29             elif self.g_of(u) > self.rhs_of(u): # overconsistent
30                 self.g[u] = self.rhs_of(u)
31                 n += 1
32                 self._trace("pop", u, k_old, "over", self.g[u])
33                 for s in self.grid.neighbors(u):
34                     self.update_vertex(s)
35             else: # underconsistent
36                 self.g[u] = INF
37                 n += 1
38                 self._trace("pop", u, k_old, "under", INF)
39                 for s in self.grid.neighbors(u) + [u]:
40                     self.update_vertex(s)
41         self.expansions += n
42         return n

```

Listing 5.3. Processing a batch of changed cells, lines 35–40 of Main (from code/ch05_dstar_lite.py).

```

1     def notify_changed_cells(self, cells, replan=True):
2         """Lines 28'-35': process cells whose blocked status flipped.
3
4         Adds h(s_last, s_start) to k_m, updates the tail of every changed

```

```

5      edge (the cell and each neighbour) and, if replan, repairs the
6      search. Returns the number of expansions.
7      """
8      if not cells:
9          return 0
10     self.k_m += self.h(self.s_last, self.start)
11     self.s_last = self.start
12     for b in cells:
13         self.update_vertex(b)
14         for u in self.grid.neighbors(b):
15             self.update_vertex(u)
16     return self.compute_shortest_path() if replan else 0

```

Listings 5.1 to 5.3 are the core of the chapter’s implementation; Listing 5.2 mirrors Algorithm 5.2 line by line, and Listing 5.3 is the change-processing part of Main, called with the cells whose status flipped after the robot has moved. The full file also contains LPA*, an A* baseline with both tie-breaking rules, the worked examples as functions and a self-test. The self-test draws random 20×20 grids for many seeds, applies random sequences of obstacle insertions and removals while the robot walks, and checks after every change that $g(s_{\text{start}})$ of D* Lite (and $g(s_{\text{goal}})$ of LPA*) equals the cost that A* computes from scratch, that the extracted path is valid and has that cost, that the queue holds exactly the inconsistent vertices with keys that are lower bounds, and that the first search expands exactly as many vertices as A* with the same tie-breaking (Theorem 5.9).

Blocking a cell changes all of its edges, in both directions

An obstacle that appears at cell b changes the edges *into* b and the edges *out of* b . Updating only the incoming edges (the natural thought: “nobody may enter b ”) leaves $rhs(b)$ and $g(b)$ finite in D* Lite, so b looks like a consistent cell with a valid goal distance; the neighbours’ rhs -values still ignore it because their edges into b cost ∞ , so the robot does not enter b today, but when the obstacle later disappears the neighbours are updated from a stale $g(b)$. Updating only the outgoing edges is wrong in the other direction: in LPA* the successors of b are corrected but b keeps a finite rhs through its unchanged incoming edges, and the backward path extraction of Proposition 5.4 happily routes *through* the obstacle. Treat a blocked cell as a vertex all of whose edges cost ∞ , and update the tail of every one of them.

Replanning can be slower than A*

Incremental search pays off when the change is small relative to the search. If the environment changes almost everywhere between two queries, for example when a fresh occupancy map arrives from a different sensor, or when the start and the goal both jump, D* Lite may expand many vertices twice (once to retract, once to recompute) and spends time in UpdateVertex on every affected neighbour. In such cases a fresh A* is faster and simpler. A robust replanning layer measures the fraction of changed cells and falls back to A* above a threshold; Exercise 5.7 asks you to find that threshold for your implementation. Remember also that the *first* search of D* Lite is more expensive than a well-tuned A* because of its tie-breaking (Section 5.8); an incremental planner pays this once and must recover it over the repairs.

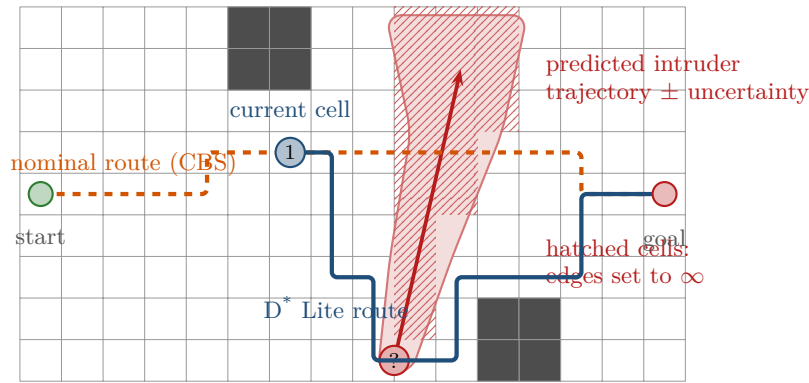


Figure 5.6. The replanning layer. A predicted intruder trajectory (red, with its uncertainty tube) is rasterised into blocked cells (hatched). The drone (blue) cannot rejoin its nominal route (dashed) with a local manoeuvre, so D* Lite is run from the drone’s current cell; only the cells around the tube are updated and the new route (solid blue) is returned to the executive for a conflict re-check.

5.11 Where this fits in the drone system

In the drone system

D* Lite is the *replanning layer* of the hybrid architecture in [Chapter 24](#). The global planner (CBS or ECBS, [Chapters 9 and 10](#)) delivers a nominal time-indexed route; the prediction layer ([Chapters 18 and 20](#)) delivers, for each unknown vehicle, a predicted trajectory with uncertainty; the local layer (ORCA or DWA, [Chapters 13 and 14](#)) executes short avoidance manoeuvres. The replanning layer is invoked when the local manoeuvre cannot safely return the drone to its nominal route: reconnection is impossible, too costly, or would violate formation or communication constraints. It then runs `ComputeShortestPath` from the drone’s *current* cell and hands the new route back to the executive, which re-checks it for conflicts with the other swarm members.

“Edge costs change” has a precise meaning here. The prediction layer inflates the predicted intruder trajectory by the intruder’s radius, the drone’s radius and a margin proportional to the prediction uncertainty ([Chapters 2 and 20](#)), and marks every grid cell that this tube touches within the safety horizon as blocked, or as expensive if soft avoidance is preferred. Each such cell is one changed vertex whose edges are updated as in [Section 5.10](#); when the prediction is revised or the intruder leaves, the cells are released again, which is a cost decrease that the same code handles. Because the drone keeps flying while this happens, its cell changes between calls, which is exactly what the key modifier was designed for. This is Week 2 of the study plan ([Appendix A](#)): implement or adapt D* Lite, change obstacles during execution, and compare with re-running A* ([Exercise 5.6](#)). [Figure 5.6](#) sketches the data flow.

5.12 Summary

Chapter summary

- Re-running A* after a small change wastes work; an incremental search keeps the previous g -values and repairs only what the change invalidated.
- LPA* stores $g(s)$ and the one-step lookahead $rhs(s)$; a vertex is locally consistent if

they agree, overconsistent if $g > rhs$ (a better value is available), underconsistent if $g < rhs$ (the old value is stale).

- The key $[\min(g, rhs) + h; \min(g, rhs)]$ orders the queue like A*'s f with g tie-breaking; using the smaller stored value processes stale vertices before the vertices that depend on them.
- `ComputeShortestPath` pops until the query vertex is consistent and no key is smaller; overconsistent pops accept rhs , underconsistent pops reset g to ∞ and update the vertex and its neighbours.
- D* Lite searches backward from the goal so that the robot may move; rhs looks at successors, expansions update predecessors, the heuristic is measured from the robot, and paths are read forward.
- The key modifier k_m accumulates $h(s_{\text{last}}, s_{\text{start}})$; it turns stale queue keys into lower bounds, which a reinsert check repairs, so the heap is never rebuilt.
- D* Lite returns the same path costs as A* from scratch, expands each vertex at most twice per repair, and behaves exactly like A* with ties broken toward the smaller g on the first search, which on grids is several times more expensive than A* with the usual tie-breaking.
- Blocking a cell changes all of its edges in both directions; the tail of every changed edge must be updated. When almost everything changes, fall back to A*.

Further reading. The original sources are the D* Lite paper [21], the journal version with the full proofs and the extension to unknown terrain [22], and the LPA* paper [23], which also contains an excellent discussion of why incremental heuristic search works. Stentz's D* [38] is the historical root. Anytime D* [26] and Field D* [10] are the two extensions most used on real robots. Textbook treatments of the underlying A* theory are in Russell and Norvig [35] and LaValle [25].

5.13 Exercises

Exercise 5.1 (★). For each of the following situations in LPA*, say whether the vertex s becomes overconsistent, underconsistent or stays consistent, and what its key is: (a) the cost of the edge from s 's best predecessor into s doubles; (b) a new predecessor with a cheaper total cost appears; (c) an edge *out of* s becomes ∞ ; (d) the best predecessor of s is popped as underconsistent and its g becomes ∞ , but s has a second predecessor with the same total cost.

Exercise 5.2 (★★ Hand trace). Repeat [Example 5.6](#), starting from the state in the left panel of [Figure 5.3](#), but let cell (1,0) become blocked instead of (3,1). Give the vertices on which `UpdateVertex` is called, their new rhs -values and keys, and the full trace of `ComputeShortestPath`. How many expansions does the repair take, and why is the answer so different from [Table 5.2](#)? Then unblock (3,1) in the state of the right panel and trace the repair; which kind of inconsistency appears now?

Exercise 5.3 (★★). Construct a graph with edge costs, a start and a goal, and a single edge-cost increase such that LPA* with keys compared by k_1 only (ties broken arbitrarily) expands some vertex three times in one call of `ComputeShortestPath`, whereas the two-component key of [Definition 5.5](#) expands every vertex at most twice.

Exercise 5.4 (★★★ Proof). Let the robot have moved from s_{last} to s_{start} and let [Algorithm 5.2](#) execute line 35. Prove that afterwards every key stored in U is a lower bound on the key

that `CalculateKey` would return for that vertex, assuming only that h satisfies the triangle inequality $h(a, c) \leq h(a, b) + h(b, c)$. Then show by an example with concrete numbers that without line 35 some stored key can be a strict *upper* bound, and explain, using the proof sketch of [Theorem 5.8](#), which invariant breaks. Finally, explain why k_m must be accumulated from s_{last} rather than recomputed as $h(s_{\text{orig}}, s_{\text{start}})$ from the original start.

Exercise 5.5 (★★). Write down LPA* for the *reversed* graph (all edges flipped, goal and start swapped) and simplify the result. Show that you obtain [Algorithm 5.2](#) with $k_m \equiv 0$ and without the reinsert check, and identify precisely which lines of `Main` are needed only because the start moves.

Exercise 5.6 (★★★ Coding). This is the coding exercise of Week 2 of the study plan ([Appendix A](#)). Implement D* Lite on a 4-connected grid (or adapt `code/ch05_dstar_lite.py`), together with an A* baseline. Let an agent execute its plan one cell per tick. At random ticks, block a random free cell (with some probability, a cell on the agent's current path) or unblock a blocked one; never block the agent's cell or the goal. After every change, replan with D* Lite and, separately, with A* from the agent's cell, and record expansions, wall-clock time and path cost for both. (a) Verify on at least 100 random runs that the path costs agree. (b) Plot expansions and time per change for both methods, as in [Figure 5.5](#), for grid sizes 30, 50 and 100. (c) Extend the grid to 8-connectivity with octile costs and check that the Manhattan heuristic must be replaced; explain what goes wrong if it is not. The milestone of the week is reached when your agent repairs its path rather than restarting every search.

Exercise 5.7 (★★★ Coding). Using your implementation from [Exercise 5.6](#), change a fraction ϕ of all free cells at once (block half of them, unblock half of the currently blocked cells) and measure the replanning time of D* Lite and of A* from scratch as a function of $\phi \in \{0.001, 0.01, 0.05, 0.1, 0.2, 0.5\}$ on a 100×100 grid. At which ϕ does D* Lite stop paying off? Explain the crossover with [Theorem 5.9](#), and propose a rule for the replanning layer of [Chapter 24](#) that chooses between the two.

Exercise 5.8 (★★). The prediction layer reports an intruder with predicted positions $\mathbf{p}_1, \dots, \mathbf{p}_H$ over the next H ticks and a growing uncertainty radius $r_t = r_0 + \sigma t$. (a) Describe how you would convert this into a set of changed cells for D* Lite, and how the set changes when the next prediction arrives one tick later. (b) D* Lite plans in space only; the intruder occupies each cell only during some ticks. Discuss what is lost by blocking the whole tube for the entire horizon, and sketch how a space-time variant ([Chapter 4](#)) could reduce the loss. (c) What must change in [Algorithm 5.2](#) if the *goal* of the drone is itself moving, and why does the backward search then lose its advantage?

Algorithm 5.1. Lifelong Planning A* (LPA*) [23]**Input:** graph G , costs c , start s_{start} , goal s_{goal} , consistent heuristic h **Output:** after every call of **ComputeShortestPath**: $g(s_{\text{goal}}) = \text{dist}(s_{\text{start}}, s_{\text{goal}})$ and a shortest path ([Theorem 5.8](#))

```

1 Function CalculateKey( $s$ ):
2   return  $[\min(g(s), rhs(s)) + h(s); \min(g(s), rhs(s))]$ 
3 Function Initialize():
4    $U \leftarrow \emptyset$ 
5   foreach  $s \in V$  do  $rhs(s) \leftarrow g(s) \leftarrow \infty$ 
6    $rhs(s_{\text{start}}) \leftarrow 0$ 
7    $U.\text{Insert}(s_{\text{start}}, [h(s_{\text{start}}); 0])$ 
8 Function UpdateVertex( $u$ ):
9   if  $u \neq s_{\text{start}}$  then  $rhs(u) \leftarrow \min_{s' \in \text{Pred}(u)} (g(s') + c(s', u))$ 
10  if  $u \in U$  then  $U.\text{Remove}(u)$ 
11  if  $g(u) \neq rhs(u)$  then  $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
12 Function ComputeShortestPath():
13  while  $U.\text{TopKey}() < \text{CalculateKey}(s_{\text{goal}})$  or  $rhs(s_{\text{goal}}) \neq g(s_{\text{goal}})$  do
14     $u \leftarrow U.\text{Pop}()$ 
15    if  $g(u) > rhs(u)$  then // overconsistent
16       $g(u) \leftarrow rhs(u)$ 
17      foreach  $s \in \text{Succ}(u)$  do  $\text{UpdateVertex}(s)$ 
18    else // underconsistent
19       $g(u) \leftarrow \infty$ 
20      foreach  $s \in \text{Succ}(u) \cup \{u\}$  do  $\text{UpdateVertex}(s)$ 
21 Function Main():
22   Initialize()
23   while true do
24     ComputeShortestPath()
25     wait for changes in edge costs
26     foreach directed edge  $(u, v)$  whose cost changed do
27       update the stored cost  $c(u, v)$ 
28       UpdateVertex( $v$ )

```

Table 5.1. First call of **ComputeShortestPath** in [Example 5.6](#). “Updated” lists the vertices on which **UpdateVertex** changed something, with their new rhs -value and key.

Step	Popped u (key)	Case	$g(u)$ after	Updated: rhs , key
1	(0, 1) [4; 0]	over	0	(1, 1): 1, [4; 1]; (0, 2): 1, [6; 1]; (0, 0): 1, [6; 1]
2	(1, 1) [4; 1]	over	1	(2, 1): 2, [4; 2]; (1, 2): 2, [6; 2]; (1, 0): 2, [6; 2]
3	(2, 1) [4; 2]	over	2	(3, 1): 3, [4; 3]; (2, 2): 3, [6; 3]; (2, 0): 3, [6; 3]
4	(3, 1) [4; 3]	over	3	(4, 1): 4, [4; 4]; (3, 2): 4, [6; 4]; (3, 0): 4, [6; 4]
5	(4, 1) [4; 4]	over	4	(4, 2): 5, [6; 5]; (4, 0): 5, [6; 5]

Table 5.2. Second call of `ComputeShortestPath` in [Example 5.6](#), after $(3, 1)$ was blocked. Steps 1–2 retract stale values (underconsistent); the remaining steps are ordinary expansions. “–” means that `UpdateVertex` changed nothing, “out” that the vertex left the queue.

Step	Popped u (key)	Case	$g(u)$ after	Updated: rhs , key
1	$(3, 1)$ [4; 3]	under	∞	–
2	$(4, 1)$ [4; 4]	under	∞	$(4, 2)$: ∞ , out; $(4, 0)$: ∞ , out
3	$(0, 0)$ [6; 1]	over	1	–
4	$(0, 2)$ [6; 1]	over	1	–
5	$(1, 0)$ [6; 2]	over	2	–
6	$(1, 2)$ [6; 2]	over	2	–
7	$(2, 0)$ [6; 3]	over	3	$(3, 0)$: 4, [6; 4]
8	$(2, 2)$ [6; 3]	over	3	$(3, 2)$: 4, [6; 4]
9	$(3, 0)$ [6; 4]	over	4	$(4, 0)$: 5, [6; 5]
10	$(3, 2)$ [6; 4]	over	4	$(4, 2)$: 5, [6; 5]
11	$(4, 0)$ [6; 5]	over	5	$(4, 1)$: 6, [6; 6]
12	$(4, 2)$ [6; 5]	over	5	–
13	$(4, 1)$ [6; 6]	over	6	–

Table 5.3. LPA* and D* Lite are mirror images. The only genuinely new ingredient of D* Lite is the key modifier k_m .

	LPA* (Algorithm 5.1)	D* Lite (Algorithm 5.2)
Search direction	from s_{start}	from s_{goal}
$g(s)$ estimates	$\text{dist}(s_{\text{start}}, s)$	$\text{dist}(s, s_{\text{goal}})$
$rhs(s)$	$\min_{s' \in \text{Pred}(s)} g(s') + c(s', s)$	$\min_{s' \in \text{Succ}(s)} c(s, s') + g(s')$
fixed $rhs = 0$ at	s_{start}	s_{goal}
expansion updates	$\text{Succ}(u)$	$\text{Pred}(u)$
edge (u, v) changed	<code>UpdateVertex</code> (v)	<code>UpdateVertex</code> (u)
heuristic	$h(s) \approx \text{dist}(s, s_{\text{goal}})$	$h(s_{\text{start}}, s) \approx \text{dist}(s_{\text{start}}, s)$
loop tests key of	s_{goal}	s_{start}
key modifier	none	k_m added to k_1
path extraction	backward from s_{goal} over Pred	forward from s_{start} over Succ

Table 5.4. Why the key modifier is needed. A queued vertex u and a vertex v inserted after the robot has moved two cells, $h(s_{\text{old}}, s_{\text{new}}) = 2$; $m = \min(g, rhs)$, and the last two columns are the keys that `CalculateKey` returns now, without and with the modifier. Without k_m the stale key of u is an overestimate and v is wrongly popped first; with $k_m = 2$ the stale key is a lower bound and the order is right.

Vertex	m	$h(s_{\text{old}}, \cdot)$	$h(s_{\text{new}}, \cdot)$	stored key	now, no k_m	now, $k_m = 2$
u (old entry)	4	6	4	[10; 4]	[8; 4]	[10; 4]
v (new entry)	6	–	3	[9; 6] or [11; 6]	[9; 6]	[11; 6]
popped first				v (no k_m)	u	u

Algorithm 5.2. D* Lite, final version [21]

Input: graph G , costs c , robot position s_{start} , goal s_{goal} , heuristic $h(\cdot, \cdot)$ consistent w.r.t. the start

Output: the robot moves along shortest paths under the current costs until it reaches s_{goal} or no path exists

```

1 Function CalculateKey( $s$ ):
2    $m \leftarrow \min(g(s), rhs(s))$ 
3   return  $[m + h(s_{\text{start}}, s) + k_m; m]$ 
4 Function Initialize():
5    $U \leftarrow \emptyset; k_m \leftarrow 0$ 
6   foreach  $s \in V$  do  $rhs(s) \leftarrow g(s) \leftarrow \infty$ 
7    $rhs(s_{\text{goal}}) \leftarrow 0$ 
8    $U.\text{Insert}(s_{\text{goal}}, [h(s_{\text{start}}, s_{\text{goal}}); 0])$ 
9 Function UpdateVertex( $u$ ):
10  if  $u \neq s_{\text{goal}}$  then  $rhs(u) \leftarrow \min_{s' \in \text{Succ}(u)} (c(u, s') + g(s'))$ 
11  if  $u \in U$  then  $U.\text{Remove}(u)$ 
12  if  $g(u) \neq rhs(u)$  then  $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
13 Function ComputeShortestPath():
14  while  $U.\text{TopKey}() < \text{CalculateKey}(s_{\text{start}})$  or  $rhs(s_{\text{start}}) \neq g(s_{\text{start}})$  do
15     $k_{\text{old}} \leftarrow U.\text{TopKey}()$ 
16     $u \leftarrow U.\text{Pop}()$ 
17    if  $k_{\text{old}} < \text{CalculateKey}(u)$  then // stale lower bound
18       $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
19    else if  $g(u) > rhs(u)$  then // overconsistent
20       $g(u) \leftarrow rhs(u)$ 
21      foreach  $s \in \text{Pred}(u)$  do UpdateVertex( $s$ )
22    else // underconsistent
23       $g(u) \leftarrow \infty$ 
24      foreach  $s \in \text{Pred}(u) \cup \{u\}$  do UpdateVertex( $s$ )
25 Function Main():
26    $s_{\text{last}} \leftarrow s_{\text{start}}$ 
27   Initialize()
28   ComputeShortestPath()
29   while  $s_{\text{start}} \neq s_{\text{goal}}$  do
30     if  $g(s_{\text{start}}) = \infty$  then return no path
31      $s_{\text{start}} \leftarrow \arg \min_{s' \in \text{Succ}(s_{\text{last}})} (c(s_{\text{last}}, s') + g(s'))$ 
32     move the robot to  $s_{\text{start}}$ 
33     scan the graph for changed edge costs
34     if any edge cost changed then
35        $k_m \leftarrow k_m + h(s_{\text{last}}, s_{\text{start}})$ 
36        $s_{\text{last}} \leftarrow s_{\text{start}}$ 
37       foreach directed edge  $(u, v)$  whose cost changed do
38         update the stored cost  $c(u, v)$ 
39         UpdateVertex( $u$ )
40       ComputeShortestPath()

```

Table 5.5. Events of [Example 5.7](#). “Updated” counts the vertices whose *rhs*-value or queue membership changed in `UpdateVertex`, “Exp.” the vertex expansions of `ComputeShortestPath` in that row; $g(s_{\text{start}})$ is the remaining path cost afterwards.

t	Robot at	Event	k_m	Updated	Exp.	$g(s_{\text{start}})$
0	(0, 3)	initial search	0	–	10	7
1	(1, 3)	move, no change	0	–	0	6
2	(2, 3)	move, no change	0	–	0	5
2	(2, 3)	(3, 3) blocked, repair	2	3	12	7
9	(6, 2)	goal reached	2	–	0	0

Table 5.6. The repair in [Example 5.7](#): `ComputeShortestPath` after (3, 3) was blocked, with the robot at (2, 3) and $k_m = 2$. Keys are $[\min(g, rhs) + h((2, 3), \cdot) + k_m; \min(g, rhs)]$; the loop stops once the robot’s cell is consistent and no queued key is smaller than $k(2, 3) = [9; 7]$. “–” means that `UpdateVertex` changed nothing, “out” that the vertex left the queue.

Step	Popped u (key)	Case	$g(u)$ after	Updated: rhs , key
1	(3, 3) [7; 4]	under	∞	–
2	(2, 3) [7; 5]	under	∞	(1, 3): 8, [9; 6]; (2, 4): ∞ , out; (2, 2): ∞ , out; (2, 3): 7, [9; 7]
3	(6, 1) [9; 1]	over	1	(6, 0): 2, [11; 2]
4	(5, 1) [9; 2]	over	2	(5, 0): 3, [11; 3]
5	(6, 4) [9; 2]	over	2	–
6	(4, 1) [9; 3]	over	3	(4, 0): 4, [11; 4]
7	(5, 4) [9; 3]	over	3	–
8	(4, 4) [9; 4]	over	4	(3, 4): 5, [9; 5]
9	(3, 4) [9; 5]	over	5	(2, 4): 6, [9; 6]
10	(1, 3) [9; 6]	under	∞	(2, 3): ∞ , out; (0, 3): ∞ , [11; 7]; (1, 4): ∞ , out; (1, 2): ∞ , out; (1, 3): 8, [11; 8]
11	(2, 4) [9; 6]	over	6	(1, 4): 7, [11; 7]; (2, 3): 7, [9; 7]
12	(2, 3) [9; 7]	over	7	(2, 2): 8, [11; 8]

Anytime Search: ARA*

This chapter is not written yet.

Part III

Multi-Agent Path Finding

The Multi-Agent Path Finding Problem

This chapter is not written yet.

Prioritized Planning and Space-Time A^*

Learning objectives

- Explain prioritized planning in one paragraph: agents are planned one after another, and each treats the paths of the agents before it as moving obstacles.
- Build a reservation table by hand, including the stay-at-target reservation, and run Cooperative A^* against it on a small grid.
- Show with small instances that the priority order matters, that no order may work even when a solution exists, and that the best order can still be suboptimal.
- State and prove the completeness result for well-formed instances.
- Implement prioritized planning for two to five agents, create deliberate conflict cases, and validate the resulting plans.
- Explain the hierarchical and windowed variants of Cooperative A^* and the idea of priority-based search.

8.1 Why this matters for a drone swarm

Picture a warehouse roof with forty inspection drones. Each drone has a start pad and an inspection point, and the airspace between them is cut into a grid of cells by the support pillars. [Chapter 7](#) taught you how to describe this situation as a multi-agent path finding (MAPF) instance and how to detect the conflicts of a plan. It also warned you that finding an optimal plan is NP-hard. Forty drones is far beyond what a joint search over all agents can handle, and even the optimal solvers of [Chapters 9](#) and [10](#) need time that grows quickly with the number of interacting agents. The operator, however, wants the drones in the air within a second.

The oldest and simplest answer is **prioritized planning**: line the drones up in some order, plan the first one as if it were alone, then plan the second one while treating the first drone's path as a moving obstacle, then the third one against the first two, and so on. Each planning step is a single-agent space-time A^* search of the kind you implemented in [Chapter 4](#). Forty drones cost forty searches. That is why prioritized planning, introduced by Erdmann and Lozano-Pérez [8] and made popular for grids by Silver's *Cooperative A^** [37], is the workhorse of video-game crowds, warehouse robots and large drone swarms. It is also the study-plan's Week 3 exercise ([Appendix A](#)): implement it for two to five agents and create deliberate conflict cases. In the hybrid architecture of [Chapter 24](#) it appears as a fast fallback global planner and as the model for the reservation logic that every later layer reuses.

The price of the speed is that the method is greedy. A drone that is planned early never makes room for a drone that is planned late. This chapter shows exactly when that matters:

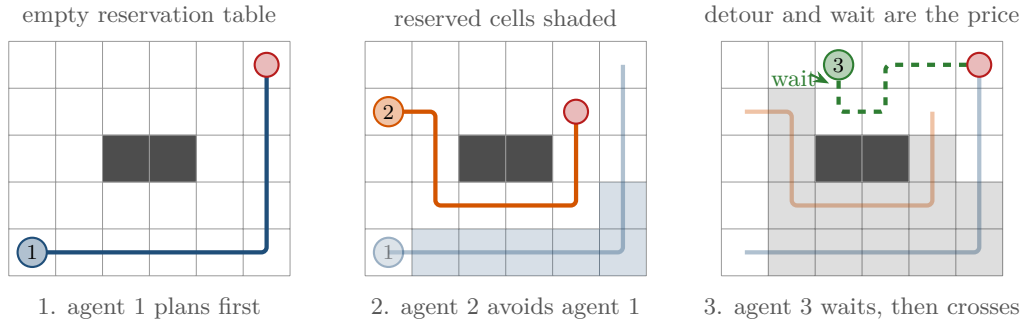


Figure 8.1. The reservation table as a space-time diagram for a corridor of five cells. Agent 1 (blue) has been planned first: the cells it occupies at each time step are reserved (shaded), and so is its goal cell for all later times. Agent 2 (orange) plans against the table: it must wait at c_4 for two steps and let agent 1 pass through the side cell before it can move. The dashed edge shows a swap that the edge reservation forbids.

the order can change the cost, the method can fail although a solution exists, and even its best order can be more expensive than the optimum. It also gives the positive result that makes prioritized planning safe in practice: on *well-formed* instances, where every agent can reach its goal without touching another agent's start or goal, prioritized planning succeeds in every order [3].

Roadmap. Section 8.2 explains the idea and the reservation table. Section 8.3 fixes the notation and Section 8.4 gives the pseudocode of prioritized planning and of Cooperative A*. ?? works through a three-agent instance in every one of its six orders. ?? collects the properties: incompleteness, suboptimality, complexity and the completeness theorem for well-formed instances. ?? covers the hierarchical and windowed variants, priority ordering heuristics with a small experiment, and priority-based search. ???? give the implementation notes and the place of the method in the drone system.

8.2 The idea in plain words

Suppose you have already planned drone 1. Its path tells you where it will be at every time step: cell v_0 at time 0, cell v_1 at time 1, and so on until it reaches its goal, where it stays. Now you plan drone 2. Instead of searching the joint space of both drones, you search only for drone 2, in space-time, and you forbid every (cell, time) pair that drone 1 occupies. Drone 1 is a moving obstacle: a wall that appears in one cell at one time step and moves on. If drone 2 wants to enter a cell that drone 1 occupies at that moment, drone 2 waits a step or goes around. When drone 2 is done, both paths are frozen, and drone 3 plans against both. The bookkeeping that stores the frozen paths is the **reservation table**: a set of (cell, time) entries that are taken, plus a set of (edge, time) entries so that two drones cannot swap cells by moving through each other.

Figure 8.1 draws the table as a space-time diagram. Three details matter and are easy to get wrong.

- **Stay at target.** Under the stay-at-target convention of Chapter 7, an agent that arrives at its goal at time T remains there for ever. Its goal cell must therefore be reserved for *all* times $t \geq T$, not only for $t = T$. Forgetting this lets a later agent drive through a parked drone.
- **Edge reservations.** Two agents that exchange cells between t and $t + 1$ never occupy the same cell at the same time, so vertex reservations alone do not catch this swapping

conflict. When agent 1 moves from u to v between t and $t + 1$, the table also reserves the reverse edge (v, u) at time t .

- **The goal of a later agent.** A later agent may only finish its path at a time T after which its goal is never reserved. If a higher-priority agent passes through that goal cell at some time $t' > T$, the later agent must wait somewhere and arrive after t' .

Key idea

Prioritized planning turns a k -agent problem into k single-agent space-time searches. Agent i sees agents $1, \dots, i - 1$ as moving obstacles stored in a reservation table, and never sees agents $i + 1, \dots, k$ at all. This is what makes it fast, and it is also the only reason it can fail.

Cooperative A* [37] is the name Silver gave to the combination of space-time A* with a reservation table. The word *cooperative* is slightly optimistic: only the later agents cooperate, by yielding to the earlier ones. Nevertheless, in open maps with few agents this one-sided cooperation is all that is needed, and the plans it produces are usually close to optimal. The trouble starts in corridors and near goals, as the counterexamples of ?? show.

8.3 Problem statement and notation

We use the classical MAPF setting of Chapter 7 and of Stern et al. [39]: an undirected graph $G = (V, E)$, usually a 4-connected grid, and k agents $a_1, \dots, a_k$ with distinct start vertices s_i and distinct goal vertices g_i . Time is discrete. In one time step an agent either moves along an edge or waits in its cell; both actions cost one. A **path** for agent i is a sequence $\pi_i = (\pi_i(0), \pi_i(1), \dots, \pi_i(T_i))$ with $\pi_i(0) = s_i$, $\pi_i(T_i) = g_i$ and consecutive vertices equal or adjacent. The agent stays at g_i afterwards, so we define $\pi_i(t) = g_i$ for all $t > T_i$. The cost of the path is its arrival time T_i and the cost of a plan is the sum of costs $\text{SoC} = \sum_i T_i$. Two paths π_i and π_j have a **vertex conflict** at time t if $\pi_i(t) = \pi_j(t)$, and a **swapping conflict** at time t if $\pi_i(t) = \pi_j(t + 1)$ and $\pi_i(t + 1) = \pi_j(t)$. A plan is valid if no pair of paths has a conflict of either kind. Following conflicts, where an agent enters the cell that another agent has just left, are allowed, as in the classical setting.

Definition 8.1 (Reservation table). A **reservation table** is a pair $R = (R_V, R_E)$ of sets

$$R_V \subseteq V \times \mathbb{N}, \quad R_E \subseteq E' \times \mathbb{N},$$

where E' contains both orientations (u, v) and (v, u) of every edge. An entry $(v, t) \in R_V$ means that some higher-priority agent occupies vertex v at time t ; an entry $((u, v), t) \in R_E$ means that some higher-priority agent moves from u to v between t and $t + 1$, so that moving from v to u in that step would be a swap. Reserving a path π with arrival time T adds $(\pi(t), t)$ for $0 \leq t \leq T$, adds $((\pi(t), \pi(t + 1)), t)$ for every move $\pi(t) \neq \pi(t + 1)$, and marks the goal $\pi(T)$ as **parked** from time T on, which stands for the infinite set of entries $\{(\pi(T), t) : t \geq T\}$.

A space-time state (v, t) is **blocked** by R if $(v, t) \in R_V$ or v is parked on at a time $\leq t$. A move from u to v at time t is blocked if $(v, t + 1)$ is blocked or $((v, u), t) \in R_E$. Waiting at u is blocked if $(u, t + 1)$ is blocked.

Definition 8.2 (Priority order and prioritized plan). A **priority order** is a permutation σ of $\{1, \dots, k\}$; agent $\sigma(1)$ has the highest priority. The **prioritized plan** for σ is obtained by planning the agents in the order $\sigma(1), \sigma(2), \dots$ where agent $\sigma(j)$ receives a shortest path that is not blocked by the reservation table containing the paths of $\sigma(1), \dots, \sigma(j - 1)$. If

some agent has no such path, the plan for σ fails.

The definition makes precise what *greedy* means here: each agent takes a path that is optimal for itself given the agents before it, and that path is never revised.

Definition 8.3 (Well-formed instance). Let $P = \{s_1, \dots, s_k, g_1, \dots, g_k\}$ be the set of all endpoints. An instance is **well-formed** if for every agent i there is a path in G from s_i to g_i that visits no endpoint of P other than s_i and g_i . In words: every agent can reach its goal without passing through the start or goal of any other agent [3].

Well-formedness is a property of the map and the endpoints alone; it does not depend on time or on the priority order, and it is cheap to test with k breadth-first searches (Chapter 3) on the graph with the other endpoints removed. Most warehouse layouts, where starts and goals are parking bays off the main aisles, are well-formed by design.

8.4 The algorithm

Algorithm 8.1 is the outer loop. It takes an order, keeps one reservation table, and calls Cooperative A* (Algorithm 8.2) once per agent. Cooperative A* is the space-time A* of Chapter 4 with three changes: successors are filtered by the reservation table, the goal test also requires that the goal is free for all later times, and the search is cut off at a time horizon $T_{\max}$ so that it terminates when no path exists.

Algorithm 8.1. Prioritized planning

Input: graph G , agents with starts s_i and goals g_i , priority order σ , horizon $T_{\max}$

Output: a conflict-free plan $(\pi_1, \dots, \pi_k)$ or *failure*

```

1 Function PrioritizedPlanning( $G, s, g, \sigma, T_{\max}$ ):
2    $R \leftarrow (\emptyset, \emptyset)$  // empty reservation table
3   for  $j \leftarrow 1$  to  $k$  do
4      $i \leftarrow \sigma(j)$ 
5      $\pi_i \leftarrow \text{CooperativeAStar}(G, s_i, g_i, R, T_{\max})$ 
6     if  $\pi_i = \text{failure}$  then
7       return failure
8      $\text{Reserve}(R, \pi_i)$  // vertices, reverse edges, parked goal
9   return  $(\pi_1, \dots, \pi_k)$ 

```

Walkthrough of Algorithm 8.1. Line 4 picks the next agent in priority order. Line 5 runs a full single-agent search; the reservation table R is the only channel through which earlier agents influence it. If the search fails (line 7) the whole procedure fails: there is no backtracking, no re-ordering and no replanning of earlier agents. The callers of ?? add restarts with new orders around this loop. Line 8 reserves the new path exactly as Definition 8.1 says.

Walkthrough of Algorithm 8.2. Line 2 computes the **goal-stay time** t_g : the last time step at which some higher-priority agent visits g . If a higher-priority agent is parked on g , then g is reserved for ever and the search cannot succeed, so the function returns failure immediately. The goal test in line 9 accepts (g, t) only if $t > t_g$: from then on nobody else will use g , so the agent can stay there. Because every action costs one, $g(v, t) = t$ and the first time a space-time state is generated is also the cheapest, which is why the algorithm never has to update g of a state in OPEN. Line 11 generates the wait action first and then the moves; line 12 discards successors whose target cell is reserved, and line 13 discards moves that would

Algorithm 8.2. Cooperative A* : space-time A* against a reservation table**Input:** graph G , start s , goal g , reservation table R , horizon $T_{\max}$, heuristic h **Output:** a shortest unblocked path from s to g or *failure*

```

1 Function CooperativeAStar( $G, s, g, R, T_{\max}$ ):
2    $t_g \leftarrow$  last time at which  $g$  is reserved in  $R$  ( $-1$  if never,  $\infty$  if parked on)
3   if  $t_g = \infty$  or  $(s, 0)$  is blocked then return failure
4   OPEN  $\leftarrow \{(s, 0)\}$  with  $g(s, 0) = 0$ ; CLOSED  $\leftarrow \emptyset$ 
5   while OPEN  $\neq \emptyset$  do
6      $(v, t) \leftarrow \arg \min_{\text{OPEN}} f = g + h(v)$ 
7     move  $(v, t)$  from OPEN to CLOSED
8     if  $v = g$  and  $t > t_g$  then
9        $\quad$  return path reconstructed from parents
10    if  $t \geq T_{\max}$  then continue
11    foreach  $v' \in \{v\} \cup \text{Succ}(v)$  do
12      if  $(v', t + 1)$  is blocked in  $R$  then continue
13      if  $v' \neq v$  and  $((v', v), t) \in R_E$  then continue
14      if  $(v', t + 1) \notin \text{OPEN} \cup \text{CLOSED}$  then
15         $\quad$   $g(v', t + 1) \leftarrow t + 1$ ; parent( $v', t + 1$ )  $\leftarrow (v, t)$ ; insert  $(v', t + 1)$  into OPEN
16  return failure

```

swap with a reserved move. The horizon test in line 10 stops the search from waiting for ever: without it, a blocked agent would generate $(s, 1), (s, 2), \dots$ without end. A safe choice of $T_{\max}$ is discussed in ??.

The heuristic h must be admissible for the static single-agent problem; the Manhattan distance is the usual first choice on 4-connected grids (Chapter 4). Reservations only remove states, so a heuristic that is admissible without reservations is admissible with them, and Algorithm 8.2 returns a path that is shortest among all unblocked paths whose arrival time is at most $T_{\max}$.

Conflict-Based Search

This chapter is not written yet.

Bounded-Suboptimal Search: ECBS

This chapter is not written yet.

Coupling and Constructive Methods: M^* , Push-and-Swap, Push-and-Rotate

This chapter is not written yet.

Part IV

Local and Reactive Collision Avoidance

Velocity Obstacles

This chapter is not written yet.

Reciprocal Avoidance: RVO and ORCA

This chapter is not written yet.

The Dynamic Window Approach

This chapter is not written yet.

Artificial Potential Fields

This chapter is not written yet.

Part V

Sampling-Based Motion Planning

Rapidly-Exploring Random Trees

This chapter is not written yet.

Optimal Sampling-Based Planning: RRT* and Informed RRT*

This chapter is not written yet.

Part VI

Tracking and Prediction

The Kalman Filter

This chapter is not written yet.

Nonlinear Filtering: EKF, UKF and Particle Filters

This chapter is not written yet.

Trajectory Prediction: From Constant Velocity to LSTMs and Transformers

This chapter is not written yet.

Part VII

Optimization and Control

Model Predictive Control

This chapter is not written yet.

Mixed-Integer Linear Programming for Planning

This chapter is not written yet.

Consensus and Formation Control

This chapter is not written yet.

Part VIII

Putting It Together

The Hybrid Collision-Avoidance Architecture

This chapter is not written yet.

Evaluating a Hybrid Planner: Experiments, Metrics and Benchmarks

This chapter is not written yet.

Part IX

Appendices

The Twelve-Week Study Plan

This appendix is the book's answer to a practical question: *in which order, and at what pace, should I work through these chapters if I want a working hybrid swarm planner in three months?* It reproduces the twelve-week training plan that the book was built from, maps every week to the chapters that teach it, and adds study notes, a capstone description, a completion checklist and a traceability table. Read it once before you start [Chapter 1](#), and return to it at the beginning of every week.

A.1 How to use this plan

Goal. The goal of the plan is to build enough theory and implementation skill to design a **hybrid swarm planner** that handles two very different kinds of trouble: *planned* conflicts between the drones you control, and *unexpected* moving obstacles such as an unknown drone crossing your formation. The target outcome after twelve weeks is a working research prototype that combines a global multi-agent path finding (MAPF) solver (conflict-based search, CBS, or its bounded-suboptimal variant ECBS, [Chapters 9](#) and [10](#)), a reactive avoidance layer (optimal reciprocal collision avoidance, ORCA, or the dynamic window approach, DWA, [Chapters 13](#) and [14](#)), a prediction layer (a Kalman filter or a long short-term memory network, LSTM, [Chapters 18](#) and [20](#)) and a replanning layer (D* Lite or A*, [Chapters 4](#) and [5](#)). [Chapter 24](#) describes how these pieces fit together and [Chapter 25](#) how to evaluate the result.

The core idea of the whole plan

Use a global planner for planned routes, a local collision-avoidance method for surprises, and a replanner only when the local response is insufficient. Every week of the plan builds one of these three abilities, or the tracking and prediction that feeds them.

Study load. Plan for approximately 6–8 hours per week. The plan prioritizes coding over passive reading, and so does this appendix: a good split is at most two hours of reading and four to six hours at the keyboard. Each week has a *coding exercise* and a *milestone*. The milestone is a sentence you should be able to say truthfully on the last day of the week; if you cannot, spend the first hour of the next week closing the gap rather than moving on.

A weekly rhythm. On the first day, read the week's chapters up to the worked example and trace that example by hand. On the next three days, write the coding exercise, using the chapter's code file as an oracle. On the fifth day, compare your implementation with the reference on the same instance, write the milestone sentence into a lab notebook, and note what you would do differently; keep the weekend as a buffer. The notebook matters more

than it looks: the capstone of Week 12, and any paper you write afterwards, will draw on the numbers, failure cases and plots you produce in Weeks 1–11.

Suggested stack. The plan assumes the following tools. All of them are free, and the book’s own code uses only the first two.

- **Python 3.10+ and NumPy.** Every reference implementation in this book is plain Python with NumPy (SciPy only for the QP and MILP solvers of [Chapters 21](#) and [22](#)). Write your own code the same way; you will read and debug it faster.
- **NetworkX** [\[14\]](#) for quick graph experiments and for checking your graph code against an independent shortest-path implementation.
- **Matplotlib** for the pictures the exercises ask for: OPEN and CLOSED sets, trajectories, velocity obstacles, benchmark curves. A plot you look at every ten minutes is worth more than a printout you look at once.
- **PyBullet or gym-pybullet-drones** [\[29\]](#) for a 3D simulator with quadrotor dynamics. You need it from Week 8 onwards and for the capstone; until then a 2D NumPy simulator is enough and much faster to iterate on.
- **Your own MAPF codebase**, if you have one. The plan was written for a reader who already owns a swarm or MAPF simulator; reuse its grid, instance and visualization code so that each week’s exercise is an addition, not a rewrite.

How the book’s code files support the exercises

Every chapter ships a Python file `code/chNN_*.py` (NN is the chapter number), and every listing in a chapter is a verbatim excerpt of that file. Each file ends with a self-test under `if __name__ == "__main__":` that runs in a few seconds and fails loudly with an `assert` if the implementation is wrong. Use the files in this order: *run* the self-test to see that the reference works; *write* your own version for the week’s exercise without looking at the reference; then *compare* both implementations on the same instance. The reference is your oracle, not your starting point. When the two disagree, the chapter’s worked example, whose numbers were produced by that same file, tells you which one is wrong.

A.2 The schedule at a glance

[Table A.1](#) is the twelve-week schedule of the training plan, with a column added that names the chapters to read. The *Learn*, *Coding exercise* and *Milestone* columns are the plan’s own words. The chapters in the *Read* column are listed in the order in which you should read them; a chapter in parentheses is background or optional depth.

Table A.1. The twelve-week schedule, mapped to the chapters of this book.

Wk	Topic	Read	Learn	Coding exercise	Milestone
1	Foundations: Dijkstra and A*	Chapters 3 and 4 (Chapters 1 and 2)	Graph search, admissible/consistent heuristics, space-time representation.	Implement grid A*. Add time as a state variable. Visualize open/closed sets.	You can explain exactly why A* returns an optimal path under the required conditions.

continued on the next page

Table A.1 (continued)

Wk	Topic	Read	Learn	Coding exercise	Milestone
2	Dynamic replanning: LPA* and D* Lite	Chapter 5 (Chapter 6)	Incremental search and what changes when obstacles appear after planning.	Implement or adapt D* Lite. Change obstacles during execution and compare runtime with rerunning A*.	Your agent can repair a path rather than restarting every search.
3	MAPF basics and conflict types	Chapters 7 and 8	Vertex conflicts, edge/swap conflicts, time-indexed paths, sum-of-costs and makespan.	Implement prioritized planning for 2–5 agents. Create deliberate conflict cases.	You can represent and detect MAPF conflicts correctly.
4	Conflict-Based Search (CBS)	Chapter 9	Constraint tree, high-level search, low-level A*, branching on conflicts.	Implement basic CBS from scratch for a grid. Test 2, 4, and 8 agents.	A working optimal CBS solver on small instances.
5	ECBS and practical MAPF	Chapter 10 (Chapter 11)	Bounded suboptimality, focal search, performance/optimality trade-off.	Add ECBS or reproduce an open implementation. Benchmark against CBS.	You can measure the cost of faster suboptimal solutions.
6	Velocity Obstacles, RVO, and ORCA	Chapters 12 and 13	Collision cones, relative velocity, reciprocal avoidance.	Build a 2D simulation with agents crossing. Compare no avoidance, VO, and ORCA.	You understand how a local velocity choice prevents a predicted collision.
7	DWA and Artificial Potential Fields	Chapters 14 and 15	Short-horizon control, local minima, oscillation, robot dynamics.	Implement DWA or APF in a continuous environment. Create failure cases.	You know when local methods work and when they fail.
8	RRT, RRT*, and continuous 3D planning	Chapters 16 and 17	Sampling, steering, collision checks, path quality in continuous space.	Implement RRT* or use a library in a 3D obstacle field. Compare to grid A*.	You can choose between graph search and sampling-based planning.
9	Tracking unknown moving obstacles	Chapters 18 and 19	Kalman/EKF tracking, state estimation, uncertainty.	Simulate an external drone observed with noisy position/velocity measurements. Track it with a Kalman filter.	Your planner receives a stable estimated state instead of raw noisy observations.
10	Trajectory prediction	Chapter 20	Constant-velocity baselines, LSTM prediction, horizon and uncertainty.	Compare constant velocity vs LSTM on trajectory sequences. Measure ADE/FDE or comparable prediction errors.	You can quantify whether learning actually improves prediction.
11	MPC and formation/communication constraints	Chapters 21 and 23 (Chapter 22)	Receding-horizon optimization, hard/soft constraints, formation error, communication radius.	Create an MPC or simplified constrained controller that follows a path while preserving a formation or distance limit.	Constraints are explicitly enforced rather than handled only after violation.

continued on the next page

Table A.1 (continued)

Wk	Topic	Read	Learn	Coding exercise	Milestone
12	Hybrid system integration	Chapters 24 and 25	Global MAPF + reactive avoidance + prediction + replanning.	Build the capstone architecture (Appendix A.5) and run controlled experiments.	A research prototype and an experiment matrix suitable for a paper/thesis chapter.

A.3 The twelve weeks on one timeline

The plan groups its algorithms into six families, labeled A to F, and asks for a different depth in each: *learn deeply* (A, B, C), *know well* (D, E) and *learn enough to integrate* (F). Figure A.1 shows how the twelve weeks are distributed over the families. Notice that eight of the twelve weeks go to the three “learn deeply” families, and that the capstone gets a full week of its own. If you are short of time, the places to compress are Week 7 (do DWA *or* APF, not both), Week 8 (use a library for RRT*) and the MILP part of Week 11 (read Chapter 22, do not implement it). Do not compress Weeks 1, 4, 6, 9 or 12: each of them builds a layer of the capstone that the later weeks assume.

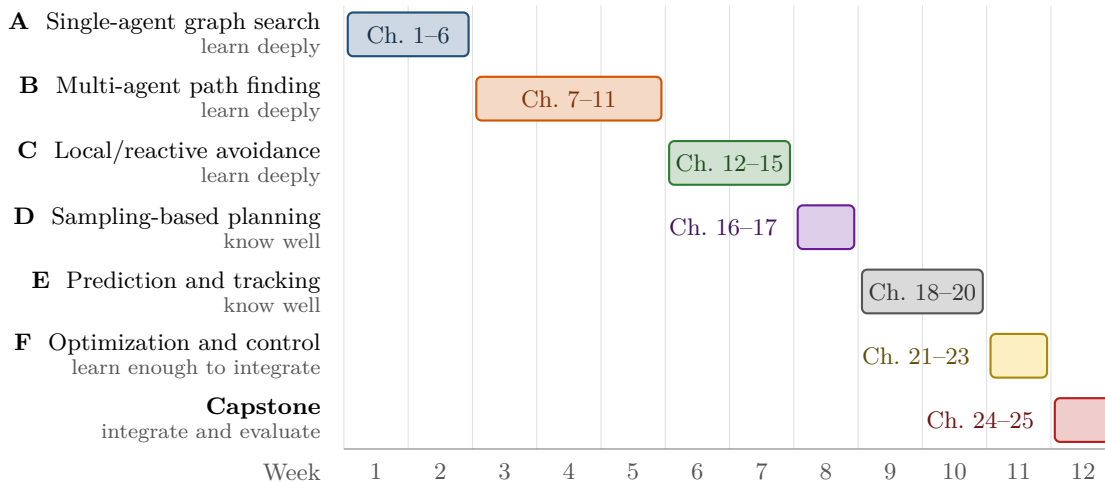


Figure A.1. The twelve weeks grouped by algorithm family. The length of a bar is the number of weeks the plan gives to the family; the label names the chapters read in those weeks (Chapters 1 and 2 are the foundations read in Week 1). Families A–C, which the plan asks you to learn deeply, take eight of the twelve weeks; the capstone has a week of its own.

A.4 Study notes, week by week

The notes below add to each row of Table A.1 what a table cell cannot say: where to put your attention, what to build first, how to know that you are done, which mistakes cost the most time, and which chapter exercises to attempt. The *load* rating, from ★ (light) to ★★★ (heavy), estimates how much of the 6–8 hours the coding exercise takes for a reader who meets the prerequisites of Chapter 2; plan lighter weeks around heavier ones. “The coding exercise of” a chapter means the exercise marked *Coding* at the end of that chapter, which is the plan’s exercise for that week.

Moving on with a red test

The most expensive mistake in a self-study plan is to carry an unfinished week forward. A space-time A^* that mishandles waiting will break the prioritized planner of Week 3, the CBS low level of Week 4 and the replanning layer of Week 12, and each time it will look like a new bug. If the milestone sentence is not true on the last day of the week, spend the first hours of the next week on it and shorten that week's exercise instead; [Appendix A.3](#) names the parts that can be compressed.

A.4.1 Week 1: Foundations – Dijkstra and A^*

Load ★★ Read [Chapters 3](#) and [4](#), with [Chapters 1](#) and [2](#) for the vocabulary and the toolbox.

Focus. Dijkstra's settled-node invariant (a node popped from OPEN with the smallest g has its final cost), and what the heuristic buys in A^* : an admissible one makes the first expansion of the goal optimal, a consistent one makes a CLOSED set safe. Give real time to the space-time state (v, t) with a wait action; it is the low-level search of every MAPF solver in Part III.

Build. Grid A^* with a pluggable heuristic (Manhattan, octile, Euclidean, zero); the same search over (v, t) with a wait action and vertex and edge constraints; a plot that colors cells by their OPEN/CLOSED status.

Done when. You can state the optimality theorem with its conditions and say what breaks when each is dropped; your space-time search waits or detours around a cell reserved at a given time; with the zero heuristic both searches expand the same nodes as Dijkstra.

Traps. Unit diagonal cost on an 8-connected grid makes the octile and Euclidean heuristics inadmissible; a space-time search without a time bound never terminates when the goal is unreachable; a search that stops the instant it reaches the goal cannot honor a later constraint on the goal cell.

Exercises. The coding exercise of [Chapter 4](#) is the deliverable. Add the coding exercise of [Chapter 3](#); that chapter's backward Dijkstra becomes the low-level heuristic of CBS in Week 4.

A.4.2 Week 2: Dynamic replanning – LPA* and D* Lite

Load ★★★ Read [Chapter 5](#); [Chapter 6](#) if time permits.

Focus. Each vertex carries g and rhs and is locally inconsistent when they differ; keys, UpdateVertex and ComputeShortestPath only repair the inconsistent vertices, in the right order. D* Lite searches backward from the goal so that a moving start needs only the key modifier k_m instead of a re-sort of the queue.

Build. Turn the Week-1 search into D* Lite; move the agent one step at a time while obstacles appear and disappear ahead of it; count expansions and time per repair against a fresh A^* from the current position.

Done when. Expansions after a small change are a small fraction of a fresh A^* , and the path cost after every repair equals the fresh A^* cost; that equality is the correctness test.

Traps. Not adding to k_m when the start moves, which leaves stale keys in the queue; updating only one endpoint of a changed edge; timing grids so small that Python overhead dominates (count expansions too); changing the map inside ComputeShortestPath.

Exercises. The coding exercise of [Chapter 5](#); then, if time permits, the coding exercise of [Chapter 6](#), since an anytime search gives the replanning layer a path within a time budget, which D* Lite alone does not promise.

A.4.3 Week 3: MAPF basics and conflict types

Load ★★ Read [Chapters 7](#) and [8](#).

Focus. The formal problem of [Chapter 7](#): agents, time steps, wait actions, the conflict types (vertex, edge or swap, following, cycle) and the two objectives (sum of costs, makespan); a swap is invisible if you compare positions only. Then [Chapter 8](#): prioritized planning as one space-time A^* per agent against a reservation table, fast but incomplete and suboptimal in general.

Build. A conflict checker that lists every conflict of a set of time-indexed paths with its type and time; prioritized planning for 2–5 agents; three deliberate conflict instances (head-on in a corridor, a crossing, an agent parked on another agent’s route).

Done when. The checker finds every planted conflict and nothing else; the planner solves the well-formed instances; you own an instance that one priority order fails and another solves.

Traps. Off-by-one in time (the position at time 0 is the start; a swap compares (u, v) at t with (v, u) at t); an agent that has arrived stays at its goal and must be reserved there for all later times; a reservation table that stores vertices but not edges.

Exercises. The coding exercises of [Chapters 7](#) and [8](#). Keep the checker: it is the conflict detector of your CBS high level next week and the “re-check inter-agent conflicts” step of the capstone.

A.4.4 Week 4: Conflict-Based Search

Load ★★★ Read [Chapter 9](#).

Focus. The constraint tree: a node holds constraints, one path per agent and the sum of costs; the high level expands the cheapest node, takes its first conflict and creates two children, each forbidding it for one agent; the low level replans only that agent with the space-time A^* of Week 1. Optimality follows because every conflict-free solution satisfies the constraint of at least one child.

Build. CBS from scratch on a grid: vertex constraints (agent, vertex, time), edge constraints (agent, edge, time), a constrained low level and the high-level search; test with 2, 4 and 8 agents on an open grid and on the Week-3 corridors.

Done when. Your plans pass the Week-3 checker; the sum of costs equals that of the reference implementation of [Chapter 9](#) on every instance; you can draw the constraint tree of a two-agent instance by hand and match it against your program’s log.

Traps. An edge constraint has a direction and a time; a low level that returns when it first pops the goal ignores later constraints on the goal cell; sorting the high level by makespan while reporting sum of costs; missing conflicts after the shorter path has ended.

Exercises. The coding exercise of [Chapter 9](#). Read about cardinal conflicts and bypass in the same chapter, but do not implement them yet; they matter when eight agents become twenty.

A.4.5 Week 5: ECBS and practical MAPF

Load ★★ Read [Chapter 10](#); [Chapter 11](#) for contrast.

Focus. Bounded suboptimality: a focal search may expand any node with $f \leq w \cdot f_{\min}$ and picks the one with the fewest conflicts; ECBS does this at both levels and guarantees a cost within a factor w of optimal. Know where the lower bound comes from (each agent’s low-level $f_{\min}$, summed).

Build. Turn your CBS into ECBS, or reproduce an open implementation as the plan allows; benchmark both for $w \in \{1.0, 1.1, 1.5, 2.0\}$: success rate under a time limit, runtime, high-level nodes and sum of costs, over several seeds.

Done when. A plot of cost against runtime as w varies, with $w = 1$ reproducing CBS exactly, and a paragraph that says with numbers what suboptimality buys.

Traps. Using the current node's f instead of the minimum over OPEN as the focal bound; taking the returned path cost instead of $f_{\min}$ as the lower bound; not growing the focal list when $f_{\min}$ rises; concluding from one instance.

Exercises. The coding exercise of Chapter 10. Read Chapter 11 to know how M^* and Push-and-Swap differ from CBS; the plan asks for awareness, not an implementation.

A.4.6 Week 6: Velocity Obstacles, RVO and ORCA

Load ★★★ Read Chapters 12 and 13.

Focus. Everything is relative: relative position, relative velocity and a Minkowski disc whose radius is the sum of the two radii. The velocity obstacle (VO) is the set of relative velocities that reach the disc within the horizon τ ; the reciprocal velocity obstacle (RVO) shares the avoidance between cooperating agents; ORCA turns it into one half-plane per neighbor, so that the new velocity is a small linear program.

Build. A 2D simulation of crossing agents (two head-on, then eight on a circle exchanging places) in three modes, no avoidance, VO and ORCA, recording collisions, minimum separation and time to goal.

Done when. No avoidance collides, VO avoids but oscillates or detours widely, ORCA is smooth and collision-free at the same speeds, and you can draw the cone and the ORCA half-plane for one pair.

Traps. The orientation of the half-plane (it must exclude the velocity obstacle); the truncated cone for finite τ is a disc plus two tangent legs, not a triangle; when the linear program is infeasible in a crowd, choose the least-violating velocity rather than stopping; symmetric instances deadlock unless you break the symmetry.

Exercises. The coding exercises of Chapters 12 and 13. Keep the simulator; Week 7 and the capstone reuse it.

A.4.7 Week 7: DWA and Artificial Potential Fields

Load ★★ Read Chapters 14 and 15.

Focus. DWA searches velocity space: the dynamic window holds the velocities reachable within one control interval, admissible velocities are those from which the robot can still stop, and a weighted sum of heading, clearance and velocity terms picks the command. Artificial potential fields (APF) have no search, only forces, attractive to the goal and repulsive from obstacles, which is why APF is fast and why it has local minima and oscillation.

Build. One of the two (DWA if your drones have real acceleration limits, APF as a two-hour baseline) in the Week-6 simulator; then failure cases: a U-shaped obstacle, a narrow passage, a dead end that a short DWA horizon cannot see.

Done when. A picture of each failure with the reason, and a tuning table of how the DWA weights or the APF gains change the behavior.

Traps. DWA terms of different units summed without normalization; skipping the braking (admissibility) check; repulsive forces that grow without bound near obstacles (clamp them); comparing the methods at different control rates.

Exercises. The coding exercise of Chapter 14 or of Chapter 15, plus the conceptual exercises of the other chapter. DWA is the plan's alternative to ORCA in the local safety layer.

A.4.8 Week 8: RRT, RRT* and continuous 3D planning

Load ★★ Read Chapters 16 and 17.

Focus. Sample, nearest, steer and collision-free plus a goal bias make RRT; the near set, parent choice and rewiring with a shrinking connection radius make RRT* asymptotically

optimal; informed sampling restricts samples to the ellipsoid that can still improve the solution. Know what probabilistic completeness and asymptotic optimality promise, and that neither says anything about finite time.

Build. RRT*, yours or a library's, in a 3D field of box or sphere obstacles; grid A^* on the same field at two or three resolutions; compare path length, time and memory.

Done when. A convergence plot of the RRT* cost against iterations, a table of A^* per resolution against RRT* per sample budget, and a one-paragraph rule for when you would choose each.

Traps. Checking the new vertex but not the segment to it; a quadratic nearest-neighbor search (use a KD-tree above a few thousand nodes); a rewire that changes a parent without updating the descendants' costs; comparing raw paths without smoothing either.

Exercises. The coding exercises of [Chapters 16](#) and [17](#). The 3D field you build here is the map of the capstone.

A.4.9 Week 9: Tracking unknown moving obstacles

Load ★★ Read [Chapters 18](#) and [19](#).

Focus. Predict with the motion model (the state moves, the covariance grows), update with the measurement (the gain weighs model against measurement). The constant-velocity model with white-noise-acceleration process noise is the workhorse; gating rejects outliers; a missing measurement is a predict without update. [Chapter 19](#) adds the extended and unscented Kalman filters (EKF, UKF) and the particle filter for targets that turn.

Build. An external drone on a curved path, observed with noisy position (optionally velocity) measurements at 10–20 Hz with dropouts; a Kalman filter with tuned $\mathbf{Q}$ and $\mathbf{R}$; the error of the estimate against the truth next to the error of the raw measurements.

Done when. The estimate beats the raw measurements, the velocity estimate extrapolates one or two seconds ahead without jitter, and the Week-6 simulator consumes the estimate and its covariance instead of the observation.

Traps. $\mathbf{R}$ below the real noise (the filter trusts noise) or $\mathbf{Q}$ too small (it lags on turns and becomes overconfident); a wrong Δt in the transition matrix; trusting the covariance when the model is violated (check the normalized innovations); tuning on one trajectory.

Exercises. The coding exercise of [Chapter 18](#); the coding exercise of [Chapter 19](#) if your intruder turns sharply.

A.4.10 Week 10: Trajectory prediction

Load ★★★ Read [Chapter 20](#).

Focus. Average and final displacement error (ADE, FDE) as the yardsticks; constant-velocity and Kalman baselines; the LSTM as a sequence model that reads past positions and writes future ones; horizon, uncertainty and the evaluation protocol.

Build. Trajectories from the Week-6 simulator (agents that avoid one another are not constant-velocity, which is what makes learning worthwhile), split by trajectory rather than by time step; both baselines; an LSTM (a deep-learning framework is fine; the book's own code stays in NumPy); ADE/FDE at 1, 2 and 3 s.

Done when. A table of ADE/FDE per horizon and method over several training seeds, and an honest conclusion: “the LSTM did not beat constant velocity on this data” is a legitimate and common result at short horizons.

Traps. Leakage between training and test trajectories; predicting absolute coordinates instead of displacements in the agent's frame; a weak baseline that uses two raw points instead of a smoothed velocity; comparing at different horizons; tuning on the test set.

Exercises. The coding exercise of [Chapter 20](#). Save the model and the baselines; the “prediction quality” axis of the capstone uses all of them.

A.4.11 Week 11: MPC and formation/communication constraints

Load ★★★ Read [Chapters 21](#) and [23](#); [Chapter 22](#) as background.

Focus. Receding horizon: solve a short optimal control problem, apply the first input, repeat. With the double-integrator model and a quadratic cost it is a quadratic program; velocity and acceleration bounds are linear; separation constraints are nonconvex and are linearized around the previous solution; slack variables keep the program feasible. From [Chapter 23](#), displacement-based formation and the formation error. Mixed-integer linear programming (MILP, [Chapter 22](#)) is the exact alternative; read it for the cost of exactness.

Build. A model predictive controller (MPC), or the simplified constrained controller the plan allows, that follows a nominal path while keeping the formation error bounded or every pairwise distance below the communication radius; log violations and slack.

Done when. Zero violations with hard constraints, logged and small slack with soft ones, a plot of the formation error over time, and a visible difference when the constraint is switched off.

Traps. A horizon too long for the time budget or too short to see a conflict coming; hard constraints that turn infeasible when an intruder closes in (use slack); linearizing around a stale trajectory; a model Δt that differs from the control interval.

Exercises. The coding exercises of [Chapters 21](#) and [23](#); the coding exercise of [Chapter 22](#) is optional.

A.4.12 Week 12: Hybrid system integration

Load ★★★ Read [Chapters 24](#) and [25](#).

Focus. The architecture and decision logic of [Appendix A.5](#): the safety horizon and the time-to-collision trigger, reconnection to the nominal path, the replanning triggers, the conflict re-check, and the experiment matrix and metrics of [Chapter 25](#).

Build. The capstone: the four layers you already own, connected behind clean interfaces, and an experiment runner that takes a scenario and a seed and writes one row of metrics per run.

Done when. The experiment matrix runs unattended from one script, every run is reproducible from its seed, and a table and plots compare local-only, replan-only and hybrid strategies under the same conditions.

Traps. One file that mixes the layers; different time steps for the discrete global plan and the continuous local layer (convert in one place); measuring success rate only; running experiments before each layer passes its own tests.

Exercises. The coding exercises of [Chapters 24](#) and [25](#).

A.5 The capstone: a hybrid collision-avoidance prototype

The capstone is the plan’s target outcome: a research prototype that combines a global multi-agent planner, a prediction layer, a local safety layer and a replanning layer. Everything you built in Weeks 1–11 is a component of it. [Chapter 24](#) designs the architecture in full, with its state machine, its safety horizon and a discussion of which guarantees survive; [Chapter 25](#) shows how to evaluate it. This section restates the plan’s specification so that you can check your prototype against it.

A.5.1 The four layers

Table A.2 lists the four layers in the plan’s words. The numbering is the plan’s; it is neither the order in which the layers act at run time (that is the decision logic below) nor the order in which you build them (that is the schedule).

Table A.2. The four layers of the capstone architecture, in the plan’s words, with the chapters that teach each layer and the weeks in which you build it.

Layer	Job	Chapters	Weeks
1 Global swarm planner (CBS/ECBS)	Plan nominal, time-indexed paths for the drones under your control. Resolve conflicts among connected swarm members before execution.	Chapters 9 and 10	3–5
2 Prediction layer (Kalman filter / LSTM)	Observe an unknown external drone. Estimate its current state and predict a short future trajectory with uncertainty.	Chapters 18 and 20	9–10
3 Local safety layer (ORCA or DWA)	When a predicted collision enters a short safety horizon, generate a temporary safe velocity or local maneuver without immediately discarding the global route.	Chapters 13 and 14	6–7
4 Replanning layer (D^* Lite or A^*)	If the local maneuver cannot safely return the drone to its nominal route, replan from the current state. Then re-check swarm conflicts and formation/communication constraints.	Chapters 4 and 5	1–2, 11

A.5.2 The decision logic

The plan suggests five steps for every controlled drone, at every control cycle. Chapter 24 turns them into a state machine with explicit triggers (a time to collision below a threshold, a reconnection cost above a bound, a violated constraint) and explains what happens to the optimality of the CBS plan once a drone leaves it.

1. Follow the nominal CBS/ECBS path while no predicted conflict is inside the safety horizon.
2. If an unknown moving object creates a near-term risk, invoke ORCA/DWA to produce a local avoidance action.
3. After the conflict clears, attempt to reconnect to the nominal path at a safe future waypoint.
4. If reconnection is impossible, too costly, or violates formation/communication constraints, run D^* Lite/ A^* from the current state.
5. Re-check inter-agent conflicts after any significant replan.

Keep the layers separable

Give each layer one entry point: the global planner takes an instance and returns time-indexed paths; the predictor takes observations and returns a predicted trajectory with covariance; the local layer takes states and predictions and returns a velocity; the replanner takes a start state and returns a path. The “avoidance strategy” axis of the experiments is then a switch, not a rewrite: local-only disables layer 4, replan-only disables layer 3, the hybrid uses both.

A.5.3 Experiments and metrics

Table A.3 lists the factors the plan asks you to vary; Chapter 25 gives the scenario generator and the statistics to report. As a full factorial design the five factors have $4 \times 4 \times 4 \times 3 \times 4 = 768$ cells before seeds; you will not run all of them. Fix every factor at a default, vary one or two at a time, and let the research question of Chapter 24 decide which.

Table A.3. The experiment factors of the capstone and their levels, in the plan’s words.

Factor	Levels	Where in this book
Controlled drones	2, 4, 8, and more if computationally feasible	Chapter 25; scaling of CBS/ECBS in Chapters 9 and 10
Unknown external drones	0, 1, 2, and multiple crossing trajectories	Chapter 25
Prediction quality	perfect state, noisy state, constant-velocity prediction, learned prediction	Chapters 18, 20 and 25
Avoidance strategy	local-only, global-replan-only, and hybrid local + replanning	Chapters 24 and 25
Constraints	no formation requirement vs formation-preserving; no communication radius vs bounded communication range	Chapters 21 and 23; during avoidance in Chapter 24

Metrics. Report, for every run, the metrics the plan lists: **collision rate**, **minimum separation**, **path length**, **travel time**, **makespan**, **sum of costs**, **replanning count**, **computation time**, **formation error** and **communication violations**. Chapter 25 defines each of them and the statistics to report over seeds; makespan and sum of costs are the MAPF objectives of Chapter 7, and the replanning count is the direct measure of the decision logic of Chapter 24, since it counts how often the local layer was not enough. Two of them deserve a warning: a collision rate of zero hides near misses, which is what minimum separation is for, and a mean computation time hides the cycle that missed its deadline, so report the maximum as well.

A.6 Completion checklist

The plan ends with ten statements. Table A.4 repeats them and names the chapters where each ability is taught. Tick a line only when you can do what it says without opening the book; together, the ten lines are a fair summary of what a thesis committee or a reviewer will expect you to know.

A.7 Traceability: from the plan to the chapters

Table A.5 maps every algorithm of the training plan to the chapter that teaches it and the week that uses it, with the plan’s priority tag; where the plan qualified a tag, the qualification is kept next to it. The plan’s one-line reason for each algorithm is in the algorithm map of Chapter 1. Use the table in two directions: to check that the book covers everything the plan asks for, and, when you open a chapter, to see how much depth the plan expects of you there.

Table A.4. The plan’s completion checklist, with the chapters that teach each item.

Statement	Where in this book
☐ I can implement and explain A^* and space-time A^* .	Chapter 4; reservation tables in Chapter 8
☐ I can explain the CBS constraint tree and implement vertex and edge constraints.	Chapter 9; conflict types in Chapter 7
☐ I understand the speed/optimality trade-off in ECBS.	Chapter 10
☐ I can explain VO/RVO/ORCA using relative position and velocity.	Chapters 12 and 13
☐ I can track an unknown drone from noisy observations.	Chapters 18 and 19
☐ I can compare a simple prediction baseline with LSTM prediction.	Chapter 20
☐ I can trigger local avoidance based on a time-to-collision or safety-horizon rule.	time to collision in Chapter 12; the trigger in Chapter 24
☐ I can reconnect to the global route or trigger D^* Lite/ A^* replanning.	Chapter 5; reconnection in Chapter 24
☐ I can enforce formation and/or communication-range constraints.	Chapters 21 and 23; during avoidance in Chapter 24
☐ I can evaluate the hybrid system with reproducible metrics and controlled scenarios.	Chapter 25

Table A.5. Every algorithm of the training plan, its priority, and where this book teaches it.

Algorithm	Priority in the plan	Chapter	Wk
A. Single-agent graph search – learn deeply			
Dijkstra	High	Chapter 3	1
A^*	Essential	Chapter 4	1
D^* Lite	Essential	Chapter 5	2
Lifelong Planning A^* (LPA*)	Medium	Chapter 5	2
Anytime Repairing A^* (ARA*)	Medium	Chapter 6	2
B. Multi-agent path finding – learn deeply			
Prioritized Planning	High	Chapter 8	3
Conflict-Based Search (CBS)	Essential	Chapter 9	4
Enhanced CBS (ECBS)	Essential	Chapter 10	5
M^*	Medium	Chapter 11	5
Push-and-Swap / Push-and-Rotate	Awareness	Chapter 11	5
C. Local/reactive collision avoidance – learn deeply			
Velocity Obstacles (VO)	Essential	Chapter 12	6
Reciprocal Velocity Obstacles (RVO)	High	Chapter 13	6
ORCA	Essential	Chapter 13	6
Dynamic Window Approach (DWA)	High	Chapter 14	7
Artificial Potential Fields (APF)	High	Chapter 15	7
D. Sampling-based motion planning – know well			
RRT	High	Chapter 16	8
RRT*	High	Chapter 17	8
Informed RRT*	Medium	Chapter 17	8

continued on the next page

Table A.5 (continued)

Algorithm	Priority in the plan	Chapter	Wk
E. Prediction and tracking – know well			
Kalman Filter	Essential	Chapter 18	9
Extended Kalman Filter (EKF)	High	Chapter 19	9
Unscented Kalman Filter (UKF)	Medium	Chapter 19	9
Particle Filter	Medium	Chapter 19	9
LSTM trajectory prediction	Essential for your planned research	Chapter 20	10
Transformer trajectory prediction	Awareness to High	Chapter 20	10
F. Optimization and control – learn enough to integrate			
Model Predictive Control (MPC)	Essential	Chapter 21	11
Mixed-Integer Linear Programming (MILP)	Medium	Chapter 22	11
Consensus / formation control	Essential for formation-preserving work	Chapter 23 (Chapter 21)	11

A.8 Reading order and primary sources

The plan also gives a reading order for the literature, reproduced in Table A.6 together with the chapter that covers each item and the primary source to read next to it. The first six items are the plan’s highest priority: they are the layers of the capstone plus the MAPF definitions that make the conflict re-check precise. To read a paper next to its chapter: read the chapter up to the end of its worked example; then read the paper’s algorithm section and compare its pseudocode line by line with the chapter’s; then read the paper’s experiments and ask which of its claims your implementation of that week reproduces. Item 10 is a book rather than a paper: LaValle’s *Planning Algorithms* [25] is the reference to keep open for anything about configuration spaces, sampling and collision checking that Chapters 2, 16 and 17 treat only as far as this book needs.

Table A.6. The plan’s reading order, mapped to chapters and to the primary sources.

#	Read	Chapter(s)	Primary source
1	A*	Chapter 4	Hart, Nilsson and Raphael [16]
2	D* Lite	Chapter 5	Koenig and Likhachev [21]
3	CBS	Chapter 9	Sharon et al. [36]
4	MAPF overview	Chapter 7	Stern et al. [39]
5	Velocity Obstacles	Chapter 12	Fiorini and Shiller [11]
6	ORCA	Chapter 13	van den Berg et al. [2]
7	DWA	Chapter 14	Fox, Burgard and Thrun [12]
8	RRT	Chapter 16	LaValle [24]
9	RRT*	Chapter 17	Karaman and Frazzoli [20]
10	Planning reference	Chapters 2, 16 and 17	LaValle, <i>Planning Algorithms</i> [25]
11	MPC	Chapter 21	Rawlings, Mayne and Diehl [33]
12	Trajectory prediction	Chapter 20	Rudenko et al. (survey) [34]; Alahi et al. (Social LSTM) [1]

After Week 12. The plan is explicit that the individual algorithms above are established, and that the research lies in how they are combined, constrained, predicted and evaluated: unknown non-cooperative drones, local avoidance that does not needlessly destroy the guarantees of the global plan, formation and communication constraints during avoidance, prediction-aware constraints that carry uncertainty, an automatic decision between local action and global replan, computation for larger swarms, and benchmarks that compare local-only, replan-only and hybrid approaches under the same conditions. [Chapter 24](#) discusses these directions and [Chapter 25](#) the benchmark; the prototype and the experiment matrix you finish in Week 12 are the starting point for both.

Mathematical Refresher

This chapter is not written yet.

Hints and Solutions to Selected Exercises

This appendix collects hints and solutions for selected exercises. Solutions are grouped by chapter; each entry names the exercise it answers.

Exercise 1.1. (a) Global planner: a planned conflict, known before take-off, resolved by CBS or ECBS. (b) Prediction layer plus local safety layer: the bird is tracked and its motion predicted; with a time to collision of 1.5 s inside the safety horizon, the local layer produces an avoidance velocity. (c) Replanning layer: the local layer can keep C clear of the hovering intruder but cannot make progress through the blocked corridor, so reconnection fails and D^* Lite replans from the current state; the new path must then be re-checked against the swarm. (d) Global planner: the zone is known before the flight, so it becomes part of the map. (e) Global planner, in its re-check role (step 5 of the decision logic): a new inter-agent conflict is a planned conflict again. (f) Prediction layer: the Kalman filter turns the noisy measurements into a state estimate and the predictor extrapolates it. (g) None of the four layers: the flight controller tracks the commanded velocity and corrects the disturbance; only if the deviation grew large enough to threaten a conflict would the local layer notice it. (h) Local safety layer with a formation constraint, and the replanner if the constraint cannot be met; how to trade the constraint against the collision risk is one of the research directions of [Chapter 1](#).

Exercise 1.4. (a) In a corridor one cell wide, two drones heading towards each other see the same situation mirrored. A reactive rule maps the current situation to an action and has no memory and no look-ahead, so both drones slow down, stop, or try to sidestep into the same wall; neither has a reason to back up to the last passing place, because backing up does not reduce the immediate collision risk and moves the drone away from its goal. They hover facing each other: a deadlock. (b) Four drones at the corners of a square, each heading for the opposite corner, all meet in the middle at the same time. A symmetric rule gives symmetric actions, so the drones either all stop at the same distance from the centre or all circle around it; nothing in the rule breaks the symmetry, and none of them reaches its goal. A global plan adds exactly what the reactive rule lacks: knowledge of the other drones' full routes and of time, which lets a planner decide that one drone waits or takes a detour *before* the encounter. A random component does break the symmetry and often resolves the deadlock in practice, but at the cost of losing every guarantee: there is no bound on how long the resolution takes, the outcome is not reproducible, and in a corridor a random choice can still oscillate for a long time. [Chapter 13](#) shows this behaviour with ORCA and [Chapter 15](#) with potential fields.

Exercise 1.5. (a) At $t = 4$ drone A is at its fifth cell $(3, 3)$ and drone C at its fifth cell $(3, 3)$: a vertex conflict. An edge conflict would need A and C to exchange cells between two consecutive steps; at $t = 3 \rightarrow 4$ the moves are $(4, 3) \rightarrow (3, 3)$ and $(3, 4) \rightarrow (3, 3)$, and at $t = 4 \rightarrow 5$ they are $(3, 3) \rightarrow (2, 3)$ and $(3, 3) \rightarrow (3, 2)$, so no swap occurs. (b) The new path of C is $(6, 4), (5, 4), (4, 4), (3, 4), (3, 4), (3, 3), (3, 2), (3, 1)$ with 7 steps. At $t = 4$ drone C is at

(3, 4) and A at (3, 3); at $t = 5$ drone C enters (3, 3) while A has moved on to (2, 3), so there is no conflict, and `find_conflicts` confirms it. The sum of costs is $10 + 9 + 7 = 26$ and the makespan stays 10. Letting A wait instead costs the same: sum of costs 26, makespan 11 because A is the longest path, so waiting C is better for the makespan. (c) The relative position is $\mathbf{p}_I(t) - \mathbf{p}_B(t) = (11 - 0.54t - 1.5 - t, 7.5 - 0.38t - 5.5) = (9.5 - 1.54t, 2 - 0.38t)$. Its squared norm is a parabola in t whose minimum lies where the derivative vanishes: $1.54(9.5 - 1.54t) + 0.38(2 - 0.38t) = 0$, that is, $t^* = (1.54 \cdot 9.5 + 0.38 \cdot 2) / (1.54^2 + 0.38^2) = 15.39 / 2.5160 \approx 6.12$. At that time the relative position is (0.08, -0.32) and the distance is ≈ 0.33 cells, slightly less than the 0.38 that the table reports at the integer time $t = 6$; checking only integer steps can miss the true closest approach. (d) With the book's code, waiting one step gives a minimum distance of 0.72 cells (at $t = 7$), two steps 0.98, and three steps 1.06, so three wait steps are needed. Such a decision would be made by the local safety layer, during the flight, from the prediction; before take-off the intruder is not known, and even if it were, its velocity is a prediction that may be wrong by the time the wait would matter.

Exercise 1.8. Hints rather than a full solution. (a) Draw the obstacles with `random.Random(seed).sample`, then draw $2k$ distinct free cells and keep them only if every start-goal pair is at a minimum Manhattan distance and breadth-first search reaches every goal; redraw the obstacles if that fails too often. For the intruder, choose a border side, a random entry point on it and a random exit point on the opposite side, and normalise the difference to a random speed; walk along the line in small steps and reject it if it enters a blocked cell. (b) Clamp the time index to the last cell so that a drone stays at its goal; a vertex conflict is $v_t^i = v_t^j$, an edge conflict is $v_t^i = v_{t+1}^j$ and $v_{t+1}^i = v_t^j$ with $v_t^i \neq v_{t+1}^i$. (c) With the book's generator on a 12×8 grid with 15% blocked cells and seeds 0 to 199, the fraction of scenarios whose independent shortest paths conflict is about 0.22 for $k = 2$, 0.60 for $k = 4$ and 0.99 for $k = 8$; your numbers will differ with the tie-breaking of your search and the minimum start-goal distance, but the trend is the same: with a handful of drones on a small map, planning independently almost always produces a collision, which is why [Chapters 7 to 9](#) exist. (d) Store the intruders as a list of position-velocity pairs and keep the check that every line is free; the book's `to_json` method shows a suitable layout.

Exercise 2.4. The time-expanded graph with horizon T has $|V| (T + 1)$ vertices, one copy of every vertex per time layer. Between two consecutive layers there are $|V|$ wait edges and, for an undirected base graph, $2|E|$ move edges (each edge $\{u, v\}$ can be traversed in both directions), so there are $T(|V| + 2|E|)$ edges in total. Every edge goes from layer t to layer $t + 1$, so no edge ever returns to an earlier layer and the graph is acyclic. A time-indexed path $(v_0, \dots, v_T)$ visits one vertex per layer, and consecutive vertices are either equal (a wait edge) or adjacent in G (a move edge); hence it is exactly a path from $(v_0, 0)$ to (v_T, T) in the time-expanded graph.

Exercise 2.5. For the plan of [Example 2.20](#) the times are $T_1 = 3$ and $T_2 = 3$, so $\text{MS} = 3$ and $\text{SoC} = 6$. For the instance in which the two objectives disagree, take a corridor of cells $(0, 1), \dots, (5, 1)$ with agent A going from $(0, 1)$ to $(5, 1)$, agent B_1 from $(1, 2)$ to $(1, 0)$ and agent B_2 from $(2, 3)$ through $(2, 2)$ to $(2, 0)$; every other cell is blocked. Without waiting, B_1 would enter cell $(1, 1)$ at $t = 1$ and B_2 would enter $(2, 1)$ at $t = 2$, exactly when A is there. If A never waits ($T_A = 5$), both crossing agents must wait one step: $T_{B_1} = 3$, $T_{B_2} = 4$, giving $\text{MS} = 5$ and $\text{SoC} = 12$. If instead A waits one step at its start, neither crossing agent is delayed: $T_A = 6$, $T_{B_1} = 2$, $T_{B_2} = 3$, giving $\text{MS} = 6$ and $\text{SoC} = 11$. No plan does better on both counts. Every agent needs at least its unobstructed travel time (5, 2 and 3 steps), so $\text{SoC} \geq 10$ with equality only if nobody waits, which is impossible because of the two conflicts. Hence $\text{SoC} \geq 11$, and a plan with $\text{SoC} = 11$ contains exactly one unit of delay; a delay of B_1

alone leaves the conflict of A with B_2 , a delay of B_2 alone leaves the conflict with B_1 , so the delayed agent must be A and the makespan is 6. Conversely, makespan 5 forces A to move without waiting, so B_1 cannot enter $(1, 1)$ before $t = 2$ and B_2 cannot enter $(2, 1)$ before $t = 3$, which gives $\text{SoC} \geq 5 + 3 + 4 = 12$. The two objectives therefore disagree on this instance.

Exercise 2.7. With $\mathbf{p} = \mathbf{p}_B - \mathbf{p}_A$ and $\mathbf{v} = \mathbf{v}_B - \mathbf{v}_A$ the squared distance is $D(t) = \|\mathbf{p} + t\mathbf{v}\|^2 = \mathbf{p} \cdot \mathbf{p} + 2t \mathbf{p} \cdot \mathbf{v} + t^2 \mathbf{v} \cdot \mathbf{v}$, a parabola in t with $D'(t) = 2 \mathbf{p} \cdot \mathbf{v} + 2t \mathbf{v} \cdot \mathbf{v}$. Setting $D'(t^*) = 0$ gives $t^* = -(\mathbf{p} \cdot \mathbf{v})/(\mathbf{v} \cdot \mathbf{v})$. Writing $\mathbf{p} = \mathbf{p}_{\parallel} + \mathbf{p}_{\perp}$ with $\mathbf{p}_{\parallel}$ along $\mathbf{v}$, the term $\mathbf{p}_{\parallel} + t^* \mathbf{v}$ vanishes, so $d_{\min} = \|\mathbf{p}_{\perp}\| = |\mathbf{p} \times \mathbf{v}| / \|\mathbf{v}\|$ because $|\mathbf{p} \times \mathbf{v}| = \|\mathbf{p}_{\parallel}\| \|\mathbf{v}\| |\sin \phi|$ is the area of the parallelogram spanned by the two vectors. For $\mathbf{p} = (4, 2)$, $\mathbf{v} = (-1, -1)$: $t^* = 6/2 = 3$, $\mathbf{p} \times \mathbf{v} = -4 + 2 = -2$, $d_{\min} = 2/\sqrt{2} = \sqrt{2}$. Along the line of motion the disc of radius R is entered at $t_c = t^* - \sqrt{R^2 - d_{\min}^2} / \|\mathbf{v}\|$, which for $R = 1.5$ gives $t_c = 3 - \sqrt{0.25}/\sqrt{2} \approx 2.646$; for $R = 1 < \sqrt{2}$ the square root is undefined and the discs never touch, in agreement with `time_to_collision` in `ch02_toolbox.py`.

Exercise 2.9. The characteristic polynomial of $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$ is $\lambda^2 - 3\lambda + 0.56 = 0$, so $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$. The eigenvector of λ_1 solves $-0.8e_x + 1.2e_y = 0$, hence $\mathbf{e}_1 \propto (3, 2)$ and the major axis makes the angle $\arctan(2/3) \approx 33.7^\circ$ with the x -axis. The 1σ semi-axes are $\sqrt{2.8} \approx 1.673$ and $\sqrt{0.2} \approx 0.447$; the 2σ ellipse doubles both. In two dimensions the mass inside the $k\sigma$ ellipse is $1 - e^{-k^2/2}$: 39.3% for $k = 1$ and 86.5% for $k = 2$ (not 68% and 95%, which are the one-dimensional values); the 400 seeded samples of Figure 2.7 give 39.8% and 85.8%.

Exercise 3.1. With $c(B, E) = 4$ the second step pushes $(6, E)$ instead of $(9, E)$, and the rest of the trace changes accordingly. The pops are $(0, A)$, $(2, B)$, $(3, C)$, $(5, C)$ stale, $(5, D)$, $(6, D)$ stale, $(6, E)$, $(8, G)$, $(9, F)$, $(10, F)$ stale and $(13, F)$ stale: 11 pops and 4 stale entries. When D is expanded, E already has the tentative cost $6 < 5 + 3$, so D pushes only $(13, F)$; when E is expanded it pushes $(10, F)$ and $(8, G)$; G then improves F to 9. The final distances are $\delta(A, B) = 2$, $\delta(A, C) = 3$, $\delta(A, D) = 5$, $\delta(A, E) = 6$, $\delta(A, G) = 8$ and $\delta(A, F) = 9$, and the shortest path to G is A, B, E, G of cost 8. The self-test of `ch03_dijkstra.py` contains this variant.

Exercise 3.2. F is pushed for the first time in step 5 with cost 13 and parent D , so a search that stops at discovery reports $\delta(A, F) = 13$ and the path A, B, C, D, F , whereas the true distance is 11 along A, B, C, D, E, G, F . For G the first push, in step 7, already carries the optimal cost 10, but only because no cheaper route to G exists; the algorithm has no way of knowing that at the time. The proof of Theorem 3.7 compares the key of u with the keys still in the heap *because the heap returned u as its minimum*. A vertex that has merely been pushed has not been returned by the heap, so nothing relates its tentative cost to the keys of the other open vertices; the only guarantee it has is $g(v) \geq \delta(s, v)$, which is the wrong direction.

Exercise 3.4. Take the graph of Figure 3.4 and add $K = 2$ to every edge: $S \rightarrow A$ costs 3, $S \rightarrow B$ costs 4, $A \rightarrow T$ costs 3 and $B \rightarrow A$ costs 0. In the shifted graph S, A, T costs 6 and S, B, A, T costs 7, so the algorithm returns S, A, T ; in the original graph these paths cost 2 and 1, and S, B, A, T is optimal. A path with k edges gains exactly kK , so the shift penalises paths with more edges. It is harmless only when every path under comparison has the same number of edges, which is the case, for example, between two layers of the time-expanded graph of Chapter 2, where every path from time 0 to time T has exactly T edges.

Exercise 3.6. (a) The backward run from G settles, in this order, $h^*(G) = 0$, $h^*(F) = 1$, $h^*(E) = 2$, $h^*(D) = 5$, $h^*(C) = 7$, $h^*(B) = 8$, $h^*(A) = 10$. Consistency holds on every edge, with equality on the edges of the backward tree: for instance $h^*(D) = 5 \leq c(D, F) + h^*(F) =$

$8 + 1$, $h^*(A) = 10 \leq c(A, C) + h^*(C) = 5 + 7$ and $h^*(A) = 10 = c(A, B) + h^*(B) = 2 + 8$. Note that $h^*(A) = \delta(A, G) = 10$ agrees with Table 3.1. (b) A move into a hatched cell costs 3 but a move out of it costs 1, so the cost of a path depends on its direction, and $\delta(T, v) \neq \delta(v, T)$ exactly when the cheapest route between v and T enters a different number of hatched cells in the two directions. Running `dijkstra(grid, T)` and `backward_dijkstra_heuristic(grid, T)` shows that the two tables differ on the following cells (row, column): (1, 1): 10 forward against 8 backward, (1, 2): 13 forward against 11 backward, (1, 3): 10 forward against 8 backward, (2, 1): 9 forward against 7 backward, (2, 3): 7 forward against 5 backward, (3, 1): 6 forward against 4 backward, (3, 2): 5 forward against 3 backward, (3, 3): 4 forward against 2 backward. (c) The constraint removes the single state $((4, 2), 6)$ from the space-time graph. Every path that respects it is still a space-time path, and dropping the time index turns it into a walk in the grid of no larger cost, so $h^*(v) = \delta(v, T)$ remains a lower bound (item 3 of Theorem 3.11). It is no longer exact: with unit costs the optimal path of Example 3.5 reaches (4, 2) at time step 6, precisely when the cell is forbidden, so the drone must wait one step or take a detour and the true constrained cost from S is 9, while $h^*(S) = 8$.

Exercise 5.1. (a) $rhs(s)$ rises to the new value (unless a second predecessor offers the old total, in which case nothing changes) while $g(s)$ keeps the old, smaller value: s is underconsistent with key $[g(s) + h(s); g(s)]$, built from the old value. (b) $rhs(s)$ drops below $g(s)$: overconsistent, key $[rhs(s) + h(s); rhs(s)]$, built from the new value. (c) $rhs(s)$ is a minimum over the edges into s , so it does not change and s stays consistent; it is the head of the changed edge that `UpdateVertex` is called on and that may become underconsistent. (d) The minimum in Equation (5.1) is attained by the second predecessor with the same total, so $rhs(s)$ and hence the state of s do not change; `UpdateVertex` still runs on s (it is a successor of the popped vertex) but removes and re-inserts nothing.

Exercise 5.2. Blocking (1, 0) changes the eight edges around it, so `UpdateVertex` runs on (1, 0), (0, 0), (2, 0) and (1, 1). Only (1, 0) changes: its rhs was 2 through (1, 1) and becomes ∞ , which equals its g , so it simply leaves the queue. (0, 0) keeps $rhs = 1$ through S , (2, 0) keeps $rhs = 3$ through (2, 1), and (1, 1) keeps $rhs = 1$; none of them ever relied on (1, 0), whose g was still ∞ . The queue now holds nine cells with keys $[6; 1]$, $[6; 1]$, $[6; 2]$, $[6; 3]$, $[6; 3]$, $[6; 4]$, $[6; 4]$, $[6; 5]$, $[6; 5]$, the goal is consistent with $g(T) = 4$, and the smallest key $[6; 1]$ is not smaller than $k(T) = [4; 4]$: `ComputeShortestPath` returns after zero expansions and the path along the middle row stands. The difference to Table 5.2 is that (3, 1) carried a finite g -value on which the goal depended, whereas (1, 0) was never expanded, so no stored value was stale. Unblocking (3, 1) in the state of the right panel calls `UpdateVertex` on (3, 1) and its four neighbours; $rhs(3, 1)$ falls from ∞ to 3 through (2, 1), so (3, 1) is overconsistent with key $[4; 3]$, while the neighbours are unaffected because $g(3, 1)$ is still ∞ . The repair pops (3, 1) (key $[4; 3]$, $g \leftarrow 3$), which lowers $rhs(T)$ from 6 to 4 and queues T with key $[4; 4]$, then pops T ($g \leftarrow 4$); two expansions, and the path is the middle row again. A cost decrease can only lower rhs -values, so only overconsistent vertices appear, and the queued side cells with keys $[6; \cdot]$ are never touched.

Exercise 5.3. Take the vertices S, u, w, T with edges $S \rightarrow u$ (cost 1), $u \rightarrow w$ (1), $w \rightarrow T$ (1) and $u \rightarrow T$ (2), start S , goal T , and the exact heuristic $h(S) = 3$, $h(u) = 2$, $h(w) = 1$, $h(T) = 0$, which stays consistent when costs increase. Both routes to T cost 3, so every vertex has $k_1 = 3$ in the first search. With k_1 only and ties broken against us, T is expanded before w , and w stays in the queue with $g(w) = \infty$ and $rhs(w) = 2$ derived from $g(u) = 1$. Now raise $c(S, u)$ to 2 and $c(u, T)$ to 10 in one batch. `UpdateVertex` gives $rhs(u) = 2$ (u underconsistent, $k_1 = 1 + 2 = 3$) and $rhs(T) = \min(1 + 10, \infty) = 11$ (T underconsistent, $k_1 = 3$); w keeps $k_1 = 2 + 1 = 3$. Pop T (underconsistent, $g(T) \leftarrow \infty$, re-queued with $rhs = 11$), then w first among the ties: it is overconsistent and accepts the stale value $g(w) = 2$, which makes

$rhs(T) = 3$ and $k_1(T) = 3$. Pop u : $g(u) \leftarrow \infty$, so $rhs(w) = \infty$ and w is underconsistent with $k_1 = 3$, while u re-enters with $rhs(u) = 2$, $k_1 = 4$. Pop w a second time ($g(w) \leftarrow \infty$; T and w both become consistent at ∞ and leave the queue), pop u ($g(u) \leftarrow 2$, $rhs(w) = 3$, $rhs(T) = 12$), pop w a *third* time ($g(w) \leftarrow 3$, $rhs(T) = 4$) and finally T with $g(T) = 4$: seven expansions and w three times. If instead the tie after the second step is broken toward T , the loop even stops with the wrong value $g(T) = 3$. With the two-component key the first search expands w ($[3; 2]$) before T ($[3; 3]$), and after the change the pops are u $[3; 1]$, w $[3; 2]$, T $[3; 3]$ (all underconsistent), then u $[4; 2]$, w $[4; 3]$, T $[4; 4]$: six expansions, each vertex twice, and the correct cost 4.

Exercise 5.4. Let s be in U with the key stored when the robot stood at s_{last} and the modifier had the value k_m : $k_1^{\text{old}} = m + h(s_{\text{last}}, s) + k_m$ with $m = \min(g(s), rhs(s))$, and $k_2^{\text{old}} = m$. Neither $g(s)$ nor $rhs(s)$ changed since, because any change goes through **UpdateVertex**, which re-inserts s with a fresh key. After line 35 the current key is $k_1 = m + h(s_{\text{start}}, s) + k_m + h(s_{\text{last}}, s_{\text{start}})$, and the triangle inequality $h(s_{\text{last}}, s) \leq h(s_{\text{last}}, s_{\text{start}}) + h(s_{\text{start}}, s)$ gives $k_1 \geq k_1^{\text{old}}$ with $k_2 = k_2^{\text{old}}$, so the stored key is a lower bound. If the batch is the second one since the key was stored, apply the argument twice: the sum of the two legs is at least the direct distance, again by the triangle inequality. For the counterexample take Table 5.4: u was stored with $[10; 4]$; the robot moves two cells toward u and, without the modifier, the current key is $[8; 4]$, so the stored key is a strict upper bound. A vertex v inserted now with $[9; 6]$ is popped before u although its current key is larger, which breaks invariant (iii) of the proof sketch of Theorem 5.8, that vertices are expanded in nondecreasing order of their current keys; v may then be expanded with an rhs -value that still depends on u , and the loop may stop with u buried under an inflated key while $g(s_{\text{start}})$ is not the true distance. Finally, k_m must be accumulated: keys stored at different times were computed with different values of k_m and different robot positions, and each of them needs the offset of the legs walked *since its insertion*. The sum of the legs is at least $h(s_{\text{last}}, s_{\text{start}})$ for every one of them, whereas the direct distance $h(s_{\text{orig}}, s_{\text{start}})$ from the original start can shrink when the robot turns back (it is 0 if the robot returns to s_{orig}), and then keys computed later would be smaller than keys stored earlier, which is the upper-bound situation above.

Exercise ??. (a) $\pi_1[1] = \pi_2[0] = B$: a following conflict only; allowed. (b) $\pi_1[1] = \pi_2[0]$ and $\pi_2[1] = \pi_1[0]$: a swapping conflict, which is also a cycle conflict with $m = 2$ and a pair of following conflicts; forbidden. (c) $\pi_1[1] = \pi_2[1] = B$: a vertex conflict at $t = 1$; forbidden. (d) Both agents traverse $A \rightarrow B$: an edge conflict, which by ??(a) contains the vertex conflicts at A ($t = 0$) and at B ($t = 1$); forbidden. (e) Agent 2 waits at B while agent 1 enters it: a following conflict that is also the vertex conflict $\langle a_1, a_2, B, 1 \rangle$ by ??(d); forbidden.

Exercise ??. π_1 arrives at its goal $(0, 2)$ at $t = 3$ and never leaves, so $\text{cost}(\pi_1) = 3$: the wait at $(0, 1)$ before the arrival is paid, the two waits at the goal are free. π_2 starts at its goal and $\text{cost}(\pi_2) = 0$. π_3 reaches $(1, 1)$ at $t = 1$, leaves it at $t = 2$ and returns at $t = 3$; the last arrival after which the agent never leaves is at $t = 3$, so $\text{cost}(\pi_3) = 3$ by ??. Hence $\text{SoC} = 3 + 0 + 3 = 6$ and $\text{MS} = 3$. π_3 does not cost 1 because under stay-at-target the agent is only “done” once it stays at its goal forever, and an agent that leaves the goal was not done.

Exercise ??. Hints. (a) Two agents crossing an empty 3×3 grid through its centre, one horizontally and one vertically, meet at the centre at $t = 1$ and nowhere else. (b) Put the agents at the two ends of a corridor of *even* length and let them exchange ends: for four cells $(0, 0), \dots, (0, 3)$ the positions at $t = 1$ are $(0, 1)$ and $(0, 2)$, at $t = 2$ they are $(0, 2)$ and $(0, 1)$, so the agents swap along the middle edge without ever sharing a cell. In a corridor of odd length the two agents meet in the middle cell instead, which is a vertex conflict and not a swap. (c) A convoy: agent 1 from $(0, 0)$ to $(0, 3)$ and agent 2 from $(0, 1)$ to $(0, 4)$ on a single

row; agent 2 always leaves the cell that agent 1 enters. (d) Agent 1 from $(0, 1)$ to $(0, 2)$ (cost 1) and agent 2 from $(0, 0)$ to $(0, 3)$: agent 2 reaches $(0, 2)$ at $t = 2$ while agent 1 is parked there. (e) Four agents on a 2×2 block, each with its goal at the next cell clockwise, all move at $t = 0$; `all_conflicts` returns the empty list because a rotation is allowed, and the plan is valid. All five cases appear in the self-test of `ch07_mapf.py`.

Exercise ??. At an integer time the two drones sit at distinct lattice vertices, and distinct vertices of a lattice with cell side ℓ are at least ℓ apart. During a step, each drone moves along a segment of length ℓ or waits, and the squared distance between two points moving at constant velocity is a convex quadratic in the fraction $\tau \in [0, 1]$ of the step, so the minimum is attained at an endpoint or at a single interior point. Enumerate the pairs of moves that a conflict-free plan allows: two waits, a wait and a move, two parallel moves, two moves on edges without a common vertex, and following moves along the same line or around a corner (moving towards the same vertex is a vertex conflict, and reversing the same edge is a swap). All cases except the last keep the distance at ℓ or more, because the segments are parallel or separated by a full cell. For the corner case take drone 1 moving from $(0, 0)$ to $(\ell, 0)$ and drone 2 leaving $(\ell, 0)$ for (ℓ, ℓ) at the same time; their positions are $(\tau\ell, 0)$ and $(\ell, \tau\ell)$, the squared distance is $\ell^2((1 - \tau)^2 + \tau^2)$, and its minimum at $\tau = \frac{1}{2}$ is $\ell^2/2$, so the distance is $\ell/\sqrt{2}$. To keep two drones of diameter d at least m apart at all times one therefore needs $\ell/\sqrt{2} \geq d + m$, that is $\ell \geq \sqrt{2}(d + m)$; forbidding following conflicts removes the corner case and $\ell \geq d + m$ suffices.

Glossary

Admissible heuristic

A heuristic that never overestimates the true cost to the goal.

Bibliography

- [1] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese. “Social LSTM: Human Trajectory Prediction in Crowded Spaces”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 961–971.
- [2] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha. “Reciprocal n -Body Collision Avoidance”. In: *Robotics Research: The 14th International Symposium ISRR*. Vol. 70. Springer Tracts in Advanced Robotics. Springer, 2011, pp. 3–19.
- [3] M. Čáp, P. Novák, A. Kleiner, and M. Selecký. “Prioritized Planning Algorithms for Trajectory Coordination of Multiple Mobile Robots”. In: *IEEE Transactions on Automation Science and Engineering* 12.3 (2015), pp. 835–849.
- [4] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.
- [6] R. Dechter and J. Pearl. “Generalized Best-First Search Strategies and the Optimality of A*”. In: *Journal of the ACM* 32.3 (1985), pp. 505–536.
- [7] E. W. Dijkstra. “A Note on Two Problems in Connexion with Graphs”. In: *Numerische Mathematik* 1 (1959), pp. 269–271.
- [8] M. Erdmann and T. Lozano-Pérez. “On Multiple Moving Objects”. In: *Algorithmica* 2 (1987), pp. 477–521.
- [9] A. Felner. “Position Paper: Dijkstra’s Algorithm versus Uniform Cost Search or a Case Against Dijkstra’s Algorithm”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. 2011.
- [10] D. Ferguson and A. Stentz. “Using Interpolation to Improve Path Planning: The Field D* Algorithm”. In: *Journal of Field Robotics* 23.2 (2006), pp. 79–101.
- [11] P. Fiorini and Z. Shiller. “Motion Planning in Dynamic Environments Using Velocity Obstacles”. In: *The International Journal of Robotics Research* 17.7 (1998), pp. 760–772.
- [12] D. Fox, W. Burgard, and S. Thrun. “The Dynamic Window Approach to Collision Avoidance”. In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33.
- [13] M. L. Fredman and R. E. Tarjan. “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms”. In: *Journal of the ACM* 34.3 (1987), pp. 596–615.
- [14] A. A. Hagberg, D. A. Schult, and P. J. Swart. “Exploring Network Structure, Dynamics, and Function Using NetworkX”. In: *Proceedings of the 7th Python in Science Conference (SciPy)* (2008), pp. 11–15.
- [15] D. Harabor and A. Grastien. “Online Graph Pruning for Pathfinding on Grid Maps”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2011.
- [16] P. E. Hart, N. J. Nilsson, and B. Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.

- [17] P. E. Hart, N. J. Nilsson, and B. Raphael. “Correction to “A Formal Basis for the Heuristic Determination of Minimum Cost Paths””. In: *SIGART Newsletter* 37 (1972), pp. 28–29.
- [18] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [19] R. E. Kalman. “A New Approach to Linear Filtering and Prediction Problems”. In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.
- [20] S. Karaman and E. Frazzoli. “Sampling-Based Algorithms for Optimal Motion Planning”. In: *The International Journal of Robotics Research* 30.7 (2011), pp. 846–894.
- [21] S. Koenig and M. Likhachev. “D* Lite”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2002, pp. 476–483.
- [22] S. Koenig and M. Likhachev. “Fast Replanning for Navigation in Unknown Terrain”. In: *IEEE Transactions on Robotics* 21.3 (2005), pp. 354–363.
- [23] S. Koenig, M. Likhachev, and D. Furcy. “Lifelong Planning A*”. In: *Artificial Intelligence* 155.1–2 (2004), pp. 93–146.
- [24] S. M. LaValle. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. Tech. rep. TR 98-11. Computer Science Department, Iowa State University, 1998.
- [25] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [26] M. Likhachev, D. Ferguson, G. Gordon, A. Stentz, and S. Thrun. “Anytime Dynamic A*: An Anytime, Replanning Algorithm”. In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2005, pp. 262–271.
- [27] M. Likhachev, G. J. Gordon, and S. Thrun. “ARA*: Anytime A* with Provable Bounds on Sub-Optimality”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 16. 2003.
- [28] D. Mellinger and V. Kumar. “Minimum Snap Trajectory Generation and Control for Quadrotors”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 2520–2525.
- [29] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig. “Learning to Fly—A Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-Agent Quadcopter Control”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021, pp. 7512–7519.
- [30] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [31] M. Phillips and M. Likhachev. “SIPP: Safe Interval Path Planning for Dynamic Environments”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 5628–5635.
- [32] I. Pohl. “Heuristic Search Viewed as Path Finding in a Graph”. In: *Artificial Intelligence* 1.3–4 (1970), pp. 193–204.
- [33] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2nd ed. Nob Hill Publishing, 2017.
- [34] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras. “Human Motion Trajectory Prediction: A Survey”. In: *The International Journal of Robotics Research* 39.8 (2020), pp. 895–935.
- [35] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.

- [36] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. “Conflict-Based Search for Optimal Multi-Agent Pathfinding”. In: *Artificial Intelligence* 219 (2015), pp. 40–66.
- [37] D. Silver. “Cooperative Pathfinding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*. 2005, pp. 117–122.
- [38] A. Stentz. “Optimal and Efficient Path Planning for Partially-Known Environments”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1994, pp. 3310–3317.
- [39] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, et al. “Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. 2019, pp. 151–158.
- [40] X. Sun, W. Yeoh, and S. Koenig. “Moving Target D* Lite”. In: *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2010.
- [41] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

Index

- C^* , optimal cost, 66
- h^* , true cost to go, 66
- A*
 - as generalisation of Dijkstra, 69
 - complexity, 73
 - consistency, 71
 - $f = g + h$, 65
 - optimality theorem, 71
 - weighted, 74
- adjacency list, 22
- anytime algorithm, 8
- Anytime D*, 88
- arrival time, 27
- backward Dijkstra, 55
- Bayes' rule, 37
- Bellman–Ford algorithm, 53
- best-first search, 65
- bidirectional search, 57
- big-O notation, 37
- binary heap, 37
- bounded suboptimality, 8
- bounding sphere, 26
- breadth-first search, 58
- capstone, 138
- centralized, 7
- closed set, 68
- closest approach
 - time of, 34
- code files
 - self-test, 131
- collision distance, 29
- completeness, 8
- completion checklist, 140
- complexity
 - Dijkstra's algorithm, 54
- configuration, 25
- configuration space, 6, 25
 - free space, 25
 - obstacle region, 25
- conflict
 - swapping, 104
 - vertex, 104
- control, 7
- conventions of the book, 16
- cooperative agent, 7
- corner cutting, 23
- covariance, 35
- covariance ellipse, 35
- cross product
 - 2D, 32
- D*
 - original (Stentz), 88
- D* Lite, 82
 - backward search, 82
 - key modifier, 83
- decentralized, 7
- decision logic
 - capstone, 139
- deliberative, 7
- Dijkstra's algorithm, 46
 - backward, 55
 - bidirectional, 57
 - complexity, 54
 - correctness, 52
 - dense graphs, 54
 - early exit, 48
 - in the drone system, 61
 - multi-source, 55
 - settling order, 52
 - unit costs, 58
- dot product, 32
- double integrator, 30
- drone
 - controlled, 2
- expansion, 49
- experiment matrix, 140
- Fibonacci heap, 54
- Field D*, 88
- floating point
 - cost comparison, 59
- Gaussian

- multivariate, 35
- global planner, 9
- goal-stay check, 105
- graph, 2
 - directed, 22
 - implicit, 22
 - undirected, 22
 - weighted, 22, 47
- grid, 2, 23
 - 4-connected, 23
 - 8-connected, 23
 - obstacle cell, 23
 - occupancy, 23
 - stretch factor, 24
- heapq, 58
- heuristic, 65
 - admissible, 56, 66
 - backward consistent, 82
 - Chebyshev, 67
 - consistent, 56, 66
 - dominance, 73
 - Euclidean, 67
 - inadmissible, 72
 - Manhattan, 67
 - monotone, 66
 - octile, 67
 - true-distance, 55
- holonomic, 30
- hybrid architecture, 9
 - capstone, 130
 - core idea, 4
- incremental search, 8, 78
- intruder, 3
- key
 - two-component, 79
- key modifier, 83
- lab notebook, 131
- lattice
 - 26-connected, 24
 - 3D, 24
 - 6-connected, 24
- lazy deletion, 38, 48
- Lifelong Planning A*, *see* LPA*
- local consistency, 78
- local safety layer, 9
- LPA*, 79
 - g-value, 78
 - key, 79
 - local consistency, 78
 - rhs-value, 78
- Mahalanobis distance, 35
- makespan, 28
- map of the book, 12
- mean, 35
- metrics
 - capstone, 140
- Minkowski sum, 25
- move action, 26
- Moving Target D* Lite, 88
- multi-source Dijkstra, 55
- negative edge cost, 53
- non-cooperative agent, 7
- non-holonomic, 30
- obstacle
 - as infinite edge cost, 78
- obstacle inflation, 25
- open set, 68
- optimality, 8
- overconsistent vertex, 78
- parent pointer, 49
- path, 23
 - cost, 23
 - length, 28
 - time-indexed, 104
- path reconstruction, 49
- pitfall
 - static intruder, 5
 - time scales, 12
- plan, 7, 27
- planned conflict, 3
- planning, 7
 - continuous, 6
 - discrete, 6
 - global, 7
 - local, 7
- point-segment distance, 32
- prediction, 8
- prediction layer, 9
- prioritized planning, 102
 - plan, 104
- priority order, 104
- priority queue, 37
 - lazy deletion, 38, 48, 68, 88
- priority tags, 12
- Python stack, 18, 131

- random vector, 35
- re-opening, 71
- reactive, 7
- reading order, 142
- reading paths, 15
- real-time, 8
- relative velocity, 34
- relaxation, 47
- replanning, 8
- replanning layer, 10
- research directions, 14, 143
- reservation table, 103
 - definition, 104
 - parked agent, 104
- rhs-value, 78
- safety horizon, 10
- search problem
 - heuristic, 66
- separation, 29
 - minimum, 29
- settled-node invariant, 52
- shortest path, 23
 - distance, 47
 - incremental, 78
 - single-source, 45
- shortest-path tree, 47
- single integrator, 30
- space-time state, 26
- stale entry, 48
- stay-at-goal convention, 27
- stopping distance, 31
- study load, 130
- study plan, 16, 130–143
 - schedule, 131
 - timeline, 133
 - week 1, 134
 - week 2, 134
 - week 3, 134
 - week 4, 135
 - week 5, 135
 - week 6, 136
 - week 7, 136
 - week 8, 136
 - week 9, 137
 - week 10, 137
 - week 11, 138
 - week 12, 138
- sum of costs, 28
- swarm, 2
- tentative cost, 48
- tie-breaking, 59, 69
 - in D* Lite, 86
 - in LPA*, 82
 - larger g first, 74
- time scale
 - global, 11
 - reactive, 11
 - replanning, 11
- time to collision, 10, 34
- time-expanded graph, 27
- time-indexed path, 3, 7, 27
- traceability table, 140
- tracking, 8
- trajectory, 28
- travel time, 28
- true-distance heuristic, 56
- underconsistent vertex, 78
- unexpected obstacle, 3
- uniform-cost search, 47
 - correctness, 53
- wait action, 26
- waypoint, 28
- weighted A*, 74
- well-formed instance, 105
- workspace, 6, 25